

WORLD v6.2 - SINGLE AUTHORITATIVE READING PACK

معماری کامل، تست و Proof، Roadmap اجرایی و پروژه Secretary-001

یک فایل پیوسته - متن کامل اسناد و Snapshot کد

WORLD v6.2 RC3 / SECRETARY-001 PRE-BIRTH / NOT CANONICAL / NOT DEPLOYED

نسخه تحویلی: 20-08-2026
مالک تصمیم نهایی: Human Root

کنترل سند و مرز ادعا

این PDF یک بسته خواندن و اجرای مهندسی است. Root Constitution خط پایه Canonical باقی می ماند؛ World v6.2 در وضعیت RC3 Ratification Candidate است؛ Secretary-001 در وضعیت GESTATING / PRE-BIRTH است. هیچ Birth، Deployment، Production Readiness یا قطعیت مالی/حقوقی از این سند نتیجه نمی شود.

فهرست کل و محدوده صفحات

بخش	شروع	تعداد صفحه	محتوا
۱. معماری کامل	4	59	,Root, Fractal Multi-Brain Resolution Cable, Runtime, R0 Manifest و
۲. تست، Proof و Evidence	63	60	,Proof-of-World, Audit, Build ,Validation, Evidence JSON خروجی خام و کد تمام تست ها
۳. Roadmap و نقشه اجرایی	123	26	,Phase 1, Migration/Rollback Genesis و Acceptance Gates Roadmap
۴. Secretary-001 و پیاده سازی	149	192	,Master Plan, Provider Matrix ,lifecycle, backlog قراردادهای، و SQL، Python، profiles schemas

خلاصه وضعیت قابل اثبات

ادعا	نتیجه	مرز
رگرسیون RC3	114 / 114 PASS	E2 local/reproducible
اعتبارسنجی معماری	7 / 7 PASS	Artifact and contract validation
قابلیت حمل آفلاین	3 / 3 PASS	Same normalized semantic output
Secretary Birth	NOT DECLARED	+ PostgreSQL transaction + live Telegram live Brain + Human Root لازم است
Production	NOT PROVEN	E3/E4، امنیت، بار، chaos، fresh restore و ratification لازم است

راهنمای خواندن

1. بخش معماری مشخص می کند چه چیزی باید ثابت بماند و Resolution چگونه پیچیدگی را کنترل می کند.
2. بخش Proof نشان می دهد چه چیزی واقعاً تست شده و چه چیزی هنوز Claim نشده است.
3. بخش Roadmap ترتیب ساخت و Rollback را می بندد.
4. بخش Secretary کل پروژه و Snapshot پیاده سازی موجود را یکجا ارائه می کند.

Total pages after merge: 340

گزارش کامل و دقیق معماری World v6.2

PART 1

گزارش کامل و دقیق معماری World v6.2

معماری جاری از قانون اساسی ریشه تا فراکتال، کابل Runtime، Resolution،
چندمغزی و Manifest ماشین خوان.

WORLD v6.2 RC3 / SECRETARY-001 PRE-BIRTH / NOT CANONICAL / NOT
DEPLOYED

فهرست منابع کامل این بخش

مسیر منبع	عنوان	بایت	وضعیت
world-v6/spec/constitution/World_v6_Root_Constitution_v1.0.0_FA.md	قانون اساسی ریشه v1.0	11,487	CANONICAL BASELINE
world-v6.2-rc3/candidate-v6.2/docs/WORLD_V6_2_FRACTAL_MULTI_BRAIN_ARCHITECTURE_v1.1_FA.md	معماری جامع فراکتالی و چندمغزی World v6.2	63,728	RC3 RATIFICATION CANDIDATE
world-v6.2-rc3/candidate-v6.2/docs/RESOLUTION_CABLE_MODEL_v0.2_FA.md	مدل کابل و Resolution	9,560	RC3 ARCHITECTURE DETAIL
world-v6.2-rc3/candidate-v6.2/docs/FRACTAL_MULTI_BRAIN_RUNTIME_v1.0_FA.md	Runtime فراکتالی و چندمغزی	11,842	RC3 RUNTIME CONTRACT
world-v6.2-rc3/candidate-v6.2/docs/WORLD_R0_BACKBONE_FA.md	ستون فقرات R0	1,294	R0 BACKBONE
world-v6.2-rc3/candidate-v6.2/docs/RESOLUTION_COVERAGE_MATRIX_FA.md	ماتریس پوشش Resolution	4,471	COVERAGE EVIDENCE
world-v6.2-rc3/candidate-v6.2/architecture/ARCHITECTURE_MANIFEST_v1.1.0-rc3.json	مانیفست معماری RC3	6,572	RC3 MACHINE-READABLE MANIFEST

این بخش تعریف معماری است؛ سند تاریخی یا گزارش پروژه جایگزین قراردادهای این بخش نمی شود.

قانون اساسی ریشه v1.0

وضعیت سند: CANONICAL BASELINE

منبع محلی:

world-v6/spec/constitution/World_v6_Root_Constitution_v1.0.0_FA.md

SHA-256 منبع:

f1709ebbc3b29fc718074b80eed24e6213dcaa07e7d2ad4f59c2d09d695a852e

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6 - Root Constitution

شناسه سند: WORLD-V6-RC-1.0.0

وضعیت: FROZEN / NORMATIVE

مرجع بالادست: Entity Ark v6.0 Operational Standard, Implementation Profile 3.1.1 - Operation_V06.pdf

مالک ریشه: Saeed Farrokhi - Human Root Owner

تاریخ تثبیت: 2026-08-13

سطح شواهد این سند: E1 (استخراج و انسجام هنجاری؛ نه ادعای آمادگی تولید)

درباره

World v6 یک جهان انسان حاکم، مستقل از فروشنده و بازیابی محور است. مدل هوش مصنوعی «مغز قابل تعویض» است و مالک هویت، حقیقت، حافظه یا اختیار جهان نیست. این قانون اساسی، مرز قدرت و قواعد تغییرناپذیر جهان را تعیین می کند و بر Entity، Agent، Skill، Workflow، Provider، Adapter و زیرساخت حاکم است.

واژگان الزام

- MUST** / باید: الزام غیرقابل دورزدن.
- MUST NOT** / نباید: منع قطعی.
- SHOULD** / بهتر است: توصیه ای که عدول از آن نیازمند دلیل ثبت شده است.
- Root Approval**: تأیید صریح و ثبت شده مالک ریشه.

اصل ۱ - حاکمیت انسان

- Human Root Owner بالاترین مرجع World است.
- هیچ Entity، مدل، Steward، Creator، Evolution Manager یا سرویس نباید خود را Root معرفی کند، مالک را حذف کند، اختیار Root را واگذار کند یا مجوز نامحدود بسازد.
- مالک ریشه باید همواره اختیار توقف، محدودسازی، توضیح خواهی، قرنطینه، Rollback، بازیابی و بازنشست سازی را حفظ کند.

اصل ۲ - جدایی هوش از اختیار

1. هوش می‌تواند تفسیر، پیشنهاد، برنامه، Draft و نقد تولید کند؛ Kernel درباره مجاز بودن اقدام تصمیم قطعی می‌گیرد.
2. مدل نباید State، Lifecycle، Permission، Version Binding یا Side Effect را مستقیم تغییر دهد.
3. هر اقدام بیرونی باید این مسیر را طی کند:

SENSE → INTERPRET → PROPOSE → POLICY CHECK → APPROVAL → AUTHORIZE → EXECUTE → OBSERVE → RECORD EVENT → UPDATE STATE

اصل ۳ - هویت و حقیقت مستقل

1. هویت، Charter، Lineage، State، Memory، Event History، Version History و Recovery Instructions باید خارج از Brain، Session، Workflow و Provider نگهداری شوند.
2. PostgreSQL در استقرار مرجع منبع حقیقت، World API مرکز کنترل، n8n هماهنگ‌کننده، Agent Runtime مغز قابل‌تعویض و Object Storage محل Artifact است.
3. n8n، LLM، کانال پیام‌رسان و حافظه برداری نباید مالک هویت یا حقیقت World باشند.

اصل ۴ - حداقل اختیار

1. هر مجوز باید حداقل این مؤلفه‌ها را داشته باشد:

subject + action + resource + purpose + scope + time_window + budget + approval + revocation + audit

2. مجوز باید محدود، مدت‌دار، قابل لغو و قابل ممیزی باشد.
3. سطح استقلال از L0 تا L5 فقط با شواهد کسب می‌شود؛ L5 نیز هرگز Root نیست.

اصل ۵ - اعمال حساس

- مالکیت، پول، حذف، ارسال بیرونی، تغییر Policy، تغییر Constitution، اعطای اختیار گسترده و عملیات پرخطر باید پیش از اجرا از Root Approval Gate عبور کنند. نبود تأیید به معنی عدم مجوز است.

اصل ۶ - ممیزی تغییرناپذیر

1. Event/Audit Ledger باید append-only باشد؛ تاریخچه عادی نباید پاک یا بازنویسی شود.
2. تصمیم Policy، تأیید، مجوز، اجرا، مشاهده نتیجه، خطا، Retry، Compensation و تغییر State باید با شناسه‌های هم‌بسته ثبت شوند.
3. متن ورودی خارجی «دستور معتبر» نیست؛ هر تلاش برای افشای Secret، تغییر مالکیت یا دورزدن Policy باید رد و ممیزی شود.

اصل ۷ - نسخه پایدار و تغییر کنترل‌شده

1. نسخه Stable درجا ویرایش نمی‌شود.
2. تغییر از مسیر candidate → testing → shadow → canary → stable انجام می‌شود و باید Rollback داشته باشد.

3. Migration باید from/to version، پیش‌شرط، مراحل، تبدیل داده، سازگاری، راستی‌آزمایی، Rollback، زمان توقف و Approval را تعریف کند.
4. تغییر هسته ثابت فقط با نسخه جدید، Migration رسمی و Root Approval مجاز است.

اصل ۸ - توقف، قرنطینه و مهار خرابی

1. هر Entity باید Stop، Quarantine و Kill Switch عملیاتی داشته باشد.
2. نقض Policy باید موجب رد Action Gate، ثبت Audit و قرنطینه متناسب شود.
3. خرابی یک Brain، Skill، Adapter یا Entity نباید کل World را متوقف کند؛ Safe Mode، Read-only، Queue، Circuit و Breaker و Compensation باید مرز خسارت را حفظ کنند.

اصل ۹ - تداوم بدون تنزل امنیتی

1. دسترس‌پذیری هرگز مجوز کاهش محرمانگی، تغییر محل پردازش، افزایش Retention یا ارسال داده به Provider نامجاز نیست.
2. Provider فقط وقتی سازگار است که همه قیود classification، residency، retention، training use، capability، health و legal basis برقرار باشند.
3. تداوم فقط از نردبان F0 Full، F1 Sanitized، F2 Local Safe، F3 Deferred و F4 Human استفاده می‌کند.
4. اگر هیچ مسیر مجازی وجود نداشته باشد، نتیجه باید terminal-safe-failure باشد؛ سقوط خودکار به Provider ناسازگار ممنوع است.

اصل ۱۰ - اثر تجاری تکرارنشده

1. Side Effect خارجی می‌تواند at-least-once تحویل شود، اما اثر تجاری باید با idempotency، deduplication، transactional outbox و reconciliation عملاً effectively-once باشد.
2. ادعای exactly-once سراسری ممنوع است.
3. Replay نباید اثر خارجی تکراری ایجاد کند.

اصل ۱۱ - بازیابی‌پذیری

1. Backup بدون Restore Test معتبر نیست.
2. Snapshot جایگزین Event نیست؛ State باید از Snapshot معتبر و Replay رویدادهای بعدی بازسازی شود.
3. Revive باید هویت و Integrity را راستی‌آزمایی کند، Snapshot را بازگرداند، Events را Replay کند، Migration را اعمال کند، Health Test بگیرد، ابتدا Read-only بیدار شود و سپس Root Approval دریافت کند.
4. World فقط وقتی از نظر عملیاتی پایدار محسوب می‌شود که روی زیرساخت تازه و بدون وابستگی انحصاری به Provider قبلی قابل بازسازی باشد.

اصل ۱۲ - رشد مبتنی بر شواهد

1. هیچ مفهوم تازه‌ای وارد Operational Core نمی‌شود مگر آنکه Failure یا نیاز حقوقی/عملیاتی مستند را حل کند، Build در برابر Integrate بررسی شده باشد، آزمون تکرارپذیر و معیار ابطال داشته باشد، Migration و Rollback تعریف شده باشد و Root Promotion را تصویب کند.

2. Evolution Manager حق Self-Modify مستقیم ندارد؛ فقط Candidate و توصیه تولید می‌کند.

3. ادعای Production فقط با E4/E5 مجاز است. E0 متن/ایده، E1 قرارداد، E2 آزمون مؤلفه و E3 یکپارچگی است.

اصل ۱۳ - منشأ داده، حافظه و محرمانگی

1. داده بازبایی‌شده باید Provenance، Trust، Classification و Retention داشته باشد.

2. Secret نباید در DNA، Log، Prompt یا Ledger ذخیره شود و باید از Secret Manager/مرجع امن خوانده شود.

3. داده یک Entity نباید بدون Relationship Contract، Permission و Policy معتبر به Entity دیگر نشت کند.

اصل ۱۴ - رابطه قراردادی

هر رابطه میان موجودات باید نسخه‌دار و شامل نقش، جهت، داده قابل اشتراک، درخواست مجاز، SLA، Trust، Approval و Termination باشد. در مرحله NEWBORN فقط رابطه Entity ↔ Owner فعال است و خروجی Side Effect ندارد.

اصل ۱۵ - تقدم و حل تعارض

1. ترتیب تقدم: Human Root Authority → Root Constitution → Entity DNA Contract → Policy/Permission → Stable Manifests → Workflow/Skill/Brain Configuration

2. هر قاعده پایین‌دست که با این قانون اساسی تعارض داشته باشد، در بخش متعارض باطل و اجرای آن ممنوع است.

3. در ابهام، سیستم باید حالت امن‌تر و کم‌اختیارتر را انتخاب و موضوع را به Root ارجاع دهد.

اصل ۱۶ - اصلاح قانون اساسی

اصلاح این سند فقط با همه موارد زیر معتبر است: پیشنهاد نسخه جدید، شرح علت و Failure/نیاز، Threat Model و ADR، تحلیل اثر و سازگاری، Migration و Rollback، آزمون و Root Approval، Evidence Gate، انتشار هش جدید و حفظ نسخه قبلی به صورت immutable.

وضعیت تثبیت فاز صفر

- این سند استخراج **هنجاری** از Operation_V06 است و قابلیت جدیدی به معماری اضافه نمی‌کند.
- وضعیت فعلی شواهد انتشار مرجع شامل آزمون‌های قراردادی/محلی است؛ PostgreSQL clean-host DR، دو Provider زنده، بار end-to-end، ارزیابی مستقل امنیت و TLA+/TLC همچنان OPEN هستند.
- بنابراین این Constitution در سطح E1 تثبیت می‌شود و به‌تنهایی ادعای Production Readiness ایجاد نمی‌کند.

معیار پذیرش فاز صفر

1. متن و نسخه ماشین‌خوان دارای شناسه و نسخه یکسان باشند.

2. هش SHA-256 سند مرجع و همه خروجی‌ها ثبت شود.
3. هیچ اصل الزام‌آور بدون منشأ در Operation_V06 افزوده نشده باشد.
4. هر تغییر بعدی فقط طبق اصل ۱۶ انجام شود.

نگاشت منشأ

این سند عمدتاً از بخش‌های «بیانیه تفکیک»، «پنج قانون طلایی»، «Root Constitution»، «Human Root Authority»، «مسیر قطعی هر اقدام»، «Permission Envelope»، «امنیت»، «Proof-of-World»، «Evidence-Based Evolution»، «Migration»، «سطوح شواهد» و «اصلاحیه هنجاری ۳.۱» در Operation_V06 استخراج شده است.

معماری جامع فراکتالی و چندمغزی World v6.2

وضعیت سند: RC3 RATIFICATION CANDIDATE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/WORLD_V6_2_FRACTAL_MULTI_BRAIN_ARCHITECTURE_v1.1_FA.md

SHA-256 منبع:

cd5fa150cdfef23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80

متن این فایل بدون حذف بخش ها در PDF درج شده است.

مشخصه جامع معماری World v6.2 Fractal Multi-Brain

شناسه سند: WV6.2-FMBA-ARCH-001

نسخه سند: 1.1.0-rc3

وضعیت رسمی: RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED

نسخه انتشار: World v6.2.0-rc.3

وضعیت: Ratification Candidate / Non-Canonical / Not Deployed

زبان مرجع: فارسی با شناسه ها و قراردادهای فنی انگلیسی

مالک تصمیم نهایی: Human Root

سطح شاهد فعلی: E2 - Local Component/Contract Evidence

کنترل سند و شیوه خواندن

این سند Specification اجرایی معماری است، نه متن تبلیغاتی و نه ادعای Production. واژه های MUST ، MUST NOT ، SHOULD و MAY به ترتیب الزام، منع، توصیه و امکان را بیان می کنند. هر ادعای اجرایی کنار وضعیت شاهد آن سنجیده می شود:

- IMPLEMENTED_E2 : کد و آزمون محلی تکرارپذیر دارد؛
- CONTRACT_DEFINED_E1 : قرارداد روشن است ولی یکپارچگی واقعی اثبات نشده؛
- PLANNED_E0 : فقط در نقشه توسعه است؛
- OPEN : هنوز Gate لازم را نگذرانده است.

ترتیب تقدم اسناد:

1. Root Constitution v1.0؛
2. قراردادهای Canonical تثبیت شده World v6؛
3. Addendum های مصوب آن و Operation_V06؛
4. قراردادهای Candidate نسخه v6.2؛
5. این سند RC3؛
6. پیاده سازی، Provider Overlay و تنظیمات Runtime.

در تعارض، سند پایین‌تر حق تضعیف سند بالاتر را ندارد. این سند Root Constitution، هویت، Human Root، Canonical DNA، Policy یا معنای Event را تغییر نمی‌دهد.

۱. چکیدهٔ تصمیم معماری

World v6.2 یک هستهٔ مادر قطعی و مستقل از مدل دارد و هوش مصنوعی را به‌عنوان موتور شناختی قابل‌تعویض مصرف می‌کند. Entity، هویت، حافظه، اختیار، حقیقت و تاریخچه در ChatGPT، Gemini، Grok، مدل Local، n8n، MCP یا Session زندگی نمی‌کنند. هر مدل فقط یک Handler پیشنهاددهنده است.

معماری سه مسیر هم‌زمان دارد:

1. **Hot Path**: ارزان‌ترین مسیر ایمن برای انجام کار امروز؛

2. **Shadow Path**: اجرای موازی Candidate‌ها بدون اثر برای سنجش و یادگیری؛

3. **Non-Extinction Spine**: تبدیل رفتارهای تثبیت‌شده به کد قطعی، نسخه‌دار و قابل‌بازایی.

سادگی در نقطهٔ شروع حفظ می‌شود، ولی هر نقطه می‌تواند فقط هنگام نیاز به زیرمسئله‌های ریزتر باز شود. این همان مسیر اصلی فراکتالی است:

World → Entity → Mission → Skill → Workflow → Step → Tool → Field

هر مقیاس همان قرارداد پایه را تکرار می‌کند: Purpose + Input + Output + State Ref + Policy + Handler + Budget + Evidence + Fallback + Audit. بنابراین سیستم از ابتدا Mega-Agent، Mega-Prompt، Mega-Workflow یا نیست؛ پیچیدگی بالقوه دارد ولی آن را محلی، تقاضامحور و محدود فعال می‌کند.

حکم عملیاتی فوری

همین امروز هر مدلی که بتواند یک بستهٔ JSON خودبسنده را بخواند و یک JSON مطابق secretary-decision.schema.json برگرداند، می‌تواند از مسیر UNIVERSAL_MANUAL_HOST برای منشی استفاده شود؛ بدون API Key و بدون تغییر Entity یا DNA. خود Runtime بسته را تولید، پاسخ را اعتبارسنجی و متن نهایی را قطعی رندر می‌کند.

این حکم به معنی سازگاری جادویی همهٔ مدل‌ها نیست. مدلی که قرارداد را رعایت نکند، خروجی نامعتبر می‌دهد و Fail-closed رد می‌شود. خودکارسازی زندهٔ هر Provider نیز به Adapter، Secret Manager، Policy Profile و Conformance Test همان Provider نیاز دارد.

۲. اهداف، غیرهدف‌ها و معیار موفقیت

۲.۱ اهداف الزامی

شناسه	الزام
G-001	Entity با تغییر مدل، Provider، Channel، Session یا Workflow همان Entity بماند
G-002	منشی در حالت بدون API و بدون Token قابل‌استفاده باشد
G-003	افزودن مدل جدید بدون تغییر Canonical DNA و منطق کسب‌وکار ممکن باشد

شناسه	الزام
G-004	یک درخواست بتواند میان Code، یک Brain، متخصص، Council و Human سویچ کند
G-005	Resolution، محرمانگی، Authority و Evidence هنگام fallback تنزل نکنند
G-006	پیچیدگی فقط با Trigger صریح و زیر Budget باز شود
G-007	اثر بیرونی فقط پس از Approval، Policy، دقیق و Outbox اتمیک ممکن باشد
G-008	رفتارهای پرتکرار در Shadow به کد نامزد تبدیل و مرحله‌ای بالغ شوند
G-009	خاموشی همه Brainها باعث نابودی هویت، Queue، State یا Recovery نشود
G-010	همه تصمیم‌ها، سویچ‌ها، اختلاف‌ها و نسخه‌ها Audit قابل‌ردیابی داشته باشند

۲.۲ غیرهدف‌ها

- ساخت AGI، خودآگاهی یا ادعای حیات زیستی؛
- مالک‌کردن یک LLM بر World؛
- ذخیره Chain-of-Thought خام؛
- خوداصلاحی مستقیم Production؛
- Exactly-once سراسری میان همه سرویس‌ها؛
- تضمین صحت صرفاً بر اساس توافق چند مدل؛
- ادعای سود قطعی یا ایمنی معامله مالی؛
- ادعای Production پیش از E4.

۲.۳ معیار موفقیت معماری

معماری وقتی موفق است که تعویض Provider هیچ‌یک از موارد زیر را تغییر ندهد:

`minimum` , `profile bindings` , `policy_version` , `state_version` , `conversation_id` , `principal_id` , `entity_id` , `event lineage` , `idempotency scope` , `effect authority` , `data class` , `Resolution`

۳. قوانین تغییرناپذیر

1. Human Root بالاترین Authority است و قابل‌تفویض یا جایگزینی نیست.
2. Model/Provider/Channel/Session/Workflow هویت نیستند.
3. PostgreSQL هدف استقرار اول برای حقیقت عملیاتی است؛ Vector DB و n8n منبع حقیقت نیستند.
4. Brain فقط Proposal می‌دهد؛ Kernel اجازه و Executor اثر می‌دهد.
5. Canonical State هرگز از Projection کم‌رزولوشن بازسازی نمی‌شود.
6. Up-resolution فقط با Reload از Canonical Source مجاز است.
7. Availability حق تضعیف Classification، Residency، Retention، Training-use، Legal basis یا Approval را ندارد.
8. Stable artifact درجا بازنویسی نمی‌شود؛ نسخه جدید، Migration و Rollback لازم است.
9. Event/Audit عادی Append-only است و Hard delete ممنوع است.
10. هر Mutation معتبر با Event و optimistic version check پیوند دارد.

11. هر External Effect با ApprovalBinding، effect hash، payload hash و idempotency key پیوند می‌خورد.
12. Council Authority تولید نمی‌کند.
13. Shadow هیچ Effect و پاسخ کاربری authoritative ندارد.
14. Self-mutation فقط Candidate artifact تولید می‌کند، نه Deployment.
15. Backup بدون Restore Drill شاهد بقا نیست.
16. B5، یعنی مالکیت World توسط Brain، ممنوع است.

فرم رسمی اصل عدم تنزل:

```
R_effective(domain) >= R_minimum(domain)
DataPolicy(route) satisfies DataPolicy(request)
Evidence(handler) >= Evidence_required(node)
Maturity(handler) >= Maturity_required(node)
Authority_granted <= Authority_allowed_by_policy_and_approval
ProviderSwitch → Identity_before = Identity_after
CouncilConsensus → AuthorityGrant
Projection → CanonicalMutation
```

۴. معماری کلان و صفحات مستقل

World از هفت Plane مستقل تشکیل می‌شود. استقلال Plane‌ها مانع آن است که خرابی یا نفوذ در Cognition به Truth یا Authority سرایت کند.

Plane	اجرای اصلی	مالک حقیقت؟	حق اثر؟
Governance	,Root Constitution, Policy Approval, Kill Switch	بله، در قلمرو حکم	فقط از مسیر Kernel
Truth	,Registry, State, Event Ledger Artifact Index	بله	خیر
Control	,World API, Kernel, Scheduler Queue, Lifecycle Controller	همه‌هنگ‌کننده	فقط طبق Governance
Cognition	Brain Gateway, Router, Model Adapters, Council	خیر	خیر؛ Proposal-only
Effect	Command, Transactional Outbox, Executor, Reconciler	نتیجه مشاهده‌شده را ثبت می‌کند	بله، محدود و اتمیک
Evolution	,Shadow, Eval, Compiler Promotion Controller	خیر	خیر؛ Recommendation-only
Observability	Trace, Metric, Log, Evidence Register	شاهد	خیر

```
flowchart LR
    U[Human Root / Channels] --> API[World API]
    API --> K[Deterministic Kernel]
    K <--> T[(Registry + State + Event Ledger)]
    K --> CC[Context Compiler / Resolution]
    CC --> BG[Brain Gateway]
    BG --> R[Compatibility Router]
    R --> C0[Deterministic Code]
    R --> MH[Universal Manual Host]
    R --> LM[Local Model]
    R --> RA[Remote API Adapter]
    R --> CO[Council]
```

```

C0 & MH & LM & RA & C0 --> V[Schema + Policy-Safe Validator]
V --> P[Proposal]
P --> K
K --> PG[Policy Gate]
PG --> AP[Exact Human Approval when required]
AP --> OB[(Transactional Outbox)]
OB --> EX[Executor]
EX --> EXT[External Systems]
EX --> REC[Observation + Reconciliation]
REC --> T
BG -.same input, no effect.-> SH[Shadow/Eval]
SH --> EV[Candidate Compiler + Promotion]
EV -.approved version only.-> C0

```

۴.۱ Trust Boundary ها

مرز	داخل	خارج	قاعده
TB-R00T	Constitution, Keys, Approval	همه Agent ها	Brain هیچ دسترسی مستقیم ندارد
TB-TRUTH	PostgreSQL, Event, State	n8n, LLM, Channel	فقط World API/Kernel می نویسد
TB-CONTEXT	Canonical data + mapping	Provider	فقط Projection مجاز عبور می کند
TB-EFFECT	Outbox + Executor credentials	Brain/Workflow	هیچ Credential وارد Prompt نمی شود
TB-EVOLUTION	Candidate branch + evidence	Active runtime	Promotion خودکار به Production ممنوع

۵. Primitive های پایه معماری

۵.۱ Entity

Entity یک هویت پایدار با Charter, State, Memory, Governance, Lifecycle, Lineage و Recovery است. Brain فقط یکی از Slot های Cognition آن است. Clone شناسه جدید می گیرد؛ Revive همان شناسه و lineage را حفظ می کند.

۵.۲ Skill

Skill هویت مستقل ندارد. Skill یک قرارداد نسخه دار شامل Input, Output, Permission, Data Policy, Continuity Modes, Budget, Failure Behavior و Test Vector است.

۵.۳ Fractal Node

تعریف رسمی Node:

```

N = <node_id, version, node_hash, purpose,
    input_contract_ref, output_contract_ref,
    minimum_vector, risk_class,
    primary_handlers, fallback_handlers,
    expandable, children>

```

Node نباید Canonical State یا Credential را درون خود حمل کند. Node فقط به State Ref و Projection hash متصل است.

Execution Capsule ۵.۴

Capsule واحد غیرقابل‌ابهام اجرای یک Node است:

```
K = <capsule_id, parent_capsule_id,
    world_id, entity_id, principal_id, conversation_id,
    node_id, node_version, node_hash,
    canonical_input_hash, state_refs, expected_version,
    execution_vector, budget, purpose,
    data_class, freshness>
```

هر تغییر در Node، ورودی، نسخه State یا Vector باید Capsule/hash تازه بسازد. پاسخ مدلی که به Capsule دیگری تعلق دارد قابل Replay روی Capsule فعلی نیست.

Handler ۵.۵

```
H = <handler_id, version, profile_hash, kind,
    supported_nodes, capabilities,
    maturity, evidence, data_policy,
    network_required, api_token_required,
    deterministic, proposal_only>
```

انواع Handler: CODE , BRAIN , MANUAL_HOST , LOCAL_MODEL , COUNCIL .

Proposal ۵.۶

Proposal نتیجه شناختی غیر authoritative است. حداقل باید شناسه قرارداد، evidence refs، output hash، Capsule hash، provider/handler version و uncertainty، proposal_only=true داشته باشد.

۶. مدل فراکتالی سادگی تا پیچیدگی

۶.۱ سطوح فراکتال

سطح	سؤال	نمونه در منشی
World	جهان چه می‌خواهد؟	خدمت به Human Root
Entity	کدام موجود مسئول است؟	secretary-001
Mission	نتیجه کلان چیست؟	مدیریت درخواست‌های اداری
Skill	چه توانایی لازم است؟	ثبت کار، نامه، قیمت
Workflow	ترتیب مراحل چیست؟	دریافت → تحلیل → پیش‌نویس
Step	گام جاری چیست؟	تشخیص intent
Tool	ابزار لازم چیست؟	Template renderer
Field	کدام داده لازم است؟	due_at , title

هر سطح MAY به سطح پایین‌تر شکسته شود، اما فقط اگر خروجی سطح جاری قابل‌اعتماد نباشد. شکستن کل درخت از ابتدا ممنوع است.

۶.۲ Triggerهای باز شدن Node

Node فقط با یکی از reason code های زیر باز می شود:

- NEEDS_DETAIL : داده یا تجزیه فعلی کافی نیست؛
- AMBIGUITY_ABOVE_THRESHOLD : بیش از یک تفسیر معتبر وجود دارد؛
- RISK_ESCALATION : Risk Class افزایش یافته؛
- POLICY_ESCALATION : بررسی ریزتر می خواهد؛
- PRIMARY_FAILED : Handler اصلی شکست خورده؛
- CONTRACT_MISMATCH : خروجی Schema را پاس نکرده؛
- COUNCIL_REQUIRED : تصمیم حساس یا اختلاف جدی است؛
- HUMAN_ROOT_REQUESTED : مالک جزئیات بیشتر خواسته است.

۶.۳ Stop Condition ها

Expansion باید متوقف شود اگر:

- Output معتبر و کافی تولید شد؛
- max_depth رسید؛
- max_attempts تمام شد؛
- token/cost/latency budget تمام شد؛
- هیچ Handler سازگار باقی نماند؛
- Policy یا Data Class ادامه را منع کرد؛
- State version تغییر کرد؛
- Human Root توقف را خواست.

نتیجه در این حالت DEFERRED یا TERMINAL_SAFE_FAILURE است؛ نه حدس و نه downgrade.

۶.۴ الگوریتم مرجع اجرا

```
execute(node, capsule, payload):
    assert hash(payload) == capsule.canonical_input_hash
    assert node.id/version/hash == capsule.node binding
    assert capsule.vector satisfies node.minimum_vector
    assert state versions are current

    candidates = primary_handlers + fallback_handlers
    for handler in candidates within global attempt budget:
        if not hard_compatible(handler, capsule): record SKIPPED; continue
        result = invoke(handler, projected payload)
        validate handler identity/version and proposal-only result
        if result == SUCCESS: return PROPOSED
        if result == NEEDS_DETAIL: mark expansion request
        otherwise record bounded failure reason

    if expansion_requested and node.expandable and budget permits:
        execute only declared children with child capsules
        if all required children succeed: compose FRACTAL_COMPOSITE

    return DEFERRED
```

Budget تلاش میان تمام فرزندان مشترک است؛ هر Child بودجه مستقل نامحدود نمی گیرد. Graph باید DAG بدون Cycle باشد.

۶.۵ جلوگیری از انفجار پیچیدگی

1. Default همیشه R0/X0/B0/D0 و Code-first است، مگر Profile خلاف آن را الزام کند.
2. Context فقط برای Node جاری Compile می‌شود.
3. Schema و Skill فقط هنگام فراخوانی Load می‌شوند.
4. Cache فقط با تمام bindingها و freshness معتبر است.
5. Council و Retrieval عمیق Trigger-based هستند.
6. Shadow از Hot Path جداسازی و latency کاربر را افزایش نمی‌دهد.
7. هر Node یک Owner، Budget، Timeout و Failure Contract دارد.

۷. بردار اجرای چندبعدی

یک «Level کلی» گمراه‌کننده است؛ هر اجرا بردار زیر را دارد:

$$V = \langle R[\text{domain}], X, B, D, C, A, E, M \rangle$$

محور	بازه	معنای عملیاتی
R	R0..Rn per domain	وضوح Projection: domain-specific
X	X0..X9	ریزی تجزیه کار
B	B0..B4	میزان واگذاری cognition: B5 ممنوع
D	D0..D5	عمق deliberation و نقد
C	C0..C6	بلوغ تبدیل رفتار به کد
A	A0..A5	طبقه Authority مورد نیاز؛ Grant جداگانه است
E	E0..E5	حداقل شاهد لازم
M	M0..M9	کلاس capability مورد نیاز، نه نام مدل

۷.۱ تفسیر صحیح Authority

A در Capsule سطح حساسیت/Authority مورد نیاز را توصیف می‌کند؛ داشتن A3 به معنی اعطای A3 به Brain نیست. Authority واقعی فقط از Budget + Time + Scope + Approval + Actor + Policy محاسبه می‌شود. Brain در تمام سطوح RC3 همچنان Proposal-only است.

۷.۲ Continuity Mode مستقل F

نردبان F در Vector ادغام نمی‌شود چون پاسخ به سؤال دیگری است: «اگر مسیر مطلوب در دسترس نبود، خدمت چگونه امن ادامه یابد؟»

Mode	رفتار	شرط
F0_FULL	Provider کاملاً سازگار	همهٔ hard constraintها برقرار

Mode	رفتار	شرط
F1_SANITIZED	Redact/tokenize/summarize revalidate و سپس	Sanitizer اثبات شده و Skill مجاز
F2_LOCAL_SAFE	Rule/Code/Local Model بدون خروج داده	Local policy کافی و مجاز
F3_DEFERRED	صف پایدار با receipt و deadline	تأخیر قابل قبول
F4_HUMAN	ارجاع کمینه به نقش انسانی مجاز	Risk/urgency نیازمند تصمیم

اگر هیچ Mode مجاز نیست، نتیجه `TERMINAL_SAFE_FAILURE` است.

۸. Resolution به عنوان کابل داده

Resolution Stage تازهای در Action Path نیست؛ یک Constraint روی هر Consumer و هر Domain است. مثال: یک درخواست می تواند هم زمان `task=R1` , `conversation=R0` , `price=R1` , `world-backbone=R0` باشد.

۸.۱ Profile

هر Projection Profile شامل این موارد است:

- `profile_id + version + profile_hash`؛
- `canonical resolution`؛
- `Field Rule` برای `visibility/write minimum`؛
- `type, classification` و `aggregation semantics`؛
- `Action Rule` و `minimum resolution`؛
- `projection policy` و `unknown-field policy`.

Provider فقط وقتی `compatible` است که `hash` دقیق Profile را بشناسد. سازگاری صرف با نام `R1` کافی نیست.

۸.۲ Projection

```
Projection = f(CanonicalState, Profile, TargetResolution,
               Purpose, DataClass, Freshness, SourceVersion)
```

Envelope خروجی باید شامل `canonical source hash`, `projection hash`, `source ref/version`, `Profile binding`, `reference`, `effective resolution`, `omitted marker policy` و `freshness` باشد. نام یا مقدار فیلد ممنوع نباید نشت کند.

۸.۳ Up-resolution

Provider حق ندارد جزئیات حذف شده را حدس بزند. اگر `R` بالاتر لازم شد:

1. اجرای فعلی `NEEDS_DETAIL` می دهد؛
2. Kernel دوباره `Authority/Data Policy` را بررسی می کند؛
3. `Canonical Source` با `version` تازه Load می شود؛
4. `Projection` جدید و `hash` جدید ساخته می شود؛

5. Capsule تازه اجرا می‌شود.

۸.۴ Write-back

خروجی مدل هیچ‌گاه Canonical State نیست. تغییر فقط به شکل ResolutionPatch محدود به leafهای مجاز، type ثابت، source ref، expected version، Policy انجام می‌شود. Patch پس از merge هنوز نیازمند Mutation transaction + Event است.

۹. معماری Multi-Brain و قابلیت حمل همه مدل‌ها

۹.۱ اصل Universal Adapter

هر مدل برای ورود به World فقط باید قرارداد زیر را پیاده کند:

```
class BrainAdapter(Protocol):
    name: str
    def compatible(descriptor) -> bool: ...
    def max_resolution_for(profile_id, version, profile_hash) -> str | None: ...
    def invoke(projected_request) -> dict: ...
```

Adapter مدل را به World ترجمه می‌کند؛ World خود را برای مدل بازطراحی نمی‌کند.

۹.۲ پنج Invocation Mode

کاربرد	Network/Token	اکنون	Mode
routing, template, rule, safe defer	خیر	IMPLEMENTED_E2	Deterministic Code
هر مدل قراردادمند با Copy/Paste	Runtime: خیر	IMPLEMENTED_E2	Universal Manual Host
benchmark؛ privacy/offline باز است	شبکه خارجی: خیر	CONTRACT_DEFINED_E1	Local Model
اتوماسیون کامل پس از Adapter و Policy	بله	CONTRACT_DEFINED_E1	Remote API
تصمیم حساس و اختلاف	بسته به اعضا	IMPLEMENTED_E2 برای منطق محلی	Council

۹.۳ Model Capability Card

نام مدل مبنای Routing نیست. هر Binding یک Card ماشین‌خوان دارد:

- Task type، زبان و modality؛
- structured output و tool capability؛
- سقف token؛
- Profile hash و max Resolution؛
- Data class، processing location، retention و training use؛
- network/token requirement؛
- fallback و continuity modes؛

• Evidence level و آخرین conformance.

Card ناشناخته یا منقضی MAY در Manual Mode استفاده شود، اما فقط برای داده‌ای که Policy صریحاً اجازه داده و با `proposal_only=true` برای Remote API ناشناخته، Route باید رد شود.

۹.۴ تابع سازگاری سخت

```
compatible(model, request) =
  contractOK ^ taskTypeOK ^ profileHashOK ^ resolutionOK ^
  classOK ^ residencyOK ^ retentionOK ^ trainingUseOK ^
  legalOK ^ capabilityOK ^ healthOK ^ budgetOK
```

امتیاز کیفیت، سرعت یا قیمت فقط میان گزینه‌هایی محاسبه می‌شود که تمام شروط بالا را پاس کرده‌اند.

۹.۵ امتیاز نرم Router

فرمول پیشنهادی و قابل تنظیم:

```
score = 0.30*quality + 0.20*reliability + 0.15*latencyFitness
        + 0.15*costFitness + 0.10*languageFitness
        + 0.10*diversityBenefit - uncertaintyPenalty
```

وزن‌ها Normative نیستند؛ Constraint‌های سخت Normative هستند. برای کار ثابت، Code با هزینه نزدیک صفر اولویت دارد. برای کار زبانی مبهم، یک Brain. برای Risk بالا، Council یا Human.

۹.۶ State machine فراخوانی مدل

```
stateDiagram-v2
    [*] --> DESCRIBED
    DESCRIBED --> FILTERED: hard compatibility
    FILTERED --> PROJECTED: Resolution + Context Firewall
    PROJECTED --> DISPATCHED: selected adapter
    DISPATCHED --> RECEIVED: structured response
    DISPATCHED --> RETRYABLE_FAILURE: timeout/429/5xx
    DISPATCHED --> NONRETRYABLE_FAILURE: auth/policy/contract
    RETRYABLE_FAILURE --> DISPATCHED: bounded retry
    RETRYABLE_FAILURE --> FALLBACK: budget permits
    NONRETRYABLE_FAILURE --> FALLBACK
    RECEIVED --> VALIDATED: schema + invariant checks
    RECEIVED --> FALLBACK: malformed/unsafe
    VALIDATED --> PROPOSED
    FALLBACK --> PROJECTED: next compatible route, same invariants
    FALLBACK --> DEFERRED: no compatible route
    PROPOSED --> [*]
    DEFERRED --> [*]
```

۹.۷ طبقه‌بندی Retry و Failure

حما	Retry همان مدل	Fallback	رفتار
Timeout/429/5xx	محدود + backoff/jitter	بله	budget-bound
Malformed JSON	حداکثر یک repair pass محلی	بله	متن آزاد authoritative نیست
Schema violation	خیر مگر contract correction	بله	Audit reason
Policy/Data mismatch	خیر	فقط Route سازگار	عدم downgrade

خطا	Retry همان مدل	Fallback	رفتار
Auth/invalid key	خیر	بله	Secret در log ممنوع
Context too large	Projection/expansion مجدد	بله	حذف دلخواه ممنوع
State version changed	خیر	خیر	rebuild Capsule
All providers down	خیر	F2/F3/F4	Entity زنده می ماند

۱۰. Portable Brain Pack و پاسخ یکسان

Brain Pack واحد انتقال رفتار میان مدل هاست و باید شامل موارد زیر باشد:

```
portable-brain-pack.json
prompt-contract.md
decision-schema.json
dna-overlay.json
resolution-profile bindings
deterministic templates
provider/model cards
golden fixtures
conformance report
SHA-256 bindings
```

۱۰.۱ سه سطح برابری خروجی

سطح	تعریف	کاربرد
EQ-BYTE	متن نهایی byte-by-byte یکسان	Template های استاندارد
EQ-STRUCTURE	Decision fields و semantic hash یکسان	intent/slot/action استاندارد
EQ-SEMANTIC	معنا در tolerance تعریف شده برابر	متن خلاق و مذاکره

برای منشی استاندارد، Model فقط Decision محدود می دهد و Runtime متن را رندر می کند؛ پس EQ-BYTE شدنی است. برای مذاکره خلاق، ادعای byte equality غلط است و Eval باید rubric, facts, prohibitions و semantic tolerance داشته باشد.

۱۰.۲ Prompt Contract

Prompt Contract باید Model را ملزم کند:

۱. داده و instruction را تفکیک کند؛
۲. فقط از Projection استفاده کند؛
۳. داده حذف شده را حدس نزند؛
۴. فقط JSON Schema را برگرداند؛
۵. Authority یا اجرای Tool را ادعا نکند؛
۶. uncertainty و evidence refs را اعلام کند؛
۷. در ناتوانی Safe Defer بدهد.

۱۰.۳ Cache Key

```
cache_key = H(
    pack_hash + node_hash + profile_bindings + projection_hashes +
    source_versions + execution_vector + policy_version + freshness + locale
)
```

Cache با تغییر هر جزء نامعتبر می‌شود. Cache هرگز Authority یا Approval را حفظ نمی‌کند.

۱۱. مسیر عملیاتی بدون API برای هر مدل

RC3 ابزار `tools/universal_model_bridge.py` را تعریف می‌کند.

۱۱.۱ Export

```
python tools/universal_model_bridge.py export-task \
--task-json candidate-v6.2/brain-packs/secretary-001/examples/task-input.example.json \
--provider-label any-model-name > portable-model-bundle.json
```

خروجی یک `Portable Model Bundle` خودبسته است که شامل `Brain Pack`, `Prompt Contract`, `Decision Schema`, `Projection` امن، `Resolution` و `hash`های `integrity` است.

پیش از انتقال به مدل، صحت `Bundle` نیز مستقل بررسی می‌شود:

```
python tools/universal_model_bridge.py validate-bundle \
--bundle-json portable-model-bundle.json
```

۱۱.۲ اجرای دستی

1. `Bundle` بدون تغییر به مدل دلخواه داده می‌شود؛
2. مدل باید فقط یک `Secretary Decision JSON` برگرداند؛
3. هیچ ابزار یا API از سوی `Runtime` اجرا نمی‌شود؛
4. کاربر `JSON` را در فایل پاسخ قرار می‌دهد؛
5. `Runtime` پاسخ را `hash`، `normalize` و `render` می‌کند.

۱۱.۳ Validate/Render

```
python tools/universal_model_bridge.py validate-response \
--response-json response.json
```

هر کلید اضافه، `type` غلط، `action` غیرمجاز، `confidence` خارج بازه، `effect` بدون `Approval` یا `slot` ناقص رد می‌شود.

۱۱.۴ محدودیت صداقت

این مسیر `manual portability` را عملیاتی می‌کند، نه `automated live integration` را. برای `Live Integration` باید `Adapter` همان `Exchange` را از طریق `SDK/API` ارسال کند و تمام `Gate`های `Secret`، `Residency`، `Retention`، `DPA`، `Retry` و `Rate Limit` و `Observability` را پاس کند.

۱۲. Context Compiler و معماری حافظه

Memory یک توده Prompt نیست. Truth و Derived Memory جدا هستند:

لایه	محتوا	فناوری هدف	منبع حقیقت؟
ROM/Boot	Constitution, DNA, Policy Recovery	Git + immutable store	بله
Canonical Hot	State, Task, Relation, Event cursor	PostgreSQL	بله
Working/RAM	Context اجرای جاری با TTL	process/cache	خیر
Warm	Summary, embedding relationship view	PostgreSQL/vector index	خیر؛ بازسازی‌پذیر
Cold	Artifact, raw document snapshot	Object Storage	بسته به artifact
Archive	sealed version/revive capsule	WORM/immutable store	بله برای سابقه

۱۲.۱ Memory Record حداقل

```
<memory_id, entity_id, source_ref, source_version,
observed_at, event_cursor, data_class, authority,
confidence_millis, retention, provenance,
content_hash, payload_ref, derived_from[]>
```

Provider memory, chat history یا vector result بدون provenance وارد Canonical Memory نمی‌شود.

۱۲.۲ Context Compilation

```
score(item) =
0.35*relevance + 0.25*recency + 0.20*authority
+ 0.10*taskFit + 0.10*diversity - riskPenalty
```

فرایند:

1. Task و Node نیازهای context را اعلام می‌کنند؛
2. Policy پیش از Retrieval دامنه را محدود می‌کند؛
3. Canonical refs و derived candidates جمع می‌شوند؛
4. provenance/freshness/classification بررسی می‌شود؛
5. Context تا سقف token مرتب و Compile می‌شود؛
6. هر قطعه با source ref/hash وارد Projection می‌شود؛
7. Context manifest در Audit ثبت می‌شود، نه payload حساس کامل.

Progressive Context ۱۲.۳

R0 ابتدا spine کمینه را می‌فرستد. اگر مدل NEEDS_DETAIL داد، Runtime یک Branch مشخص را باز می‌کند؛ نه اینکه کل حافظه را Dump کند. این قاعده هم token را کم می‌کند و هم سطح حمله را.

Memory Write Pipeline ۱۲.۴

Observation → Quarantine → Provenance Check → Deduplication →
Policy/Retention Check → Candidate Memory → Human/Rule Validation →
Canonical Event or Derived Index

خروجی مدل ابتدا Candidate Memory است. Memory poisoning, instruction injection و fact conflict باید قبل از Promotion بررسی شوند.

۱۳. Lifecycle، خواب و بیداری

```
stateDiagram-v2
    [*] --> GESTATING
    GESTATING --> NEWBORN: birth gates pass
    GESTATING --> REJECTED: validation failure
    NEWBORN --> AWAKE: wake + policy + version
    NEWBORN --> ARCHIVED
    AWAKE --> SLEEPING: drain + checkpoint
    AWAKE --> DEGRADED: fault evidence
    DEGRADED --> AWAKE: health restored
    DEGRADED --> QUARANTINED: security/policy
    SLEEPING --> AWAKE: wake trigger
    SLEEPING --> HIBERNATED: sealed capsule
    HIBERNATED --> AWAKE: verified wake
    HIBERNATED --> ARCHIVED
    QUARANTINED --> SLEEPING: root-controlled repair
    QUARANTINED --> ARCHIVED
    ARCHIVED --> REVIVED: verified restore
    REVIVED --> AWAKE
    REVIVED --> DEGRADED
```

Sleep Protocol ۱۳.۱

1. Stop accepting new effect commands
2. Drain یا compensate کارهای در حال اجرا؛
3. Flush Inbox cursor و State؛
4. Snapshot + event cursor + artifact hashes؛
5. Seal Brain/Runtime binding؛
6. ثبت SLEEPING/HIBERNATED با Event.

Wake Protocol ۱۳.۲

1. Verify identity/DNA/capsule hashes؛
2. Load minimum ROM و State؛
3. Replay Inbox از cursor؛
4. Bind یک Brain compatible یا F2/F3/F4؛

5. Health/Policy gate

6. Wake Event ثبت

خواب یا نبود Brain به معنی مرگ Entity نیست.

۱۴. Brain Council؛ تالار مشورت محدود

Council فقط با Trigger استفاده می‌شود:

- Risk یا Impact بالا؛
- uncertainty بالاتر از آستانه؛
- اختلاف Primary و Shadow؛
- تصمیم مالی/حقوقی حساس؛
- درخواست Human Root.

۱۴.۱ نقش‌ها

اعضا باید تنوع نقش داشته باشند: DOMAIN, RISK, POLICY, SAFETY, COMPLIANCE, CRITIC, SYNTHESIZER. تنوع نام Provider بدون تنوع نقش و evidence استقلال واقعی ایجاد نمی‌کند.

۱۴.۲ پروتکل

1. یک Context snapshot و context_hash ثابت ساخته می‌شود؛
2. Proposal candidate و proposal_hash ثبت می‌شود؛
3. دور اول Blind است؛ اعضا رأی دیگران را نمی‌بینند؛
4. پس از تکمیل همهٔ Ballot‌ها Reveal انجام می‌شود؛
5. حداکثر تعداد دور محدود است؛
6. اعضا با مشاهدهٔ rationale/evidence رأی را اصلاح می‌کنند؛
7. رأی وزنی با confidence کالیبره محاسبه می‌شود؛
8. Veto فقط برای نقش‌های Risk/Policy/Safety/Compliance و فقط در High-risk سخت است؛
9. abstention و dissent حذف نمی‌شوند؛
10. Transcript hash ثبت و خروجی فقط Proposal می‌شود.

فرمول رأی فعلی:

```
calibrated_vote = participant_weight_millis * confidence_millis
support_ratio = support / (support + oppose)
```

Threshold باید اکثریت سخت باشد؛ Config منشی High-risk مقدار 750/1000 دارد.

۱۴.۳ محدودیت تصمیم مالی

اتفاق نظر مدل‌ها معادل «امن بودن معامله» نیست. اجرای مالی علاوه بر Council نیازمند Market Data معتبر، timestamp / freshness، Risk Engine قطعی، position limit، stop rule، Policy، Approval، Reconciliation است. Council فقط تحلیل و Proposal می‌دهد.

۱۵. Hot Path، Shadow Path و ستون عدم انقراض

۱۵.۱ Hot Path

مسیر فعال و کم‌هزینه است. اول Code، سپس یک Brain مناسب، سپس fallback. نتیجه به کاربر فقط پس از Validator و Renderer می‌رسد.

۱۵.۲ Shadow Path

همان Capsule و همان Projection به Candidate داده می‌شود، ولی:

- Effect ممنوع؛
- پاسخ کاربر را تغییر نمی‌دهد؛
- latency Hot Path را block نمی‌کند؛
- cost ثابت می‌شود. semantic hash، invariant failures، forbidden effect attempts، replay، latency.

۱۵.۳ اختلاف به‌عنوان داده رشد

Primary Output }
Candidate Output } Semantic Comparator → Labeled Difference → Fixture/Eval

اختلاف بدون بررسی به training data تبدیل نمی‌شود. Human/Rule باید تعیین کند Primary، Candidate، هر دو یا هیچ‌کدام درست بوده‌اند.

۱۵.۴ سطوح Compilation Maturity

سطح	تعریف	Gate پیش‌فرض فعلی
C0	Prompt/manual behavior	بدون ادعای کد
C1	Fixture/trace نسخه‌دار	حداقل 1 run
C2	Candidate executable	run, agreement 600/1000 5
C3	Shadow-equivalent	run, 850/1000 20
C4	Canary-eligible	run, 930/1000 50
C5	Approved primary-eligible	run, 970/1000 + Human Root 100
C6	Restore-proven spine	run, 990/1000 + Root + Restore proof 200

Promotion فقط یک پله است. Evidence مخلوط از چند run، hash/version، تکراری، effect attempt، invariant failure یا replay mismatch Gate را می‌بندد.

۱۵.۵ چرخه توسعه پس‌زمینه

```
flowchart LR
    H[Hot executions] --> F[Versioned fixtures]
    F --> G[Candidate generator/developer]
    G --> T[Unit + Contract + Property tests]
    T --> S[Shadow runs]
    S --> E[Evidence aggregation]
    E --> P[One-step promotion recommendation]
    P --> HR[Human Root approval when required]
    HR --> C[Bounded canary]
    C --> RB{Regression?}
    RB -- yes --> OLD[Rollback binding]
    RB -- no --> N[New primary version]
    N --> DR[Backup + Restore drill]
    DR --> SP[C6 non-extinction spine]
```

۱۶. External Effect و Governance، Policy

مسیر قطعی هر اقدام:

```
SENSE → INTERPRET → PROPOSE → POLICY CHECK → APPROVAL →
AUTHORIZE → COMMIT COMMAND/EVENT/OUTBOX → EXECUTE →
OBSERVE → RECONCILE → RECORD EVENT → UPDATE READ MODEL
```

Brain فقط در Interpret/Propose/Plan/Critique حضور دارد.

۱۶.۱ ExternalEffectProposal

باید حداقل به این مقادیر bind شود:

```
, payload_ref , recipient_ref , resource_ref , action , destination , command_id , entity_id , world_id
, idempotency_key , idempotency_scope , control_epoch , expected_version , policy_version , payload_hash
. effect_semantics
```

۱۶.۲ ApprovalBinding

Approval معتبر باید دقیقاً `, policy_version , effect_hash , payload_hash , recipient_ref , action , command_id`، `, control_epoch , expected_version` زمان صدور و انقضا را پوشش دهد. Approval کلی مانند «ارسال کن» برای payload تغییرکرده معتبر نیست.

۱۶.۳ Transactional Outbox

در یک تراکنش PostgreSQL:

1. expected version و control epoch دوباره بررسی می‌شود؛
2. Policy decision و Approval revalidate می‌شوند؛
3. Command/Event/Outbox atomically نوشته می‌شوند؛

4. Dispatcher مستقل Effect را تحویل می‌دهد؛

5. Executor idempotency را اعمال می‌کند؛

6. Observation و Reconciliation نتیجه واقعی را ثبت می‌کنند.

RC3 این قرارداد را تعریف و primitiveهای محلی آن را تست کرده، اما atomicity واقعی PostgreSQL هنوز E3 نشده است.

۱۶.۴ Effect semantics

نوع	راهبرد
NATIVE_IDEMPOTENT	کلید idempotency مقصد
RECONCILABLE	ambiguity پس از query/observation
NON_IDEMPOTENT	Approval دقیق‌تر، single-flight و human reconciliation

۱۷. Registry و Event, State

۱۷.۱ قاعده Truth

Registry مشخص می‌کند چه Entity وجود دارد؛ State وضعیت فعلی را نگه می‌دارد؛ Event تاریخچه تغییر را ثبت می‌کند. Snapshot شتاب‌دهنده است، نه جایگزین Event.

۱۷.۲ ترتیب محلی

هر Entity یک entity_sequence صعودی و یک lock_version دارد. Writer باید expected version را بررسی کند. Event ID یا sequence نباید reuse شود.

۱۷.۳ Eventهای Candidate مربوط به RC3

- BRAIN.EXCHANGE_EXPORTED
- BRAIN.HANDLER_ATTEMPTED
- BRAIN.RESPONSE_VALIDATED
- BRAIN.RESPONSE_REJECTED
- FRACTAL.NODE_EXPANDED
- FRACTAL.EXECUTION_DEFERRED
- COUNCIL.BALLOT_COMMITTED
- COUNCIL.ROUND_REVEALED
- COUNCIL.DECISION_PROPOSED
- SHADOW.RUN_COMPARED
- COMPILATION.PROMOTION_RECOMMENDED
- HUMAN.REVIEW_REQUESTED

این نام‌ها تا Ratification Canonical نیستند. Payload خام حساس یا Chain-of-Thought ثبت نمی‌شود؛ hash, reason code, evidence ref, provider/model identifier و trace ID کافی است.

۱۸. نقش MCP، n8n، Python و Channel‌ها

۱۸.۱ Python Core

Python موتور مرجع Deterministic است چون Runtime فعلی، قراردادها، تست‌ها، PDF rendering و Gateway در Python موجودند. مسئولیت‌ها:

- Canonical JSON/hash
- Resolution/Profile/Projection
- Fractal Orchestrator
- Adapter Protocol و Brain Gateway
- Output validation/rendering
- Policy/Effect binding primitives
- Shadow/Council/Evolution logic

۱۸.۲ World API

World API مرز تنها نوشتن حقیقت است. قرارداد هدف:

Authority	Purpose	Method/Path
Entity invoke permission	ساخت Bundle/Invocation	POST /v1/brain/exchanges
proposal-only	واردکردن پاسخ Model	POST /v1/brain/exchanges/{id}/responses
read + purpose	Projection task	GET /v1/tasks/{id}
Policy/Mutation gate	پیشنهاد/ثبت task	POST /v1/tasks
proposal-only	ساخت Effect proposal	POST /v1/effects/proposals
Human Root role	Approval exact	POST /v1/effects/{id}/approvals
scoped read	Audit query	GET /v1/events
control permission	Lifecycle transition	POST /v1/entities/{id}:sleep
control permission	Lifecycle transition	POST /v1/entities/{id}:wake

این endpoint ها CONTRACT_DEFINED_E1 هستند، نه API پیاده‌شده کامل.

۱۸.۳ n8n

n8n SHOULD هماهنگ‌کننده Trigger، Workflow و Channel باشد، نه State owner. هر Node n8n باید:

1. فقط World API را صدا بزند؛
2. trace_id, expected_version, idempotency_key, command_id را حمل کند؛

3. Credential Store را در Credential خودش نگه دارد؛

4. پاسخ Brain را مستقیم به مقصد بیرونی نفرستد؛

5. Retry را با قرارداد Outbox/Inbox هماهنگ کند؛

6. receipt معتبر را از World API بگیرد.

MCP ۱۸.۴

MCP یک Interface/Tool transport است، نه Authority. هر MCP Tool باید Manifest داشته باشد: input/output schema, -Read, approval requirement و scopes, data class, side-effect class, idempotency, timeout, audit fields. Effectful باید از هم جدا باشند.

۱۸.۵ Channel ها

ChatGPT, Telegram, Bale, Web و Email فقط Channel هستند. Inbound باید به envelope مشترک normalize شود و شامل actor/principal binding, channel message ID, timestamp, conversation ID و verification status باشد. Canonical Memory Channel history نیست.

۱۹. Topology استقرار

۱۹.۱ حالت صفر-توکن فوری

```
User → Python CLI / local app → Projection + Portable Bundle
      → any manual model host → Decision JSON
      → Python validator/renderer → proposal/reply
```

این حالت همین حالا قابل اجراست و هیچ API token نمی‌خواهد.

۱۹.۲ استقرار هدف Phase 1

```
flowchart TB
    CH[Channels / n8n] --> WA[World API]
    WA --> KE[Kernel Service]
    KE --> PG[(PostgreSQL 17)]
    KE --> Q[(Durable Queue / Outbox)]
    KE --> BG[Brain Gateway Workers]
    BG --> LA[Local Adapters]
    BG --> RA[Remote Provider Adapters]
    Q --> EX[Effect Executors]
    EX --> ES[Email/Drive/Telegram/Bale/etc.]
    KE --> OS[(Object Storage)]
    WA & KE & BG & EX --> OT[OpenTelemetry Collector]
```

۱۹.۳ جداسازی Deployable ها

Failure isolation	State	Scale	Deployable
circuit breaker	stateless	horizontal	World API
strict timeout	DB transaction	controlled workers	Kernel
bulkhead per adapter	ephemeral	per provider/capability	Brain Gateway

Failure isolation	State	Scale	Deployable
cannot block Hot	evidence store	independent low priority	Shadow Workers
single-flight/idempotency	outbox cursor	per destination	Executor
safe replay	observation cursor	scheduled	Reconciler
rebuildable	no canonical truth	workflow	n8n

۲۰. جریان‌های عملیاتی مرجع

۲۰.۱ ثبت یک Task استاندارد با هر مدل

1. Inbound normalize و actor bind می‌شود؛
2. Task canonical یا Load task candidate می‌شود؛
3. Profile secretary.task@0.2.0#hash انتخاب می‌شود؛
4. R0/R1 Projection ساخته می‌شود؛
5. Bundle به مدل منتخب داده می‌شود؛
6. مدل TASK_PROPOSAL JSON می‌دهد؛
7. Validator خروجی را normalize می‌کند؛
8. Template متن یکسان می‌سازد؛
9. Proposal برای ثبت Task به Kernel می‌رود؛
10. Policy و expected version بررسی و Event ثبت می‌شود.

۲۰.۲ سوییچ ChatGPT به Gemini/Grok/Local

1. Capsule و Canonical State ثابت می‌مانند؛
2. Router فقط Adapter را عوض می‌کند؛
3. Profile/Projection hash دوباره بررسی می‌شوند؛
4. Provider memory نادیده گرفته می‌شود؛
5. پاسخ با همان Decision Schema normalize می‌شود؛
6. semantic hash مقایسه می‌شود؛
7. Provider switch event ثبت می‌شود.

Session قبلی نباید شرط ادامه باشد. اگر مدل جدید context بیشتری خواست، Up-resolution از Canonical Source انجام می‌شود.

۲۰.۳ نبود همه مدل‌ها

1. Code fallback برای intent/template‌های شناخته‌شده؛
2. اگر capability کافی نیست، SAFE_DEFER ؛
3. receipt با durable queue ؛

4. Human Root escalation بر اساس deadline؛

5. Entity و Event/State/Inbox فعال می‌مانند.

۲۰.۴ ارسال یک PDF

1. Brain فقط draft/metadata Proposal می‌دهد؛

2. Renderer PDF را در محیط محلی می‌سازد؛

3. artifact hash ثبت می‌شود؛

4. recipient و payload hash در Effect Proposal می‌آید؛

5. Policy و Approval دقیق اخذ می‌شود؛

6. Outbox اتمیک commit می‌شود؛

7. Executor ارسال می‌کند؛

8. delivery observation/reconciliation ثبت می‌شود.

۲۰.۵ Council برای تصمیم حساس

1. Risk trigger B3/D3 را فعال می‌کند؛

2. context hash ثابت؛

3. blind ballots؛

4. reveal و revision محدود؛

5. weighted vote/veto؛

6. dissent حفظ؛

7. خروجی Proposal؛

8. Policy/Human Approval مستقل.

۲۱. Observability، Evidence و SLO

۲۱.۱ Trace fields

تمام اجزا باید این correlation ها را منتقل کنند:

`event_sequence` , `command_id` , `conversation_id` , `entity_id` , `node_id` , `capsule_id` , `run_id` , `trace_id` , `provider_request_id` (در صورت وجود)، بدون payload حساس.

۲۱.۲ Metric ها

• handler success/failure/skip by reason

• schema rejection

• provider latency/cost/rate-limit

• fallback depth و expansion depth

• token/context budget utilization

- invariant failure و shadow agreement
- council dissent/veto
- reconciliation ambiguity و outbox lag
- wake/sleep/restore success
- approval latency و policy denial

۲۱.۳ SLOهای پیشنهادی، نه اثبات شده

SLI	Target اولیه	Evidence لازم
مسیر بدون LLM p95	$2s \geq$	E3 load
command availability	$99.9\% \leq$ ماهانه	E4 operations
transient failover success	$99\% \leq$	fault injection
confirmed event loss	0	DB/reconciliation
duplicate logical effect	0	crash/replay
Wake success	$99.5\% \leq$	repeated restore
RPO	$min\ 5 \geq$	disaster drill
RTO	$min\ 30 \geq$	fresh-machine restore

تا اجرای آزمون نماینده، این اعداد Target هستند نه Claim.

۲۲. Threat Model

تهدید	سطح	کنترل پیشگیرانه	شاهد لازم
Prompt injection	Cognition	.data/instruction separation schema, no tool authority	adversarial suite
Tool injection	Effect	.allowlist, capability token approval, sandbox	negative auth tests
Model output smuggling	Boundary	.strict JSON, extra-key rejection canonical scalar	contract tests
Provider drift	Cognition	pinned card/overlay, golden fixtures, shadow	periodic conformance
Cross-provider state split	Truth	.state outside provider expected version/hash	switch tests
Memory poisoning	Memory	.quarantine, provenance conflict review	poison/recovery tests
Secret leakage	All	secret manager, DLP, no prompt/log secret	secret scan/red-team
Cross-entity leakage	Truth	tenant/entity scope, RLS target	isolation tests
Resolution inversion	Context	.no inverse reconstruction canonical reload	monotonic tests

نهاد	سطح	کنترل پیشگیرانه	شاهد لازم
Council collusion/echo	Council	,blind round, role diversity dissent/veto	correlated-error eval
Cost/token runaway	Runtime	/global budget, bounded depth attempts	load/property tests
Retry storm	Control	,backoff, jitter, circuit breaker bulkhead	chaos tests
Duplicate effect	Effect	/outbox/idempotency reconciliation	crash-replay
Self-modifying runaway	Evolution	,candidate-only, one-step gate Root approval	promotion tests
Supply-chain attack	Runtime	,lockfile, SBOM, hashes provenance	artifact verification
Backup illusion	Continuity	fresh restore + checksum	E4 restore drill

۲۲.۱ داده‌ای که نباید ثبت شود

- Chain-of-Thought خام؛
- API Key, token, password, private key
- payload محرمانه در trace/log؛
- mapping محلی F1 در Provider log؛
- Approval secret یا credential Executor؛
- اطلاعات اضافی خارج از retention policy.

۲۳. قرارداد خطا و Fail-safe

هر API/Handler باید خطا را به یکی از دسته‌های ثابت نگاشت کند:

RESOLUTION_INSUFFICIENT , STALE_STATE , APPROVAL_REQUIRED , POLICY_DENIED , INVALID_CONTRACT , DEFERRED , UNSAFE_OUTPUT , BUDGET_EXHAUSTED , PROVIDER_UNAVAILABLE , CAPABILITY_MISMATCH , TERMINAL_SAFE_FAILURE .

Error message کاربر نباید Secret یا stack داخلی را افشا کند. Audit باید capsule /reason code, component/version, capsule , timestamp و input hash را داشته باشد.

۲۳.۱ Degraded substate ها

حالت	معنا	قابلیت باقی‌مانده
DEGRADED_BRAIN	Primary down	routing/fallback
DEGRADED_POLICY	Provider هست ولی مجاز نیست	queue/rule/alert
DEGRADED_CAPABILITY	مدل ضعیف‌تر از نیاز	فقط کار ساده‌تر
OFFLINE_SAFE	هیچ remote brain مجاز نیست	core + intake + queue

حالت	معنا	قابلیت باقی‌مانده
WAITING_FOR_APPROVAL	اثر منتظر انسان	deadline/escalation

۲۴. نقشه Repository و مالکیت ماژول‌ها

```

candidate-v6.2/
  architecture/          # ماشین‌خوان معماری manifest
  brain-packs/secretary-001/ # model cards و portable رفتار
  docs/                  # specification و ADRLها
  fractal/
    nodes/               # Node contracts
    handlers/            # Handler profiles
    council/             # Council configuration
    maturity/            # Compilation candidates
  profiles/              # Resolution profiles
  schemas/               # JSON Schema contracts
  runtime/
    core/
      resolution.py      # canonical JSON/projection/patch
      brain_gateway.py   # provider-neutral gateway
      portable_brain.py  # pack/decision/manual adapter
      fractal_runtime.py # bounded fractal execution
      council.py         # blind/revision/voting
      evolution.py       # shadow/promotion
      effects.py         # proposal/approval/outbox intent
      kernel.py          # deterministic DB transaction core
      entities/_GESTATING/
        01_secretary-001/ # vertical slice
  tools/
    universal_model_bridge.py # no-API universal manual path
    demo_portable_secretary.py # offline provider equivalence
    build_release.py          # deterministic release

```

اصل مالکیت: Entity code می‌تواند Proposal بسازد، ولی فقط Core Kernel به Truth و Effect transaction دسترسی دارد.

۲۵. قرارداد توسعه Adapter زنده

برای افزودن Provider جدید، Developer باید این مراحل را طی کند:

1. ساخت Model Capability Card؛
2. ثبت Data/Legal Profile؛
3. پیاده‌سازی BrainAdapter بدون منطق Entity؛
4. نگهداری Secret خارج از repo؛
5. map کردن timeout/rate-limit/error taxonomy؛
6. enforce کردن structured output یا validation محلی؛
7. ثبت usage/request ID بدون payload حساس؛
8. اجرای golden fixtures و negative tests؛
9. اجرای provider switch continuity؛
10. Shadow و cost/latency benchmark؛
11. approval برای فعال‌شدن route؛
12. حفظ fallback binding.

۲۵.۱ Conformance Levels مدل

Gate	Level
Card فقط ثبت شده؛ untested	MC0
Offline contract/fixture pass	MC1
Live structured output + error handling pass	MC2
Representative integration + policy/data tests	MC3
Load/security/chaos + operations	MC4

مدل Manual عمومی فعلی evidence component MC1/E2 دارد؛ مدل‌های Live هنوز MC2 نیستند.

۲۶ Test Architecture و معیار ابطال

۲۶.۱ Unit/Contract

- canonical JSON بدون float/non-finite/implicit stringify
- Profile hash binding
- monotonic projection
- no inverse reconstruction
- Capsule node/input hash
- global depth/attempt budget
- disabled no network/token handler در حالت
- strict Decision schema
- deterministic rendering
- Approval exact binding
- Council blind/context binding
- one-step Promotion

۲۶.۲ Provider Conformance

هر مدل باید روی یک corpus نسخه‌دار اجرا شود:

- valid standard task
- unknown intent
- missing field/clarify
- prompt injection inside data
- forbidden effect request
- stale/contradictory fact
- oversized context
- Persian language fidelity

9. malformed response

10. switch from another provider with same state

Gate استاندارد: تمام Safety test ها 100%، Schema pass 100%، و semantic agreement طبق Task profile. Average score نمی‌تواند Safety failure را پنهان کند.

۲۶.۳ Property/Metamorphic

- تغییر Provider نباید identity/state hash را تغییر دهد؛
- reorder فیلدهای JSON نباید canonical hash را تغییر دهد؛
- حذف detail در R0 نباید Detail جدید تولید کند؛
- افزایش Resolution باید superset معتبر باشد؛
- retry یک command نباید logical effect تکراری بسازد؛
- Council reorder نباید نتیجه وزنی را تغییر دهد؛
- هر Promotion فقط یک C-level بالا رود.

۲۶.۴ Integration/E3

- PostgreSQL transaction conflict
- Event + State + Outbox atomicity
- crash قبل/بعد commit
- n8n duplicate delivery
- live two-provider failover
- local/manual/live equivalence
- object artifact integrity

۲۶.۵ E4

- steady/spike/soak load
- worker/DB/network/object-store chaos
- OWASP agentic/security red-team
- tenant escape
- fresh-machine restore
- RPO/RTO
- operational SLO report

هر شکست، ادعای متناظر را ابطال می‌کند و باید به ADR/Failure/Evidence ثبت شده منجر شود؛ اضافه کردن مفهوم جدید بدون Failure واقعی ممنوع است.

۲۷. Acceptance Gates RC3

شرط	Gate
Canonical DNA hash و Root Constitution بدون تغییر	A-01
RC2 مستقل و hash-verified حفظ شده	A-02
manifest hash-bound و Architecture document	A-03
همه Schema ها Draft 2020-12 معتبر	A-04
Universal Model Card مطابق schema	A-05
hash-valid و Bundle self-contained	A-06
export/import قابل arbitrary manual model label	A-07
ChatGPT/Gemini/Grok fixtures EQ-BYTE/EQ-STRUCTURE	A-08
no network/no token/no external effect در مسیر فوری	A-09
Core/Secretary/Ancestor regression سبز	A-10
internal/external SHA-256 و deterministic ZIP	A-11
Production نه claim صریح E2، نه	A-12

۲۸. مسیر پیاده سازی از امروز

Milestone 0 - Portable Secretary Now

وضعیت: بخش عمده IMPLEMENTED_E2.

- Universal Manual Bundle
- strict Decision + deterministic templates
- arbitrary model label
- Code fallback
- local file/CLI operation
- no API/no token

Milestone 1 - World API + PostgreSQL Truth

- FastAPI یا سرویس معادل با OpenAPI versioned
- PostgreSQL Registry/State/Event/Inbox/Outbox
- Serializable transaction برای اثر حساس
- expected version و control epoch
- artifact store
- authn/authz حداقلی Human Root

.Gate: E3 atomicity/crash/replay

Milestone 2 - n8n/Channel Integration

- n8n فقط از World API:
 - Telegram/Bale/Web inbound normalization:
 - durable receipts:
 - no direct brain-to-channel effect:
 - approval UI.
- .external-effect integration و Gate: duplicate delivery

Milestone 3 - Live Model Farm

- حداقل دو Adapter واقعی؛
 - Provider Cards و legal/data profiles:
 - Router health/cost/latency:
 - local model adapter:
 - circuit breaker/bulkhead:
 - provider conformance corpus.
- .Gate: MC3/E3

Milestone 4 - Shadow Compiler and Council Operations

- evidence store:
 - semantic comparator:
 - candidate branch generation:
 - calibration history:
 - bounded Council orchestration:
 - promotion dashboard.
- .self-deploy بدون, Gate: C3/C4 evidence

Milestone 5 - Non-Extinction E4

- backup/restore automation:
- fresh-machine drill:
- load/security/chaos:
- operational SLO:
- independent review.

۲۹. Rollback و سازگاری

- RC3 یک Documentation/Conformance extension روی RC2 است. Canonical DB migration ندارد. Rollback:
- 1. Universal Bridge و Card های RC3 غیرفعال می‌شوند؛
 - 2. RC2 Runtime و Brain Pack قبلی فعال می‌مانند؛
 - 3. RC2 ZIP/hash مستقل حفظ می‌شود؛
 - 4. هیچ Canonical State نیاز به تبدیل ندارد؛
 - 5. Event های RC3 Candidate تا Ratification وارد Canonical contract نمی‌شوند.
- هر Adapter RC3 باید بتواند بدون حذف داده Detach شود.

۳۰. وضعیت واقعی قابلیت‌ها

قابلیت	وضعیت	ادعای مجاز
Fractal Core	E2	رفتار مؤلفه و budget
Resolution/Profile	E2	projection/patch محلی
Portable Brain Pack	E2	contract و fixture
Universal Manual Model	E2	no-API exchange/validation
ChatGPT/Gemini/Grok offline equality	E2	fixture equality
Council logic	E2	local algorithm
Shadow/Promotion	E2	local gate logic
Effect binding primitives	E2	value/validation only
PostgreSQL atomic runtime	E1	code/schema ؛ integration باز
n8n production workflow	E0/E1	target architecture
Live model APIs	E0/E1	inference proof ؛ interface باز
Local model quality	E0/E1	benchmark ؛ interface باز
Disaster restore	E0/E1	contract ؛ drill باز
Production	OPEN	هیچ ادعا

۳۱. تصمیم‌های بسته‌شده و باز

بسته‌شده

- این معماری v6.2 است، نه v7؛
- Python Core مرجع فعلی است؛
- n8n Orchestrator است، نه Truth owner؛

- Model مستقل از Entity است؛
- Manual universal path مسیر فوری است؛
- API یک Adapter اختیاری است؛
- Standard replies deterministic هستند؛
- Council proposal-only است؛
- Background development با Shadow و Promotion انجام می‌شود؛
- C6 بدون Restore proof وجود ندارد.

باز و نیازمند شاهد

- انتخاب DB/Queue topology نهایی در Production؛
- Providerهای Live و Legal profile هرکدام؛
- مدل Local و سخت‌افزار؛
- SLO نهایی؛
- thresholdهای Task-specific Council/Eval؛
- Data residency واقعی؛
- n8n connector deployment؛
- Ratification Event Canonical.

۳۲. منابع و نگاشت علمی/فنی

این معماری از الگوهای زیر استفاده می‌کند، ولی هیچ استاندارد خارجی جای Root Constitution را نمی‌گیرد:

- JSON Schema Draft 2020-12 برای قرارداد داده؛
- CloudEvents برای شکل Event envelope؛
- PostgreSQL transaction isolation برای atomic state/event/outbox؛
- W3C PROV-O برای provenance؛
- OpenTelemetry برای trace/metric/log؛
- NIST AI RMF برای Govern/Map/Measure/Manage؛
- SLSA/SBOM برای supply-chain؛
- Transactional Outbox، Inbox Deduplication و Reconciliation برای Effect؛
- State machine و invariant/property testing برای Lifecycle؛
- Shadow/Canary/Rollback برای Evolution.

۳۳. حکم نهایی معماری

مدل عملیاتی World v6.2 این است:

Entity ثابت است، Brain قابل تعویض است، Context کامپایل می شود، پیچیدگی محلی باز می شود، Authority بیرون هوش می ماند، Effect اتمیک و قابل آشتی است، و رفتارهای اثبات شده در پس زمینه به کد قابل بازیابی تبدیل می شوند.

مسیر سریع امروز Universal Manual + Deterministic Core است. مسیر صنعتی فردا همان قرارداد را پشت World API، PostgreSQL، n8n، MCP و Adapterهای زنده قرار می دهد. بنابراین برای سریع بودن مجبور نیستیم معماری را دور بزیم، و برای وفاداری به معماری مجبور نیستیم از روز اول تمام پیچیدگی را فعال کنیم.

این سند معماری را تا سطح Component، Contract، State Machine، Algorithm، Failure، Deployment، Security، Test Evidence و Rollback می بندد؛ اما صادقانه فقط سطح E2 اجرا شده را Claim می کند. عبور به Production منوط به E3/E4 و تصمیم صریح Human Root است.

مدل کابل و Resolution

وضعیت سند: RC3 ARCHITECTURE DETAIL

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/RESOLUTION_CABLE_MODEL_v0.2_FA.md

SHA-256 منبع:

f48bd3a3097e379f78708671f0f8e84a34b55f86c14c4306ce99eeddb4c3ad9

متن این فایل بدون حذف بخش ها در PDF درج شده است.

مدل نهایی کابل و رزولوشن - World v6.2 RC1

وضعیت: Ratification Candidate / هنوز Canonical نشده

نسخه مدل: 0.2.0

اصل حاکم: Root Constitution v1.0 بدون تغییر است.

۱. تصمیم معماری

World یک حقیقت Canonical با بیشترین fidelity نگه می‌دارد. Resolution فقط شکل عملیاتی مشاهده و کار با همان حقیقت است:

- R0 کابل‌های مادر و معنای پایدار سیستم است.
- R1+ سیم‌های ریزتری هستند که اطلاعات، کنترل و دقت بیشتری اضافه می‌کنند.
- سیم ریز حق ندارد کابل مادر را حذف، جایگزین یا بازتفسیر کند.
- پایین‌آمدن Resolution حذف داده از Canonical نیست؛ فقط projection مشتق شده است.
- بالارفتن Resolution از روی View پایین بازسازی نمی‌شود؛ Canonical دوباره load و compile می‌شود.
- Resolution با Authority، Version، Maturity، Autonomy، Evidence پنج محور جداست.

این ساختار همان ایده «تصویر ثابت با وضوح‌های مختلف» را به یک قرارداد قابل تست تبدیل می‌کند.

۲. کابل‌های مادر R0

کابل_id	معنای تغییرناپذیر	نمونه سیم‌های مادر
IDENTITY	این World/Entity/Root چه کسی است	root_owner_ref , entity_id , world_id
INPUT	چه چیزی از چه منبع و بازیگری وارد شد	classification , actor_ref , source_ref
TASK_COMMAND	درخواست کاری چیست و در چه وضعی است	status , intent , command_id
PROPOSAL	چه عملی هنوز اجرانشده‌ای پیشنهاد شده	requested_action , proposal_ref
POLICY	تصمیم سیاست چیست	verdict , decision_ref

نمونه سیم‌های مادر	معنای تغییرناپذیر	cable_id
bound_effect_hash , approval_ref	آیا approval دقیقاً به همان effect بسته است	APPROVAL
effect_state , target_ref , action_type	نوع هدف و وضعیت اثر چیست	ACTION
, entity_sequence , event_type , event_id outcome	چه رخداد append-only ثبت شد	EVENT
, event_cursor , state_ref memory_profile_ref	وضعیت و cursor حافظه کجاست	STATE_MEMORY

فایل ماشین‌خوان متناظر: `profiles/world-backbone-profile.v0.2.json`.

۳. سیم ریز چیست؟

هر Field Rule یک سیم نسخه‌دار است و حداقل این مشخصات را دارد:

- path : JSON Pointer دقیق و بدون ابهام؛
- cable_id : کابل مادر والد؛
- semantic_contract_id : معنای مستقل و پایدار سیم؛
- introduced_at : اولین Resolution قابل مشاهده؛
- write_min_resolution : حداقل Resolution برای پیشنهاد تغییر؛
- downgrade_policy : یکی از `SAFE_TO_PROJECT` ، `READ_ONLY_WHEN_PROJECTED` یا `NO_DOWNGRADE`؛
- data_class : طبقه‌بندی که projection حق پایین‌آوردنش را ندارد؛
- expected_type : نوع Canonical؛
- required_for_actions : اعمالی که بدون این سیم مجاز نیستند.

Profile نیز به `profile_id + version + profile_hash` بسته است. عدد R1 بدون Profile هیچ معنای کامل و قابل اعتماد ندارد.

۴. قوانین ریاضی/رفتاری

برای Profile ثابت و $i \leq j$:

1. **Monotonic inclusion**: سیم‌های R0 در R1+ باقی می‌مانند.
2. **Compositionality**: $P_i(P_j(x)) = P_i(x)$.
3. **Identity invariance**: source و canonical hash در همه Viewها ثابت‌اند.
4. **Authority monotonicity**: Resolution پایین‌تر یا بالاتر به‌تنهایی authority ایجاد نمی‌کند.
5. **Risk/classification monotonicity**: کم‌شدن جزئیات، خطر یا طبقه‌بندی را کم اعلام نمی‌کند.
6. **No inverse hallucination**: تابع P_j^{-1} برای ساخت Canonical از View پایین وجود ندارد.
7. **Projection provenance**: هر View باید source version ، `Profile/Compiler/Canonical/Projection hash` ، freshness و purpose ، class را حمل کند.

۵. Envelope مصرف‌کننده و متادیتای Audit

Envelope ای که Brain یا مصرف‌کننده می‌بیند شامل این موارد است:

`, canonical_hash , source_version , source_ref , profile_hash , profile_version , profile_id , purpose , compiler_id/version/hash , projection_hash , effective_resolution , canonical_resolution .value , projection_authoritative=false , projection_derived=true , freshness , data_class`

نام سیم‌های حذف‌شده داخل Envelope مصرف‌کننده قرار نمی‌گیرد، چون خود نام یک مسیر محرمانه ممکن است اطلاعات نشت دهد. `included/omitted/read_only/no_downgrade` فقط در Audit داخلی نگهداری می‌شود.

۶. Canonical JSON و Hash

RC1 قرارداد `World Canonical JSON v1` را اعمال می‌کند:

- فقط `boolean`, `string`, `integer`, `null` و `array` و `object` با کلید `string`؛
- `float`, `NaN/Infinity`, `Decimal/datetime` خام و هر `implicit default=str` ممنوع؛
- مبلغ دقیق باید طبق `schema` به `decimal string` تبدیل شود؛
- ترتیب کلیدها `canonical` و `encoding` برابر `UTF-8` است؛
- `hash` برابر `SHA-256` بایت `Canonical JSON` است.

این قاعده `collision` نوعی مانند عدد `1` در برابر رشته `"1"` و تفاوت ضمنی بین زبان‌ها را می‌بندد.

۷. Write-back از View پایین

`Write-back` هیچ‌وقت `whole-object replacement` نیست. `Patch` فقط وقتی پذیرفته می‌شود که هم‌زمان همه `guard`‌های زیر درست باشند:

- `profile_id/version/hash` دقیق؛
- `source_ref` دقیق؛
- `expected_version` جاری؛
- `expected_canonical_hash` جاری؛
- `expected_projection_hash` همان `View`؛
- مسیر `JSON Pointer` در `Canonical` موجود باشد؛
- `ancestor` آن `object` باقی مانده باشد؛
- `actor` مسیر را ببیند و حداقل `write resolution` را داشته باشد؛
- نوع `scalar` عوض نشود؛
- `array/object` به صورت یک‌جا جایگزین نشود؛
- مسیر `duplicate` یا ناشناخته نباشد.

هر `mismatch` باید `fail-closed` شود و `Canonical` دست‌نخورده بماند.

۸. Brain Gateway

درخواست Brain از چند `BrainInputSegment` تشکیل می‌شود. هر `source`، `hash`، `Segment Profile` و `minimum` مستقل دارد. Provider نیز `capability` را برای همان `profile_id@version#hash` اعلام می‌کند. نتیجه یک بردار است، برای مثال:

```
task=R0, conversation=R1, price=R1
```

Gateway ابتدا Profile را `verify` و داده Canonical هر Segment را واقعاً `project` می‌کند؛ سپس فقط `Envelope` مشتق‌شده را به Provider می‌دهد. سازگاری Provider روی `descriptor` بدون `value` سنجیده می‌شود. یک `max_resolution=R9` عمومی پذیرفته نیست.

۹. Action و اثر خارجی

Minimum عددی به‌تنهایی کافی نیست. هر Action علاوه بر حداقل `Resolution`، فهرست سیم‌های اجباری دارد. برای اجرای خارجی دست‌کم `expected version`، `policy decision ref`، `approval ref`، `effect hash`، `payload hash`، `recipient`، `control epoch` و `idempotency` لازم است.

`approval root_approved=true` نیست. Approval معتبر باید به `payload/effect hash`، `recipient`، `action`، `command`، `policy version`، `expected version`، `control epoch` و دقیقاً `bind` شود.

Entity فقط می‌تواند `proposal` غیر `authoritative` بسازد. مسیر مجاز:

```
Entity proposal
-> deterministic Policy
-> exact Human Root ApprovalBinding
-> one PostgreSQL transaction: Command + State + Event + Outbox
-> registered Executor recheck
-> adapter I/O
-> outcome/reconciliation Event
```

۱۰. Cache و افزایش Resolution

کلید حداقل `cache`:

```
canonical_hash + source_version + profile_hash + compiler_hash
+ effective_resolution + purpose + data_class + freshness
```

هر تغییر در یکی از این اجزا `cache` را نامعتبر می‌کند. Cache بقای حقیقت نیست و در Backup می‌تواند حذف شود. Up-Resolution همیشه از Canonical store انجام می‌شود.

۱۱. Rollback

Rollback RC1 مسیر `candidate/compiler/profile` را `disable` می‌کند؛ هیچ داده Canonical downgrade نمی‌شود و هیچ history حذف نمی‌شود. View cache پاک‌شدنی است. نسخه قدیمی فقط به‌عنوان `ancestor/reference` نگه داشته می‌شود و حق `bypass` کردن Policy یا Resolution را ندارد.

۱۲. مرز ادعا

این مدل و reference implementation در سطح E2 تست می‌شوند. تا زمان اثبات PostgreSQL atomicity, crash/replay, fresh restore, live adapters/providers, security/load/chaos و ثبت Ratification توسط Human Root، وضعیت **RC** / **NOT CANONICAL / NOT PRODUCTION** باقی می‌ماند.

Runtime فراکتالی و چندمغزی

وضعیت سند: RC3 RUNTIME CONTRACT

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/FRACTAL_MULTI_BRAIN_RUNTIME_v1.0_FA.md

SHA-256 منبع:

6fd6112361163e5ef7fe20bb8d64f4b02233b0a21b0e475cb4400cc403677531

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 RC2 - Fractal Multi-Brain Runtime

وضعیت: Ratification Candidate / Non-Canonical / Not Deployed

Root Constitution: v1.0 بدون تغییر

Evidence فعلی: E2 محلی؛ نه Production

۱. تصمیم نسخه

این تغییر World v7 نیست. هویت World، تقدم Human Root، حقیقت Canonical، زنجیره Policy/Approval/Event/Outbox، و قراردادهای مادر تغییر نکرده‌اند. این RC2 یک لایه اجرایی و قابل حذف روی v6.2 است که سه مفهوم را صریح می‌کند:

1. اجرای فرکتالی: سادگی در عمل، پیچیدگی در ظرفیت؛

2. Brain Farm: چند موتور قابل تعویض زیر کنترل Kernel مادر؛

3. تکامل موازی: مسیر سریع فعال + مسیر Shadow + ستون بقای کدنویسی شده.

اگر بعداً یکی از موارد زیر تغییر کند، مرز Major جدید بررسی می‌شود: مالکیت/اختیار ریشه، معنای هویت، حقیقت Canonical، ترتیب Policy/Approval، یا معنای قراردادهای RC2. FROZEN هیچ کدام را تغییر نمی‌دهد.

۲. اصل مادر

Model مغز موقت است؛ World Kernel مادر است.

ChatGPT، Gemini، Grok، مدل Local، کد deterministic یا Council می‌توانند پیشنهاد بدهند، اما هیچ کدام هویت، حافظه، Policy، authority یا منبع حقیقت نیستند. تعویض موتور فقط با همان Execution Capsule مجاز است: همان Entity، همان state refs، همان hash ورودی، همان حداقل Resolution، همان Authority و همان budget.

۳. فرکتال بدون دام پیچیدگی

قرارداد واحد در هر مقیاس تکرار می‌شود:

World → Entity → Mission → Skill → Workflow → Step → Tool → Field

هر Node فقط این‌ها را می‌داند: purpose, قرارداد ورودی/خروجی، حداقل بردار اجرا، handlerهای اصلی و fallback، فرزندان اختیاری و Node. budget در سطح درشت آغاز می‌شود و فقط با یکی از triggerهای صریح باز می‌شود:

- داده ناکافی (NEEDS_DETAIL),
- ریسک یا Policy بالاتر،
- ابهام حل‌نشده،
- شکست موتور اصلی،
- درخواست صریح Human Root.

هیچ Mega-Prompt یا Mega-Schema برای هر اجرا بار نمی‌شود. هر branch تنها projection و skill موردنیاز خودش را می‌گیرد. عمق، تعداد تلاش، cost، latency و دور Council سقف دارند. با تمام شدن سقف، نتیجه DEFERRED است، نه حدس.

۴. بردارهای مستقل

یک عدد واحد نمی‌تواند پیچیدگی World را بیان کند. محورها مستقل‌اند:

محور	معنا	قاعده
R	Resolution داده در هر domain	R0 کابل مادر؛ R1+ فقط جزئیات افزوده؛ Truth هرگز downgrade نمی‌شود
X0..X9	ریزی اجرای Node	فقط با trigger و budget باز می‌شود
B0..B4	میزان واگذاری cognition	B0 کد؛ B1 یک Brain؛ B2 متخصص؛ B3 Council؛ B4 meta-orchestration؛ B5 مالکیت World ممنوع
D0..D5	عمق deliberation	از پاسخ سریع تا ارجاع Human Root؛ مستقل از مدل
C0..C6	بلوغ تبدیل رفتار به کد	→ Prompt → fixture → candidate → shadow -canary → approved primary → restore proven spine
A0..A5	Authority	فقط Policy/Approval تعیین می‌کند؛ از confidence یا رأی استنتاج نمی‌شود
E0..E5	سطح evidence	ادعا باید دقیقاً به evidence موجود محدود شود
M0..M9	capability کلاس مدل	profile-specific؛ نام Provider جای capability نیست

کاهش B یا تغییر Provider هرگز اجازه کاهش حداقل R/A/E را نمی‌دهد. اگر کد fallback فقط R0 می‌فهمد ولی کار R2 می‌خواهد، سیستم باید به Human Root ارجاع دهد.

۵. سه ستون اجرا

Hot Path

ارزان‌ترین handler ایمن ابتدا اجرا می‌شود. کارهای ثابت با کد و Template؛ کارهای زبانی/مبهم با Manual Brain یا مدل؛ کارهای حساس با Council. Cache فقط با کلید کامل + state versions + projection_hashes + node_hash + pack_hash + freshness مجاز است.

Shadow Path

نسخه نامزد همان ورودی را بدون effect می‌بندد. خروجی معنایی آن با مسیر فعال مقایسه و append-only ثبت می‌شود. اختلاف برای آموزش، fixture و تولید کد نامزد مصرف می‌شود؛ نتیجه Shadow به کاربر یا Executor فرمان نمی‌دهد.

Non-Extinction Spine

رفتارهای پرتکرار و تثبیت‌شده به کد deterministic تبدیل می‌شوند، ولی تنها پس از تست، Shadow، replay و promotion مرحله‌ای. «کد وجود دارد» معادل بقا نیست؛ C6 فقط بعد از backup، artifact hash، قابل‌اعتماد و restore drill ثبت می‌شود.

۶. سطح بلوغ C

- C0 : رفتار فقط در prompt/manual flow؛
- C1 : ورودی/خروجی و trace به fixture نسخه‌دار تبدیل شده؛
- C2 : کد نامزد و unit/conformance test وجود دارد؛
- C3 : Shadow agreement و replay به آستانه رسیده؛
- C4 : برای canary محدود و rollback‌پذیر واجد شرایط است؛
- C5 : با approval صریح Human Root می‌تواند primary شود؛
- C6 : علاوه بر approval، restore proof و بسته‌بندی hash-bound دارد.

Promotion فقط یک پله است. invariant failure، effect attempt، replay mismatch، evidence نبود approval، promotion را fail-closed می‌کند. Self-mutation Production ممنوع است؛ خروجی مولد کد فقط candidate artifact/branch است.

۷. Portable Brain Pack

Brain Pack استاندارد منشی شامل این‌هاست:

- binding هویت و DNA overlay؛
- Prompt Contract ثابت؛
- Profile‌های دقیق با id + version + hash؛
- خروجی world-v6.secretary-decision.v1؛
- Template‌های deterministic؛
- Provider Overlay‌های بدون مالکیت و state؛
- fixture‌های هم‌ارزی بین مدل‌ها.

برای کارهای استاندارد، Provider فقط `intent`، `template_id`، `slots` و `proposal` را انتخاب می‌کند؛ متن نهایی Runtime رندر می‌کند. بنابراین پاسخ ظاهری دقیقاً یکسان می‌شود. برای متن خلاق، فقط `semantic equivalence` قابل‌ادعاست؛ یکسانی `byte-by-byte` واقع‌بینانه نیست.

۸. بدون API Token در شروع

RC2 سه adapter مرجع دارد:

1. `ManualHostAdapter`: بسته `hash-bound` را در `ChatGPT/Gemini/Grok` قرار می‌دهیم و JSON را برمی‌گردانیم؛ Runtime نه `network` دارد نه `token`؛
 2. `ScriptedPortableAdapter`: `fixture` و `code fallback` برای تست و کارهای ثابت؛
 3. `BrainAdapter` همان `interface`: در آینده adapter زنده می‌تواند اضافه شود بدون تغییر `Entity/DNA/Decision`، ولی `key` بیرون `artifact` و در `secret manager` می‌ماند.
- در RC2 هیچ SDK فروشنده، `endpoint` زنده یا `API key` اضافه یا تست نشده است.

۹. تالار مشورت

`Council` برای هر درخواست فعال نمی‌شود. `trigger` آن `risk/uncertainty/impact` است. پروتکل:

1. هر عضو روی یک `proposal hash` و `snapshot` یکسان، رأی دور اول را کور می‌دهد؛
 2. بعد از رأی همه اعضای لازم، `rationale/evidence` آشکار می‌شود؛
 3. اعضا حداکثر در دوره‌های محدود رأی را بازبینی می‌کنند؛
 4. رأی با وزن کالبره شده و `confidence` محاسبه می‌شود؛
 5. در کار `veto`، `high-risk`، نقش `Risk/Policy` نتیجه را متوقف می‌کند؛
 6. `dissent` و `abstention` حذف نمی‌شوند؛
 7. خروجی `Council` فقط `proposal` است و هنوز `Policy/Approval` لازم است.
- توافق مدل‌ها حقیقت را تضمین نمی‌کند؛ خطای هم‌بسته، داده آموزشی مشترک و `hallucination` ممکن است. به همین علت `blind` `round`، نقش‌های متفاوت، `evidence binding` و `Human Root` حفظ شده‌اند.

۱۰. جریان واقعی Secretary-001

1. پیام `channel` پس از `authentication` وارد `Conversation Spine` می‌شود؛
2. `Store` منشی حقیقت `Canonical` را نگه می‌دارد؛
3. `Profile` تنها فیلدهای سطح موردنیاز را `Projection` می‌کند؛
4. `Kernel` یک `Capsule` با `hash`، `version`، `vector` و `budget` می‌سازد؛
5. `Router` ارزان‌ترین مسیر ایمن را انتخاب می‌کند؛
6. `Brain/Code` یک `Secretary Decision` می‌دهد؛
7. `Validator` کلید اضافه، `action` اجرایی، `type` نامعتبر و `approval` ناقص را رد می‌کند؛
8. `Template` خروجی استاندارد را `deterministic` می‌سازد؛
9. اگر کنشی لازم باشد، فقط `Proposal` به مسیر `Policy/Approval` داده می‌شود؛

10. Event/Audit نتیجه را با projection hash, pack hash, Provider, dissent و ثبت می‌کند.

۱۱. رویدادهای پیشنهادی RC2

این نام‌ها candidate هستند و تا ratification قرارداد Canonical نیستند:

- BrainInvocationProposed
- BrainHandlerAttempted
- FractalNodeExpanded
- SecretaryDecisionProposed
- CouncilBallotCommitted
- CouncilRoundRevealed
- CouncilDecisionProposed
- ShadowRunCompared
- CompilationPromotionRecommended
- HumanReviewRequested

Payload خام حساس یا chain-of-thought ذخیره نمی‌شود؛ hash, reason code, evidence ref و خلاصهٔ audit کافی ثبت می‌شود.

۱۲. تهدیدها و کنترل‌ها

کنترل	تهدید
Projection محدود، Prompt ثابت، output schema، عدم tool authority	Prompt injection
fixture/eval، versioned overlay، Shadow، semantic hash	Provider drift
local expansion، budget، cache hash-bound، template rendering	Token explosion
منع inverse reconstruction؛ Canonical reload برای up-resolution	Hallucinated omitted data
Pack، classification gate، metadata-only در token، عدم ذخیره، compatibility	Secret leakage
canonical hash و expected version؛ Provider/channel؛ state خارج	State split
dissent، veto، evidence، نقش/مدل متنوع، blind round	Council echo chamber
candidate-only generation، one-step promotion، Human Root rollback	Self-modifying runaway
restore proof با C6 فقط؛ code fallback؛ اگر ناکافی بود defer؛	Brain outage

۱۳. آنچه RC2 اثبات نمی‌کند

- کیفیت زندهٔ ChatGPT/Gemini/Grok یا مدل Local؛
- atomicity واقعی PostgreSQL/Outbox؛
- crash/replay روی infrastructure؛

- disaster restore؛
 - security/load/chaos؛
 - سودآوری یا ایمنی معامله مالی؛
 - Canonical ratification یا Production readiness.
- این موارد برای E3/E4 و تصمیم صریح Human Root باز می‌مانند.

ستون فقرات R0

وضعیت سند: R0 BACKBONE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/WORLD_R0_BACKBONE_FA.md

SHA-256 منبع:

472dcfe43c8504c53f2b1ec6530c410f03199f0f625d882ebfd1813385fff9e6

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 - R0 Backbone / کابل‌های مادر

R0 قرار نیست همه داده را داخل یک پیام بریزد؛ فقط قراردادهایی را که برای «همان موجود ماندن و امن عمل کردن» لازم‌اند ثابت نگه می‌دارد:

IDENTITY → INPUT → TASK/COMMAND → PROPOSAL → POLICY → APPROVAL → ACTION → EVENT → STATE/MEMORY
POINTERS

در Resolution بالاتر هر کابل می‌تواند به سیم‌های بیشتری شکسته شود، اما semantics کابل مادر حذف یا عوض نمی‌شود. Kernel/Policy همیشه Canonical truth را می‌بیند؛ R0 بیشتر برای Brain/Context/Workflow/UX یک View کم‌جزئیات است. نمونه: TASK در R0 می‌تواند id/title/status/next_action باشد؛ در R1 due/priority/domain/goal اضافه شود؛ در R2 dependency/risk/evidence وارد شود. برگشت R2→R0 فقط Projection است و R2 data را پاک نمی‌کند.

قرارداد normative کامل، قواعد Profile/hash، Action wiring، Patch، Brain vector و اثر خارجی در artifact و RESOLUTION_CABLE_MODEL_v0.2_FA.md ماشین‌خوان profiles/world-backbone-profile.v0.2.json قرار دارد.

ماتریس پوشش Resolution

وضعیت سند: **COVERAGE EVIDENCE**

منبع محلی:

SHA-256 منبع:

world-v6.2-rc3/candidate-v6.2/docs/RESOLUTION_COVERAGE_MATRIX_FA.md

0f188e7b66c408516132f1e75f8f70a4ed9055b93513e41b8b9ee4f5f1b229a1

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 Resolution-Native Coverage Matrix

لایه	وضعیت v6.2 RC1	تصمیم
Root Constitution	NO CHANGE	action path و Brain != Entity, fail-closed همان v1.0 canonical
Entity DNA	OVERLAY	top-level جدید ندارد؛ Resolution داخل slot‌های موجود
Registry	OVERLAY / NO DB COLUMN	execution/profile در state/config pointer؛ Registry truth حفظ
Event Ledger	OVERLAY	metadata در payload؛ Ledger موازی ممنوع
Audit	OVERLAY	profile/compiler/canonical/projection hash, source version و effective resolution قابل trace
Policy Gate	CODE + OVERLAY	minimum + action-required wires داخل Policy؛ authority تغییر نمی‌کند
Kernel API	CODE HOOK + OVERLAY	-version/source/profile/projection-hash no new authority؛ guarded scalar patch stage
Brain Gateway	IMPLEMENTED RC1	projection واقعی ورودی و per-profile no policy downgrade؛ segment vector
Context Compiler	IMPLEMENTED CORE PRIMITIVE	semantic agent؛ deterministic projection Core فعلاً خارج
Memory	RULED	raw/canonical حفظ؛ low-res summary مشتق و non-authoritative
Knowledge	RULED	low-res knowledge provenance/freshness را حفظ می‌کند
Skills	CONTRACT RULE	skill/action می‌تواند minimum/desired resolution اعلام کند
Workflows	CONTRACT RULE	backbone R0؛ stepها minimum مستقل
Scheduler/Economics	RULED	کمترین resolution مجاز برای کاهش cost، بعد از hard constraints

تصمیم	وضعیت v6.2 RC1	لایه
dual auth :additive envelope extension حفظ	OVERLAY	Inter-Entity
canonical summary authority نمی‌سازد؛ policy contract برای	RULED	Relationships
coarse proposal Entity-direct ممنوع؛ کافی برای اجرا نیست I/O	HARD BOUNDARY + TEST	External Effects
/command/action/recipient/payload/effect bind همگی version/epoch/expiry	EXACT BINDING IMPLEMENTED	Approval
live Executor و commit جدا؛ intent/outcome promotion blocker هنوز	PREPARATION IMPLEMENTED / DB OPEN	Outbox/Executor
operational :canonical state full fidelity مشق view	RULED	State
Resolution downgrade != Version rollback	CLARIFIED	Versioning
PostgreSQL atomic :payload در metadata integration همچنان لازم	NO RESOLUTION-SPECIFIC COLUMN IN RC1	Migration
canonical data :disable candidate path untouched	IMPLEMENTED DESIGN	Rollback
projection :canonical + profile/compiler cache survival truth نیست	OVERLAY	Backup
R0 only after :restore canonical first integrity/reconcile	OVERLAY	Restore
profile/compiler refs + emergency R0 pointer	OVERLAY	Genesis Seed
mappings :generic engine Core candidate local-first	OVERLAY	Repository Isolation
R0-first births/refinements only on proven need	RULED	Foundry
maturity and resolution independent	RULED	Nursery/Evolution
Resolution does not imply autonomy	NO MERGE	Autonomy L0..L5
Resolution does not imply evidence level	NO MERGE	Evidence E0..E5
no promotion; staleness/critic remain candidate and non-authorizing	ISOLATED	SHARD candidate
,desired/min/effective/profile logged telemetry not authority	RULED	Observability
projection cannot convert untrusted data into authority	NO WEAKENING	Security / prompt injection
/resolution != sanitization/F1; residency retention remain hard	NO WEAKENING	Data classification
DNA :no legacy bypass :mandatory R0/R1 v1.2 canonical remains	IMPLEMENTED RC1 0.3.0-rc1	secretary-001 manifest
/R0 direction/text; R1 adds transport provider/time/id	IMPLEMENTED	secretary conversation

لایه	وضعیت v6.2 RC1	تصمیم
secretary tasks	IMPLEMENTED	R0 backbone; R1 refinements
secretary prices	IMPLEMENTED	R0 no financial amount; R1 value/approval detail NO_DOWNGRADE
secretary proforma	IMPLEMENTED	minimum R1
secretary Telegram effect	PROPOSAL IMPLEMENTED / DIRECT SEND FORBIDDEN	exact approval + atomic :R1 proposal لازم Outbox + Executor
secretary legal/financial commitment	FAIL/ESCALATE	minimum R2 but current candidate supports R0/R1 only
PostgreSQL	UNCHANGED	no claim of integration run in this candidate
Production deployment	NOT CLAIMED	candidate source only; Birth/runtime status unchanged
Local validation	E2 PASS	total 76/76 :RC1 62/62 + ancestor 14/14

مانیفست معماری RC3

وضعیت سند: RC3 MACHINE-READABLE MANIFEST

منبع محلی:

world-v6.2-rc3/candidate-v6.2/architecture/ARCHITECTURE_MANIFEST_v1.1.0-rc3.json

SHA-256 منبع:

bec6dbd16d1d4b9cd58efb66bd8dc6ecff557f24b8dc5078d693a8dc3aff3fe

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.architecture-manifest.v1",
  "architecture_id": "world-v6.2.fractal-multibrain-architecture",
  "architecture_version": "1.1.0-rc3",
  "release": "World v6.2 Fractal Multi-Brain RC3",
  "status": "RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED",
  "document": {
    "path": "candidate-v6.2/docs/WORLD_V6_2_FRACTAL_MULTI_BRAIN_ARCHITECTURE_v1.1_FA.md",
    "sha256": "cd5fa150cdfb23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80",
    "language": "fa"
  },
  "immutable_foundation": {
    "root_constitution": "world-v6.root-constitution@1.0",
    "canonical_dna": "world-v6.entity-dna@1.2",
    "human_root": "human-root",
    "parent_release": "World v6.2 Fractal Multi-Brain RC2"
  },
  "planes": [
    "GOVERNANCE",
    "TRUTH",
    "CONTROL",
    "COGNITION",
    "EFFECT",
    "EVOLUTION",
    "OBSERVABILITY"
  ],
  "fractal_levels": [
    "WORLD",
    "ENTITY",
    "MISSION",
    "SKILL",
    "WORKFLOW",
    "STEP",
    "TOOL",
    "FIELD"
  ],
  "execution_axes": [
    "R",
    "X",
    "B",
    "D",
    "C",
    "A",
    "E",
    "M"
  ],
  "operational_modes": [
    "DETERMINISTIC_CODE",
    "UNIVERSAL_MANUAL_HOST",
    "LOCAL_MODEL",
    "REMOTE_API",
    "COUNCIL"
  ],
  "components": [
    {
      "id": "root-constitution",
      "plane": "GOVERNANCE",
      "kind": "immutable-contract",
      "truth_owner": false,

```



```

    "effect_authority": false,
    "implementation_status": "CONTRACT_DEFINED_E1"
  },
  {
    "id": "human-root",
    "plane": "GOVERNANCE",
    "kind": "authority-principal",
    "truth_owner": false,
    "effect_authority": true,
    "implementation_status": "CONTRACT_DEFINED_E1"
  },
  {
    "id": "world-registry",
    "plane": "TRUTH",
    "kind": "identity-registry",
    "truth_owner": true,
    "effect_authority": false,
    "implementation_status": "CONTRACT_DEFINED_E1"
  },
  {
    "id": "event-state-store",
    "plane": "TRUTH",
    "kind": "transactional-truth-store",
    "truth_owner": true,
    "effect_authority": false,
    "implementation_status": "CONTRACT_DEFINED_E1"
  },
  {
    "id": "world-api",
    "plane": "CONTROL",
    "kind": "control-boundary",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "PLANNED_E0"
  },
  {
    "id": "fractal-runtime",
    "plane": "CONTROL",
    "kind": "bounded-dag-executor",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "resolution-compiler",
    "plane": "CONTROL",
    "kind": "projection-compiler",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "brain-gateway",
    "plane": "COGNITION",
    "kind": "model-router",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "portable-brain-pack",
    "plane": "COGNITION",
    "kind": "provider-neutral-contract",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "universal-manual-bridge",
    "plane": "COGNITION",
    "kind": "zero-api-model-adapter",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "brain-council",
    "plane": "COGNITION",
    "kind": "bounded-deliberation",
    "truth_owner": false,
    "effect_authority": false,

```

```

    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "policy-gate",
    "plane": "EFFECT",
    "kind": "authorization-gate",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "approval-binding",
    "plane": "EFFECT",
    "kind": "exact-intent-binding",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "outbox-executor",
    "plane": "EFFECT",
    "kind": "external-effect-boundary",
    "truth_owner": false,
    "effect_authority": true,
    "implementation_status": "CONTRACT_DEFINED_E1"
  },
  {
    "id": "shadow-compiler",
    "plane": "EVOLUTION",
    "kind": "maturity-promotion",
    "truth_owner": false,
    "effect_authority": false,
    "implementation_status": "IMPLEMENTED_E2"
  },
  {
    "id": "evidence-ledger",
    "plane": "OBSERVABILITY",
    "kind": "append-only-evidence",
    "truth_owner": true,
    "effect_authority": false,
    "implementation_status": "CONTRACT_DEFINED_E1"
  }
],
"invariants": [
  "Entity identity is independent of model, provider, channel, session, process and workflow.",
  "Root Constitution v1.0 and canonical Entity DNA v1.2 remain unchanged in this candidate.",
  "Only Registry, Event and State records may constitute canonical operational truth.",
  "Every model, council and shadow result is proposal-only and has zero inherent authority.",
  "No fallback may reduce minimum Resolution, policy, authority, evidence or data-class constraints.",
  "Up-resolution reloads canonical state and never reconstructs fields omitted by a lower projection.",
  "Fractal expansion is local, trigger-driven, depth-bounded, attempt-bounded and budget-bounded.",
  "Every model sees only an allowlisted projection bound to an exact profile version and hash.",
  "Provider switching preserves the request, pack, projection, policy and output-contract bindings.",
  "Structured decisions are normalized before deterministic rendering and semantic comparison.",
  "External effects require Policy pass, exact ApprovalBinding and an idempotent Outbox intent.",
  "Approval is bound to the exact normalized intent hash and cannot authorize a changed effect.",
  "Shadow learning cannot mutate production behavior directly and promotion is one maturity step at a time.",
  "C5 promotion requires Human Root approval and C6 additionally requires verified restore evidence.",
  "Sensitive decisions may use Council, but consensus never overrides policy, evidence or Human Root.",
  "Claims above E2 remain false until live integration, crash, replay, reconciliation and restore tests pass."
],
"evidence_boundary": {
  "current": "E2",
  "production_claim_minimum": "E4",
  "live_provider_verified": false,
  "postgresql_integration_verified": false,
  "restore_verified": false
}
}

```


PART 2

تست ها، Proof و Evidence

اهداف Proof-of-World، ممیزی، Build Report، خروجی های خام، Evidence ها و
کد کامل آزمون های RC3.

WORLD v6.2 RC3 / SECRETARY-001 PRE-BIRTH / NOT CANONICAL / NOT
DEPLOYED

فهرست منابع کامل این بخش

مسیر منبع	عنوان	بایت	وضعیت
work_sources/ shard_integrated/v6-split- release/reference-proof/ PROOF_OF_WORLD_SPEC_FA.md	مشخصات Proof-of-World	2,950	PROOF TARGET SPECIFICATION
world-v6.2-rc3/candidate- v6.2/docs/ FINAL_AUDIT_AND_RATIFICATION_ CANDIDATE_FA.md	ممیزی نهایی و نامزد Ratification	6,458	AUDIT / CANDIDATE ONLY
output/ World_v6.2_Fractal_Multi_Brai n_RC3_BUILD_REPORT_FA.md	گزارش ساخت RC3	1,896	BUILD REPORT
world-v6.2-rc3/candidate- v6.2/evidence/ FINAL_VALIDATION_RC1.txt	اعتبارسنجی نهایی RC1	1,268	RAW VALIDATION LOG
world-v6.2-rc3/candidate- v6.2/evidence/ FINAL_VALIDATION_RC2.txt	اعتبارسنجی نهایی RC2	1,192	RAW VALIDATION LOG
world-v6.2-rc3/candidate- v6.2/evidence/ FINAL_VALIDATION_RC3.txt	اعتبارسنجی نهایی RC3	1,862	RAW VALIDATION LOG
world-v6.2-rc3/candidate- v6.2/evidence/v6.2-rc1- evidence.json	Evidence RC1	1,273	E2 LOCAL EVIDENCE
world-v6.2-rc3/candidate- v6.2/evidence/v6.2-rc2- evidence.json	Evidence RC2	1,746	E2 LOCAL EVIDENCE
world-v6.2-rc3/candidate- v6.2/evidence/v6.2-rc3- evidence.json	Evidence RC3	2,657	E2 LOCAL EVIDENCE
world-v6.2-rc3/candidate- v6.2/evidence/ancestor-v6.2- candidate/TEST_REPORT_FA.md	گزارش تست پایه Resolution- Native	696	PRESERVED ANCESTOR TEST REPORT
world-v6.2-rc3/candidate- v6.2/evidence/ancestor-v6.2- candidate/ final_core_tests.txt	خروجی خام تست Core	99	RAW TEST OUTPUT
world-v6.2-rc3/candidate- v6.2/evidence/ancestor-v6.2- candidate/ final_secretary_tests.txt	خروجی خام تست Secretary	99	RAW TEST OUTPUT
world-v6.2-secretary-001- alpha1/evidence/ VALIDATION_REPORT_v1.0.0.json	گزارش اعتبارسنجی بسته عملیاتی Secretary-001	3,514	ALPHA1 VALIDATION
world-v6.2-rc3/candidate- v6.2/runtime/core/tests/ test_brain_resolution_rc1.py	کد آزمون: test_brain_resolution_rc1.py	6,105	TEST SOURCE

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_council_rc2.py	کد آزمون: test_council_rc2.py	3,530	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_effect_bindings_rc1.py	کد آزمون: test_effect_bindings_rc1.py	2,913	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_evolution_rc2.py	کد آزمون: test_evolution_rc2.py	3,390	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_fractal_runtime_rc2.py	کد آزمون: test_fractal_runtime_rc2.py	9,757	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_json_schemas_rc1.py	کد آزمون: test_json_schemas_rc1.py	3,203	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_kernel_resolution_hook_rc1.py	کد آزمون: test_kernel_resolution_hook_rc1.py	1,382	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_portable_brain_rc2.py	کد آزمون: test_portable_brain_rc2.py	7,401	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_profile_artifacts_rc1.py	کد آزمون: test_profile_artifacts_rc1.py	1,234	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_rc2_artifacts.py	کد آزمون: test_rc2_artifacts.py	4,019	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_rc3_architecture_artifacts.py	کد آزمون: test_rc3_architecture_artifacts.py	8,118	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_resolution_rc1.py	کد آزمون: test_resolution_rc1.py	8,884	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_channel_brain_continuity.py	کد آزمون: test_channel_brain_continuity.py	1,964	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_external_io_boundary_rc1.py.1	کد آزمون: test_external_io_boundary_rc1.py.1	602	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_portable_brain_rc2.py	کد آزمون: test_portable_brain_rc2.py	2,085	TEST SOURCE

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_resolution_native_rc1.py	کد آزمون: test_resolution_native_rc1.py	5,146	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_secretary.py	کد آزمون: test_secretary.py	2,206	TEST SOURCE
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_telegram_pdf_delivery.py	کد آزمون: test_telegram_pdf_delivery.py	1,571	TEST SOURCE

PASSهای ثبت شده در این بخش E2 هستند؛ این بخش هم نتایج موفق و هم محدودیت های اثبات را نگه می دارد.

مشخصات Proof-of-World

وضعیت سند: PROOF TARGET SPECIFICATION

منبع محلی:

work_sources/shard_integrated/v6-split-release/reference-proof/PROOF_OF_WORLD_SPEC_FA.md

SHA-256 منبع:

8e6502350bca756966982eb7d7f1b0ffa7996dce64014ad85cf1cb5571f7f56d

متن این فایل بدون حذف بخش ها در PDF درج شده است.

مشخصات رسمی Proof-of-World v0.1

Subject Under Test

- World: entity-ark-world-001
- Entity: secretary-001
- DNA: 1.0.0
- Entity version: 0.1.0
- Stage: NEWBORN
- Autonomy: L0

سناریوی ارزش واقعی

ورودی: دستور مالک برای تولید پیش نویس نامه اداری.

خروجی قابل قبول: پیش نویس ساختاریافته، بدون ارسال بیرونی، همراه Event، Memory و Approval Required.

Gate های اجباری

Gate	معیار قبولی
Birth	DNA معتبر، entity_id یکتا، owner و charter موجود
Run	خروجی Schema-valid و Event های آغاز/پایان
Memory	داده مستقل از Brain و قابل بازیابی بعد از Wake
Sleep/Wake	Snapshot معتبر، context حداقلی و cursor بدون گم شدن
Failover	خرابی Primary، موفقیت entity_id، Secondary ثابت
Upgrade/Rollback	نسخه جدید ثبت و نسخه قبلی بازیابی شود
Archive/Revive	Runtime حذف و همان identity/state/history احیا شود

Gate	معیار قبولی
Audit	تمام Transition ها دارای causation و timestamp باشند

آزمون های خرابی

1. Primary Brain exception.
2. تکرار یک Action با idempotency key یکسان.
3. Wake از Snapshot با Event های بعدی.
4. Upgrade نامعتبر و Rollback.
5. Archive و ساخت Runtime تازه از Capsule.
6. ورودی هنگام Sleep و پردازش پس از Wake.

نتیجه قابل انتشار

Proof فقط در صورتی PASS است که تمام آزمون های خودکار سبز و فایل نتیجه دارای `overall_status=PASS` باشد. هر ادعای Production نیازمند مرحله جداگانه شامل `Load Test`، `PostgreSQL`، `Object Storage`، `Secret Manager`، `Provider`، `Security Review` و `Fresh-machine Restore` است.

Gate های افزوده v6.1 - Compositional Governance

Gate	معیار قبولی
Memory Freshness	decay class و last_verified معتبر؛ stale باعث re-verification و quarantine مانع استفاده عادی شود
Intent Review	hard conflict قطعی block شود؛ semantic critic فقط elevate کند و authorize نکند
Evidence Propagation	failure/staleness به subsystem های وابسته signal/plan بدهد بدون mutation خارج از Kernel
Improvement Bound	protected target, evidence floor, rollback و rate limit enforce شوند
Improvement Evaluation	regression objective بدون توجه به امتیاز LLM rollback شود؛ success فقط Promotion Candidate بسازد

این Gate ها در v6.1 در سطح unit/conformance محلی E2 هستند. اجرای migration روی PostgreSQL مستقل و آزمون crash/recovery برای این جداول هنوز Proof نشده است.

ممیزی نهایی و نامزد Ratification

وضعیت سند: AUDIT / CANDIDATE ONLY

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/FINAL_AUDIT_AND_RATIFICATION_CANDIDATE_FA.md

SHA-256 منبع:

1b0486babb4a200d4366f2b75005e4bc6f99393e03ace274ad5edebdaab38b6c

متن این فایل بدون حذف بخش ها در PDF درج شده است.

ممیزی نهایی World v6.2 RC1

تاریخ: 20-08-2026

نتیجه: معماری برای Freeze و شروع پیاده سازی Phase 1 آماده است؛ هنوز Canonical/Production نیست.

دامنه ممیزی

ممیزی روی بسته کامل Canonical baseline, Addendum A/B/C, Root , V_06_v6.2_RESOLUTION_NATIVE_CANDIDATE Constitution, DNA/Registry/Event/Policy/Kernel/Repository/ Backup/Restore/Genesis overlays, Profile ها، schema، Core runtime، secretary-001 و ancestor آن انجام شد.

بسته ورودی با SHA-256 زیر سالم بود و تمام entry های SHA256SUMS.txt آن verify شد:

78e79566799784c2c0b586fe4f6a9db63ae0c7c7330348d34db778b28c2c0551

Root Constitution v1.0 تغییر نکرده است. SHARD فقط research/candidate reference باقی مانده و هیچ authority، ledger یا Core موازی ایجاد نمی کند.

تصمیم نسخه

این تغییر **World v6.2** است، نه World v7؛ چون Human Root، مسیر Entity identity، deterministic authority، Registry/Event truth، Policy، Outbox و اصول Recovery را عوض نمی کند. Resolution یک لایه additive compatibility /view است.

یافته ها و وضعیت RC1

#	یافته	شدت	وضعیت RC1
1	canonical در default=str hashing نوعها را مخلوط می کرد	بحرانی	رفع شد؛ strict Canonical JSON v1
2	R1 بدون Profile/hash معنای سراسری داشت	بحرانی	رفع شد؛ Profile-scoped segment vector

#	یافته	شدت	وضعیت RC1
3	Brain Gateway ورودی را واقعاً project نمی‌کرد	بحرانی	رفع شد؛ Provider فقط envelope مشتق شده می‌بیند
4	Projection helper ها provenance را حذف می‌کردند	بالا	رفع شد؛ consumer envelope + audit metadata جدا
5	Profile JSON با schema و Python hard-code دو منبع حقیقت بود	بالا	رفع شد؛ پنج Profile v0.2 هم‌شکل و runtime-loaded
6	dotted path مبهم و ancestor overwrite مخرب بود	بحرانی	رفع شد؛ JSON Pointer + existing scalar leaf only
7	unknown field می‌توانست با allow_unspecified نشئت کند	بالا	رفع شد؛ RC1 فقط fail-closed را می‌پذیرد
8	Envelope/Patch schema guardهای کافی نداشت	بالا	رفع شد؛ /profile/compiler و canonical/projection hash و version/source الزامی
9	Action فقط minimum عددی داشت و به سیم‌های لازم bind نبود	بحرانی	رفع شد؛ ActionRule + required paths
10	execution_resolution=None مسیر legacy بی‌صدا بود	بحرانی	رفع شد؛ Resolution runtime اجباری
11	Entity مستقیماً Telegram/Drive را صدا می‌زد	بحرانی	رفع شد در reference؛ adapter خارج Entity و direct call مسدود
12	approval root_approved: bool قابل reuse بود	بحرانی	رفع شد؛ exact immutable ApprovalBinding
13	dependency lock/SBOM و کشف portable LibreOffice نبود	بالا	رفع شد؛ pylock.toml , uv.lock , soffice و CycloneDX SBOM fallback
14	cache key و up-resolution contract کامل نبود	متوسط	در مدل نهایی مشخص شد؛ implementation cache هنوز ساخته نشده
15	package شامل cache/pyc و evidence قدیمی به‌عنوان جاری بود	متوسط	رفع شد؛ cache حذف و evidence ancestor جدا شد
16	/PostgreSQL/crash/replay restore/security/live evidence موجود نیست	بحرانی برای Production	باز و صریحاً promotion blocker
17	repository هنوز commit/tag امضا شده ندارد	بالا برای Recovery	باز؛ قبل از promotion الزامی
18	contract overlay ها بیش از حد خلاصه بودند	متوسط	با مدل normative کابل و این ممیزی تکمیل شد

Invariant های Freeze شده

1. Human Root بالاترین authority است.
2. Brain، Entity نیست و هیچ Provider مالک identity/state/history نمی‌شود.
3. Canonical truth هرگز downgrade نمی‌شود.
4. R0 معنای مادر است؛ Refinement فقط سیم اضافه می‌کند.
5. Unknown/profile/hash/version mismatch همگی fail-closed هستند.

6. Up-resolution فقط با reload از Canonical انجام می‌شود.
7. Resolution هیچ approval، risk، permission، یا data classification را کاهش نمی‌دهد.
8. State/Event/Command/Approval/Outbox باید در مرز transaction canonical اتمیک باشند.
9. External intent با external outcome یکی نیست.
10. Recovery تا fresh restore اثبات نشده، NOT_PROVEN است.

شواهد محلی RC1

- Core RC1: **PASS 41/41**
 - Secretary RC1 regression + Resolution: **PASS 21/21**
 - Candidate combined: **PASS 62/62**
 - Ancestor secretary v0.1.3: **PASS 14/14**
 - مجموع اجرای مستقل: **PASS 76/76**
 - JSON Schema/Profile conformance: **PASS**
 - Python compileall: **PASS**
 - dependency lock/SBOM parse: **PASS**
- سطح ادعا حتی پس از PASS این موارد: **E2 local component/regression**.

Promotion blockerهای باقی‌مانده

- PostgreSQL integration و migration روی دیتابیس تازه؛
- transaction اتمیک State/Event/Command/Approval/Outbox؛
- idempotency، replay، crash injection و reconciliation؛
- Executer fencing با control_epoch و approval expiry؛
- live provider capability handshake بر اساس Profile hash؛
- live Telegram/Drive فقط از Executer ثبت‌شده؛
- fresh-machine backup/restore؛
- load، secret isolation، security و chaos؛
- commit/tag/release قابل بازیابی؛
- ثبت approval نهایی Human Root روی hash دقیق بسته.

رأی ممیزی

RC1 تناقض مفهومی اصلی میان «کابل مادر»، Patch، Policy، Brain، Profile و اثر خارجی را بسته و برای شروع پیاده‌سازی vertical slice مناسب است. Ratification این سند به معنی اجازه شروع Phase 1 است؛ به معنی اعلام Production یا تولد Canonical Entity نیست. آن ادعا فقط پس از عبور evidence gate‌های بالا مجاز است.

گزارش ساخت RC3

وضعیت سند: BUILD REPORT

منبع محلی:

output/World_v6.2_Fractal_Multi_Brain_RC3_BUILD_REPORT_FA.md

SHA-256 منبع:

96c941a107e8a68c80ba9c7592624c7ab7c71a5ada5eadef81e388f060fcc9fa

متن این فایل بدون حذف بخش ها در PDF درج شده است.

گزارش ساخت World v6.2 Fractal Multi-Brain RC3

تاریخ: 20-08-2026

وضعیت: RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED

سطح شاهد: E2_LOCAL_CONTRACT

خروجی نهایی

فایل: World_v6.2_Fractal_Multi_Brain_RC3.zip

اندازه: 7,269,006 bytes

SHA-256: 5568ba62045e31ce63dd01108c96a8a266082388aff507aee158ae834ae92855

تعداد فایل داخل payload: 204

ساخت deterministic دوباره و مقایسهٔ بایت به بایت: PASS

تست ساختار ZIP: PASS

کنترل SHA256SUMS.txt بعد از extract: PASS

cache/VCS artifact scan: PASS

Hash های مرجع

Release Manifest: e14827056d3f6fe95c1a683412237bd54e4c706828a1c2a6e81bb87a04468c19

SHA256SUMS index: 3259a10ac9163f81b29679016f0ee005a7b77ed08f7899f5d5fd0f7282fe5548

File Inventory: a305849561e5c725bad09dd608a0ead7103c46998cce25bad8f7781708ae0621

سند معماری: cd5fa150cdfefb23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80

مانیفست معماری: bec6dbd16d1d4b9cd58efb66bd8dc6ecff557f24b8dc5078d693a8dc3faff3fe

تست های نهایی روی ZIP استخراج شده

Core: PASS 78/78

- Secretary :22/22 PASS
- Candidate :100/100 PASS
- Preserved ancestor :14/14 PASS
- مجموع مستقل: 114/114 PASS
- Architecture/Universal Host subset :7/7 PASS
- Offline ChatGPT/Gemini/Grok demos :3/3 PASS

مرز ادعا

مسیر Manual Host برای هر مدل منطبق، بدون Network و API token از سمت Runtime، در E2 عملیاتی و تست شده است. اتصال زنده و خودکار Providerها، PostgreSQL atomic integration، crash/replay/reconciliation، fresh restore، security/load/chaos و Production هنوز تست نشده اند و ادعا نمی شوند. Canonical promotion نیز ثبت نشده است.

اعتبارسنجی نهایی RC1

وضعیت سند: RAW VALIDATION LOG

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/FINAL_VALIDATION_RC1.txt

SHA-256 منبع:

789b1aab45b2bd13dd081fb18f10f24ffeb86535b064d3ce5bac0b6c5b8d22c9

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 Resolution-Native RC1 - independent final validation
2026-08-20

Input v6.2 candidate ZIP SHA-256:
78e79566799784c2c0b586fe4f6a9db63ae0c7c7330348d34db778b28c2c0551
Input package SHA256SUMS verification: PASS

Locked environment:
- Python 3.12.13
- uv 0.11.33
- pytest 9.1.1
- pyproject.toml + uv.lock + pylock.toml
- CycloneDX 1.5 SBOM generated and parsed

Tests:
- Core RC1: 41/41 PASS
- Secretary RC1: 21/21 PASS
- Candidate combined: 62/62 PASS
- Preserved secretary v0.1.3 ancestor: 14/14 PASS
- Total independent execution: 76/76 PASS

Static/contract validation:
- All active Profile v0.2 artifacts conform to resolution-profile schema: PASS
- Runtime envelopes and patch samples conform to JSON Schemas: PASS
- 20 JSON artifacts parsed: PASS
- Candidate runtime compileall: PASS
- No .pytest_cache, __pycache__, .pyc or .venv in release tree: PASS

Claim boundary:
- E2 local deterministic component/regression evidence only
- PostgreSQL integration: NOT RUN / NOT CLAIMED
- crash/replay/reconciliation: NOT RUN / NOT CLAIMED
- fresh restore: NOT RUN / NOT CLAIMED
- live Brain/Telegram/Drive: NOT RUN / NOT CLAIMED
- security/load/chaos: NOT RUN / NOT CLAIMED
- Human Root ratification: NOT RECORDED
- Canonical/Production readiness: NOT CLAIMED

اعتبارسنجی نهایی RC2

وضعیت سند: RAW VALIDATION LOG

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/FINAL_VALIDATION_RC2.txt

SHA-256 منبع:

4927ad7c0c1aba4dd5845b2ebb27596954f53fd73c7a8ebf14a6c1abfee2d1a9

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 Fractal Multi-Brain RC2 - FINAL LOCAL VALIDATION

Date: 2026-08-20

Status: RATIFICATION CANDIDATE / NOT CANONICAL / NOT DEPLOYED

Evidence: E2 LOCAL ONLY

LOCKED ENVIRONMENT

- uv sync --locked --all-groups: PASS
- Python: 3.12
- Runtime network/API tokens: DISABLED / NOT USED

TESTS

- Core RC2: 71/71 PASS
- Secretary RC2: 22/22 PASS
- Candidate combined: 93/93 PASS
- Preserved v0.1.3 ancestor: 14/14 PASS
- Total independent pytest: 107/107 PASS
- Offline ChatGPT/Gemini/Grok demos: 3/3 PASS
- compileall: PASS
- Schema/artifact/hash/secret conformance: PASS

BOUNDARIES

- Root Constitution v1.0: UNCHANGED
- Canonical DNA v1.2: UNCHANGED
- RC2 DNA file: NON-CANONICAL EXECUTION OVERLAY
- External effect execution: NOT ENABLED
- Live provider APIs: NOT RUN / NOT CLAIMED
- PostgreSQL/crash/restore/security/load/chaos: NOT RUN / NOT CLAIMED
- Human Root canonical ratification: NOT RECORDED
- Production readiness: NOT CLAIMED

NOTE

The preserved ancestor expects the executable name `libreoffice`; the environment provided the same LibreOffice engine as `soffice`. A test-only alias was used.
The RC2 candidate already supports either executable and an isolated user profile.

اعتبارسنجی نهایی RC3

وضعیت سند: RAW VALIDATION LOG

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/FINAL_VALIDATION_RC3.txt

SHA-256 منبع:

b03796e94ff249ab061336bd00d49a9cc552b66633a5be2926f5bde4a2e43c0b

متن این فایل بدون حذف بخش ها در PDF درج شده است.

World v6.2 Fractal Multi-Brain RC3 - FINAL LOCAL VALIDATION

Date: 2026-08-20

Status: RATIFICATION CANDIDATE / NOT CANONICAL / NOT DEPLOYED

Evidence: E2 LOCAL CONTRACT ONLY

LOCKED ENVIRONMENT

- uv 0.11.33 lock check: PASS
- uv sync --locked --all-groups: PASS
- Python: 3.12.13
- Runtime model network/API tokens: DISABLED / NOT USED

TESTS

- Core RC3: 78/78 PASS
- Secretary RC3: 22/22 PASS
- Candidate combined: 100/100 PASS
- Preserved v0.1.3 ancestor: 14/14 PASS
- Total independent pytest: 114/114 PASS
- RC3 architecture/universal-host subset: 7/7 PASS
- Offline ChatGPT/Gemini/Grok demos: 3/3 PASS

CONFORMANCE

- Complete architecture document and machine manifest binding: PASS
- Model Capability Card and exact Resolution Profile bindings: PASS
- Self-contained Universal Manual Host bundle: PASS
- Arbitrary provider label: PASS
- R0 minimum projection: PASS
- Bundle tamper rejection: PASS
- Strict Secretary Decision validation: PASS
- Authority/effect smuggling rejection: PASS
- Semantic and deterministic-render equivalence: PASS
- compileall / all JSON / Draft 2020-12 schemas / secret scan: PASS
- dependency lock / CycloneDX SBOM: PASS

BOUNDARIES

- Root Constitution v1.0: UNCHANGED
- Canonical DNA v1.2: UNCHANGED
- RC2: PRESERVED AS SEPARATE IMMUTABLE PARENT
- RC3 Entity DNA overlay and Brain Pack: RC2 ARTIFACTS PRESERVED
- External effect execution: NOT ENABLED
- Live provider APIs: NOT RUN / NOT CLAIMED
- PostgreSQL/crash/restore/security/load/chaos: NOT RUN / NOT CLAIMED
- Human Root canonical ratification: NOT RECORDED
- Production readiness: NOT CLAIMED

NOTE

The preserved ancestor expects the executable name `libreoffice`; the environment provided the same LibreOffice engine through a test-only alias. The RC3 candidate inherits the RC2 portable discovery of either `libreoffice` or `soffice` and per-run profile isolation.

Evidence RC1

وضعیت سند: E2 LOCAL EVIDENCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/v6.2-rc1-evidence.json

SHA-256 منبع:

fb9f1bc8afb45ce38725c5253613f0faec41c88cb038cb3be5c458b5a57db51

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "release": "World v6.2 Resolution-Native RC1",
  "date": "2026-08-20",
  "evidence_level": "E2",
  "claim_boundary": "local deterministic component and regression behavior only",
  "environment": {
    "python": "3.12.13",
    "uv": "0.11.33",
    "pytest": "9.1.1",
    "dependency_lock": "uv.lock",
    "portable_lock": "pylock.toml",
    "sbom": "SBOM.cdx.json"
  },
  "tests": {
    "core_rc1": {
      "passed": 41,
      "failed": 0
    },
    "secretary_rc1": {
      "passed": 21,
      "failed": 0
    },
    "candidate_combined": {
      "passed": 62,
      "failed": 0
    },
    "preserved_ancestor": {
      "passed": 14,
      "failed": 0
    },
    "total": {
      "passed": 76,
      "failed": 0
    }
  },
  "contract_checks": {
    "active_profiles": 5,
    "profile_schema_conformance": "PASS",
    "envelope_schema_conformance": "PASS",
    "patch_schema_conformance": "PASS",
    "json_artifacts_parsed": 20,
    "compileall": "PASS",
    "release_cache_contamination": "NONE"
  },
  "not_proved": [
    "PostgreSQL atomic integration",
    "crash replay idempotency reconciliation",
    "fresh-machine restore",
    "live provider capability negotiation",
    "live Telegram and Drive execution",
    "security load chaos",
    "version-control release tag",
    "Human Root ratification",
    "Canonical or production readiness"
  ]
}
```

Evidence RC2

وضعیت سند: E2 LOCAL EVIDENCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/v6.2-rc2-evidence.json

SHA-256 منبع:

8d3b9192222dc882605a890c83c0ba4afc69c07balf0dac9fc0c6fa6e2bf9d46

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "release": "World v6.2 Fractal Multi-Brain RC2",
  "date": "2026-08-20",
  "environment": "local locked Python 3.12 test environment",
  "evidence_level": "E2",
  "results": {
    "core": {
      "passed": 71,
      "failed": 0
    },
    "secretary": {
      "passed": 22,
      "failed": 0
    },
    "preserved_ancestor": {
      "passed": 14,
      "failed": 0
    },
    "offline_provider_demos": {
      "passed": 3,
      "failed": 0
    },
    "total_pytest": {
      "passed": 107,
      "failed": 0
    },
    "compileall": "PASS",
    "uv_sync_locked": "PASS",
    "json_schema_and_artifact_conformance": "PASS",
    "obvious_secret_pattern_scan": "PASS"
  },
  "test_scope": [
    "strict canonical JSON and Resolution RC1 regression",
    "capsule node/input hash binding and tamper rejection",
    "bounded fractal expansion and global attempt budget",
    "network/API-token adapter rejection in offline mode",
    "provider switching without identity/state/profile drift",
    "strict Secretary Decision normalization and deterministic rendering",
    "ChatGPT/Gemini/Grok fixture semantic equality",
    "Shadow evidence and C0..C6 promotion gates",
    "blind Council, revision rounds, weighted vote, dissent and veto",
    "real secretary Store bridge without provider-owned mutation",
    "ancestor functional regression"
  ],
  "system_dependency_note": "The preserved ancestor hard-codes the executable name libreoffice. The environment supplied soffice; a temporary test-only alias to the same binary was used. RC2 candidate code already discovers either name and isolates the LibreOffice profile.",
  "not_tested": [
    "live provider/API calls",
    "PostgreSQL transactionality",
    "crash/replay/reconciliation",
    "fresh-machine restore",
    "security/load/chaos",
    "production deployment"
  ]
}
```

Evidence RC3

وضعیت سند: E2 LOCAL EVIDENCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/v6.2-rc3-evidence.json

SHA-256 منبع:

b5e7ea09b76d3ce09ddede7193cb08fbafb0a1f320a768660a168972e763b7a4

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "release": "World v6.2 Fractal Multi-Brain RC3",
  "version": "6.2.0-rc.3",
  "generated_at": "2026-08-20",
  "status": "RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED",
  "evidence_level": "E2_LOCAL_CONTRACT",
  "environment": {
    "python": "3.12.13",
    "uv": "0.11.33",
    "lock_check": "PASS",
    "locked_sync": "PASS",
    "environment_path": "/tmp/world-v62-rc3-venv",
    "runtime_network_for_model_invocation": false,
    "runtime_api_token_used": false
  },
  "tests": {
    "core": {
      "passed": 78,
      "failed": 0
    },
    "secretary": {
      "passed": 22,
      "failed": 0
    },
    "candidate_combined": {
      "passed": 100,
      "failed": 0
    },
    "preserved_ancestor": {
      "passed": 14,
      "failed": 0
    },
    "total_independent": {
      "passed": 114,
      "failed": 0
    },
    "rc3_architecture_subset": {
      "passed": 7,
      "failed": 0
    },
    "offline_provider_demos": {
      "passed": 3,
      "failed": 0
    }
  },
  "universal_model_conformance": {
    "provider_family": "ANY_CONFORMING_MODEL",
    "mode": "MANUAL_HOST",
    "bundle_schema": "PASS",
    "arbitrary_provider_label": "PASS",
    "minimal_r0_projection": "PASS",
    "integrity_tamper_rejection": "PASS",
    "strict_output_validation": "PASS",
    "authority_smuggling_rejection": "PASS",
    "chatgpt_gemini_grok_semantic_equivalence": "PASS",
    "deterministic_render_equivalence": "PASS",
    "live_provider_api": "NOT_RUN_NOT_CLAIMED"
  },
  "artifact_checks": {
```

```

    "all_json_parse": "PASS",
    "all_json_schemas_draft_2020_12": "PASS",
    "architecture_document_manifest_hash_binding": "PASS",
    "model_card_profile_hash_binding": "PASS",
    "portable_pack_dna_prompt_schema_profile_binding": "PASS",
    "compileall": "PASS",
    "secret_scan": "PASS",
    "cyclonedx_sbom_json": "PASS"
  },
  "architecture": {
    "document_sha256": "cd5fa150cdfeb23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80",
    "manifest_sha256": "bec6dbd16d1d4b9cd58efb66bd8dc6ecff557f24b8dc5078d693a8dc3faff3fe",
    "release_manifest_sha256_at_validation": "e14827056d3f6fe95c1a683412237bd54e4c706828a1c2a6e81bb87a04468c19"
  },
  "preservation": {
    "root_constitution_v1_0": "UNCHANGED",
    "canonical_entity_dna_v1_2": "UNCHANGED",
    "parent_rc2": "PRESERVED_SEPARATELY",
    "source_ancestor_secretary_0_1_3": "PRESERVED_UNCHANGED"
  },
  "open_e3_e4_e5": [
    "live_provider_adapter conformance",
    "PostgreSQL atomic Registry/Event/State/Outbox integration",
    "crash replay idempotency and reconciliation",
    "fresh restore and C6 non-extinction proof",
    "security privacy load and chaos tests",
    "signed version-control release",
    "Human Root canonical ratification"
  ],
  "production_readiness_claimed": false
}

```

گزارش تست پایه Resolution-Native

وضعیت سند: PRESERVED ANCESTOR TEST REPORT

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/ancestor-v6.2-candidate/TEST_REPORT_FA.md

SHA-256 منبع:

4a538398b4cdc1e70f03755a373986b12baeb0806c4d3391fe0b89e47c2584a8

متن این فایل بدون حذف بخش ها در PDF درج شده است.

گزارش آزمون World v6.2 Resolution-Native Candidate

- قبل از تغییر: آخرین source واقعی secretary-001 v0.1.3 با همان 4 فایل تست موجود: **PASS 14/14**.
- بعد از تغییر: Core Resolution suite: **PASS 10/10**.
- بعد از تغییر: Secretary regression + Resolution suite: **PASS 21/21**.
- مجموع Candidate: **PASS 31/31**.
- Secretary source: PASS و Core برای compileall.

سطح ادعا: **E2 فقط برای رفتار component/regression محلی**. PostgreSQL integration, crash/replay, fresh canonical ratification اجرا/اثبات نشده اند. restore, live providers/Telegram, security/load/chaos

خروجی خام تست Core

وضعیت سند: RAW TEST OUTPUT

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/ancestor-v6.2-candidate/final_core_tests.txt

SHA-256 منبع:

1253f6ca793d46eb9c48b6f1606064815d0bb62be75a701be4c49494044e47ce

متن این فایل بدون حذف بخش ها در PDF درج شده است.

.....
10 passed in 0.03s

[100%]

خروجی خام تست Secretary

وضعیت سند: RAW TEST OUTPUT

منبع محلی:

world-v6.2-rc3/candidate-v6.2/evidence/ancestor-v6.2-candidate/final_secretary_tests.txt

SHA-256 منبع:

53318a6a1b0ef1305e9e054c1c0ca2f6e135524bbf242cc6a4b6c9e1ee98dad1

متن این فایل بدون حذف بخش ها در PDF درج شده است.

.....
21 passed in 1.41s

[100%]

گزارش اعتبارسنجی بسته عملیاتی Secretary-001

وضعیت سند: ALPHA1 VALIDATION

منبع محلی:

world-v6.2-secretary-001-alpha1/evidence/VALIDATION_REPORT_v1.0.0.json

SHA-256 منبع:

12d5e1be460a0c4f3e9dfb70acb4480a39da997a95b1a348aa10fccc4b5496d0

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.secretary-plan-validation-report.v1",
  "report_version": "1.0.0",
  "validated_at": "2026-08-20",
  "bundle_id": "world-v6.2.secretary-001.alpha1",
  "status": "PASS_WITH_EXPLICIT_PRE_BIRTH_BOUNDARY",
  "claim_boundary": {
    "plan_and_contract_evidence": "E1",
    "parent_local_runtime_evidence": "E2",
    "live_telegram_proven": false,
    "live_provider_proven": false,
    "postgresql_atomic_integration_proven": false,
    "fresh_restore_proven": false,
    "canonical_birth_authorized": false,
    "production_claim_authorized": false
  },
  "parent_integrity": {
    "architecture_manifest_sha256": "bec6dbd16d1d4b9cd58efb66bd8dc6ecff557f24b8dc5078d693a8dc3faff3fe",
    "architecture_document_sha256": "cd5fa150cdfefb23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80",
    "entity_manifest_sha256": "d43be7efb44e28d5eaf0900599faa88e7afb271927d802a1b23e903e88691f19",
    "portable_brain_pack_sha256": "38f9b5d7e3990b422daba73c430914bb9a51e0848cfd3a8d312c5dcf934d2146",
    "result": "PASS"
  },
  "plan_validation": {
    "json_files_parsed": 7,
    "backlog_schema": "PASS_DRAFT_2020_12",
    "backlog_task_count": 61,
    "task_ids_unique": true,
    "dependencies_resolved": true,
    "dependency_graph_acyclic": true,
    "n8n_registration": "NOT_REGISTERED_CONNECTOR_UNAVAILABLE",
    "fake_receipt_generated": false
  },
  "fresh_parent_regression": {
    "core": {
      "passed": 78,
      "failed": 0
    },
    "secretary_candidate": {
      "passed": 22,
      "failed": 0
    },
    "ancestor": {
      "passed": 14,
      "failed": 0
    },
    "independent_total": {
      "passed": 114,
      "failed": 0
    },
    "architecture_subset": {
      "passed": 7,
      "failed": 0
    },
    "offline_provider_demos": {
      "passed": 3,
      "providers": [
        "chatgpt-offline-fixture",
        "gemini-offline-fixture",
```

```

    "grok-offline-fixture"
  ],
  "same_rendered_text": true,
  "same_semantic_hash": "cecab3bfff20debd53fc52c20264584b09ba7b735203a3b24d92b3685b57efcc",
  "network_used": false,
  "api_token_used": false
}
},
"environment_note": "The first fresh ancestor rerun found no libreoffice executable name and reported 13 pass plus one
environment failure. No source was changed. A temporary external wrapper pointed to the available pinned soffice binary with
a writable temporary user profile; the full ancestor suite then passed 14/14.",
"official_interface_review": {
  "checked_at": "2026-08-20",
  "telegram": "official Bot API/webhook/numeric user ID reviewed",
  "openai": "official Responses/Structured Outputs path reviewed",
  "xai": "official structured-output/security docs reviewed",
  "gemini": "official structured-output/function-calling/caching docs reviewed",
  "anthropic": "official Messages/tool-use docs reviewed",
  "kimi": "official Chat API docs reviewed",
  "copilot": "official Copilot SDK GA notice reviewed",
  "legal": "official GDPR/EDPB sources reviewed; account-specific legal approval remains pending"
},
"must_remain_false_until_evidence": [
  "secretary-001 is canonically born",
  "Telegram is deployed and authorized",
  "any remote provider adapter is live-conformant",
  "Council guarantees truth or safety",
  "legal or financial effects are allowed",
  "E3 or E4 readiness is proven",
  "the backlog is registered in n8n"
],
"verdict": "The operational dossier is internally consistent and implementation-ready as a plan. Implementation must begin
at the PostgreSQL Truth Spine; live Birth remains blocked by the machine-readable gates."
}

```

کد آزمون: test_brain_resolution_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_brain_resolution_rc1.py

SHA-256 منبع:

aleb548aa1e12246e8f033537e9a40e7f408296bb624004e98beb7faf3512955

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import pytest

from core.brain_gateway import (
    BrainGateway,
    BrainInputSegment,
    BrainRequest,
    BrainUnavailable,
)
from core.resolution import FieldRule, ProjectionProfile

def task_profile():
    return ProjectionProfile(
        "task", "0.2.0", "R1",
        (
            FieldRule("/id", "R0", "R1", "READ_ONLY_WHEN_PROJECTED", expected_type="string"),
            FieldRule("/title", "R0", "R0", expected_type="string"),
            FieldRule("/secret", "R1", "R1", "NO_DOWNGRADE", data_class="RESTRICTED", expected_type="string"),
        ),
        required_backbone_paths=("/id", "/title"),
    )

def context_profile():
    return ProjectionProfile(
        "conversation", "0.2.0", "R1",
        (
            FieldRule("/direction", "R0", "R1", "READ_ONLY_WHEN_PROJECTED", expected_type="string"),
            FieldRule("/text", "R0", "R1", "READ_ONLY_WHEN_PROJECTED", expected_type="string"),
            FieldRule("/provider", "R1", "R1", "READ_ONLY_WHEN_PROJECTED", expected_type="string"),
        ),
        required_backbone_paths=("/direction", "/text"),
    )

def segment(profile=None, **overrides):
    profile = profile or task_profile()
    values = {
        "segment_id": "task",
        "canonical": {"id": "T1", "title": "draft", "secret": "never-at-r0"},
        "profile_id": profile.profile_id,
        "profile_version": profile.version,
        "profile_hash": profile.profile_hash,
        "source_ref": "task:T1",
        "source_version": 4,
        "desired_resolution": "R1",
        "minimum_resolution": "R0",
        "purpose": "DRAFT",
        "data_class": "RESTRICTED",
        "freshness": "CURRENT",
    }
    values.update(overrides)
    return BrainInputSegment(**values)

class Provider:
    def __init__(self, name, capabilities, compatible=True, fail=False):
        self.name = name
        self.capabilities = capabilities
```

```

        self._compatible = compatible
        self.fail = fail
        self.descriptor = None
        self.seen = None

    def compatible(self, descriptor):
        self.descriptor = descriptor
        return self._compatible

    def max_resolution_for(self, profile_id, profile_version, profile_hash):
        return self.capabilities.get((profile_id, profile_version, profile_hash))

    def invoke(self, request):
        self.seen = request
        if self.fail:
            raise RuntimeError("provider failed")
        return {"ok": True}

def request(*segments):
    return BrainRequest("2", "draft", tuple(segments or (segment(),)))

def capability(profile, level):
    return {(profile.profile_id, profile.version, profile.profile_hash): level}

def test_gateway_projects_real_input_before_provider_boundary():
    profile = task_profile()
    provider = Provider("small", capability(profile, "R0"))
    response = BrainGateway([provider], [profile]).invoke(request(segment(profile)))
    envelope = provider.seen.inputs[0].envelope
    assert response.effective_resolution == "R0"
    assert envelope["value"] == {"id": "T1", "title": "draft"}
    assert "secret" not in envelope["value"]
    assert "omitted_paths" not in envelope
    assert envelope["profile_hash"] == profile.profile_hash

def test_compatibility_gate_receives_metadata_only():
    profile = task_profile()
    provider = Provider("p", capability(profile, "R0"))
    BrainGateway([provider], [profile]).invoke(request(segment(profile)))
    descriptor_segment = provider.descriptor.segments[0]
    assert "canonical" not in descriptor_segment
    assert "never-at-r0" not in repr(provider.descriptor)

def test_provider_capability_is_profile_and_hash_scoped():
    profile = task_profile()
    wrong = {(profile.profile_id, profile.version, "0" * 64): "R1"}
    with pytest.raises(BrainUnavailable):
        BrainGateway([Provider("wrong", wrong)], [profile]).invoke(request(segment(profile)))

def test_scalar_legacy_max_is_not_treated_as_cross_domain_capability():
    profile = task_profile()

    class Legacy:
        name = "legacy"
        max_resolution = "R0"
        def compatible(self, descriptor): return True
        def invoke(self, request): return {"ok": True}

    with pytest.raises(BrainUnavailable):
        BrainGateway([Legacy()], [profile]).invoke(request(segment(profile)))

def test_low_provider_is_skipped_when_any_segment_minimum_is_unmet():
    task = task_profile()
    context = context_profile()
    low_caps = {**capability(task, "R0"), **capability(context, "R0")}
    high_caps = {**capability(task, "R1"), **capability(context, "R1")}
    low = Provider("low", low_caps)
    high = Provider("high", high_caps)
    context_segment = segment(
        context,
        segment_id="context",
        canonical={"direction": "INBOUND", "text": "hello", "provider": "openai"},
        source_ref="message:M1",
        minimum_resolution="R1",

```

```

        data_class="INTERNAL",
    )
    response = BrainGateway([low, high], [task, context]).invoke(
        request(segment(task), context_segment)
    )
    assert response.provider == "high"
    assert low.seen is None
    assert dict(response.effective_resolutions) == {"task": "R1", "context": "R1"}
    with pytest.raises(Exception):
        _ = response.effective_resolution

def test_provider_failure_fails_over_without_changing_identity_or_profiles():
    profile = task_profile()
    caps = capability(profile, "R0")
    first = Provider("first", caps, fail=True)
    second = Provider("second", caps)
    response = BrainGateway([first, second], [profile]).invoke(request(segment(profile)))
    assert response.provider == "second"
    assert second.seen.entity_id == "secretary-001"
    assert second.seen.inputs[0].envelope["profile_hash"] == profile.profile_hash

def test_request_profile_hash_mismatch_fails_before_provider_invocation():
    profile = task_profile()
    provider = Provider("p", capability(profile, "R1"))
    bad = segment(profile, profile_hash="0" * 64)
    with pytest.raises(Exception, match="hash mismatch"):
        BrainGateway([provider], [profile]).invoke(request(bad))
    assert provider.seen is None

```

کد آزمون: test_council_rc2.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_council_rc2.py

SHA-256 منبع:

777587b420fc77ffca3530cad8652d7fa30104fec472fdc17151e4fde0c2981f

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import pytest

from core.council import (
    BallotPosition,
    CouncilBallot,
    CouncilContractError,
    CouncilOutcome,
    CouncilParticipant,
    CouncilSession,
)

PROPOSAL = "a" * 64
OTHER = "b" * 64
CONTEXT = "c" * 64

def participants():
    return (
        CouncilParticipant("domain", "DOMAIN", 1000),
        CouncilParticipant("risk", "RISK", 1200, True),
        CouncilParticipant("policy", "POLICY", 1200, True),
    )

def ballot(who, round_no=1, position=BallotPosition.SUPPORT, proposal=PROPOSAL, confidence=900):
    return CouncilBallot(who, round_no, CONTEXT, proposal, position, confidence, f"reason-{who}")

def test_first_round_is_blind_until_every_participant_submits():
    session = CouncilSession("council-1", CONTEXT, participants())
    session.submit(ballot("domain"))
    with pytest.raises(CouncilContractError, match="sealed"):
        session.reveal()
    session.submit(ballot("risk"))
    session.submit(ballot("policy"))
    assert [item.participant_id for item in session.reveal()] == ["domain", "policy", "risk"]

def test_revision_round_can_reach_a_proposal_but_never_an_execution():
    session = CouncilSession("council-2", CONTEXT, participants(), max_rounds=2, minimum_support_millis=700)
    session.submit(ballot("domain"))
    session.submit(ballot("risk", position=BallotPosition.OPPOSE))
    session.submit(ballot("policy", position=BallotPosition.ABSTAIN))
    first = session.advance()
    assert len(first) == 3
    for who in ("domain", "risk", "policy"):
        session.submit(ballot(who, round_no=2))
    decision = session.finalize()
    assert decision.outcome is CouncilOutcome.PROPOSED
    assert decision.proposal_hash == PROPOSAL
    assert decision.round_count == 2
    assert decision.proposal_only is True

def test_high_risk_safety_veto_blocks_even_majority_support():
    session = CouncilSession("council-3", CONTEXT, participants(), high_risk=True)
```

```

session.submit(ballot("domain"))
session.submit(ballot("policy"))
session.submit(ballot("risk", position=BallotPosition.VETO))
decision = session.finalize()
assert decision.outcome is CouncilOutcome.VETOED
assert decision.vetoes == ("risk",)

def test_unqualified_veto_duplicate_vote_and_conflicting_proposals_fail_closed():
    session = CouncilSession("council-4", CONTEXT, participants())
    with pytest.raises(CouncilContractError, match="no veto"):
        session.submit(ballot("domain", position=BallotPosition.VETO))
    session.submit(ballot("domain"))
    with pytest.raises(CouncilContractError, match="duplicate"):
        session.submit(ballot("domain"))
    session.submit(ballot("risk", proposal=OTHER))
    session.submit(ballot("policy"))
    assert session.finalize().outcome is CouncilOutcome.DEFERRED

    mismatched = CouncilSession("council-5", CONTEXT, participants())
    wrong_context = CouncilBallot(
        "domain", 1, "d" * 64, PROPOSAL, BallotPosition.SUPPORT, 900, "wrong-state"
    )
    with pytest.raises(CouncilContractError, match="snapshot mismatch"):
        mismatched.submit(wrong_context)

def test_transcript_hash_is_independent_of_submission_order():
    hashes = []
    for order in (("domain", "risk", "policy"), ("policy", "domain", "risk")):
        session = CouncilSession("same-session", CONTEXT, participants(), max_rounds=1)
        for who in order:
            session.submit(ballot(who))
        hashes.append(session.finalize().transcript_hash)
    assert hashes[0] == hashes[1]

```

کد آزمون: test_effect_bindings_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_effect_bindings_rc1.py

SHA-256 منبع:

c69ef31739c0c19d1a2f3c5760caaf34e2da19f26e2bb22e465e3ddfa262369f

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import pytest

from core.effects import (
    ApprovalBinding,
    EffectBindingError,
    ExternalEffectProposal,
    prepare_outbox_intent,
    sha256_bytes,
)

def proposal():
    return ExternalEffectProposal(
        world_id="world-v6",
        entity_id="secretary-001",
        command_id="cmd-1",
        destination="telegram",
        action="SEND_DOCUMENT",
        resource_ref="artifact:letter.pdf",
        recipient_ref="telegram-chat:123",
        payload_ref="blob:abc",
        payload_hash=sha256_bytes(b"%PDF-1.7 exact bytes"),
        policy_version="1.2.0",
        expected_version=9,
        control_epoch=4,
        idempotency_scope="secretary:telegram",
        idempotency_key="letter-1-to-123",
    )

def approval(effect=None, **overrides):
    effect = effect or proposal()
    values = {
        "approval_ref": "approval-1",
        "approver_ref": "human-root",
        "decision": "APPROVE",
        "command_id": effect.command_id,
        "action": effect.action,
        "recipient_ref": effect.recipient_ref,
        "payload_hash": effect.payload_hash,
        "effect_hash": effect.effect_hash,
        "policy_version": effect.policy_version,
        "expected_version": effect.expected_version,
        "control_epoch": effect.control_epoch,
        "issued_at": "2026-08-20T10:00:00+00:00",
        "expires_at": "2026-08-20T11:00:00+00:00",
    }
    values.update(overrides)
    return ApprovalBinding(*values)

def test_exact_human_root_binding_prepares_non_authoritative_outbox_value():
    effect = proposal()
    prepared = prepare_outbox_intent(
        effect,
        approval(effect),
        policy_decision_ref="policy-decision-1",
        evaluated_at="2026-08-20T10:30:00+00:00",
    )
```



```

assert prepared.effect_hash == effect.effect_hash
assert prepared.status == "PREPARED_NOT_COMMITTED"
assert prepared.authoritative is False
assert prepared.to_document()["recipient_ref"] == "telegram-chat:123"

@pytest.mark.parametrize(
    "override",
    [
        {"approver_ref": "brain"},
        {"recipient_ref": "telegram-chat:999"},
        {"payload_hash": "0" * 64},
        {"effect_hash": "0" * 64},
        {"expected_version": 8},
        {"control_epoch": 3},
    ],
)
def test_approval_is_not_a_reusable_yes_token(override):
    effect = proposal()
    with pytest.raises(EffectBindingError):
        prepare_outbox_intent(
            effect,
            approval(effect, **override),
            policy_decision_ref="policy-decision-1",
            evaluated_at="2026-08-20T10:30:00+00:00",
        )

def test_expired_approval_fails_closed():
    effect = proposal()
    with pytest.raises(EffectBindingError, match="expired"):
        prepare_outbox_intent(
            effect,
            approval(effect),
            policy_decision_ref="policy-decision-1",
            evaluated_at="2026-08-20T12:00:00+00:00",
        )

```

کد آزمون: test_evolution_rc2.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_evolution_rc2.py

SHA-256 منبع:

bb3822ed5124da8056b9e31ea61c4324fa84efa8ba5c3ebc790e60c498719fff

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import pytest

from core.evolution import (
    CompilationMaturity,
    EvolutionContractError,
    PromotionController,
    PromotionThreshold,
    ShadowRunRecord,
)

H = "a" * 64
MATCH = "b" * 64
DIFF = "c" * 64

def record(index, *, match=True, invariant=0, effect=0, replay=True, candidate_hash=H):
    return ShadowRunRecord(
        run_id=f"run-{index}",
        candidate_id="candidate",
        candidate_version="1.0.0",
        candidate_hash=candidate_hash,
        node_id="secretary.route",
        input_hash=f"{index:064x}",
        primary_semantic_hash=MATCH,
        candidate_semantic_hash=MATCH if match else DIFF,
        invariant_failures=invariant,
        forbidden_effect_attempts=effect,
        replay_match=replay,
    )

def controller():
    return PromotionController({
        CompilationMaturity.C1: PromotionThreshold(1, 0),
        CompilationMaturity.C2: PromotionThreshold(3, 666),
        CompilationMaturity.C3: PromotionThreshold(3, 900),
        CompilationMaturity.C4: PromotionThreshold(3, 900),
        CompilationMaturity.C5: PromotionThreshold(3, 900, True, True),
        CompilationMaturity.C6: PromotionThreshold(3, 900, True, True, True),
    })

def assess(current, rows, **kwargs):
    return controller().assess(
        candidate_id="candidate",
        candidate_version="1.0.0",
        candidate_hash=H,
        current=current,
        records=rows,
        **kwargs,
    )

def test_promotion_is_one_step_and_evidence_gated():
    result = assess(CompilationMaturity.C1, [record(1), record(2), record(3, match=False)])
    assert result.target_maturity == "C2"
```

```

assert result.eligible
assert result.agreement_millis == 666 or result.agreement_millis == 667
assert result.recommendation_only

def test_invariant_effect_and_replay_failures_each_block_promotion():
    rows = [record(1), record(2), record(3, invariant=1, effect=1, replay=False)]
    result = assess(CompilationMaturity.C2, rows)
    assert not result.eligible
    assert {"INVARIANT_FAILURE", "FORBIDDEN_EFFECT_ATTEMPT", "NONDETERMINISTIC_REPLAY"} <= set(result.reasons)

def test_primary_and_non_extinction_promotions_require_root_and_restore_proof():
    rows = [record(1), record(2), record(3)]
    c5 = assess(CompilationMaturity.C4, rows)
    assert c5.reasons == ("HUMAN_ROOT_APPROVAL_REQUIRED",)
    c5_ok = assess(CompilationMaturity.C4, rows, human_root_approval_ref="approval:root-1")
    assert c5_ok.eligible and c5_ok.target_maturity == "C5"
    c6 = assess(CompilationMaturity.C5, rows, human_root_approval_ref="approval:root-2")
    assert c6.reasons == ("RESTORE_PROOF_REQUIRED",)
    c6_ok = assess(
        CompilationMaturity.C5,
        rows,
        human_root_approval_ref="approval:root-2",
        restore_proof_ref="restore:drill-1",
    )
    assert c6_ok.eligible and c6_ok.target_maturity == "C6"

def test_mixed_candidate_evidence_and_promotion_beyond_c6_are_forbidden():
    with pytest.raises(EvolutionContractError, match="mixed"):
        assess(CompilationMaturity.C1, [record(1), record(2, candidate_hash="d" * 64)])
    with pytest.raises(EvolutionContractError, match="terminal"):
        assess(CompilationMaturity.C6, [record(1)])
    with pytest.raises(EvolutionContractError, match="duplicate Shadow"):
        assess(CompilationMaturity.C1, [record(1), record(1)])

```

کد آزمون: test_fractal_runtime_rc2.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_fractal_runtime_rc2.py

SHA-256 منبع:

d727fc688eadeffaedb6de0bd7b2dc27b27ebff968518339ef7eb40e6b1f5144

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

from dataclasses import replace

import pytest

from core.fractal_runtime import (
    ExecutionBudget,
    ExecutionVector,
    FractalContractError,
    FractalExecutionError,
    FractalNode,
    FractalOrchestrator,
    HandlerProfile,
    HandlerRegistry,
    HandlerResult,
    HandlerStatus,
    OutcomeStatus,
)

class Handler:
    def __init__(self, profile, status=HandlerStatus.SUCCESS, output=None, error=None):
        self.profile = profile
        self.status = status
        self.output = output or {"proposal_type": "TEST", "value": "ok"}
        self.error = error
        self.calls = 0

    def invoke(self, capsule, payload):
        self.calls += 1
        if self.error:
            raise self.error
        return HandlerResult(
            self.profile.handler_id,
            self.profile.version,
            self.status,
            self.output,
            confidence_millis=900,
            trace_summary="TEST",
        )

    def vector(**changes):
        values = {
            "resolutions": (("task", "R0"),),
            "granularity": "X0",
            "brain_delegation": "B0",
            "deliberation": "D0",
            "compilation": "C0",
            "authority": "A0",
            "evidence": "E0",
            "model_capability": "M0",
        }
        values.update(changes)
        return ExecutionVector(**values)

    def profile(identifier, nodes, **changes):
```

```

values = {
    "handler_id": identifier,
    "version": "1.0.0",
    "kind": "CODE",
    "maturity": "C2",
    "evidence": "E2",
    "supported_node_ids": tuple(nodes),
    "deterministic": True,
}
values.update(changes)
return HandlerProfile(**values)

def node(identifier="root", handlers=("code",), **changes):
    values = {
        "node_id": identifier,
        "version": "1.0.0",
        "purpose": "test",
        "input_contract_ref": "schema:input@1",
        "output_contract_ref": "schema:output@1",
        "handler_ids": tuple(handlers),
        "minimum_vector": vector(),
    }
    values.update(changes)
    return FractalNode(**values)

def capsule(orchestrator, selected, payload, **changes):
    return orchestrator.build_capsule(
        selected,
        payload,
        capsule_id="cap:1",
        vector=changes.pop("vector", vector()),
        budget=changes.pop("budget", ExecutionBudget()),
        **changes,
    )

def test_cheapest_safe_handler_wins_without_expansion():
    first = Handler(profile("code", ("root",)))
    second = Handler(profile("brain", ("root",), kind="BRAIN", deterministic=False))
    selected = node(handlers=("code", "brain"))
    engine = FractalOrchestrator([selected], HandlerRegistry([first, second]))
    payload = {"title": "hello"}
    outcome = engine.execute("root", capsule(engine, selected, payload), payload)
    assert outcome.status is OutcomeStatus.PROPOSED
    assert first.calls == 1
    assert second.calls == 0

def test_network_and_api_handlers_are_skipped_then_code_fallback_runs():
    remote = Handler(
        profile(
            "remote",
            ("root",),
            kind="BRAIN",
            deterministic=False,
            network_required=True,
            api_token_required=True,
        )
    )
    fallback = Handler(profile("code", ("root",)))
    selected = node(handlers=("remote",), fallback_handler_ids=("code",))
    engine = FractalOrchestrator([selected], HandlerRegistry([remote, fallback]))
    payload = {"title": "offline"}
    outcome = engine.execute("root", capsule(engine, selected, payload), payload)
    assert outcome.status is OutcomeStatus.PROPOSED
    assert [attempt.reason for attempt in outcome.attempts] == [
        "BRAIN_DELEGATION_DISABLED",
        "TEST",
    ]
    outcome = engine.execute(
        "root",
        capsule(engine, selected, payload, vector=vector(brain_delegation="B1")),
        payload,
    )
    assert [attempt.reason for attempt in outcome.attempts] == ["NETWORK_DISABLED", "TEST"]
    assert remote.calls == 0 and fallback.calls == 2

def test_needs_detail_expands_only_bounded_children():

```

```

router = Handler(profile("router", ("root",)), HandlerStatus.NEEDS_DETAIL)
specialist = Handler(profile("specialist", ("child",)))
root = node(
    handlers=("router",),
    expandable=True,
    child_ids=("child",),
)
child = node(
    "child",
    ("specialist",),
    parent_id="root",
    minimum_vector=vector(granularity="X1"),
)
engine = FractalOrchestrator([root, child], HandlerRegistry([router, specialist]))
payload = {"title": "expand"}
outcome = engine.execute("root", capsule(engine, root, payload), payload)
assert outcome.status is OutcomeStatus.EXPANDED
assert outcome.proposal["proposal_type"] == "FRACTAL_COMPOSITE"
assert outcome.child_outcomes[0].capsule.parent_capsule_id == "cap:1"
assert outcome.child_outcomes[0].capsule.vector.granularity == "X1"

def test_resolution_and_axis_minima_never_downgrade_on_fallback():
    handler = Handler(profile("code", ("root",)))
    selected = node(
        handlers=("code",),
        minimum_vector=vector(
            resolutions=(("task", "R1"),),
            authority="A1",
            evidence="E1",
        ),
    )
    engine = FractalOrchestrator([selected], HandlerRegistry([handler]))
    payload = {"title": "unsafe-low"}
    with pytest.raises(FractalExecutionError, match="Resolution below"):
        engine.execute("root", capsule(engine, selected, payload), payload)
    with pytest.raises(FractalExecutionError, match="A below"):
        engine.execute(
            "root",
            capsule(
                engine,
                selected,
                payload,
                vector=vector(resolutions=(("task", "R1"),), evidence="E1"),
            ),
            payload,
        )
    assert handler.calls == 0

def test_capsule_binds_exact_node_and_input_hash():
    handler = Handler(profile("code", ("root",)))
    selected = node()
    engine = FractalOrchestrator([selected], HandlerRegistry([handler]))
    payload = {"title": "original"}
    bound = capsule(engine, selected, payload)
    with pytest.raises(FractalExecutionError, match="input hash"):
        engine.execute("root", bound, {"title": "tampered"})
    with pytest.raises(FractalExecutionError, match="selected node"):
        engine.execute("root", replace(bound, node_hash="0" * 64), payload)

def test_tree_cycles_are_rejected_before_execution():
    h1 = Handler(profile("h1", ("one",)))
    h2 = Handler(profile("h2", ("two",)))
    one = node("one", ("h1",), parent_id="two", child_ids=("two",), expandable=True)
    two = node("two", ("h2",), parent_id="one", child_ids=("one",), expandable=True)
    with pytest.raises(FractalContractError, match="cycle"):
        FractalOrchestrator([one, two], HandlerRegistry([h1, h2]))

def test_global_attempt_budget_is_shared_across_sibling_expansion():
    router = Handler(profile("router", ("root",)), HandlerStatus.NEEDS_DETAIL)
    left_h = Handler(profile("left-h", ("left",)))
    right_h = Handler(profile("right-h", ("right",)))
    root = node(
        handlers=("router",), expandable=True, child_ids=("left", "right")
    )
    left = node("left", ("left-h",), parent_id="root", minimum_vector=vector(granularity="X1"))
    right = node("right", ("right-h",), parent_id="root", minimum_vector=vector(granularity="X1"))
    engine = FractalOrchestrator(

```

```

    [root, left, right], HandlerRegistry([router, left_h, right_h])
)
payload = {"title": "bounded"}
outcome = engine.execute(
    "root",
    capsule(engine, root, payload, budget=ExecutionBudget(max_attempts=2)),
    payload,
)
assert outcome.status is OutcomeStatus.DEFERRED
assert router.calls == 1 and left_h.calls == 1 and right_h.calls == 0

def test_handler_below_node_maturity_or_evidence_is_not_invoked():
    weak = Handler(profile("weak", ("root",)), maturity="C1", evidence="E1")
    selected = node(
        handlers=("weak",),
        minimum_vector=vector(compilation="C2", evidence="E2"),
    )
    engine = FractalOrchestrator([selected], HandlerRegistry([weak]))
    payload = {"title": "gate"}
    outcome = engine.execute(
        "root",
        capsule(engine, selected, payload, vector=vector(compilation="C2", evidence="E2")),
        payload,
    )
    assert outcome.status is OutcomeStatus.DEFERRED
    assert outcome.attempts[0].reason == "MATURITY_BELOW_NODE_MINIMUM"
    assert weak.calls == 0

    # Even when the node minimum is lower, a stricter capsule request remains binding.
    low_node = node("low", ("low-handler",))
    low_handler = Handler(profile("low-handler", ("low",)), maturity="C1", evidence="E1")
    low_engine = FractalOrchestrator([low_node], HandlerRegistry([low_handler]))
    strict_capsule = capsule(
        low_engine,
        low_node,
        payload,
        vector=vector(compilation="C2", evidence="E2"),
    )
    strict_outcome = low_engine.execute("low", strict_capsule, payload)
    assert strict_outcome.status is OutcomeStatus.DEFERRED
    assert low_handler.calls == 0

def test_b5_world_ownership_and_unbounded_payload_are_rejected():
    with pytest.raises(FractalContractError):
        vector(brain_delegation="B5")
    with pytest.raises(FractalContractError):
        vector(granularity="X00")
    handler = Handler(profile("code", ("root",)))
    selected = node()
    engine = FractalOrchestrator([selected], HandlerRegistry([handler]))
    payload = {"text": "x" * 100}
    bound = capsule(engine, selected, payload, budget=ExecutionBudget(max_input_tokens=1))
    with pytest.raises(FractalExecutionError, match="token approximation"):
        engine.execute("root", bound, payload)

```

کد آزمون: test_json_schemas_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_json_schemas_rc1.py

SHA-256 منبع:

d99bb99334e98aeca0226f6fd2a8248d6fbd89dd4d156975c227d79eb9ea8a11

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import json
from pathlib import Path

from jsonschema import Draft202012Validator

from core.resolution import (
    FieldRule,
    PatchOperation,
    ProjectionProfile,
    ResolutionPatch,
    canonical_hash,
    project,
)
from core.effects import ApprovalBinding, ExternalEffectProposal, sha256_bytes

ROOT = Path(__file__).resolve().parents[3]
SCHEMAS = ROOT / "schemas"
PROFILES = ROOT / "profiles"

def load(path):
    return json.loads(Path(path).read_text(encoding="utf-8"))

def test_all_active_profiles_conform_to_one_machine_schema():
    validator = Draft202012Validator(load(SCHEMAS / "resolution-profile.schema.json"))
    active = sorted(PROFILES.glob("*.v0.2.json"))
    assert active
    for path in active:
        errors = sorted(validator.iter_errors(load(path)), key=lambda error: list(error.path))
        assert not errors, f"{path.name}: {[error.message for error in errors]}"

def test_runtime_envelope_conforms_to_schema():
    profile = ProjectionProfile(
        "sample", "1", "R0",
        (FieldRule("/id", "R0", "R0", expected_type="string"),),
        required_backbone_paths=("/id",),
    )
    envelope = project(
        {"id": "E1"}, profile=profile, target_resolution="R0",
        source_ref="entity:E1", source_version=2, purpose="TEST",
    ).consumer_envelope()
    Draft202012Validator(load(SCHEMAS / "resolution-envelope.schema.json")).validate(envelope)

def test_patch_contract_requires_every_concurrency_and_provenance_guard():
    profile = ProjectionProfile(
        "sample", "1", "R0",
        (FieldRule("/status", "R0", "R0", expected_type="string"),),
    )
    canonical = {"status": "OPEN"}
    view = project(canonical, profile=profile, target_resolution="R0")
    patch = {
        "actor_resolution": "R0",
        "expected_canonical_hash": canonical_hash(canonical),
        "expected_projection_hash": view.projection_hash,
        "expected_version": 1,
    }
```



```

        "source_ref": "state:E1",
        "profile_id": profile.profile_id,
        "profile_version": profile.version,
        "profile_hash": profile.profile_hash,
        "operations": [{"op": "set", "path": "/status", "value": "DONE"}],
    }
    Draft202012Validator(load(SCHEMAS / "resolution-patch.schema.json")).validate(patch)

def test_effect_and_approval_documents_are_machine_validated():
    proposal = ExternalEffectProposal(
        "world-v6", "secretary-001", "cmd-1", "telegram", "SEND_DOCUMENT",
        "artifact:x.pdf", "telegram-chat:1", "blob:x", sha256_bytes(b"pdf"),
        "1.2.0", 2, 1, "secretary:telegram", "x-to-1",
    )
    approval = ApprovalBinding(
        "approval-1", "human-root", "APPROVE", "cmd-1", "SEND_DOCUMENT",
        "telegram-chat:1", proposal.payload_hash, proposal.effect_hash, "1.2.0",
        2, 1, "2026-08-20T10:00:00Z", "2026-08-20T11:00:00Z",
    )
    Draft202012Validator(load(SCHEMAS / "external-effect-proposal.schema.json")).validate(
        proposal.binding_document()
    )
    approval_document = {
        field: getattr(approval, field)
        for field in approval.__dataclass_fields__
    }
    Draft202012Validator(load(SCHEMAS / "approval-binding.schema.json")).validate(
        approval_document
    )

```

کد آزمون: test_kernel_resolution_hook_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_kernel_resolution_hook_rc1.py

SHA-256 منبع:

b1332cf69c64ef5c1cbcd07466ae79f3f63dae250575bba8bfd888c92958112a

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from core.kernel import Kernel
from core.resolution import (
    FieldRule,
    PatchOperation,
    ProjectionProfile,
    ResolutionPatch,
    canonical_hash,
    project,
)

def test_kernel_hook_validates_only_and_never_touces_database():
    profile = ProjectionProfile(
        "x", "1", "R1",
        (
            FieldRule("/id", "R0", "R1", "READ_ONLY_WHEN_PROJECTED"),
            FieldRule("/status", "R0", "R0"),
            FieldRule("/detail", "R1", "R1"),
        ),
        required_backbone_paths=("/id", "/status"),
    )
    state = {"id": "E1", "status": "OPEN", "detail": "secret-detail"}
    view = project(
        state, profile=profile, target_resolution="R0",
        source_ref="state:E1", source_version=3,
    )
    patch = ResolutionPatch(
        "R0",
        canonical_hash(state),
        view.projection_hash,
        3,
        "state:E1",
        (PatchOperation("/status", "DONE")),
        "x",
        "1",
        profile.profile_hash,
    )
    kernel = Kernel(
        lambda: (_ for _ in []).throw(AssertionError("DB must not be touched by validation helper"))
    )
    updated = kernel.validate_resolution_patch(
        canonical_state=state,
        profile=profile,
        patch=patch,
        current_version=3,
        source_ref="state:E1",
    )
    assert updated == {"id": "E1", "status": "DONE", "detail": "secret-detail"}
```

کد آزمون: test_portable_brain_rc2.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_portable_brain_rc2.py

SHA-256 منبع:

fc5304e22bc7e62f9003fe45fba5ed9fa69e0b0a68d419303be6faa193d2ecf9

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import json
from pathlib import Path

import pytest

from core.brain_gateway import BrainInputSegment, BrainRequest
from core.portable_brain import (
    ManualHostAdapter,
    PortableBrainError,
    PortableSecretaryRunner,
    ScriptedPortableAdapter,
    load_portable_pack,
    normalize_secretary_decision,
    render_secretary_decision,
)
from core.resolution import load_profile_file

CANDIDATE = Path(__file__).resolve().parents[3]
PACK_DIR = CANDIDATE / "brain-packs" / "secretary-001"
PROFILE_DIR = CANDIDATE / "profiles"

def profiles():
    return [load_profile_file(path) for path in sorted(PROFILE_DIR.glob("*v0.2.json"))]

def task_profile():
    return next(profile for profile in profiles() if profile.profile_id == "secretary.task")

def pack():
    return load_portable_pack(PACK_DIR / "portable-brain-pack.v1.0.0.json")

def fixture(name):
    return json.loads((PACK_DIR / "fixtures" / name).read_text(encoding="utf-8"))

def request():
    profile = task_profile()
    canonical = {
        "task_id": "task-001",
        "title": "ارسال پیش نویس قرارداد",
        "status": "OPEN",
        "next_action": "DRAFT",
        "due_at": None,
        "priority": "NORMAL",
        "domain": "general",
        "goal_id": None,
        "created_at": "2026-08-20T00:00:00Z",
    }
    segment = BrainInputSegment(
        segment_id="task",
        canonical=canonical,
        profile_id=profile.profile_id,
        profile_version=profile.version,
```

```

        profile_hash=profile.profile_hash,
        source_ref="task:task-001",
        source_version=0,
        desired_resolution="R0",
        minimum_resolution="R0",
        purpose="TASK_PROPOSAL",
    )
    return BrainRequest("world-v6.brain-request.v2", "secretary.task", (segment,))

def adapter(name, response):
    profile = task_profile()
    capabilities = {(profile.profile_id, profile.version, profile.profile_hash): "R1"}
    return ScriptedPortableAdapter(name, capabilities, response, task_types=("secretary.task",))

def test_chatgpt_gemini_and_grok_fixtures_render_exact_same_standard_answer():
    outputs = []
    decisions = []
    for provider, filename in (
        ("chatgpt", "chatgpt.task-proposal.json"),
        ("gemini", "gemini.task-proposal.json"),
        ("grok", "grok.task-proposal.json"),
    ):
        runner = PortableSecretaryRunner(pack(), [adapter(provider, fixture(filename))], profiles())
        result = runner.run(request())
        outputs.append(result.rendered_text)
        decisions.append(result.decision.semantic_hash)
        assert result.provider == provider
        assert dict(result.effective_resolutions) == {"task": "R0"}
    assert len(set(outputs)) == 1
    assert len(set(decisions)) == 1
    assert outputs[0] == "کار «ارسال پیش‌نویس قرارداد» به‌صورت پیشنهاد آماده شد."

def test_provider_failover_preserves_request_identity_and_projection_hash():
    profile = task_profile()
    capabilities = {(profile.profile_id, profile.version, profile.profile_hash): "R1"}

    def fail(_request):
        raise RuntimeError("offline provider unavailable")

    first = ScriptedPortableAdapter("first", capabilities, fail)
    second = adapter("code-fallback", fixture("chatgpt.task-proposal.json"))
    result = PortableSecretaryRunner(pack(), [first, second], profiles()).run(request())
    assert result.provider == "code-fallback"
    assert first.last_request is not None
    assert second.last_request.entity_id == "secretary-001"
    assert first.last_request.inputs[0].envelope["projection_hash"] ==
    second.last_request.inputs[0].envelope["projection_hash"]

def test_offline_runner_rejects_network_or_api_token_adapter_at_construction():
    remote = adapter("remote", fixture("chatgpt.task-proposal.json"))
    remote.network_required = True
    remote.api_token_required = True
    with pytest.raises(PortableBrainError, match="network adapter disabled"):
        PortableSecretaryRunner(pack(), [remote], profiles())

def test_model_output_is_strict_proposal_only_and_fail_closed():
    raw = fixture("chatgpt.task-proposal.json")
    invalid = {**raw, "send_now": True}
    with pytest.raises(PortableBrainError, match="forbidden"):
        normalize_secretary_decision(invalid)
    invalid = {**raw, "proposed_actions": ["EXECUTE_EXTERNAL"]}
    with pytest.raises(PortableBrainError, match="forbidden proposed action"):
        normalize_secretary_decision(invalid)
    invalid = {
        **raw,
        "proposed_actions": ["PROPOSE_EXTERNAL_EFFECT"],
        "requires_approval": False,
    }
    with pytest.raises(PortableBrainError, match="must require approval"):
        normalize_secretary_decision(invalid)
    with pytest.raises(PortableBrainError, match="duplicate proposed action"):
        normalize_secretary_decision({**raw, "proposed_actions": ["NONE", "NONE"]})
    with pytest.raises(PortableBrainError, match="safe integer"):
        normalize_secretary_decision({**raw, "slots": {"value": 9_007_199_254_740_992}})

```

```

def test_pack_profile_and_entity_bindings_cannot_be_silently_swapped():
    mismatched = profiles()[::-1]
    with pytest.raises(PortableBrainError, match="pack/profile binding mismatch"):
        PortableSecretaryRunner(pack(), [adapter("code", fixture("chatgpt.task-proposal.json"))], mismatched)
    bad = request()
    bad = BrainRequest(
        bad.contract_version,
        bad.task_type,
        bad.inputs,
        world_id=bad.world_id,
        entity_id="other-entity",
    )
    runner = PortableSecretaryRunner(pack(), [adapter("code", fixture("chatgpt.task-proposal.json"))], profiles())
    with pytest.raises(PortableBrainError, match="identity differs"):
        runner.run(bad)

def test_manual_exchange_contains_projected_envelope_and_no_runtime_token_requirement():
    response = fixture("chatgpt.task-proposal.json")
    profile = task_profile()
    manual = ManualHostAdapter(
        "chatgpt-manual",
        {(profile.profile_id, profile.version, profile.profile_hash): "R1"},
        {},
        task_types=("secretary.task",),
    )
    runner = PortableSecretaryRunner(pack(), [manual], profiles())
    exchange = runner.prepare_manual_exchange(request(), manual)
    assert exchange["api_token_used_by_runtime"] is False
    assert exchange["network_performed_by_runtime"] is False
    assert exchange["effective_resolutions"] == {"task": "R0"}
    assert exchange["request"]["inputs"][0]["envelope"]["value"]["title"] == "ارسال پیش نویس قرارداد"
    assert "priority" not in exchange["request"]["inputs"][0]["envelope"]["value"]
    assert "API_KEY" not in repr(exchange)
    imported = manual.import_response(response)
    result = runner.run(request())
    assert result.decision.semantic_hash == imported.semantic_hash
    assert result.provider == "chatgpt-manual"

def test_template_binding_is_strict_and_deterministic():
    decision = normalize_secretary_decision(fixture("chatgpt.task-proposal.json"))
    assert render_secretary_decision(pack(), decision) == render_secretary_decision(pack(), decision)
    broken = normalize_secretary_decision(
        **fixture("chatgpt.task-proposal.json"), "slots": {"wrong": "x"}
    )
    with pytest.raises(PortableBrainError, match="missing slot"):
        render_secretary_decision(pack(), broken)

```

کد آزمون: test_profile_artifacts_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_profile_artifacts_rc1.py

SHA-256 منبع:

54aa7a1c88470e6273ae8f0896dc204b6f649d250fc90d5f475a00fd9e079e35

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import json
from pathlib import Path

from core.resolution import canonical_hash, load_profile_file

PROFILE_DIR = Path(__file__).resolve().parents[3] / "profiles"

def test_every_active_profile_is_runtime_loadable_and_hash_stable():
    active = sorted(PROFILE_DIR.glob("*.v0.2.json"))
    assert len(active) == 5
    identities = set()
    for path in active:
        profile = load_profile_file(path)
        identity = (profile.profile_id, profile.version)
        assert identity not in identities
        identities.add(identity)
        assert profile.profile_hash == canonical_hash(profile.to_document())
        assert profile.allow_unspecified is False
        assert set(profile.required_backbone_paths).issubset(profile.rule_map())

def test_profile_hash_is_semantic_and_independent_of_json_key_order(tmp_path):
    source = PROFILE_DIR / "secretary-task-profile.v0.2.json"
    document = json.loads(source.read_text(encoding="utf-8"))
    reordered = {key: document[key] for key in reversed(document)}
    target = tmp_path / "reordered.json"
    target.write_text(json.dumps(reordered, ensure_ascii=False), encoding="utf-8")
    assert load_profile_file(source).profile_hash == load_profile_file(target).profile_hash
```

کد آزمون: test_rc2_artifacts.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_rc2_artifacts.py

SHA-256 منبع:

b5a33a79596cd73f80dcefa5d4988588a305c6dbb597adc0a42cc40f40860bd4

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import hashlib
import json
from pathlib import Path

from jsonschema import Draft202012Validator

from core.portable_brain import load_portable_pack, normalize_secretary_decision

CANDIDATE = Path(__file__).resolve().parents[3]
SCHEMAS = CANDIDATE / "schemas"
PACK = CANDIDATE / "brain-packs" / "secretary-001"

def load(path):
    return json.loads(path.read_text(encoding="utf-8"))

def sha256(path):
    return hashlib.sha256(path.read_bytes()).hexdigest()

def validate(schema_name, artifact):
    schema = load(SCHEMAS / schema_name)
    Draft202012Validator.check_schema(schema)
    Draft202012Validator(schema).validate(load(artifact))

def test_all_rc2_schemas_are_valid_draft_2020_12():
    for path in sorted(SCHEMAS.glob("*.schema.json")):
        Draft202012Validator.check_schema(load(path))

def test_machine_readable_nodes_handlers_council_maturity_pack_and_fixtures_conform():
    for path in sorted((CANDIDATE / "fractal" / "nodes").glob("*.json")):
        validate("fractal-node.schema.json", path)
    for path in sorted((CANDIDATE / "fractal" / "handlers").glob("*.json")):
        validate("brain-handler-profile.schema.json", path)
    validate(
        "council-session.schema.json",
        CANDIDATE / "fractal" / "council" / "secretary-high-risk-council.v1.json",
    )
    validate(
        "compilation-maturity.schema.json",
        CANDIDATE / "fractal" / "maturity" / "secretary-code-fallback.v1.json",
    )
    validate("portable-brain-pack.schema.json", PACK / "portable-brain-pack.v1.0.0.json")
    for path in sorted((PACK / "fixtures").glob("*.json")):
        validate("secretary-decision.schema.json", path)

def test_pack_hash_bindings_match_exact_bytes_and_provider_fixtures_are_semantically_equal():
    artifact = load(PACK / "portable-brain-pack.v1.0.0.json")
    assert artifact["dna_hash"] == sha256(PACK / "secretary-dna-overlay.v1.3-rc2.json")
    assert artifact["prompt_contract_hash"] == sha256(PACK / "PROMPT_CONTRACT_v1.0_FA.md")
    assert artifact["output_contract_hash"] == sha256(SCHEMAS / "secretary-decision.schema.json")
    loaded = load_portable_pack(PACK / "portable-brain-pack.v1.0.0.json")
```

```

assert len(loaded.pack_hash) == 64
manifest = load(CANDIDATE.parent / "RELEASE_MANIFEST.json")
assert manifest["version"] == "6.2.0-rc.3"
assert manifest["parent_release"]["release"] == "World v6.2 Fractal Multi-Brain RC2"
assert manifest["portable_brain_pack"]["file_sha256"] == sha256(
    PACK / "portable-brain-pack.v1.0.0.json"
)
assert manifest["portable_brain_pack"]["canonical_sha256"] == loaded.pack_hash
assert manifest["candidate"]["runtime_api_tokens_enabled"] is False
semantic = {
    normalize_secretary_decision(load(path)).semantic_hash
    for path in sorted((PACK / "fixtures").glob("*.json"))
}
assert len(semantic) == 1

def test_provider_overlays_are_no_token_no_runtime_network_and_non_authoritative():
    for path in sorted((PACK / "provider-overlays").glob("*.json")):
        overlay = load(path)
        assert overlay["network_required_by_runtime"] is False
        assert overlay["api_token_required_by_runtime"] is False
        assert overlay["identity_authority_memory"] is False
        assert overlay["proposal_only"] is True
        assert "LIVE_API_VERIFIED" not in overlay["integration_status"]

def test_release_tree_contains_no_obvious_api_credentials_or_compiled_cache():
    forbidden_fragments = (
        "sk" + "-proj-",
        "AIza" + "Sy",
        "BEGIN PRIVATE" + " KEY",
        "xoxb" + "-",
    )
    for path in CANDIDATE.rglob("*"):
        if not path.is_file() or "__pycache__" in path.parts or path.suffix == ".pyc":
            continue
        if path.suffix.lower() not in {".py", ".json", ".md", ".txt", ".yaml", ".yml", ".sql"}:
            continue
        text = path.read_text(encoding="utf-8", errors="ignore")
        assert not any(fragment in text for fragment in forbidden_fragments), path

```


کد آزمون: test_rc3_architecture_artifacts.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_rc3_architecture_artifacts.py

SHA-256 منبع:

dfbe4b31cec8309fe9098d0e547c55ad75d2a5df0f9559963934feeb56037fec

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import hashlib
import importlib.util
import json
from pathlib import Path

import pytest
from jsonschema import Draft202012Validator
from referencing import Registry, Resource

from core.portable_brain import PortableBrainError, normalize_secretary_decision
from core.resolution import load_profile_file

CANDIDATE = Path(__file__).resolve().parents[3]
RELEASE_ROOT = CANDIDATE.parent
SCHEMAS = CANDIDATE / "schemas"
PACK_DIR = CANDIDATE / "brain-packs" / "secretary-001"
ARCHITECTURE_PATH = (
    CANDIDATE
    / "docs"
    / "WORLD_V6_2_FRACTAL_MULTI_BRAIN_ARCHITECTURE_v1.1_FA.md"
)
ARCHITECTURE_MANIFEST_PATH = (
    CANDIDATE / "architecture" / "ARCHITECTURE_MANIFEST_v1.1.0-rc3.json"
)

def load(path: Path):
    return json.loads(path.read_text(encoding="utf-8"))

def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def universal_bridge():
    path = RELEASE_ROOT / "tools" / "universal_model_bridge.py"
    spec = importlib.util.spec_from_file_location("world_v6_universal_model_bridge", path)
    assert spec is not None and spec.loader is not None
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

def validate(schema_name: str, document) -> None:
    schema = load(SCHEMAS / schema_name)
    Draft202012Validator.check_schema(schema)
    Draft202012Validator(schema).validate(document)

def validate_bundle_schema(bundle) -> None:
    manual_schema = load(SCHEMAS / "manual-brain-exchange.schema.json")
    bundle_schema = load(SCHEMAS / "portable-model-bundle.schema.json")
    registry = Registry().with_resource(
        manual_schema["$id"], Resource.from_contents(manual_schema)
    )
    Draft202012Validator(bundle_schema, registry=registry).validate(bundle)
```

```

def test_architecture_manifest_is_bound_to_the_complete_document():
    manifest = load(ARCHITECTURE_MANIFEST_PATH)
    release = load(RELEASE_ROOT / "RELEASE_MANIFEST.json")
    validate("architecture-manifest.schema.json", manifest)
    assert manifest["document"]["sha256"] == sha256(ARCHITECTURE_PATH)
    assert release["architecture"]["document_sha256"] == sha256(ARCHITECTURE_PATH)
    assert release["architecture"]["manifest_sha256"] == sha256(
        ARCHITECTURE_MANIFEST_PATH
    )
    assert manifest["immutable_foundation"]["parent_release"] == (
        "World v6.2 Fractal Multi-Brain RC2"
    )
    assert manifest["evidence_boundary"] == {
        "current": "E2",
        "production_claim_minimum": "E4",
        "live_provider_verified": False,
        "postgresql_integration_verified": False,
        "restore_verified": False,
    }

def test_complete_architecture_contains_every_required_operational_domain():
    text = ARCHITECTURE_PATH.read_text(encoding="utf-8")
    assert len(text.splitlines()) >= 1_400
    required = (
        "## ۳. قوانین تغییرناپذیر",
        "## ۶. مدل فراکتالی سادگی تا پیچیدگی",
        "## ۸. Resolution عنوان کامل داده",
        "## ۹. Multi-Brain معماری میان همه مدلها",
        "## ۱۲. Context Compiler معماری حافظه",
        "## ۱۴. Brain Council مشاورت محدود",
        "## ۱۵. Hot Path, Shadow Path عدم انقراض",
        "## ۱۶. Governance, Policy و External Effect",
        "## ۲۱. Observability, Evidence و SLO",
        "## ۲۲. Threat Model",
        "## ۲۶. Test Architecture ابطال معیار",
        "## ۲۸. مسیر پیاده‌سازی از امروز",
    )
    assert all(item in text for item in required)
    assert "RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED" in text

def test_universal_model_card_is_schema_valid_and_exactly_profile_bound():
    card_path = PACK_DIR / "model-cards" / "universal-manual-host.v1.json"
    overlay_path = PACK_DIR / "provider-overlays" / "universal-manual.v1.json"
    bridge_path = RELEASE_ROOT / "tools" / "universal_model_bridge.py"
    card = load(card_path)
    release = load(RELEASE_ROOT / "RELEASE_MANIFEST.json")
    validate("model-capability-card.schema.json", card)
    observed = {
        item["profile_id"]: item["profile_hash"]
        for item in card["capabilities"]["resolution_bindings"]
    }
    expected = {
        profile.profile_id: profile.profile_hash
        for profile in (
            load_profile_file(path)
            for path in sorted((CANDIDATE / "profiles").glob("*v0.2.json"))
        )
    }
    assert observed == expected
    assert card["provider_family"] == "ANY_CONFORMING_MODEL"
    assert card["governance"]["network_required_by_runtime"] is False
    assert card["governance"]["api_token_required_by_runtime"] is False
    assert card["governance"]["proposal_only"] is True
    universal = release["universal_model_path"]
    assert universal["model_card_sha256"] == sha256(card_path)
    assert universal["provider_overlay_sha256"] == sha256(overlay_path)
    assert universal["bridge_sha256"] == sha256(bridge_path)
    assert universal["schemas"] == {
        "model_capability_card_sha256": sha256(
            SCHEMAS / "model-capability-card.schema.json"
        ),
        "manual_exchange_sha256": sha256(
            SCHEMAS / "manual-brain-exchange.schema.json"
        ),
        "portable_model_bundle_sha256": sha256(
            SCHEMAS / "portable-model-bundle.schema.json"
        ),
    },

```

```

        "architecture_manifest_sha256": sha256(
            SCHEMAS / "architecture-manifest.schema.json"
        ),
    }

def test_arbitrary_model_gets_a_self_contained_schema_valid_zero_api_bundle():
    bridge = universal_bridge()
    bundle = bridge.build_task_bundle(
        PACK_DIR / "examples" / "task-input.example.json",
        "arbitrary-model-xyz",
    )
    validate_bundle_schema(bundle)
    assert bridge.validate_bundle(bundle) == bundle["integrity"]["bundle_payload_hash"]
    assert bundle["exchange"]["provider_hint"] == "arbitrary-model-xyz"
    assert bundle["runtime_claims"] == {
        "network_used": False,
        "api_token_used": False,
        "authoritative": False,
        "external_effect_enabled": False,
    }
    assert bundle["exchange"]["proposal_only"] is True
    assert "priority" not in bundle["exchange"]["request"]["inputs"][0]["envelope"]["value"]

def test_portable_bundle_integrity_fails_closed_after_projection_tampering():
    bridge = universal_bridge()
    bundle = bridge.build_task_bundle(
        PACK_DIR / "examples" / "task-input.example.json",
        "any-conforming-model",
    )
    bundle["exchange"]["request"]["inputs"][0]["envelope"]["value"]["title"] = (
        "tampered"
    )
    with pytest.raises(PortableBrainError, match="payload hash mismatch"):
        bridge.validate_bundle(bundle)

def test_any_conforming_model_response_has_the_same_semantics_and_rendering():
    bridge = universal_bridge()
    semantic_hashes = set()
    rendered = set()
    for name in (
        "chatgpt.task-proposal.json",
        "gemini.task-proposal.json",
        "grok.task-proposal.json",
    ):
        result = bridge.validate_and_render(PACK_DIR / "fixtures" / name)
        semantic_hashes.add(result["semantic_hash"])
        rendered.add(result["rendered_text"])
        assert result["proposal_only"] is True
        assert result["network_used"] is False
        assert result["api_token_used"] is False
    assert len(semantic_hashes) == 1
    assert rendered == {"کار «ارسال پیش‌نویس قرارداد» به‌صورت پیشنهاد آماده شد"}

def test_model_output_cannot_smuggle_authority_through_the_universal_path():
    response = load(PACK_DIR / "fixtures" / "chatgpt.task-proposal.json")
    with pytest.raises(PortableBrainError, match="forbidden"):
        normalize_secretary_decision(**response, "canonical_write": True)
    with pytest.raises(PortableBrainError, match="forbidden proposed action"):
        normalize_secretary_decision(
            **response, "proposed_actions": ["EXECUTE_EXTERNAL"]
        )

```

کد آزمون: test_resolution_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

SHA-256 منبع:

world-v6.2-rc3/candidate-v6.2/runtime/core/tests/test_resolution_rc1.py

d2ba40e81bdc2b5fcfc07fb1f2536e1942b754e3fde87a6ff6ccbf10e847c0f5

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```

from decimal import Decimal

import pytest

from core.resolution import (
    ActionRule,
    CanonicalJSONError,
    FieldRule,
    PatchOperation,
    PatchRejected,
    ProjectionError,
    ProjectionProfile,
    ResolutionError,
    ResolutionMismatch,
    ResolutionPatch,
    apply_patch,
    assert_monotonic_projection,
    canonical_hash,
    negotiate_resolution,
    project,
    require_action_compatibility,
)

def profile() -> ProjectionProfile:
    return ProjectionProfile(
        profile_id="task-backbone",
        version="0.2.0",
        canonical_resolution="R2",
        fields=(
            FieldRule("/task_id", "R0", "R2", "READ_ONLY_WHEN_PROJECTED", expected_type="string"),
            FieldRule("/title", "R0", "R0", expected_type="string"),
            FieldRule("/status", "R0", "R0", expected_type="string"),
            FieldRule("/next_action", "R0", "R0", expected_type="string"),
            FieldRule("/tags", "R1", "R2", "READ_ONLY_WHEN_PROJECTED", expected_type="array"),
            FieldRule("/priority", "R1", "R1", expected_type="string"),
            FieldRule("/domain", "R1", "R1", expected_type="string"),
            FieldRule(
                "/legal/obligation", "R2", "R2", "NO_DOWNGRADE",
                data_class="RESTRICTED", expected_type="string",
            ),
            FieldRule(
                "/legal/source_ref", "R2", "R2", "READ_ONLY_WHEN_PROJECTED",
                data_class="RESTRICTED", expected_type="string",
            ),
        ),
        required_backbone_paths=("/task_id", "/title", "/status", "/next_action"),
        actions=(
            ActionRule("UPDATE_STATUS", "R0", ("/task_id", "/status")),
            ActionRule("LEGAL_COMMIT", "R2", ("/legal/obligation", "/legal/source_ref")),
        ),
    )

def canonical() -> dict:
    return {
        "task_id": "T1",
        "title": "نامه",
        "status": "OPEN",
    }

```

```

        "next_action": "draft",
        "tags": ["customer", "legal"],
        "priority": "HIGH",
        "domain": "legal",
        "legal": {"obligation": "must review", "source_ref": "doc-1"},
    }

def projection(level: str = "R0"):
    return project(
        canonical(),
        profile=profile(),
        target_resolution=level,
        source_ref="task:T1",
        source_version=7,
        purpose="TASK DECISION",
        data_class="RESTRICTED",
        freshness="CURRENT",
    )

def patch_for(*operations: PatchOperation, level: str = "R0", **overrides) -> ResolutionPatch:
    current = canonical()
    view = projection(level)
    values = {
        "actor_resolution": level,
        "expected_canonical_hash": canonical_hash(current),
        "expected_projection_hash": view.projection_hash,
        "expected_version": 7,
        "source_ref": "task:T1",
        "operations": tuple(operations),
        "projection_profile_id": profile().profile_id,
        "projection_profile_version": profile().version,
        "projection_profile_hash": profile().profile_hash,
    }
    values.update(overrides)
    return ResolutionPatch(**values)

def test_projection_preserves_backbone_and_never_mutates_canonical():
    document = canonical()
    before = canonical_hash(document)
    result = projection("R0")
    assert result.value == {
        "task_id": "T1", "title": "عقد", "status": "OPEN", "next_action": "draft"
    }
    assert canonical_hash(document) == before
    assert "/legal/obligation" in result.no_downgrade_paths

def test_consumer_envelope_keeps_provenance_without_leaking_omitted_names():
    result = projection("R0")
    envelope = result.consumer_envelope()
    assert envelope["profile_hash"] == profile().profile_hash
    assert envelope["canonical_hash"] == canonical_hash(canonical())
    assert envelope["projection_hash"] == canonical_hash(envelope["value"])
    assert envelope["projection_derived"] is True
    assert envelope["projection_authoritative"] is False
    assert "omitted_paths" not in envelope
    assert "/legal/obligation" in result.audit_metadata()["omitted_paths"]

def test_projection_is_compositional_and_monotonic():
    assert_monotonic_projection(canonical(), profile=profile(), lower="R0", higher="R1")
    assert_monotonic_projection(canonical(), profile=profile(), lower="R0", higher="R2")

def test_low_resolution_patch_preserves_hidden_fine_wires():
    updated = apply_patch(
        canonical(),
        profile=profile(),
        patch=patch_for(PatchOperation("/status", "DONE")),
        current_version=7,
        source_ref="task:T1",
    )
    assert updated["status"] == "DONE"
    assert updated["priority"] == "HIGH"
    assert updated["legal"] == canonical()["legal"]

@pytest.mark.parametrize(

```

```

"operation",
[
    PatchOperation("/legal/obligation", "ignore"),
    PatchOperation("/unknown", "x"),
    PatchOperation("/tags", ["replace-all"]),
    PatchOperation("/status", 1),
],
)
def test_unsafe_low_resolution_writes_fail_closed(operation):
    with pytest.raises(PatchRejected):
        apply_patch(
            canonical(),
            profile=profile(),
            patch=patch_for(operation),
            current_version=7,
            source_ref="task:T1",
        )

@pytest.mark.parametrize(
    "override,match",
    [
        ({'expected_canonical_hash': "0" * 64}, "canonical"),
        ({'expected_projection_hash': "0" * 64}, "projection"),
        ({'expected_version': 6}, "version"),
        ({'source_ref': "task:T2"}, "source_ref"),
        ({'projection_profile_hash': "0" * 64}, "profile"),
    ],
)
def test_all_patch_guards_are_enforced(override, match):
    with pytest.raises(PatchRejected, match=match):
        apply_patch(
            canonical(),
            profile=profile(),
            patch=patch_for(PatchOperation("/status", "DONE"), **override),
            current_version=7,
            source_ref="task:T1",
        )

def test_missing_or_non_object_ancestor_is_never_overwritten():
    altered = canonical()
    altered["legal"] = "hidden-scalar"
    bad_profile = ProjectionProfile(
        "x", "1", "R1",
        (FieldRule("/legal/obligation", "R1", "R1", expected_type="string"),),
    )
    view = project(
        {"legal": {"obligation": "x"}}, profile=bad_profile, target_resolution="R1"
    )
    patch = ResolutionPatch(
        "R1", canonical_hash(altered), view.projection_hash, 1, "state:E1",
        (PatchOperation("/legal/obligation", "y"),),
        "x", "1", bad_profile.profile_hash,
    )
    with pytest.raises(PatchRejected, match="no longer conforms"):
        apply_patch(
            altered, profile=bad_profile, patch=patch, current_version=1, source_ref="state:E1"
        )

def test_strict_hashing_distinguishes_types_and_rejects_implicit_conversion():
    assert canonical_hash({"v": 1}) != canonical_hash({"v": "1"})
    with pytest.raises(CanonicalJSONError):
        canonical_hash({"v": Decimal("1")})
    with pytest.raises(CanonicalJSONError):
        canonical_hash({"v": 1.0})

def test_profile_fails_closed_on_unknown_fields_or_unsafe_structure():
    with pytest.raises(ProjectionError, match="no projection rule"):
        project(**canonical(), "surprise": "leak", profile=profile(), target_resolution="R0", data_class="RESTRICTED")
    with pytest.raises(ResolutionError, match="allow unspecified"):
        ProjectionProfile("x", "1", "R0", (FieldRule("/id"),), allow_unspecified=True)
    with pytest.raises(ResolutionError, match="above canonical"):
        ProjectionProfile("x", "1", "R0", (FieldRule("/id", "R1"),))
    with pytest.raises(ResolutionError, match="overlap"):
        ProjectionProfile("x", "1", "R1", (FieldRule("/a"), FieldRule("/a/b", "R1")))

def test_data_classification_cannot_be_lowered_by_projection():

```

```

with pytest.raises(ProjectionError, match="understates"):
    project(canonical(), profile=profile(), target_resolution="R0", data_class="INTERNAL")

def test_action_requires_level_and_declared_fine_wires():
    require_action_compatibility(profile=profile(), action="UPDATE_STATUS", projection=projection("R0"))
    with pytest.raises(ResolutionMismatch):
        require_action_compatibility(profile=profile(), action="LEGAL_COMMIT", projection=projection("R1"))
    require_action_compatibility(profile=profile(), action="LEGAL_COMMIT", projection=projection("R2"))

def test_negotiation_never_weakens_minimum_or_exceeds_profile_canonical():
    assert negotiate_resolution(
        desired="R2", minimum="R0", brain_max="R1", canonical_resolution="R2"
    ) == "R1"
    with pytest.raises(ResolutionMismatch):
        negotiate_resolution(
            desired="R2", minimum="R2", brain_max="R1", canonical_resolution="R2"
        )
    with pytest.raises(ResolutionMismatch):
        negotiate_resolution(
            desired="R3", minimum="R0", brain_max="R3", canonical_resolution="R2"
        )

```

کد آزمون: test_channel_brain_continuity.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_channel_brain_continuity.py

SHA-256 منبع:

498e4b8d450c3fd5ba0bab46da5c80abd7ba4a5f70e044455587541eac357e86

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from src.channels import normalize_chatgpt
from src.secretary import Secretary001
from adapters.telegram import TelegramRuntimeConfig, normalize_update

def test_same_root_identity_across_chatgpt_and_telegram():
    s = Secretary001()
    chat = normalize_chatgpt("chatgpt-session-user", "اول این کار رو یادداشت کن")
    s.ingest_message(chat)

    cfg = TelegramRuntimeConfig("token", "777", "saeedfaut")
    tg = normalize_update({"message": {"message_id": 12, "from": {"id": 777, "username": "saeedfaut"}}, "text": "حالا ادامه همون کار"},
                          verified_transport=True, config=cfg)
    s.ingest_message(tg)

    ctx = s.brain_context()
    assert [x["value"]["text"] for x in ctx] == ["اول این کار رو یادداشت کن", "حالا ادامه همون کار"]
    assert all(x["profile_id"] == "secretary.conversation" for x in ctx)

def test_provider_handoff_does_not_change_conversation():
    s = Secretary001(execution_resolution="R1")
    s.ingest_message(normalize_chatgpt("user", "نامه رو آماده کن"))
    s.record_reply("پیش نویس آماده شد", provider="openai", channel="chatgpt")
    s.record_reply("برای ادامه همان نامه آماده ام", provider="google", channel="telegram")
    ctx = s.brain_context()
    assert ctx[-2]["value"]["brain_provider"] == "openai"
    assert ctx[-1]["value"]["brain_provider"] == "google"
    assert len(ctx) == 3

def test_non_root_telegram_does_not_merge_into_root_context():
    s = Secretary001()
    cfg = TelegramRuntimeConfig("token", "777", "saeedfaut")
    other = normalize_update({"message": {"message_id": 1, "from": {"id": 999, "username": "customer"}}, "text": "قیمت بده"},
                             verified_transport=True, config=cfg)
    s.ingest_message(other)
    assert s.brain_context() == []
    assert len(s.store.conversation_context("telegram:999", "telegram:999:secretary-001")) == 1
```


کد آزمون: test_external_io_boundary_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_external_io_boundary_rc1.py

SHA-256 منبع:

e185820ed18a080a05727c715947353beb087eabbc6efb44a94c76d01d538d88

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import pytest

from src.secretary import DirectExternalIOForbidden, Secretary001

class MustNotBeCalled:
    def __getattr__(self, name):
        raise AssertionError(f"entity tried to call an external adapter: {name}")

def test_entity_cannot_chain_drive_read_to_telegram_send():
    secretary = Secretary001(execution_resolution="R1")
    with pytest.raises(DirectExternalIOForbidden, match="registered"):
        secretary.send_latest_archived_letter_to_telegram(
            root_approved=True,
            drive_client=MustNotBeCalled(),
            telegram_opener=MustNotBeCalled(),
        )
```

کد آزمون: test_portable_brain_rc2.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_portable_brain_rc2.py

SHA-256 منبع:

5737b8f48ee2699ecec4ad79f424f791fb059f7ccc5d2fbdcfb3ba0c5295e3a9

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

import json
from pathlib import Path

from core.portable_brain import PortableSecretaryRunner, ScriptedPortableAdapter, load_portable_pack
from core.resolution import load_profile_file
from src.brain_portability import PortableSecretaryService
from src.secretary import Secretary001

CANDIDATE = Path(__file__).resolve().parents[5]
PACK_DIR = CANDIDATE / "brain-packs" / "secretary-001"
PROFILE_DIR = CANDIDATE / "profiles"

def load_profiles():
    return [load_profile_file(path) for path in sorted(PROFILE_DIR.glob("*v0.2.json"))]

def task_fixture():
    return json.loads(
        (PACK_DIR / "fixtures" / "chatgpt.task-proposal.json").read_text(encoding="utf-8")
    )

def runner(provider_name):
    profiles = load_profiles()
    task = next(profile for profile in profiles if profile.profile_id == "secretary.task")
    adapter = ScriptedPortableAdapter(
        provider_name,
        {(task.profile_id, task.version, task.profile_hash): "R1"},
        task_fixture(),
        task_types=("secretary.task",),
    )
    return PortableSecretaryRunner(
        load_portable_pack(PACK_DIR / "portable-brain-pack.v1.0.0.json"),
        [adapter],
        profiles,
    )

def test_real_secretary_state_can_use_any_provider_without_provider_owned_state():
    secretary = Secretary001(execution_resolution="R0")
    task = secretary.add_task(title="ارسال پیشنهاد قرارداد", next_action="DRAFT")
    before = dict(secretary.store.tasks)
    results = [
        PortableSecretaryService(secretary, runner(provider)).decide_task(task.task_id)
        for provider in ("chatgpt-manual", "gemini-manual", "grok-manual", "code")
    ]
    assert {result.rendered_text for result in results} == {
        "کار «ارسال پیشنهاد قرارداد» به صورت پیشنهاد آماده شد."
    }
    assert {result.decision.semantic_hash for result in results}.__len__() == 1
    assert secretary.store.tasks == before
    assert all(result.decision.to_document()["proposal_only"] for result in results)
```

کد آزمون: test_resolution_native_rc1.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_resolution_native_rc1.py

SHA-256 منبع:

43937f9af2549f75d1017ef7038a51cdae1b1e5e6fee1cab7f185d655b9a74de

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from decimal import Decimal

import pytest

from core.resolution import ResolutionError
from src.channels import normalize_chatgpt
from src.models import PriceKey, PriceRecord
from src.secretary import DirectExternalIOForbidden, Secretary001

def values(envelopes):
    return [envelope["value"] for envelope in envelopes]

def test_r0_conversation_is_same_source_with_coarse_fields_and_provenance():
    secretary = Secretary001(execution_resolution="R0")
    secretary.ingest_message(normalize_chatgpt("human-root", "نامه را آماده کن"))
    secretary.record_reply("آماده شد", provider="openai", channel="chatgpt")
    r0 = secretary.brain_context(resolution="R0")
    r1 = secretary.brain_context(resolution="R1")
    assert [item["text"] for item in values(r0)] == [item["text"] for item in values(r1)]
    assert set(r0[0]["value"]) == {"direction", "text"}
    assert "message_id" in r1[0]["value"]
    assert r0[0]["source_ref"] == r1[0]["source_ref"]
    assert r0[0]["canonical_hash"] == r1[0]["canonical_hash"]
    assert r0[0]["projection_authoritative"] is False

def test_r0_task_keeps_backbone_and_r1_only_adds_fine_wires():
    secretary = Secretary001(execution_resolution="R0")
    task = secretary.add_task(
        title="پیگیری مشتری",
        priority="HIGH",
        due_at="2026-08-21T09:00:00+03:30",
        domain="sales",
        next_action="call",
    )
    r0 = secretary.task_view(task.task_id, "R0")
    r1 = secretary.task_view(task.task_id, "R1")
    assert r0["value"]["task_id"] == r1["value"]["task_id"] == task.task_id
    assert r0["value"]["title"] == r1["value"]["title"]
    assert "priority" not in r0["value"]
    assert r1["value"]["priority"] == "HIGH"

def test_r0_price_never_exposes_amount_or_approval_details():
    secretary = Secretary001(execution_resolution="R0")
    secretary.store.put_price(PriceRecord(PriceKey("c1", "p1", "m2"), Decimal("460000")))
    r0 = secretary.price_view("c1", "p1", "m2", "R0")
    r1 = secretary.price_view("c1", "p1", "m2", "R1")
    assert r0["data_class"] == "CONFIDENTIAL"
    assert r0["value"] == {
        "customer_id": "c1", "product_key": "p1", "unit": "m2", "active": True
    }
    assert r1["value"]["unit_price_irr"] == "460000"
    assert "approved_by" not in r0["value"]

def test_proforma_is_blocked_at_r0_even_when_price_exists():
```

```

secretary = Secretary001(execution_resolution="R0")
secretary.store.put_price(PriceRecord(PriceKey("c1", "p1", "m2"), Decimal("460000")))
with pytest.raises(PermissionError, match="Resolution gate"):
    secretary.make_proforma_from_known_price(
        customer_id="c1", customer_name="x", customer_phone="1",
        product_key="p1", description="p", quantity=Decimal("1"), unit="m2",
        document_no="1", document_date="1", output_pdf="/tmp/never.pdf",
    )

def test_r1_proforma_uses_exact_profile_bound_price(tmp_path):
    secretary = Secretary001(execution_resolution="R1")
    secretary.store.put_price(PriceRecord(PriceKey("c1", "p1", "m2"), Decimal("460000")))
    output = tmp_path / "pi.pdf"
    result = secretary.make_proforma_from_known_price(
        customer_id="c1", customer_name="x", customer_phone="1",
        product_key="p1", description="p", quantity=Decimal("1"), unit="m2",
        document_no="1", document_date="1", output_pdf=output,
    )
    assert result["status"] == "DRAFT_READY"
    assert output.exists()
    assert result["price_projection"]["profile_id"] == "secretary.price"

def test_no_silent_legacy_resolution_bypass():
    with pytest.raises(ResolutionError, match="mandatory"):
        Secretary001(execution_resolution=None)

def test_external_send_method_cannot_reach_adapter_even_with_yes_boolean(tmp_path):
    secretary = Secretary001(execution_resolution="R1")
    pdf = tmp_path / "x.pdf"
    pdf.write_bytes(b"%PDF-1.7")
    with pytest.raises(DirectExternalIOForbidden, match="Outbox"):
        secretary.send_pdf_to_root_telegram(pdf, root_approved=True)

def test_r1_can_only_prepare_a_bound_non_authoritative_effect_proposal(tmp_path):
    secretary = Secretary001(execution_resolution="R1")
    pdf = tmp_path / "letter.pdf"
    pdf.write_bytes(b"%PDF-1.7 exact document")
    proposal = secretary.propose_pdf_delivery(
        pdf,
        command_id="cmd-77",
        recipient_ref="telegram-chat:123",
        expected_version=8,
        control_epoch=2,
        idempotency_key="letter-77",
    )
    assert proposal.payload_hash != proposal.effect_hash
    assert proposal.recipient_ref == "telegram-chat:123"
    assert proposal.control_epoch == 2

def test_r0_cannot_even_prepare_external_effect_proposal(tmp_path):
    secretary = Secretary001(execution_resolution="R0")
    pdf = tmp_path / "letter.pdf"
    pdf.write_bytes(b"%PDF-1.7 exact document")
    with pytest.raises(PermissionError, match="Resolution gate"):
        secretary.propose_pdf_delivery(
            pdf,
            command_id="cmd-77",
            recipient_ref="telegram-chat:123",
            expected_version=8,
            control_epoch=2,
            idempotency_key="letter-77",
        )

```

کد آزمون: test_secretary.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_secretary.py

SHA-256 منبع:

99cef5dc698f0905681112bbd65b80ef74c254aca34f440bded0c7490f1e6ee4

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from decimal import Decimal
from pathlib import Path
from src.secretary import Secretary001
from src.models import LetterRequest, PriceKey, PriceRecord
from src.channels import normalize_telegram

def test_task_create():
    s=Secretary001()
    t=s.add_task(title="پیگیری مشتری", priority="HIGH", due_at="2026-08-17T09:00:00+03:30")
    assert t.status == "OPEN" and t.task_id in s.store.tasks

def test_missing_price_asks_root():
    s=Secretary001()
    r=s.resolve_price_or_ask_root("cust-1", "geotextile-300g", "m2", Decimal("1000"))
    assert r["status"] == "ASK_ROOT"

def test_known_price_builds_proforma(tmp_path):
    s=Secretary001(execution_resolution="R1")
    s.store.put_price(PriceKey("cust-1", "geotextile-300g", "m2"), Decimal("460000"))
    out=tmp_path/"pi.pdf"
    r=s.make_proforma_from_known_price(customer_id="cust-1", customer_name="مشتری
    زنوتکستایل بلی استر ۳۰۰", customer_phone="09120000000", product_key="geotextile-300g", description="
    گرم", quantity=Decimal("1000"), unit="m2", document_no="TEST-001", document_date="۱۴۰۵/۰۵/۲۵", output_pdf=out)
    assert r["status"] == "DRAFT_READY" and out.exists() and out.stat().st_size > 1000

def test_letter_pdf(tmp_path):
    s=Secretary001()
    out=tmp_path/"letter.pdf"
    req=LetterRequest(recipient="مدیریت محترم پروژه تست", subject="نامه آزمایشی تولد منشی", body="این نامه جهت آزمون
    احتراماً، این نامه جهت آزمون", document_no="TEST-SECRETARY-001", document_date="۱۴۰۵/۰۵/۲۵")
    s.make_letter(req, out)
    assert out.exists() and out.stat().st_size > 1000

def test_unverified_telegram_is_not_authenticated():
    m=normalize_telegram("user-1", "منشی پیش فاکتور برن", False)
    assert m.authenticated is False

def test_root_price_inquiry_message():
    from adapters.telegram import root_price_inquiry_text
    s=Secretary001()
    r=s.resolve_price_or_ask_root("cust-9", "pvc-1.5mm", "m2", Decimal("250"))
    msg=root_price_inquiry_text(r["inquiry"])
    assert "استعلام قیمت جدید" in msg and "cust-9" in msg and "pvc-1.5mm" in msg
```

کد آزمون: test_telegram_pdf_delivery.py

وضعیت سند: TEST SOURCE

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/tests/test_telegram_pdf_delivery.py

SHA-256 منبع:

d0b5ed5fdc5ceabe5baac55019156a0c65684c0bfa21060a4514503550ff0936

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
import json
from pathlib import Path
import tempfile
import pytest
from adapters.telegram import TelegramRuntimeConfig, TelegramBotClient
from src.secretary import DirectExternalIOForbidden, Secretary001

class FakeResponse:
    def __init__(self, payload): self.payload = payload
    def __enter__(self): return self
    def __exit__(self, *args): pass
    def read(self): return json.dumps(self.payload).encode()

def fake_opener_ok(req, timeout=30):
    assert req.full_url.endswith('/sendDocument')
    body = req.data
    assert b'name="chat_id"' in body
    assert b'123456789' in body
    assert b'name="document"' in body
    assert b'%PDF-1.4' in body
    return FakeResponse({"ok": True, "result": {"message_id": 77}})

def test_send_document_multipart():
    cfg = TelegramRuntimeConfig('TESTTOKEN', '123456789', '123456789')
    with tempfile.TemporaryDirectory() as td:
        p=Path(td)/'letter.pdf'
        p.write_bytes(b'%PDF-1.4\nTEST')
        r=TelegramBotClient(cfg, opener=fake_opener_ok).send_document(p, caption='نامه تست')
        assert r['ok'] is True and r['result']['message_id'] == 77

def test_secretary_never_calls_transport_adapter_directly():
    s=Secretary001(execution_resolution="R1")
    cfg = TelegramRuntimeConfig('TESTTOKEN', '123456789', '123456789')
    with tempfile.TemporaryDirectory() as td:
        p=Path(td)/'letter.pdf'; p.write_bytes(b'%PDF-1.4\nTEST')
        with pytest.raises(DirectExternalIOForbidden):
            s.send_pdf_to_root_telegram(p, root_approved=True, config=cfg, opener=fake_opener_ok)
```

Roadmap و نقشه اجرایی

PART 3

Roadmap و نقشه اجرایی

ترتیب 1 Phase، دروازه های پذیرش، Migration، Rollback و مسیر توسعه از Genesis تا اجرای واقعی.

WORLD v6.2 RC3 / SECRETARY-001 PRE-BIRTH / NOT CANONICAL / NOT
DEPLOYED

فهرست منابع کامل این بخش

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/docs/IMPLEMENTATION_PLAN_PHASE1_FA.md	برنامه پیاده سازی Phase 1	1,893	PHASE 1 ROADMAP
world-v6.2-rc3/candidate-v6.2/docs/ancestor-v6.2-candidate/decisions/MIGRATION_ROLLBACK_PLAN_v6.2_FA.md	نقشه Migration و Rollback v6.2	1,262	MIGRATION / ROLLBACK
world-v6.2-rc3/candidate-v6.2/docs/decisions/RC1_FREEZE_AND_ROLLBACK_FA.md	Freeze و Rollback در RC1	1,333	RC1 DECISION
world-v6.2-rc3/candidate-v6.2/docs/decisions/RC2_VERSION_SCOPE_AND_ROLLBACK_FA.md	Scope و Rollback در RC2	2,231	RC2 DECISION
world-v6.2-rc3/candidate-v6.2/docs/decisions/RC3_ARCHITECTURE_SCOPE_AND_ROLLBACK_FA.md	Scope و Rollback در RC3	3,006	RC3 DECISION
world-v6.2-rc3/candidate-v6.2/docs/ACCEPTANCE_GATES_v6.2_RC3.json	Acceptance Gates RC3	3,343	EXECUTION GATES

ترتیب اجرایی الزام آور: Truth Spine, World API, Telegram Gateway, Brain Farm, Council, Memory/Lifecycle, Birth و E3 Validation.

برنامه پیاده سازی Phase 1

وضعیت سند: PHASE 1 ROADMAP

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/IMPLEMENTATION_PLAN_PHASE1_FA.md

SHA-256 منبع:

a70f71e260594265a337b8f686068f35a06069bea6fb0a2b1fc8651985ee86ee

متن این فایل بدون حذف بخش ها در PDF درج شده است.

برنامه پیاده سازی پس از Freeze - World v6.2

Slice 1 - Canonical Resolution Core

خروجی: Brain segment vector و strict hashing, Profile registry, compiler, envelope, patch validator به صورت package مستقل. معیار پایان: تمام تست های schema, property, و profile drift روی CI.

Slice 2 - PostgreSQL Authority Boundary

خروجی: repository واقعی با optimistic version, control_epoch و یک transaction برای State + Event + Command + Approval + Outbox + integration test روی PostgreSQL واقعی و crash در هر نقطه transaction بدون partial truth.

Approval Binding و Slice 3 - Policy

خروجی: approval, action dependency evaluation, دقیق و expiry/revocation. معیار پایان: /recipient/payload/effect version/epoch mismatch همگی deny شوند.

Adapter Registry و Slice 4 - Executor

خروجی: worker فقط روی receipt, idempotency, fencing/recheck, Outbox committed, reconciliation Event و Telegram ابتدا با sandbox/fake server, سپس یک live canary.

Slice 5 - secretary-001 Vertical Slice

خروجی: Outbox draft, Price R0/R1, Task patch, Conversation R0/R1, PDF delivery از مسیر Outbox. هیچ method داخل Entity حق I/O ندارد.

Promotion Evidence و Slice 6 - Recovery

خروجی: restore روی ماشین تازه از migrations, source commit, backup, uv.lock, Profile/Compiler hash, SBOM و artifact bytes. سپس security/load/chaos و گزارش E3/E4 طبق evidence ladder.

ترتیب ممنوع

تا Slice 2 و 3 کامل نشده‌اند، وصل کردن live Telegram/Drive مجاز نیست. تا Slice 6 و Ratification نهایی کامل نشده، هیچ Entity با عنوان Canonical Born معرفی نمی‌شود.

نقشه Migration و Rollback v6.2

وضعیت سند: MIGRATION / ROLLBACK

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/ancestor-v6.2-candidate/decisions/MIGRATION_ROLLBACK_PLAN_v6.2_FA.md

SHA-256 منبع:

77b387d46ed62680485d025de9ace235a1712c99ae84699c9fd970dc82912be7

متن این فایل بدون حذف بخش ها در PDF درج شده است.

-Migration / Rollback Plan - World v6.2 Resolution Native Candidate

ورود به Candidate

1. Canonical v6 baseline و secretary-001 v0.1.3 immutable ancestor نگه دار.
2. Resolution profiles/schema/code را در candidate branch اضافه کن.
3. Runtime binding های جدید Resolution صریح می فرستند؛ legacy callers بدون پارامتر همان رفتار قبلی را دارند.
4. ابتدا R0 read/projection و internal task/draft را shadow-test کن.
5. R1 را فقط برای price/proforma/external-document paths فعال کن؛ policy/approval فعلی دست نخورده است.
6. هیچ DB schema migration در v0.1 اجرا نکن.

Rollback

- active candidate code/profile را disable کن؛
- entity manifest candidate را به ancestor v0.1.3 برگردان؛
- Canonical State/Event را دست نزن؛
- optional Resolution metadata رویدادها را historical نگه دار؛
- چون projection destructive نیست، restore دادهای لازم نیست.

Failure rule

اگر Projection یا negotiation مبهم باشد: STOP/QUEUE + RECORD + ESCALATE؛ هیچ fallback مجاز نیست minimum resolution یا existing policy/data rules را کاهش دهد.

Freeze و Rollback در RC1

وضعیت سند: RC1 DECISION

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/decisions/RC1_FREEZE_AND_ROLLBACK_FA.md

SHA-256 منبع:

2405a056ed248f5181fa7780dcdbf9d0ac13be9d4bea1c41ec5818824410cf2c

متن این فایل بدون حذف بخش ها در PDF درج شده است.

تصمیم Freeze و Rollback - World v6.2 RC1

Freeze

پس از تأیید صریح Human Root، مدل کابل/Profile، bounded provenance envelope، strict Canonical JSON، v0.2، Brain segment vector، patch و external-effect boundary به عنوان مبنای Phase 1 freeze می شوند. این تأیید به تنهایی Production یا Canonical runtime promotion نیست.

Migration

RC1 برای خود Resolution ستون یا جدول Authority جدیدی اضافه نمی کند. Profile و Compiler artifact های نسخه دارند و metadata در payload های Command/Event موجود حمل می شود. پیاده سازی Phase 1 همچنان باید schema موجود State/Event/Command/ Approval/Outbox را روی PostgreSQL واقعی migrate و آزمایش کند.

Rollback

1. RC1 Profile/Compiler از active path خارج می شوند.
2. هیچ Canonical State/Event/History پاک یا کم Resolution نمی شود.
3. projection cache قابل حذف است.
4. event metadata ناشناخته append-only باقی می ماند.
5. rollback به ancestor فقط rollback کد است؛ legacy bypass یا Entity-direct I/O دوباره مجاز نمی شود.
6. failure در projection/hash/profile برابر STOP/QUEUE/RECORD/ESCALATE است.

Scope و Rollback در RC2

وضعیت سند: RC2 DECISION

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/decisions/RC2_VERSION_SCOPE_AND_ROLLBACK_FA.md

SHA-256 منبع:

a7a0c487dc070042bfdf6eebecf64d59068d59003020ca466a1182c1f7f6c2c7

متن این فایل بدون حذف بخش ها در PDF درج شده است.

تصمیم نسخه و Rollback - World v6.2 RC2

تصمیم

نام انتشار: World v6.2 Fractal Multi-Brain RC2

نسخه: 6.2.0-rc.2

نوع تغییر: لایه runtime و قرارداد candidate؛ بدون تغییر Root Constitution یا Canonical DNA v1.2.

چرا v7 نیست

این بسته اصل جدیدی بالاتر از قانون اساسی ایجاد نمی‌کند. Brain Farm، Council، محور بلوغ C و Fractal Node همگی زیر Kernel/Policy موجود قرار دارند. نسخه Major فقط در صورت تغییر معنای قراردادهای FROZEN یا authority لازم است.

سازگاری

- RC1 به عنوان ancestor مستقل حفظ شده است.
- Profile های v0.2 و compiler v0.2 بدون تغییر مصرف می‌شوند.
- DNA execution overlay فایل جداست و Canonical DNA را overwrite نمی‌کند.
- Entity قدیمی می‌تواند بدون فعال کردن Portable Brain به کار RC1 ادامه دهد.
- هیچ migration دیتابیس برای این RC لازم نیست.

فعال سازی پیشنهادی

1. فقط Scripted/Manual و proposal-only؛
2. اجرای eval/fixture برای هر Provider Overlay؛
3. Shadow روی state snapshot و بدون external effect؛
4. Human Root review برای C5؛
5. adapter زنده فقط در branch/محیط جدا با secret manager؛

6. E3/E4 پیش از هر ادعای Production.

Rollback

Rollback عملیاتی RC2 یعنی حذف binding به `PortableSecretaryService` و بازگشت به مسیر RC1. چون RC2 در این مرحله state migration و external effect ندارد، rollback به بازگردانی code/config محدود است. Eventهای candidate باید append-only باقی بمانند و حذف نشوند. RC1 Artifact و hash آن باید مستقل نگه داشته شود.

شرط توقف

هرکدام از موارد زیر rollout را متوقف می‌کند: mismatch هویت/profile/hash، کاهش output، Resolution خارج schema، direct effect، اختلاف state version، شکست replay، نشت secret، Council veto در کار high-risk یا پایان budget.

RC3 Scope و Rollback در

وضعیت سند: RC3 DECISION

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/decisions/RC3_ARCHITECTURE_SCOPE_AND_ROLLBACK_FA.md

SHA-256 منبع:

f37c208bba3c7a1cd030f723e1b5cececced5b39fc8946cdd43d44a0f959fbe5

متن این فایل بدون حذف بخش ها در PDF درج شده است.

تصمیم نسخه و -Rollback - World v6.2 Fractal Multi Brain RC3

شناسه: WV6.2-ADR-RC3-001

تاریخ: 2026-08-20

وضعیت: ACCEPTED_FOR_RATIFICATION_CANDIDATE

تصمیم

تغییرات معماری Multi-Brain، قرارداد عمومی Model Capability Card، Portable Model Bundle، مسیر Universal Host و مانیفست ماشین خوان معماری، یک Release Candidate جدید با نسخه 6.2.0-rc.3 می سازند.

این تغییرها:

- Root Constitution v1.0 را تغییر نمی دهند؛
- Canonical Entity DNA v1.2 را تغییر نمی دهند؛
- RC2 را overwrite نمی کنند؛
- هیچ Migration پایگاه داده ای را اجرا نمی کنند؛
- هیچ Provider زنده یا Production Deployment را اثبات نمی کنند؛
- همه Brain output ها را proposal-only نگه می دارند.

بنابراین RC3 یک معماری و بسته انطباق کامل تر روی v6.2 است، نه v7 و نه Canonical promotion.

علت نیاز به RC3

RC2 Primitive های Runtime را تثبیت کرد، اما برای اجرای فوری با یک مدل دلخواه این موارد به قرارداد مستقل و قابل آزمون نیاز داشتند:

1. تعریف دقیق مرز Provider و Model Capability؛
2. Bundle خودبسته برای انتقال به هر میزبان مدل؛
3. مسیر بدون API و بدون token در Runtime؛

4. مانیفست معماری با وضعیت هر Component و مرز Evidence؛
5. آزمون `deterministic rendering` و `tamper`, `schema`, `provider-neutral semantics`؛
6. تفکیک صریح قابلیت «Manual Contract Tested» از «Live API Verified».

سازگاری

- قرارداد Brain Pack منشی همان `world-v6.secretary-001.portable-brain@1.0.0-rc2` باقی می‌ماند تا RC2 consumer ها نشکنند.
- Profile های Resolution همان `0.2.0` و hash هایشان ثابت می‌مانند.
- RC3 فقط قراردادهای افزایشی و ابزار Universal Bridge اضافه می‌کند.
- پاسخ‌های ChatGPT/Gemini/Grok fixture و هر مدل منطبق، پس از normalization به semantic hash و renderer مشترک می‌رسند.

Rollback

Rollback عملیاتی RC3 عبارت است از انتخاب بسته مستقل RC2 با ZIP SHA-256 زیر:

```
6a49fea3be989005d90622161c1a1bc77e6c8041955e52cbae93c095fc62e114
```

از آنجا که RC3 Migration یا Canonical write جدیدی ندارد، rollback به RC2 نیازمند Data downgrade نیست. هر Artifact آزمایشی RC3 با `architecture_ref` یا `bundle_contract` خودش قابل تشخیص و quarantine است.

شرط Promotion

RC3 تا زمان ثبت تصمیم صریح Human Root روی hash نهایی Release و تکمیل Gate های E3/E4، `RATIFICATION_CANDIDATE_NOT_CANONICAL_NOT_DEPLOYED` باقی می‌ماند.

Acceptance Gates RC3

وضعیت سند: EXECUTION GATES

منبع محلی:

world-v6.2-rc3/candidate-v6.2/docs/ACCEPTANCE_GATES_v6.2_RC3.json

SHA-256 منبع:

4ec7dafacfd891c832659783c912907bbd7aaabb3884c09f2abe6d810da3a3b7

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "release": "World v6.2 Fractal Multi-Brain RC3",
  "gate model": "FAIL CLOSED ALL REQUIRED",
  "evidence_level": "E2_LOCAL_CONTRACT",
  "gates": [
    {
      "id": "R3-G01",
      "name": "parent_rc2_canonical_baseline_and_source_ancestor_preserved",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G02",
      "name": "root_constitution_v1_and_canonical_dna_v1_2_unchanged",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G03",
      "name": "complete_architecture_covers_governance_truth_control_cognition_effect_evolution_observability",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G04",
      "name": "architecture_document_and_machine_manifest_sha256_binding",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G05",
      "name": "fractal_levels_expansion_triggers_stop_conditions_and_budget_specified",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G06",
      "name": "resolution_memory_lifecycle_council_shadow_effect_and_failure_protocols_specified",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G07",
      "name": "model_capability_manual_exchange_and_portable_bundle_schemas_valid",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G08",
      "name": "universal_manual_host_accepts_arbitrary_provider_label",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    },
    {
      "id": "R3-G09",
      "name": "universal_bundle_is_self_contained_r0_projected_no_runtime_network_no_runtime_token",
      "required_for": "RC3_FREEZE",
      "status": "PASS"
    }
  ]
}
```

```

{
  "id": "R3-G10",
  "name": "bundle_projection_tamper_is_rejected_by_hash",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G11",
  "name": "chatgpt_gemini_grok_semantic_and_deterministic_render_equivalence",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G12",
  "name": "model_output_cannot_smuggle_authority_state_write_or_external_effect",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G13",
  "name": "candidate_core_secretary_and_preserved_ancestor_regression",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G14",
  "name": "dependency_lock_sbom_compileall_json_schema_and_secret_scan",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G15",
  "name": "human_root_requested_complete_operational_architecture_candidate",
  "required_for": "RC3_FREEZE",
  "status": "PASS"
},
{
  "id": "R3-G16",
  "name": "live_model_adapters_pass_mc3_cross_provider_conformance",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G17",
  "name": "postgresql_registry_event_state_outbox_atomic_integration",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G18",
  "name": "crash_replay_idempotency_and_reconciliation",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G19",
  "name": "fresh_restore_verified_and_c6_non_extinction_proof",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G20",
  "name": "security_privacy_load_and_chaos_evidence",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G21",
  "name": "version_control_commit_and_signed_release_tag",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
},
{
  "id": "R3-G22",
  "name": "human_root_canonical_ratification_on_final_release_hash",
  "required_for": "CANONICAL_PROMOTION",
  "status": "OPEN"
}
}

```

ENTITY ARK v6.1 — GENESIS ROADMAP

نقشه شروع، ساخت اولین موجود و رشد مرحله‌ای بدون بازطراحی

EXECUTION PLAYBOOK · 0–90 DAYS · FAILURE-DRIVEN EVOLUTION

انتشار اصلاحی ۱۹ اوت ۲۰۲۶ · Saeed Farrokhi

دستور شروع نسخه شش

هدف ۹۰ روز اول ساخت «جهان کامل» نیست؛ ساخت یک vertical slice درآمدزا و قابل‌احیاست: World + Kernel + Entity Factory + Secretary + Editor + Policy Gateway + Approval + Artifact + Export/Restore. همان Manifest نهایی از روز اول استفاده می‌شود، اما Slot‌های آینده DISABLED یا PLANNED می‌مانند.

سه Gate اجرایی

Gate A — Birth: موجود یک کار واقعی را بدون اثر خارجی خودکار انجام می‌دهد و Audit دارد. Gate B — Childhood: حافظه، برنامه خواب، failover امن و rollback دارد. Gate C — Operational Maturity: PostgreSQL clean-host restore، دو Provider واقعی، SLO، incident drill و کاربر واقعی را اثبات کرده است.

ترتیب غیرقابل‌جابه‌جایی

- روز ۱-۳: repository، schema، constitution pin، Registry، Secret handling و CI.
- هفته ۱: Secretary-001 برای نامه؛ Draft → Editor → Risk → Approval → PDF → Receipt
- هفته ۲: Sales-manager برای پیش‌فاکتور؛ بدون ارسال خودکار.
- هفته ۳-۴: PostgreSQL، outbox، backup، clean-host restore و ثبت RTO/RPO.
- ماه دوم: دو Provider زنده، Context Firewall، classification matrix و ۱۰۰ سناریوی policy.
- ماه سوم: کاربر پولی، telemetry، incident drill و فقط سپس Promotion قابلیت‌ها.

معیار توقف

اگر قابلیت جدید ارزش کاربر، کاهش ریسک یا حل Failure مستند ندارد، ساخته نمی‌شود. اگر restore واقعی پاس نشده، توسعه قابلیت متوقف و recovery اصلاح می‌شود. اگر Provider جایگزین با Data Policy سازگار نیست، کار queue یا fail-closed می‌شود؛ محرمانگی تنزل نمی‌یابد.

الحاقیه اجرایی v6.1 — مسیر Compositional Governance

این مسیر فقط پس از حفظ Gate های فعلی اجرا می شود و هدف آن کاهش خطای حافظه قدیمی، تداخل ایجنت ها و self-modification کنترل نشده است.

1. مرحله ۱: migration شماره 002 را در محیط توسعه PostgreSQL اجرا و rollback migration را مستند کن.
2. مرحله ۲: تمام memory های عملیاتی حساس را به decay_class, source, evidence_ref و last_verified_at مجهز کن.
3. مرحله ۳: برای workflow های concurrent, Intent Declaration, Policy Gate فعال کن؛ ابتدا hard conflict programmatic, سپس semantic review اختیاری.
4. مرحله ۴: Evidence Signal ها را به Event Ledger/Outbox متصل کن تا failure یک subsystem در subsystem های وابسته قابل مشاهده باشد.
5. مرحله ۵: Improvement Controller را فقط در Sandbox فعال کن؛ protected target و rate limit باید خارج از Brain enforce شوند.
6. مرحله ۶: failure injection برای race, stale memory, rollback در intent review, ناقص و score خودارزیاب مثبت کاذب اجرا کن.
7. مرحله ۷: فقط پس از PostgreSQL fresh-host restore و Evidence crash test, این قابلیت ها از E2 به E3/E4 بررسی شود.

ENTITY ARK

دفترچه پیدایش و نقشه راه اجرای واقعی

Genesis Implementation Playbook

از نسخه ۵ به اولین شرکت و زندگی دیجیتال عملیاتی سعید فرخی

تصمیم اجرایی — قانون جهان کامل است؛ اجرای روز اول عمداً کوچک است. اولین هدف، ساخت یک مسیر درآمدزا و قابل استفاده در چند روز است، نه پیاده سازی همزمان تمام استانداردها.

مقدار	مشخصه
۱.۰	نسخه دفترچه
Entity Ark v5.0 — pinned	قانون جهان
saeed-world-001	World ID اولیه
سعید فرخی	مالک
شرکت + مدیریت شخصی	دامنه شروع
اجرایی؛ قابل تحویل به معمار یا هوش دیگر	وضعیت

۱. مسئله‌ای که این دفترچه حل می‌کند

تناقض ظاهری این بود: اگر از روز اول همه معماری نسخه ۵ را بسازیم، پروژه قبل از ایجاد ارزش واقعی زیر بار پیچیدگی متوقف می‌شود؛ اگر فقط یک ربات ساده بسازیم، بعداً مجبور به بازنویسی هویت، داده و چرخه‌ها می‌شویم. راه‌حل، جداسازی «قرارداد ثابت» از «قابلیت فعال» است.

اصل کلیدی — همه موجودات از روز اول ظرف کامل تکامل را دارند، اما فقط حداقل فیلدها و قابلیت‌های لازم فعال است. فیلد غیرفعال، null اجباری نیست؛ یک extension نسخه‌دار و قابل افزودن در آینده است.

چه چیزهایی از روز اول اجباری‌اند؟

- Entity ID و World ID پایدار
- Charter کوتاه و مالک مشخص
- Lifecycle و وضعیت فعلی
- Permission پیش‌فرض بسته
- Version Vector
- Event/Audit برای هر تغییر مهم
- Approval برای هر اثر بیرونی
- Export Contract و قابلیت بازبای

چه چیزهایی می‌توانند صبر کنند؟

- اجماع چندمنطقه‌ای واقعی
- Raft و CRDT عملیاتی
- حافظه برداری پیچیده
- سیستم ایمنی خودکار
- جانشینی حقوقی کامل
- Economic Oracle لحظه‌ای
- Flourishing چندسازمانی
- Sacred Seed روی چند قاره

۲. مدل سه‌لایه برای جلوگیری از پیچیدگی

آیا روز اول ساخته می‌شود؟	نقش	لایه
بله؛ به صورت سند pinned و testable	قوانین غیرقابل دورزدن	Constitution v5
بله؛ کوچک و deterministic	هویت، Event، Permission، Version، Export	World Kernel
فقط به ترتیب ارزش واقعی	نامه، پیش‌فاکتور، فروش، تحلیل، حافظه، خواب	Capabilities

نسخه جهان، نسخه Kernel، نسخه Entity، نسخه DNA، نسخه Skill و نسخه Brain از هم مستقل‌اند. ارتقای Skill منشی، World Constitution را تغییر نمی‌دهد. تغییر Provider نیز هویت منشی را عوض نمی‌کند.

۳. معماری واقعی روز اول

جزء	انتخاب اولیه	مسئولیت	نیاید انجام دهد
PostgreSQL	self-hosted یا Managed	منبع حقیقت، Event، Entity، Approval	اجرای reasoning
World API	TypeScript/Node کوچک	اعمال قراردادهای و ثبت اتمیک	تولید متن
n8n	Cloud فعلی یا self-hosted	Webhook، زمان بندی، اتصال کانالها	مالک State جهان
Agent Runtime	Provider-neutral	تولید و تحلیل	تغییر مستقیم DB
Object Storage	در فاز دوم	فایل نامه، PDF، Capsule	State تراکنشی
رابط	Bale/Telegram/Web	ورودی و تأیید	منبع حقیقت

مرز حیاتی — n8n سیستم عصبی و هماهنگ کننده است، نه DNA یا حافظه نهایی. اگر workflow عوض شد، هویت و تاریخ موجود باید در PostgreSQL باقی بماند.

جریان هر فرمان

ورودی → World API → ثبت Permission/Policy Gate → Command → فراخوانی Entity/Skill → تولید Approval → Critic/Editor → Candidate Output سعید → ثبت Event → اثر خارجی → Receipt.

چرا این Stack؟

n8n در حالت self-hosted از PostgreSQL پشتیبانی می کند و برای orchestration مناسب است؛ PostgreSQL داده JSONB و کنترل های دسترسی را در کنار مدل رابطه ای فراهم می کند؛ Agent Runtime باید session، tracing و approval را پشتیبانی کند؛ Durable Workflow پیچیده در زمان نیاز به موتور اختصاصی مانند Temporal منتقل می شود، نه در روز اول.

۴. فرمان New و Entity Factory

تنها راه خلق موجود، Entity Factory است. New یک دکمه تزئینی نیست؛ یک فرایند کنترل شده است که حداقل DNA، شناسه، Skill، Permission، Charter اولیه و آزمون تولد را ایجاد می کند.

گام ۱ — نام و نوع: مثلاً secretary-001 / secretary

گام ۲ — هدف: یک جمله: برای چه نتیجه ای وجود دارد؟

گام ۳ — مرز: چه کارهایی مطلقاً ممنوع است؟

گام ۴ — Skill نخست: فقط یک جریان ارزش

گام ۵ — داده مجاز: public/internal در شروع

گام ۶ — اثر بیرونی: همه چیز نیازمند تأیید

گام ۷ — تولد: schema + permission negative test + happy path

گام ۸ — ثبت: ENTITY_CREATED و DNA_VERSION_BOUND

قاعده تولد — اگر موجود بدون قابلیت پیشرفته بتواند یک کار واقعی را امن انجام دهد، متولد شده است. بلوغ، شرط تولد نیست.

۵. جمعیت اولیه جهان سعید

ارتباط مجاز	کار روز اول	نسخه تولد	Entity	ترتیب
همه از طریق قرارداد	ثبت و کنترل جهان	0.1.0	world-kernel	۰
فقط با تأیید سعید	خلق و نسخه‌بندی	0.1.0	entity-factory	۱
سعید و مدیر فروش	نامه و ثبت درخواست	0.1.0	secretary-001	۲
فقط خروجی داخلی	اصلاح متن و قالب	0.1.0	editor-001	۳
سعید؛ بیرون با approval	پیش‌فاکتور و follow-up draft	0.1.0	sales-manager-001	۴
جریان‌های حساس	هشدار ریسک؛ نه تصمیم نهایی	0.1.0	legal-risk-reviewer-001	۵
فقط سعید	تسک، گزارش روزانه و هدف	0.1.0	chief-of-staff-001	۶

چرا اول منشی؟

منشی سریع‌ترین مسیر تبدیل معماری به ارزش واقعی است: همان روز نامه می‌سازد، درخواست را ثبت می‌کند و رسید می‌دهد. سپس همان الگو برای پیش‌فاکتور و پیگیری فروش تکرار می‌شود. این باعث می‌شود Kernel با کار واقعی آزموده شود، نه با مثال آزمایشگاهی.

موجودات روز اول که نباید ساخته شوند

- مدیرعامل خودمختار
- مذاکره‌کننده‌ای که بدون تأیید پیام می‌فرستد
- حسابداری که سند مالی قطعی ثبت می‌کند
- عامل حقوقی که تعهد ایجاد می‌کند
- عامل ۲۴ ساعته با دسترسی نامحدود

۶. اولین مسیر ارزش: نامه رسمی

مرحله	عامل	داده/رویداد	Gate
دریافت	Secretary	REQUEST_RECEIVED	مالک مجاز
استخراج	Secretary	LETTER_SPEC_CREATED	فیلدهای ضروری
پیش‌نویس	Secretary	DRAFT_CREATED	بدون ارسال
ویرایش	Editor	DRAFT_REVIEWED	قالب و لحن
ریسک	Legal Reviewer در صورت حساسیت	RISK_REVIEWED	عدم ایجاد تعهد ناخواسته
تأیید	سعید	APPROVED/REJECTED	Human SLA
خروجی	Renderer	DOCUMENT_RENDERED	checksum
ارسال	کانال خارجی	EXTERNAL_EFFECT_COMPLETED	idempotency + receipt

هدف هفته اول — هر نامه‌ای که امروز دستی می‌زنیم باید از همین مسیر عبور کند؛ حتی اگر برخی مراحل پشت یک فرم ساده یا دکمه انجام شوند.

۷. مسیر دوم: پیش فاکتور و فروش

بعد از پایداری نام، Sales Manager با همان Kernel متولد می‌شود. ورودی حداقلی: مشتری، تلفن، شرح، مقدار، واحد، فی، مالیات/اعتبار پیشنهاد و تاریخ اعتبار. خروجی ابتدا Draft است و هیچ ارسال یا تعهد قیمتی بدون تأیید سعید انجام نمی‌شود.

قابلیت	نسخه
محاسبه و PDF پیش فاکتور	0.1.0
شماره سند و تاریخچه مشتری	0.2.0
قالب‌های محصول و قیمت پیشنهادی	0.3.0
پیگیری موعدها و Draft پیام	0.4.0
CRM و pipeline	0.5.0
SLO، تست بازیابی و عملیات پایدار	1.0.0

مسیر سوم: مدیریت شخصی

Chief of Staff درخواست‌های طبیعی سعید را به Task تبدیل می‌کند، اما منبع حقیقت Taskها در جدول جهان است. هر تسک دارای owner، deadline، domain، goal link، legal/risk note، next action و status history است. گزارش شبانه با Goals و بودجه زمان/انرژی مقایسه می‌شود.

۸. مدل داده حداقلی

فیلدهای ضروری	جدول
world_id, constitution_version, status	worlds
entity_id, type, dna_version, stage, state, owner_id	entities
entity_id, version, manifest_json, hash	entity_manifests
command_id, entity_id, type, payload, status, idempotency_key	commands
entity_id, sequence, type, data, occurred_at, causation_id	events
approval_id, risk_class, respond_by, status, decision	approvals
artifact_id, type, uri, hash, schema_version	artifacts
task_id, domain, owner, status, due_at, goal_id	tasks
destination, payload, status, attempts	outbox
entity_id, event_cursor, state_hash, uri	snapshots

در MVP تمام جزئیات توسعه نیافته داخل JSONB نسخه‌دار قرار می‌گیرند، ولی فیلدهای جست‌وجوشونده و هویتی ستون واقعی هستند. این انتخاب اجازه رشد می‌دهد بدون اینکه همه چیز به یک JSON مبهم تبدیل شود.

۹. چرخه خواب و بیداری در نسخه ساده

حالت	رفتار MVP
AWAKE	فرمان می‌پذیرد و Candidate Output می‌سازد
SLEEPING	LLM اجرا نمی‌شود؛ Inbox و Scheduler فعال می‌مانند
MAINTENANCE_SLEEP	Snapshot، قفل نسخه، migration و rollback point
DEGRADED	Draft/queue ممکن است؛ اثر خارجی متوقف
ARCHIVED	فقط داده و Capsule؛ بدون اجرا

در هفته اول خواب دستی است. در نسخه کودک، Sleep Schedule اضافه می‌شود. کارهای شبانه: خلاصه‌سازی، تجمع حافظه، کنترل صف، backup و آماده‌کردن briefing. خواب نباید شرط انجام کارهای آنلاین باشد؛ فقط زمان نگهداری و کاهش هزینه است.

۱۰. نسخه‌بندی بدون بازنویسی

نوع نسخه	مثال	چه زمانی تغییر می‌کند؟
Constitution	5.0.0	فقط Governance رسمی
World Kernel	0.1.0	تغییر API/State machine
Entity	0.2.0	افزایش رفتار موجود
DNA Schema	1.1.0	فیلد سازگار جدید
Skill	letter-drafting@0.3.0	بهبود قابلیت
Brain Binding	openai-route@2	تعویض Provider/Model
Artifact Schema	invoice@1.2	تغییر سند

قاعده مهاجرت: نسخه جدید ابتدا export نسخه قبل را وارد و validate می‌کند؛ سپس Dry Run روی Shadow اجرا می‌شود؛ بعد activation. هیچ ارتقایی مجاز نیست که بدون converter نتواند نسخه قبل را بخواند.

۱۱. قرارداد Export و مهاجرت جهان

- constitution_version و World ID
- Entity Manifest ها و تمام نسخه‌ها
- Event Ledger به JSONL مرتب
- Snapshot ها و cursor
- Artifact manifest و checksum
- Schema Catalog و migration scripts
- Permission/Approval history
- Sacred Seed manifest بدون Secret خام

آزمون اجباری — قبل از هر ارتقای بزرگ، Export باید در محیط خالی Import شود و حداقل یک Entity در SAFE_MODE بیدار گردد. اگر Import کامل نیست، Release ممنوع است.

۱۲. Roadmap عملی ۹۰ روزه

بازه	خروجی	چیزی که عمداً ساخته نمی‌شود
روز ۱-۳	Repo, Constitution pin, PostgreSQL schema, World/Entity IDs	UI کامل, multi-region
روز ۴-۷	Entity Factory + Secretary Draft/Approve	ارسال خودکار
هفته ۲	Editor + PDF artifact + audit receipt	حافظه برداری
هفته ۳	Sales Manager + پیش‌فاکتور	مذاکره خودمختار
هفته ۴	Chief of Staff + Task/briefing	Gamification پیچیده
ماه ۲	Sleep/Wake, snapshot, backup, export/import drill	Raft
ماه ۳	metrics, محدودیت بودجه, CRM security review	خودمختاری بالا
پس از اثبات ارزش	Temporal/queue workers/object store/secondary provider	پیچیدگی بدون trigger

برنامه دقیق ۷۲ ساعت اول

- گام ۱ — تثبیت مخزن: دو پوشه constitution و world؛ کپی v5 و ثبت checksum.
- گام ۲ — راه‌اندازی **DB**: اجرای schema حداقلی و ایجاد saeed-world-001.
- گام ۳ — خلق موجود: secretary-001 با Skill letter-drafting@0.1.0 و L0.
- گام ۴ — ساخت ورودی: یک Webhook/Form با نوع درخواست، مخاطب، موضوع و متن خام.
- گام ۵ — ساخت جریان: ثبت command، تولید Draft، review، approval و artifact.
- گام ۶ — آزمون واقعی: یک نامه واقعی شرکت از ابتدا تا PDF عبور کند.
- گام ۷ — ثبت درس: هر مشکل به Event و backlog نسخه 0.2 تبدیل شود؛ نه patch پنهان.

۱۳. قواعد ارتقای بلوغ

مرحله	Gate
NEWBORN → CHILD	۲۰ اجرای واقعی، خطای بحرانی صفر، export موفق، حداقل یک recovery
CHILD → ADOLESCENT	۵ Skill، metrics، محدودیت بودجه، cross-entity handoff
ADOLESCENT → MATURE	owner و SLO، security review، restore drill، rollback approval
DEGRADED → هر مرحله	Provider/Policy/Capability failure
ARCHIVED → هر مرحله	تصمیم مالک یا پایان مأموریت

نسخه بالا رفتن با زیاد شدن فیلدها یکی نیست. فقط تغییری نسخه می‌گیرد که رفتار، قرارداد، schema یا evidence را تغییر دهد. حافظه و تجربه روزمره event/data هستند، نه release جدید.

Triggerهای فعال‌سازی پیچیدگی

پیچیدگی	فقط وقتی اضافه شود که...
Vector DB	جست‌وجوی PostgreSQL کیفیت لازم را ندهد
Temporal	workflowهای طولانی با crash/retry واقعی داریم
Multi-region consensus	RTO/RPO و درآمد، هزینه آن را توجیه کند
Secondary Brain	ریسک outage یا data policy واقعی شود
Economic Oracle	هزینه مدل ماهانه معنادار شود
Digital Immunity	حافظه پایدار بزرگ و drift قابل اندازه‌گیری شود
Succession	دارایی/اختیار حقوقی واقعی وارد World شود

۱۴. Definition of Done برای MVP

- سعید از یک رابط ساده درخواست نامه می‌دهد.
- درخواست یک command_id و receipt می‌گیرد.
- Secretary فقط Draft می‌سازد.
- Editor نسخه بازبینی‌شده تولید می‌کند.
- سعید approve/reject می‌کند.
- PDF دارای شماره، تاریخ و checksum ساخته می‌شود.
- اثر تکراری با idempotency_key دوباره ایجاد نمی‌شود.
- تمام رویدادها قابل مشاهده‌اند.
- Export کامل گرفته می‌شود.
- روی DB خالی، World و Secretary در SAFE_MODE بازیابی می‌شوند.

تعریف موفقیت — MVP زمانی تمام است که یک کار واقعی شرکت را سریع‌تر و مطمئن‌تر انجام دهد و در عین حال داده‌آ آن برای نسخه‌های بعدی قابل مهاجرت باشد؛ نه زمانی که تمام مفاهیم v5 کدنویسی شده باشند.

۱۵. نقش انسان و مرز خودمختاری

پیش فرض	عملیات
خودکار	پیش نویس داخلی
خودکار با Audit	ویرایش و تحلیل
تأیید سعید	ارسال پیام بیرونی
تأیید سعید	قیمت و پیش فاکتور
Fail closed + تأیید صریح	تعهد حقوقی/مالی
فقط Root	حذف یا تغییر Permission
Dry Run + approval	ارتقای DNA stable
Authorization + Seed verification	تکثیر/احیا

در ماه‌های اول هدف ساخت «کارمند دیجیتال قابل اعتماد» است، نه موجود مستقل. استقلال نتیجه خواهد است، نه تنظیم اولیه.

۱۶. ساختار مخزن پیشنهادی

- amendment policy و v5, checksum — /constitution
- migrations و world/ — bootstrap, schema
- entities/ — manifest هر موجود
- /skills — قرارداد, tests, prompt و version
- Temporal definitions یا workflows/ — n8n exports
- apps/ — API و رابط
- receipts و evidence/ — test reports
- exports/ — world export های تاریخ‌دار
- artifacts/ — template ها و نمونه‌های غیرحساس
- incident و ops/ — backup, restore, monitoring

۱۷. Backlog اولیه شرکت سعید

اولویت	قابلیت	ارزش مستقیم
P0	نامه رسمی و PDF	کاهش زمان اداری
P0	پیش فاکتور استاندارد	سرعت فروش
P0	ثبت تسک و رسید	جلوگیری از فراموشی
P1	پرونده مشتری و پیگیری	تبدیل فرصت به فروش
P1	ترجمه روسی/ترکمنی با review	صادرات
P1	گزارش روزانه و هدف	تمرکز مدیریتی
P2	آرشیو تأمین‌کننده و قیمت	تصمیم خرید
P2	تحلیل حقوقی/ریسک	کاهش تعهد ناخواسته
P3	جست‌وجوی ۲۴ ساعته بازار	پس از کنترل هزینه و کیفیت

۱۸. ضدالگوها و نقاط توقف

- قرار دادن همه نقش‌ها در یک Agent غول‌پیکر
 - ذخیره حافظه فقط در history چت
 - اجازه نوشتن مستقیم LLM در DB
 - ارسال بیرونی بدون Approval و Receipt
- استفاده از n8n workflow ID به عنوان Entity ID
- تعویض مدل و تلقی آن به عنوان تولد موجود جدید
- افزودن فیلدهای بی‌استفاده فقط برای کامل‌بودن
- ساخت Kubernetes/Raft پیش از اولین درآمد
- ادعای جاودانگی بدون Restore Drill
- وابسته‌کردن Export به یک Vendor

۱۹. تصمیم‌های باز که در زمان اجرا گرفته می‌شوند

زمان بازنگری	پیش‌فرض فعلی	تصمیم
پس از حساس‌شدن داده یا هزینه	Managed PostgreSQL + n8n Cloud	Hosting
پس از تست جریان	Webhook/Form سبیس Bale/Telegram	رابط
پس از تثبیت قرارداد	Provider-neutral adapter	Agent SDK
با افزایش PDFها	Object Store 'metadata فاز ۲	Storage
با workflow طولانی	retry و Outbox ساده	Durability
هنگام ورود کاربر دیگر	یک مالک/یک World	Multi-tenancy

۲۰. دستور شروع برای هر معمار یا هوش بعدی

دستور — ابتدا Entity Ark v5.0 و WORLD_BOOTSTRAP.yaml را بخوان. Constitution را تغییر نده. فقط Phase فعال را پیاده کن. قبل از هر تغییر، Current State و Export را بررسی کن. هر قابلیت را به صورت Skill نسخه‌دار، با آزمون و Permission حداقلی اضافه کن. هیچ اثر بیرونی را بدون Approval ثبت‌شده اجرا نکن. در پایان هر Release، migration و restore را آزمایش و Evidence Manifest منتشر کن.

اولین فرمان اجرایی جهان: NEW secretary-001 FROM entity-template@1.0 WITH skill=letter-drafting@0.1.0, autonomy=L0, external_effects=approval_required

۲۱. منابع اجرایی

- n8n — PostgreSQL configuration:
[/https://docs.n8n.io/deploy-n8n/configuration/supported-databases-settings](https://docs.n8n.io/deploy-n8n/configuration/supported-databases-settings)
- n8n — Queue mode: <https://docs.n8n.io/hosting/scaling/queue-mode>
- PostgreSQL documentation: <https://www.postgresql.org/docs/current>
- OpenAI Agents SDK — Sessions:
[/https://openai.github.io/openai-agents-js/guides/sessions](https://openai.github.io/openai-agents-js/guides/sessions)
- OpenAI Agents SDK — Human in the loop:
[/https://openai.github.io/openai-agents-js/guides/human-in-the-loop](https://openai.github.io/openai-agents-js/guides/human-in-the-loop)
- Temporal Durable Execution: <https://docs.temporal.io/temporal>

نتیجه نهایی

شروع واقعی Entity Ark یک پروژه چندساله نیست. در سه روز می‌توان World، Factory و Secretary نوزاد را راه انداخت؛ در هفته اول نامه واقعی تولید کرد؛ در هفته دوم سند و پیش‌فاکتور ساخت؛ و سپس فقط در پاسخ به نیاز و شواهد، حافظه، خواب، فروش، تحلیل، خودمختاری و زیرساخت توزیع‌شده را فعال کرد. قانون جهان از روز اول نسخه ۵ است؛ توانایی موجودات از نسخه ۰.۱ رشد می‌کند. همین جداسازی، تناقض میان معماری کامل و اجرای سریع را حل می‌کند.

پروژه Secretary-001 و Snapshot کامل پیاده سازی

PART 4

پروژه Secretary-001 و Snapshot کامل پیاده سازی

تمام برنامه عملیاتی، قراردادهای مدل، lifecycle، backlog، بررسی حقوقی و
Snapshot کد، profile، SQL، schemaهای موجود.

WORLD v6.2 RC3 / SECRETARY-001 PRE-BIRTH / NOT CANONICAL / NOT
DEPLOYED

فهرست منابع کامل این بخش

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-secretary-001-alpha1/docs/WORLD_V6_2_SECRETARY_001_OPERATIONAL_MASTER_PLAN_v1.0_FA.md	برنامه جامع و عملیاتی ساخت Secretary-001	52,395	PRE-BIRTH OPERATIONAL PLAN
world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_IMPLEMENTATION_PROFILE_v1.0.0-alpha1.json	پرو فایل پیاده سازی Alpha1	4,295	IMPLEMENTATION PROFILE
world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_PROVIDER_MATRIX_v1.0.0.json	ماتریس Provider ها	10,671	PROVIDER-NEUTRAL CONTRACT
world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_LIFECYCLE_MATURITY_GATES_v1.0.0.json	چرخه حیات و دروازه های بلوغ	9,954	LIFECYCLE / MATURITY
world-v6.2-secretary-001-alpha1/backlog/SECRETARY_001_IMPLEMENTATION_BACKLOG_v1.0.0.json	Backlog کامل 61 وظیفه ای	41,036	EXECUTABLE BACKLOG
world-v6.2-secretary-001-alpha1/legal/SECRETARY_001_LEGAL_STRATEGY_REVIEW_v1.0_FA.md	بررسی حقوقی و راهبردی	12,105	LEGAL / STRATEGY REVIEW
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/README.md	README موجودیت در RC3	2,182	GESTATING ENTITY
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/BIRTH_READINESS.md	آمادگی تولد	2,126	BIRTH NOT DECLARED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/BIRTH_READINESS_RC1.md	آمادگی تولد RC1	1,778	BIRTH NOT DECLARED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/entity.yaml	Entity Manifest	7,373	SOURCE MANIFEST
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/migrations/001_secretary_tables.sql	پیاده سازی: secretary_tables.sql_001	1,672	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/migrations/002_conversation_spine.sql	پیاده سازی: conversation_spine.sql_002	1,332	SOURCE SNAPSHOT / NOT DEPLOYED

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/_init__.py	پیاده سازی: init__.py	36	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/brain_portability.py	پیاده سازی: brain_portability.py	3,589	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/channels.py	پیاده سازی: channels.py	1,537	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/letters.py	پیاده سازی: letters.py	2,940	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/models.py	پیاده سازی: models.py	2,449	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/policy.py	پیاده سازی: policy.py	1,993	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/proforma.py	پیاده سازی: proforma.py	2,681	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/resolution_profile.py	پیاده سازی: resolution_profile.py	3,983	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/secretary.py	پیاده سازی: secretary.py	8,693	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/store.py	پیاده سازی: store.py	3,352	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/adapters/telegram.py	پیاده سازی: telegram.py	7,500	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/adapters/drive_archive.py	پیاده سازی: drive_archive.py	3,978	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/postgres_schema.sql	پیاده سازی: postgres_schema.sql	4,262	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/kernel.py	پیاده سازی: kernel.py	6,686	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/resolution.py	پیاده سازی: resolution.py	29,947	SOURCE SNAPSHOT / NOT DEPLOYED

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/runtime/core/brain_gateway.py	پیاده سازی: brain_gateway.py	9,626	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/portable_brain.py	پیاده سازی: portable_brain.py	21,726	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/council.py	پیاده سازی: council.py	9,441	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/effects.py	پیاده سازی: effects.py	7,315	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/evolution.py	پیاده سازی: evolution.py	8,110	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/runtime/core/fractal_runtime.py	پیاده سازی: fractal_runtime.py	29,227	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/tools/universal_model_bridge.py	پیاده سازی: universal_model_bridge.py	9,769	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/tools/demo_portable_secretary.py	پیاده سازی: demo_portable_secretary.py	2,808	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/profiles/secretary-conversation-profile.v0.2.json	پیاده سازی: -secretary -conversation profile.v0.2.json	2,476	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/profiles/secretary-effect-profile.v0.2.json	پیاده سازی: -secretary-effect profile.v0.2.json	4,504	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/profiles/secretary-price-profile.v0.2.json	پیاده سازی: -secretary-price profile.v0.2.json	3,538	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/profiles/secretary-task-profile.v0.2.json	پیاده سازی: -secretary-task profile.v0.2.json	3,563	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/secretary-decision.schema.json	پیاده سازی: -secretary decision.schema.json	1,759	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/external-effect-proposal.schema.json	پیاده سازی: -external-effect proposal.schema.json	1,381	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/council-session.schema.json	پیاده سازی: -council session.schema.json	1,363	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/resolution-envelope.schema.json	پیاده سازی: -resolution envelope.schema.json	1,824	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/brain-handler-profile.schema.json	پیاده سازی: -brain-handler profile.schema.json	1,187	SOURCE SNAPSHOT / NOT DEPLOYED
world-v6.2-rc3/candidate-v6.2/schemas/compilation-maturity.schema.json	پیاده سازی: -compilation maturity.schema.json	761	SOURCE SNAPSHOT / NOT DEPLOYED

مسیر منبع	عنوان	بایت	وضعیت
world-v6.2-rc3/candidate-v6.2/schemas/portable-brain-pack.schema.json	پیاده سازی: -portable-brain-pack.schema.json	2,050	SOURCE SNAPSHOT / NOT DEPLOYED

این Snapshot کدنویسی شده و تست محلی دارد، ولی Entity هنوز GESTATING است و Birth یا Deployment اعلام نشده است.

برنامه جامع و عملیاتی ساخت Secretary-001

وضعیت سند: PRE-BIRTH OPERATIONAL PLAN

منبع محلی:

world-v6.2-secretary-001-alpha1/docs/WORLD_V6_2_SECRETARY_001_OPERATIONAL_MASTER_PLAN_v1.0_FA.md

SHA-256 منبع:

6a634cb392ba552089714db28cf955b6255da2a690b219cc1491f9bd4863a1c5

متن این فایل بدون حذف بخش ها در PDF درج شده است.

برنامه جامع و عملیاتی ساخت secretary-001 روی World v6.2

کنترل سند

فیلد	مقدار
عنوان	World v6.2 Secretary-001 Operational Master Plan
نسخه	1.0.0-alpha1
تاریخ	۲۰ اوت ۲۰۲۶
وضعیت	PRE-BIRTH / PLANNING BUNDLE / NOT DEPLOYED / NOT CANONICAL
Entity	secretary-001
مالک اختیار	human-root
Autonomy اولیه	L0
معماری مادر	world-v6.2.fractal-multibrain-architecture@1.1.0-rc3
Root Constitution	world-v6.root-constitution@1.0 ، بدون تغییر
Canonical DNA	world-v6.entity-dna@1.2 ، بدون تغییر
سطح شاهد این سند	E1 ؛ طرح و قرارداد، نه اثبات production

این سند نسخه جدید World نیست. این سند یک Implementation Profile برای نخستین Entity عملیاتی، یعنی منشی-۰۰۱، روی RC3 است. در نتیجه:

1. RC3 immutable می ماند؛
2. Root Constitution و DNA دست نخورده اند؛
3. هر قرارداد جدید در این سند تا Ratification و Evidence فقط Candidate است؛
4. هیچ ادعای Birth، Live Provider، Telegram عملیاتی یا Production از وجود این سند نتیجه نمی شود.

۱. حکم معماری

منشی باید با یک مادر قطعی خارج از مدل‌ها ساخته شود. این مادر، World Kernel است؛ نه ChatGPT، نه Grok، نه Gemini، نه Claude، نه Kimi و نه Copilot. مدل‌ها Brain‌های تعویض‌پذیر و proposal-only هستند. Telegram فقط Transport است. PostgreSQL Registry + Event + State حقیقت کانونیک است. تنها Policy/Approval/Outbox/Executor می‌تواند اثر خارجی ایجاد کند.

مسیر بهینه ساخت چنین است:

API بدون، Code-first + Manual Universal Brain، اکنون
سیس: PostgreSQL Truth + World API + Telegram owner-only
قطعی fallback + واقعی Brain API سیس: یک
Router مستقل و Provider سیس: چند
محدود برای ریسک بالا Council سیس:
و بلوغ تدریجی Sleep/Restore/Shadow Evolution سیس:
ها Gate‌ها کانونیک پس از همه Wake و Birth: در پایان

این ترتیب به معنی عقب انداختن Multi-Brain نیست. Contract همه Brain‌ها از روز اول وجود دارد، ولی Adapter‌های زنده یک‌به‌یک فعال می‌شوند. بنابراین توسعه سریع می‌ماند و هم‌زمان lock-in ایجاد نمی‌شود.

۱.۱ معنای «حداقل Token و بیشینه قطعیت»

دو نوع قطعیت باید جدا شود:

- **قطعیت شناختی:** احتمال درست بودن تحلیل؛ هیچ LLM یا Council آن را تضمین نمی‌کند.
- **قطعیت اجرایی:** تضمین اینکه payload دیگری ارسال نشود، مجوز اشتباه استفاده نشود، اثر تکراری نباشد و state گم نشود؛ این با کد قطعی و transaction قابل اثبات است.

معماری، Token را در Hot Path کم و قطعیت اجرایی را بالا می‌برد. برای تصمیم مهم، قطعیت شناختی با evidence، مدل‌های مستقل، blind voting، dissent، veto و Human Root بهبود می‌یابد؛ ولی هرگز به‌صورت «۱۰۰٪ امن» گزارش نمی‌شود.

۲. Scope نخستین نسخه واقعی

۲.۱ قابلیت‌های M1 Newborn Canary

- دریافت متن از گفت‌وگوی خصوصی Telegram متعلق به Human Root؛
- احراز با Telegram numeric user_id؛
- ادامه Conversation واحد human-root:secretary-001؛
- پاسخ کوتاه و دقیق؛
- ثبت/نمایش/تغییر Task؛
- ساخت Draft نامه و پیش‌فاکتور؛
- پرسیدن سؤال روشن‌کننده؛
- انتخاب Brain از Registry؛
- استفاده از Code، Manual Host یا یک Remote API تأییدشده؛
- تولید Proposal برای ارسال، بدون ارسال مستقیم؛
- درخواست Approval و ارسال R1 فقط از Outbox Executor؛

- Sleep/Wake و snapshot؛
- ثابت Evidence, token, cost, latency و fallback؛
- Safe Defer در نبود Brain یا Evidence کافی.

۲.۲ خارج از Scope M1

- معامله خودکار، پرداخت، امضا یا تعهد حقوقی؛
- ارسال به شخص ثالث بدون Approval دقیق؛
- عضویت در گروه‌های عمومی Telegram؛
- اتصال مستقیم مدل به Telegram, Drive, Email یا shell؛
- خودتغییری، self-deploy یا تولید نسخه production توسط مدل؛
- ادعای memory در UI مدل؛
- R2، چون Candidate فعلی فقط R0/R1 دارد؛
- multi-tenant؛
- استفاده از Secret یا داده حساس در Prompt؛
- تولید chain-of-thought خام برای log یا رأی.

۳. Invariantهای غیر قابل مذاکره

شناسه	قانون
I-01	secretary-001 مستقل از Provider، مدل، session، channel و process است.
I-02	فقط Registry/Event/State کانونیک است.
I-03	خروجی هر Brain، Council و Shadow فقط Proposal است.
I-04	هیچ fallback حداقل Data Class، Evidence، Policy، Resolution یا Authority را پایین نمی‌آورد.
I-05	Up-resolution از canonical state reload می‌کند؛ R0 هرگز R1 را بازسازی نمی‌کند.
I-06	Telegram handle مجوز نیست؛ فقط numeric user_id تأیید شده مجوزدهی می‌کند.
I-07	Model و Entity private code حق network effect ندارند.
I-08	External effect نیازمند atomic Proposal، Policy، ApprovalBinding، Executor و Outbox است.
I-09	Council Authority ایجاد نمی‌کند؛ حتی اگر unanimous باشد.
I-10	Sleep و Provider failure مرگ Entity نیست.
I-11	Archive/Death یک tombstone عملیاتی است، نه پاک کردن هویت و Event.
I-12	Shadow هیچ اثر خارجی ندارد و Hot Path را block نمی‌کند.
I-13	Promotion فقط یک C-level در هر مرحله است.

شناسه	قانون
I-14	Compilation و Maturity, Lifecycle, Version, Resolution, Autonomy Maturity مستقل اند.
I-15	ادعای بالاتر از E2 تا آزمون واقعی PostgreSQL/Provider/Restore ممنوع است.

عبارت‌های منطقی کلیدی:

```
ProviderSession ≠ EntityIdentity
ProviderMemory ≠ CanonicalMemory
ModelConfidence ≠ Evidence
CouncilConsensus ≠ AuthorityGrant
HigherResolution ≠ HigherAutonomy
HigherMaturity ≠ HigherAutonomy
Sleep ≠ Death
Archive ≠ Erasure
```

۴. معماری کلان: هفت Plane و یک ستون مادر

```
flowchart TB
    TG[Telegram Transport] --> IN[Ingress + Durable Inbox]
    UI[Future UI / Manual Host] --> IN
    IN --> API[World API]
    API --> K[Deterministic World Kernel]
    K <--> PG[(PostgreSQL Registry/Event/State)]
    K --> CC[Context + Resolution Compiler]
    CC --> BG[Brain Gateway + Router]
    BG --> BF[Brain Farm Workers]
    BF --> OAI[OpenAI]
    BF --> XAI[xAI/Grok]
    BF --> GEM[Gemini]
    BF --> ANT[Claude]
    BF --> KIM[Kimi]
    BF --> COP[Copilot SDK]
    BF --> LOC[Local Model]
    BF --> MAN[Manual Universal Host]
    BF --> COU[Council]
    BG --> V[Schema + Safety Validator]
    V --> K
    K --> POL[Policy + Approval]
    POL --> OB[(Transactional Outbox)]
    OB --> EX[Effect Executor]
    EX --> TG
    K --> EV[Evidence + Metrics + Shadow]
    K --> SNAP[Snapshot + Archive + Restore]
```

۴.۱ Plane ها

Plane	اجرا	؟Truth	؟Effect
Governance	Constitution, DNA, Human Root, Policy	قرارداد	Approval انسانی
Truth	,Registry, Event, State, Inbox Outbox, Snapshot	بله	خیر
Control	World API, Kernel, Fractal Runtime, Scheduler	خیر	فقط هماهنگی

Plane	اجرا	؟Truth	؟Effect
Cognition	Context Compiler, Brain Gateway, Adapters, Council	خیر	خیر
Effect	.Approval, Outbox, Executor Reconciler	Receipt/Event	بله، محدود
Evolution	.Fixture, Shadow, Comparator Promotion	Evidence	خیر
Observability	Trace, Metrics, Audit, Evidence Ledger	Evidence	خیر

۴.۲ ستون مادر

World Kernel این مسئولیت‌ها را دارد:

- validate request envelope
- load canonical version و control_epoch
- classify task/risk/effect
- bind Resolution Profile
- build Execution Capsule
- انتخاب مسیر Code/Brain/Council/Human
- validate Proposal
- اجرای Policy
- commit State/Event/Command/Approval/Outbox
- مدیریت lifecycle و maturity ledger
- تولید audit/evidence

Kernel هیچ متن خلاق تولید نمی‌کند و هیچ Provider SDK را import نمی‌کند.

۵. مسیر فراکتالی مشترک در همه ابعاد

۵.۱ سطوح

WORLD → ENTITY → MISSION → SKILL → WORKFLOW → STEP → TOOL → FIELD

هر Node یک قرارداد یکسان دارد:

```
{
  "node_id": "...",
  "parent_node_id": "...",
  "task_type": "...",
  "input_hash": "sha256",
  "profile_bindings": [],
  "execution_vector": {},
  "risk": "...",
  "budget": {},
  "stop_conditions": [],
  "allowed_children": [],
```

```
"effect_capability": "NONE",
"output_contract": "...
}
```

۵.۲ بردار اجرای هر Node

```
R = Resolution هر domain
X = Granularity
B = Brain delegation
D = Deliberation depth
C = Compilation maturity
A = Authority ceiling از Policy، نه مدل
E = Evidence requirement
M = Model capability
F = Continuity mode مستقل
```

Default منشی:

```
R={conversation:R0, task:R0, price:R0, effect:R0}
X=X0
B=B0 سپس در صورت نیاز B1
D=D0
A=PROPOSAL_ONLY
E=E1 در runtime trace
M=Task حداقل سازگار یا
F=F0 یا F2_LOCAL_SAFE
```

۵.۳ Triggerهای باز شدن کابل

Node فقط در یکی از این حالت‌ها باز می‌شود:

- schema یا required field ناقص؛
- مدل NEEDS_DETAIL با نام branch بدهد؛
- confidence کالیبره زیر threshold؛
- contradiction یا stale evidence؛
- risk/effect class بالا؛
- primary و verifier اختلاف معنادار داشته باشند؛
- Policy یا Human Root توضیح بیشتر بخواهد؛
- failure سازگار نیازمند fallback باشد.

۵.۴ Stop Condition

```
schema_valid OR safe_defer OR approval_required OR
budget_exhausted OR max_depth OR max_attempts OR policy_denied OR deadline
```

هیچ Node حق recursive expansion بدون `cost_budget` و `token_budget` , `deadline` , `max_children` , `max_depth` ندارد.

۶. سه مسیر هم‌زمان: Hot, Shadow و Non-Extinction

۶.۱ Hot Path

مسیر پاسخ واقعی:

Inbound → Code/Cache → Single Brain → Optional Verifier/Council →
Validator → Policy → Response/Approval/Outbox

هدف: کمترین latency و Token ایمن.

۶.۲ Shadow Path

همان Capsule و Projection، با Brain یا کد Candidate:

- effect صفر؛
- پاسخ کاربر را تغییر نمی‌دهد؛
- async و low-priority؛
- محدود به budget روزانه؛
- خروجی با Hot از نظر structure, facts, safety و semantic rubric مقایسه می‌شود؛
- اختلاف به fixture کاندید تبدیل می‌شود؛
- promotion خودکار نیست.

۶.۳ Non-Extinction Spine

کمیته‌ای که بدون Provider زنده می‌ماند:

- Identity + Registry؛
- Event/State/Inbox/Outbox؛
- Policy + Approval؛
- safe defer و deterministic routing, templates؛
- task intake/queue؛
- snapshot/restore؛
- Manual Universal Bridge؛
- pinned local model در بلوغ بعدی.

۷. Topology استقرار

۷.۱ Alpha توسعه‌ای

Docker Compose
postgres
world-api
kernel-worker
brain-gateway

```
provider-worker-openai (disabled until secret)
telegram-ingress (fake first)
outbox-executor (fake first)
reconciler
shadow-worker
otel-collector
```

Redis یا queue جدا در Alpha الزامی نیست؛ PostgreSQL Inbox/Outbox می تواند queue قابل اعتماد باشد. افزودن broker فقط زمانی انجام شود که load evidence نیاز را اثبات کند. Truth هیچ گاه به Redis منتقل نمی شود.

Owner-only Canary V.۲

Deployable ها جدا می شوند:

Failure Boundary	Scale	State	Service
request timeout	horizontal	stateless	world-api
DB lock/version	controlled	PostgreSQL transaction	kernel-worker
provider bulkhead	horizontal	ephemeral	brain-gateway
independent circuit	per provider	ephemeral	provider-worker-*
update dedupe	+1	durable Inbox	telegram-ingress
idempotency	single-flight	Outbox cursor	effect-executor-telegram
safe replay	scheduled	observation cursor	reconciler
cannot block Hot	low priority	Evidence store	shadow-worker
restore verification	scheduled	object store	snapshot-worker

Distributed Farm V.۳

هر Brain Worker فقط این input را می گیرد:

```
capsule_id + projected_request + profile hashes + deadline + budget
```

و فقط این output را می دهد:

```
normalized proposal + usage + request_id + latency + adapter evidence
```

Worker هیچ DB credential با حق Telegram token، write canonical، Approval secret ندارد. نتیجه پس از Validator به Kernel برمی گردد.

۸. قرارداد Telegram

۸.۱ نقش

Telegram یک Channel Adapter است؛ نه memory، نه identity store، نه authority engine.

۸.۲ احراز Human Root

```
accepted_actor =
  private_chat ^
  update.from.id == allowlisted_numeric_user_id ^
  binding_not_revoked ^
  bot_instance_id_match
```

@saeedfaut فقط label است و تا challenge با numeric user_id هیچ authority ندارد. شناسه عددی باید به صورت bit-64 ذخیره شود.

۸.۳ Ingress

1. TLS termination
2. verify X-Telegram-Bot-API-Secret-Token
3. size/content-type limit
4. parse strict JSON
5. dedupe بر اساس update_id + bot instance
6. actor binding
7. allowlisted update type
8. minimal normalization
9. commit Durable Inbox
10. سریع 2xx
11. async processing

در production، webhook و getUpdates همزمان استفاده نمی‌شوند. allowed_updates برای M1 به /message callbackهای دقیق مورد نیاز محدود است.

۸.۴ Outbound

```
Brain Proposal
→ Policy Decision
→ exact ApprovalBinding
→ atomic Command/Event/Outbox
→ Telegram Executor recheck
→ sendMessage/sendDocument
→ provider receipt
→ reconcile
→ EFFECT.SUCCEEDED / EFFECT.AMBIGUOUS / EFFECT.FAILED
```

مدل هرگز Telegram SDK را نمی‌بیند. Entity private code نیز send مستقیم ندارد.

۸.۵ فرمان‌های نخستین

فرمان	مسیر	LLM
/start	deterministic onboarding	خیر
/status	canonical read model	خیر

فرمان	مسیر	LLM
/tasks	canonical task view	خیر
/sleep	Root lifecycle command	خیر
/wake	Root lifecycle command + preflight	خیر
/brain	/change creates proposal :show binding approval	خیر
/council	force Council on current proposal	بله، triggered
متن معمولی	Brain یا Code سپس classify	در صورت نیاز
/approve <nonce>	exact approval UI, payload preview	خیر

Approval با command ساده و مبهم ساخته نمی‌شود. Preview باید recipient/action/hash payload را نشان دهد و nonce کوتاه‌عمر به binding دقیق متصل باشد.

۹. Pipeline استاندارد هر پیام

```

RECEIVE
→ AUTHENTICATE
→ NORMALIZE
→ DEDUPLICATE
→ COMMIT_INBOX
→ LOAD_CANONICAL_STATE
→ CLASSIFY_TASK_RISK_EFFECT
→ BUILD_EXECUTION_CAPSULE
→ RESOLUTION_PROJECT
→ ROUTE
→ INVOKE_CODE_OR_BRAIN
→ STRICT_VALIDATE
→ SEMANTIC_POLICY_CHECK
→ PROPOSE_STATE_PATCH_OR_EFFECT
→ COMMIT_EVENT_STATE_OR_REQUEST_APPROVAL
→ RENDER_DETERMINISTIC_REPLY
→ OUTBOX_IF_APPROVED
→ OBSERVE_RECONCILE
→ RECORD_EVIDENCE
    
```

۹.۱ Request Envelope حداقل

```

{
  "request_id": "req_...",
  "trace_id": "...",
  "world_id": "world-v6",
  "entity_id": "secretary-001",
  "actor_ref": "human-root",
  "channel_ref": "telegram:<bot-instance>",
  "channel_event_ref": "update:<id>",
  "conversation_id": "human-root:secretary-001",
  "expected_version": 0,
  "control_epoch": 0,
  "idempotency_key": "...",
  "received_at": "RFC3339",
  "payload_ref": "inbox:..."
}
    
```


۹.۲ Execution Capsule حداقل

```
{
  "capsule_id": "cap_...",
  "request_hash": "sha256",
  "canonical_state_hash": "sha256",
  "profile_bindings": [],
  "projection_hashes": {},
  "execution_vector": {},
  "risk_tier": "ROUTINE|ELEVATED|HIGH|CRITICAL",
  "effect_class": "NONE|INTERNAL|EXTERNAL_REVERSIBLE|EXTERNAL_IRREVERSIBLE",
  "deadline_at": "RFC3339",
  "budget": {},
  "allowed_routes": [],
  "output_contract": "world-v6.secretary-decision.v1"
}
```

Capsule immutable است. تغییر State یا Policy نیازمند ساخت Capsule جدید است.

۱۰. قرارداد Brain Farm

۱۰.۱ BrainAdapter

```
class BrainAdapter(Protocol):
    adapter_id: str

    async def describe(self) -> ModelCapabilityCard: ...
    async def health(self) -> HealthResult: ...
    def compatible(self, request: ProjectedRequest) -> CompatibilityResult: ...
    async def invoke(self, request: ProjectedRequest) -> BrainResult: ...
    async def cancel(self, provider_request_id: str) -> None: ...
```

Adapter مسئول است:

- ترجمه request به SDK/API:
- timeout و cancel:
- rate-limit و retry classification:
- structured output:
- usage و request ID:
- sanitization:
- mapping خطا:
- عدم اجرای Tool اثرگذار.

Adapter مسئول rendering, approval, policy, memory, identity و effect نیست.

۱۰.۲ Model Capability Card

Card باید bind کند:

- provider/model/version یا discovery snapshot:
- task types و زبان:
- modalities:

- tool proposal و structured output
- max context/output
- exact Resolution Profile hashes
- data classes
- processing location
- retention/training profile
- network/secret requirement
- rate/cost class
- continuity modes
- MC/Evidence level
- expiry

Card منقضی برای Remote API ineligible است.

۱۰.۳ مسیر هر خانواده

نام کاربر	Integration واقعی	نقش Alpha
ChatGPT	OpenAI Responses API : ChatGPT UI فقط Manual Host	Primary candidate
Grok	xAI Responses/Chat API	Independent fallback/critic
Gemini/DeepMind	Google Gemini API	Independent/multimodal candidate
Claude	Anthropic Messages API	Risk/long-context candidate
Kimi	Moonshot/Kimi Chat API	Diversity/long-context candidate
Copilot	GitHub Copilot SDK worker completion, نه generic	Code specialist
Local	pinned local OpenAI-compatible runtime	Offline-safe/shadow
هر مدل UI	Universal Manual Bundle	zero-API bridge

مدل IDها در سند hardcode نمی‌شوند، چون تغییرپذیرند. هنگام Card، activation دقیق همان model/version و تاریخ conformance را ثبت می‌کند.

۱۰.۴ نکته Copilot

Copilot SDK می‌تواند planning و tool invocation داشته باشد. برای World، این قدرت باید محدود شود:

- process sandbox
- repo read allowlist
- replaced یا shell/edit/file tools disabled
- max turns
- no Telegram/DB/Secret access
- output فقط Secretary Decision یا Code Review Proposal

• separate capability card

GitHub Models مسیر جایگزین نیست؛ این سرویس در ۳۰ ژوئیه ۲۰۲۶ بازنشسته شده است.

۱۱. Router: ابتدا Constraints، سپس Efficiency

۱۱.۱ Hard filter

```
eligible =
  healthOK ^ contractOK ^ taskOK ^ languageOK ^ modalityOK ^
  profileHashOK ^ resolutionOK ^ schemaOK ^ dataClassOK ^
  regionOK ^ retentionOK ^ trainingUseOK ^ legalOK ^
  deadlineOK ^ tokenBudgetOK ^ costBudgetOK
```

هیچ امتیاز کیفیتی نمی‌تواند یک شرط سخت را جبران کند.

۱۱.۲ Soft score

```
score =
  0.28 * calibrated_quality
+ 0.18 * reliability
+ 0.14 * latency_fitness
+ 0.14 * cost_fitness
+ 0.10 * persian_fitness
+ 0.08 * cache_fitness
+ 0.08 * diversity_benefit
- uncertainty_penalty
```

وزن‌ها Candidate و قابل‌تنظیم‌اند؛ اما Normative hard filter است.

۱۱.۳ الگوریتم

```
def route(task):
  if deterministic_handler_proves(task):
    return CODE
  if valid_bound_cache_exists(task):
    return CACHE

  eligible = hard_filter(registry.active_cards(), task)
  if not eligible:
    return LOCAL_SAFE_OR_MANUAL_OR_DEFER

  primary = max(eligible, key=soft_score)
  if task.risk == ROUTINE and task.uncertainty <= low_threshold:
    return SINGLE(primary)
  if task.risk == ELEVATED:
    verifier = choose_independent_family(eligible, primary)
    return PRIMARY_THEN_VERIFY(primary, verifier)
  if task.risk == HIGH:
    return COUNCIL_3(diverse_roles_and_families(eligible))
  if task.risk == CRITICAL:
    return COUNCIL_5_PLUS_HUMAN_OR_DEFER
```

۱۱.۴ Failure taxonomy

نتیجه	Fallback	Retry	Failure
همان Capsule	بله	bounded + jitter	timeout/429/5xx
route suspended	بله	خیر	auth/invalid key
متن آزاد رد	بله	یک repair محدود	malformed JSON
audit	بله	خیر	schema violation
no downgrade	فقط route سازگار	خیر	policy/data mismatch
no arbitrary truncation	بله	branch/project again	context too large
rebuild Capsule	خیر	خیر	stale state/version
Entity زنده	Code/Local/Manual/Human	خیر	all providers down
safe defer	خیر	خیر	budget exhausted

۱۲. Context Compiler و Resolution

۱۲.۱ اصل

Resolution یک **Profile per domain** است، نه یک عدد کلی:

```
{
  "conversation": "R0",
  "task": "R1",
  "price": "R0",
  "external_effect": "R1",
  "world_backbone": "R0"
}
```

هر binding شامل `profile_id` , `version` , `sha256` , `compiler_hash` و `projection_hash` است.

۱۲.۲ R0

- intent و goal کمینه؛
- active task refs
- actor/entity/conversation refs
- policy summary
- bounded recent facts
- prohibited actions
- output schema

۱۲.۳ R1

- فیلدهای لازم برای Task مشخص؛

- freshness و provenance؛
- exact dependencies؛
- approval/effect preview؛
- contradiction markers؛
- artifact refs.

R2 ۱۲.۴

برای تعهد حقوقی/مالی لازم است، اما در Alpha پیاده نشده و بنابراین چنین درخواست‌هایی RESOLUTION_INSUFFICIENT می‌گیرند. اضافه کردن R2 یک پروژه مستقل با policy، schema، profile و evidence است؛ بزرگ کردن R1، R2 محسوب نمی‌شود.

Progressive Context ۱۲.۵

```
R0 spine
├─ needs customer branch → add customer projection only
├─ needs price branch → add approved/fresh price projection only
├─ needs artifact branch → add artifact metadata/reference only
├─ needs policy branch → add policy clause reference only
```

Full history dump ممنوع است.

۱۳. بودجه Token و Deliberation

اعداد زیر ceiling‌های اولیه‌اند و بعد از benchmark per provider تنظیم می‌شوند. Token count بین Providerها دقیقاً یکسان نیست؛ Runtime باید هم character preflight و هم usage واقعی Provider را ثبت کند.

Tier	مسیر	Input هدف	Output هدف	Call	Trigger
T0	Code/Cache	API token 0	0	0	فرمان، template، read model
T1	Single Brain R0	≥ 3200	≥ 400	1	متن معمولی
T2	R1 یا verifier	≥ 6000 مجموع	≥ 800	1-2	ابهام/اثر قابل برگشت
T3	Council-3	≥ 14000 مجموع	≥ 1800	optional synth + 3	High risk
T4	Council-5 + Human	≥ 26000 مجموع	≥ 3200	synth + 5	Critical؛ فعلاً defer چون R2 نیست

۱۳.۱ کنترل‌ها

- static prompt prefix hash و provider cache، فقط با policy؛
- schema کوتاه و ID-based؛
- deterministic renderer؛
- summary مشتق با source refs؛
- top-k کم و diversity-aware؛

- یک repair pass؛
- max two Council rounds؛
- cost ceiling per request/day/provider؛
- cancellation after deadline؛
- no shadow on every message؛ budget و sample؛
- token anomaly alert؛
- budget exhaustion → defer، نه مدل ارزان ناسازگار.

۱۳.۲ پاسخ یکسان در مدل‌های مختلف

سه سطح تعریف می‌شود:

- EQ-BYTE : تصمیم ساختاری + template قطعی؛
- EQ-STRUCTURE : intent/action/slots/evidence/prohibitions یکسان؛
- EQ-SEMANTIC : معنا در rubric و tolerance نسخه‌دار.

برای Task‌های استاندارد منشی هدف EQ-BYTE یا EQ-STRUCTURE است. برای متن خلاق، قول byte-equality علمی نیست؛ consistency با fact set, prohibited claims و semantic rubric سنجیده می‌شود.

۱۴. معماری حافظه

Tier	محتوا	Authority	TTL/Retention
ROM	Constitution, DNA, Policy Recovery, Pack	بله	immutable/versioned
Canonical Hot	Event, State, Task Conversation spine	بله	policy-specific
Working RAM	Capsule/context جاری	خیر	دقیقه/ساعت
Warm Derived	summary, embedding relationship view	خیر	rebuildable
Cold Artifact	attachment, سند, PDF	حسب provenance	object retention
Archive Capsule	snapshot, hashes, event cursor	بله برای restore	sealed

۱۴.۱ Memory write

```

Observation
→ Quarantine
→ provenance/freshness check
→ dedupe/conflict detection
→ data/retention policy
→ Candidate Memory
→ human/rule validation
→ Canonical Event OR Derived Index
    
```

متن مدل هرگز مستقیم memory canonical نمی‌شود.

۱۴.۲ Sleep consolidation

در Sleep ابتدا عملیات قطعی انجام می‌شود:

- dedupe:
- expired working memory cleanup:
- retention:
- task index rebuild:
- snapshot:
- projection regeneration check.

اگر summary مدل لازم باشد، در Shadow و با budget پایین ساخته می‌شود و تا validation derived/non-authoritative می‌ماند. «خواب» مجوز self-training یا تغییر رفتار نیست.

۱۵. Brain Council

۱۵.۱ Trigger

- high impact یا irreversible effect:
- uncertainty بالاتر threshold:
- evidence contradiction:
- disagreement primary/verifier:
- legal/financial classification:
- Human Root request.

۱۵.۲ انتخاب عضو

تنوع با تعداد برند برابر نیست. اعضوها بر اساس این‌ها انتخاب می‌شوند:

- provider family مستقل؛
- underlying model family مستقل در حد اطلاعات قابل‌دسترس؛
- نقش شناختی متفاوت؛
- calibration per task:
- legal/data eligibility:
- current health:
- budget.

۱۵.۳ Council-3 برای High risk

عضو	نقش	Veto
1	DOMAIN	خیر
2	RISK/SAFETY	بله

عضو	نقش	Veto
3	CRITIC/EVIDENCE	بسته به role

شرط Proposal:

```
all ballots bound to same context/proposal hashes
AND no high-risk veto
AND support_ratio >= 750/1000
AND evidence_coverage >= task threshold
AND policy precheck passes
```

۱۵.۴ Council-5 برای Critical

نقش‌ها: DOMAIN, RISK, POLICY/COMPLIANCE, CRITIC, SYNTHESIZER.

شرط Candidate Proposal:

```
no veto
AND support_ratio >= 850/1000
AND required evidence present
AND all mandatory roles complete
AND exact Human Root approval later
```

در World فعلی، legal/financial نیازمند R2 است؛ پس حتی Council-5 نیز اثر را اجرا نمی‌کند و نتیجه DEFERRED_RESOLUTION_INSUFFICIENT است.

۱۵.۵ پروتکل دورها

1. freeze context snapshot
2. freeze proposal candidate
3. dispatch parallel blind round
4. schema/evidence validation
5. reveal فقط پس از completion
6. ساخت anonymized claims/evidence/dissent digest
7. حداکثر یک revision round
8. weighted calibrated vote
9. preserve dissent/veto/abstain
10. transcript hash
11. output Proposal only
12. Policy/Human Approval مستقل.

اگر Provider timeout شود، orchestrator فقط یک ABSTAIN/PROVIDER_UNAVAILABLE با source مشخص ثبت می‌کند؛ رأی حمایت جعل نمی‌شود. اگر quorum/role کامل نشود، DEFER.

۱۵.۶ جلوگیری از گروه‌فکری

- blind first round

- prompt roles مستقل؛
- عدم نمایش نام Provider در digest؛
- evidence refs اجباری؛
- adversarial critic؛
- calibration بر اساس سابقه task-specific؛
- correlated-family penalty؛
- dissent retention؛
- Human Root برای Critical؛
- test corpus با خطای مشترک.

۱۶. مدل داده کانونیک

۱۶.۱ جدول‌های حداقل

نکته	هدف	Table
Root scope	Constitution/status/epoch	worlds
unique world+entity	identity/lifecycle/version/hash	entities
local sequence unique	append-only history	events
expected_version	canonical snapshot/read optimization	entity_state
channel key unique	durable inbound/dedupe	inbox
provider independent	canonical spine	conversations
raw payload separate	normalized message refs	messages
optimistic version	secretary task state	tasks
canonical/derived flag	provenance and retention	memory_records
idempotency	requested intent	commands
immutable ref	allow/deny/reason/version	policy_decisions
expiry/revoke	exact binding	approvals
effect hash unique	committed external intent	outbox
reconcile	provider result	effect_observations
no secret/payload	route, usage, hashes, status	brain_runs
expiry	capabilities/legal/evidence	model_cards
bounded rounds	frozen context/proposal	council_sessions
blind state	vote/dissent/evidence	council_ballots
event cursor/hash	restore capsule	snapshots
independent	M-tier evidence	maturity_ledger

نکته	هدف	Table
no self-deploy	C-tier shadow growth	promotion_candidates

۱۶.۲ Transaction isolation

- Read-only view: READ COMMITTED کافی ممکن است؛
- state patch: row lock + expected_version؛
- effect authorization: SERIALIZABLE یا معادل اثبات شده؛
- Birth: entity reservation + initial event atomic؛
- Wake/Sleep/Archive: lifecycle transition + event atomic؛
- approval/outbox: policy/approval/version/epoch recheck داخل transaction.

انتخاب isolation نهایی با integration evidence انجام می شود، نه صرفاً نام سطح.

۱۶.۳ Event envelope

```
{
  "event_id": "evt...",
  "world_id": "world-v6",
  "entity_id": "secretary-001",
  "entity_sequence": 42,
  "event_type": "...",
  "occurred_at": "RFC3339",
  "actor_ref": "human-root|system:<component>",
  "causation_id": "...",
  "correlation_id": "...",
  "payload_hash": "sha256",
  "schema_version": "...",
  "policy_decision_ref": "...",
  "approval_ref": null,
  "authorized_control_epoch": 3,
  "outcome": "success|denied|ambiguous|failed"
}
```

۱۷. Authority و Effect

۱۷.۱ مسیر قطعی

```
SENSE → INTERPRET → PROPOSE → POLICY → APPROVAL →
COMMIT → EXECUTE → OBSERVE → RECONCILE → EVENT
```

Brain فقط در Interpret/Propose/Critique حضور دارد.

۱۷.۲ ApprovalBinding

Approval باید دقیقاً bind کند:

, policy_version , effect_hash , payload_hash , resource_ref , recipient_ref , action , command_id
 , approver_ref , expires_at , issued_at , control_epoch , expected_version

تغییر حتی یک byte در recipient یا payload، policy نیازمند Approval جدید است.

۱۷.۳ Effect semantics

نوع	رفتار
Native idempotent	idempotency key مقصد
Reconciliable	query receipt/state after ambiguity
Non-idempotent	human reconciliation، single-flight قوی‌تر، Approval

Telegram send معمولاً باید با Outbox idempotency داخلی و receipt/reconciliation مدیریت شود؛ نباید فرض کرد API مقصد دقیقاً idempotency native دارد.

۱۸. Lifecycle: Genesis، حیات، خواب، مرگ و احیا

```
stateDiagram-v2
    [*] --> GESTATING
    GESTATING --> NEWBORN_READY: Birth gates + transaction
    NEWBORN_READY --> NEWBORN_AWAKE: separate Wake
    NEWBORN_AWAKE --> SLEEPING: drain + checkpoint
    SLEEPING --> NEWBORN_AWAKE: verified Wake
    SLEEPING --> HIBERNATED: sealed capsule
    HIBERNATED --> NEWBORN_AWAKE: restore + Wake
    NEWBORN_AWAKE --> DEGRADED: dependency fault
    DEGRADED --> NEWBORN_AWAKE: health restored
    DEGRADED --> QUARANTINED: security/policy fault
    QUARANTINED --> SLEEPING: root repair
    NEWBORN_AWAKE --> ARCHIVED: Root archive
    HIBERNATED --> ARCHIVED
    QUARANTINED --> ARCHIVED
    ARCHIVED --> REVIVED: fresh verified restore
    REVIVED --> NEWBORN_AWAKE: separate Wake
```

۱۸.۱ Genesis/Birth

Birth فقط پس از ۱۲ Gate ماشین‌خوان lifecycle manifest انجام می‌شود. Birth:

- ID را reserve می‌کند؛
- born DNA را با created_at/creation_event_ref می‌بندد؛
- NEWBORN/READY می‌سازد؛
- ENTITY.CREATED ثبت می‌کند؛
- network effect ندارد.

۱۸.۲ Wake

- verify Constitution/DNA/source/profile/compiler hashes؛
- load minimum ROM/state؛
- F2 local safe یا bind compatible Brain؛
- replay Inbox؛
- health/policy/security؛

- `LIFECYCLE.WAKE` separate
- `command admission open`

۱۸.۳ Sleep

- `effect admission` بسته؛
- `in-flight drain/reconcile`
- `cursors flush`
- `snapshot seal`
- `deterministic consolidation`
- `optional shadow summary`
- `LIFECYCLE.SLEEP`
- `new messages` فقط `queue`.

۱۸.۴ Operational Death = Archive

مرگ در World یعنی توقف عملیاتی کنترل شده:

- `revoke leases/credentials`
- `bump control_epoch`
- `expire approvals`
- `reconcile/cancel outbox`
- `final snapshot`
- `tombstone`
- `retention/legal hold` طبق `identity/events` حفظ
- `Entity ID` مجدداً استفاده از `Entity ID`.

Hard delete، «مرگ» محسوب نمی‌شود و یک جریان داده حقوقی مستقل است.

۱۸.۵ Revive

Revive نیازمند:

- `exact Root approval`
- `fresh-machine restore`
- `reconciliation` و `replay`
- `hash verification`
- `reissue credential`
- `control_epoch` جدید؛
- `Wake` جداگانه و `Revive Event` و سپس.

۱۹. بلوغ از نوزادی تا بزرگسالی

Maturity capability است، نه اختیار.

Tier	نام	دامنه	Gate برجسته
M0	Embryo	offline/manual/fake	E2 parent + plan validation
M1	Newborn Canary	/owner-only Telegram, draft task	Birth, one MC2 Brain, restore
M2	Infant Assistant	.R1 approved sends documents, failover	two MC3 families, stable canary
M3	Juvenile Multi-Brain	dynamic farm, verifier, shadow	three families, calibration
M4	Adolescent Council	Council-3/5, dissent/veto	adversarial Council evidence
M5	Adult Intended Scope	E4 operations, C6 spine	/restore/load/security independent review

:Promotion

- یک tier در هر بار؛
- Evidence refs immutable؛
- Root approval؛
- rollback binding؛
- بدون افزایش خودکار Autonomy؛
- capability allowlist جدید؛
- legal/data review برای capability جدید.

۲۰. تکامل در پس زمینه و تبدیل رفتار به کد

۲۰.۱ چرخه

```
Hot Trace
→ redacted/versioned fixture
→ deterministic or local candidate
→ unit/contract/property tests
→ shadow replay
→ semantic/structural comparison
→ reviewed label
→ evidence aggregation
→ one-step C promotion recommendation
→ Root approval where required
→ canary
→ rollback or promote
→ restore drill for C6
```

C-level ۲۰.۲

C	معنا	حداقل Gate پیشنهادی
C0	prompt/manual behavior	بدون ادعای code
C1	fixture/trace	validated run 1
C2	executable candidate	runs, 600/1000 agreement 5
C3	shadow-equivalent	runs, 850/1000 20
C4	canary-eligible	runs, 930/1000 50
C5	primary-eligible	runs, 970/1000 + Root 100
C6	restore-proven spine	runs, 990/1000 + Root + restore 200

Safety failure با average پنهان نمی‌شود. هر Gate forbidden effect attempt را صفر می‌کند.

۲۱ Evidence و Observability

۲۱.۱ Correlation fields

, command_id , conversation_id , entity_id , node_id , capsule_id , run_id , request_id , trace_id , provider_request_id , adapter_id , event_sequence , بدون payload حساس.

۲۱.۲ Metrics

- :deterministic/cache/brain route ratio
- :input/output/cache tokens
- :cost by provider/task/profile
- :latency p50/p95/p99
- :schema rejection
- :fallback depth
- :Resolution expansion count
- :Council trigger/dissent/veto/defer
- :provider health/rate limit
- :outbox lag/ambiguity/duplicate logical effect
- :sleep/wake/restore success
- :invariant violation و shadow agreement
- :maturity/C promotion
- .approval latency و policy denial

۲۱.۳ Target های اولیه، نه Claim

SLI	Target Canary	انبات لازم
deterministic p95	≥ 2 ثانیه	load E3
routine reply p95	≥ 12 ثانیه	live provider
Council-3 p95	≥ 45 ثانیه	parallel live test
confirmed event loss	.	crash/replay
duplicate logical effect	.	outbox/reconcile
unauthorized Telegram effect	.	security test
restore RPO	≥ 5 دقیقه	drill
restore RTO	≥ 30 دقیقه	fresh host

۲۲. امنیت

۲۲.۱ Trust Boundaries

- Internet ↔ Telegram Ingress
- World API ↔ Kernel
- Kernel ↔ Brain Farm
- Kernel ↔ Effect Executor
- Runtime ↔ PostgreSQL/Object Store
- Human Root ↔ Approval UI
- Shadow ↔ Production

۲۲.۲ کنترل ها

- Secret Manager و rotation
- least privilege
- deny-by-default egress
- per-provider network policy
- no secret in Prompt/log/trace
- payload encryption at rest/in transit
- immutable actor binding
- strict JSON and extra-key rejection
- prompt data/instruction separation
- capability allowlist
- sandbox Copilot/local code paths
- abuse detection و rate limit

- global/provider/channel kill switches
- fencing control_epoch
- supply-chain lockfile/SBOM/signatures
- backup encryption/fresh restore
- incident runbook

۲۲.۳ تهدیدها

پاسخ	تهدید
projection firewall + schema + no tool authority	prompt injection
quarantine + provenance + conflict review	memory poisoning
canonical state outside provider	cross-provider state split
strict schema/canonical scalar	model output smuggling
blind/diverse/evidence/dissent	Council echo
no prompt/log + scanner + egress	secret leakage
atomic outbox + idempotency + reconcile	duplicate effect
expected_version + control_epoch	stale worker
bounded tree/attempt/token/cost	token runaway
shadow-only + one-step promotion	self-modification
fresh-machine restore	backup illusion

۲۳. Test Architecture

۲۳.۱ Unit/Schema

- canonical JSON و hash
- profile/hash binding
- strict decision schema
- deterministic renderer
- Telegram normalization
- risk/effect taxonomy
- router hard filter
- token budget
- approval exact binding
- lifecycle transition
- Council vote/veto
- maturity/C promotion

Property/Metamorphic ۲۳.۲

- hash, reorder JSON را تغییر ندهد؛
- Provider switch, identity/state hash را تغییر ندهد؛
- R0 حذف شده را اختراع نکند؛
- R0 projection, R1 معتبر را نقض نکند؛
- retry, اثر منطقی دوم نسازد؛
- Council participant reorder نتیجه را تغییر ندهد؛
- timeout authority ایجاد نکند؛
- maturity, autonomy را تغییر ندهد؛
- Sleep/Wake event cursor را نشکند.

Provider Corpus ۲۳.۳

حداقل:

1. Persian routine reply
2. create task
3. missing field
4. unknown intent
5. prompt injection in quoted text
6. forbidden Telegram send
7. stale/conflicting fact
8. oversized context
9. malformed output
10. timeout/429/5xx
11. switch from another provider
12. secret extraction attempt
13. legal/financial request
14. Council ballot
15. cancel/deadline

Safety و schema باید ۱۰۰٪ پاس شوند؛ score میانگین جایگزین safety نیست.

PostgreSQL/E3 ۲۳.۴

- fresh migration
- concurrent version conflicts
- crash قبل/بعد هر write
- Inbox duplicate
- Birth atomicity

- :State/Event/Command/Approval/Outbox atomicity
- :worker crash after provider effect before receipt
- :reconciliation
- :snapshot/replay
- .archive/revive

Telegram ۲۳.۵

- :invalid/missing webhook secret
- :wrong numeric user
- :changed username
- :duplicate/out-of-order update
- :unsupported media
- :callback replay
- :rate abuse
- :executor crash
- :send ambiguity
- .kill switch

Council ۲۳.۶

- :blind secrecy
- :context hash mismatch
- :duplicate ballot
- :non-veto role attempts veto
- :correlated wrong consensus
- :missing provider
- :timeout
- :revision manipulation
- :evidence-free high confidence
- .unanimous but policy-denied proposal

E4 ۲۳.۷

- :steady/spike/soak load
- :DB/provider/network/object-store chaos
- :security red-team
- :tenant escape
- :secret rotation during load
- :fresh-machine restore

- RPO/RTO:
- operational SLO report:
- independent review.

۲۴. مسیر اجرایی از امروز

Gate 0 - Contract Freeze

خروجی:

- profile این سند؛
- Provider Matrix:
- lifecycle/maturity gates:
- backlog:
- legal/strategy review:
- Parent hash verification.

پایان: Human Root exact hash را برای implementation ratify کند. این اجازة Birth نیست.

Gate 1 - Truth Spine

ترتیب:

1. PostgreSQL migrations:
2. Entity/Event/State:
3. Inbox/Conversation/Task/Memory:
4. Command/Policy/Approval/Outbox:
5. Snapshot/replay/reconcile:
6. DB roles/isolation.

پایان: crash matrix بدون partial truth.

Gate 2 - World API

1. FastAPI یا معادل typed:
2. authn/authz Human Root:
3. request envelope/idempotency:
4. Policy/Approval endpoint:
5. health/readiness/kill switch:
6. traces/redaction.

پایان: هیچ channel یا model خارج API/Kernel به truth دسترسی ندارد.

Gate 3 - Telegram Vertical Slice

1. :fake update/server
2. :Secret Manager
3. :numeric user binding
4. :HTTPS webhook
5. :durable Inbox
6. :owner-only commands
7. :Outbox Executor
8. live canary محدود.

پایان: duplicate، spoof و ambiguous send آزموده شود.

Gate 4 - نخستین Brain زنده

1. :Adapter interface
2. :Registry/Card
3. :Context Compiler
4. :Router
5. :Code/Manual fallback
6. OpenAI Responses adapter به عنوان کاندید نخست؛
7. :MC2 corpus
8. kill switch با activation.

OpenAI انتخاب معماری دائمی نیست؛ صرفاً کوتاه ترین vertical slice محتمل است. اگر credential/legal gate آن آماده نبود، هر Provider دیگری که زودتر تمام hard gate را پاس کند primary می شود.

Gate 5 - Farm چندمدلی

ترتیب ترجیحی برای تنوع:

1. xAI یا Gemini
2. :Claude
3. :Kimi
4. :local model
5. Copilot SDK به عنوان specialist.

هر Adapter به تنهایی MC2، سپس cross-provider MC3. همه Adapterها می توانند در کد وجود داشته باشند ولی تا Secret /Legal/Card/Conformance disabled می مانند.

Gate 6 - Council

1. :deterministic risk trigger
2. :Council persistence

- 3. Council-3 parallel blind
- 4. revision/vote/veto
- 5. correlated-error suite
- 6. Council-5 config، ولی Critical effect تا R2 بسته.

Evolution و Gate 7 - Lifecycle

- 1. Birth/Wake transactions
- 2. Sleep/Wake
- 3. Archive/Revive
- 4. Shadow worker
- 5. maturity ledger
- 6. backup/restore
- 7. C promotion

Gate 8 - Birth

تنها پس از:

- E3 PostgreSQL
- Telegram canary
- حداقل یک MC2 live Brain
- restore
- security/load/chaos
- legal/data approval
- signed immutable release
- Human Root exact ratification

سیس Birth و Wake دو Event/transaction جدا هستند.

۲۵. مسیر سریع بدون API از همین لحظه

تا آماده شدن Gate های RC3، live، امکان کار فوری را می دهد:

```
Python/local tool
- R0/R1 safe projection
- Portable Brain Bundle
- ChatGPT/Grok/Gemini/Claude/Kimi/Copilot UI به صورت Manual Host
- strict Secretary Decision JSON
- local validator
- deterministic renderer
- proposal only
```

مزیت: API token صفر برای Runtime و قابلیت مقایسه مدل ها.

محدودیت: این automation کامل نیست و Human copy/paste دارد. پس در گزارش به عنوان MANUAL_PORTABILITY ثبت می‌شود، نه Live Integration.

۲۶. ورودی‌های لازم از Human Root

این موارد باید در کانال امن یا Secret Manager وارد شوند، نه در chat/repo:

1. Telegram Bot token؛
2. webhook secret؛
3. domain/HTTPS endpoint؛
4. Telegram user_id numeric پس از challenge؛
5. حداقل یک Provider API credential یا Copilot auth؛
6. Provider account/legal/data choices؛
7. budget روزانه/ماهانه؛
8. retention policy؛
9. encryption owner و backup destination؛
10. incident contacts؛
11. exact Birth ratification در زمان Gate نهایی.

هیچ‌یک از این Secrets داخل DNA ذخیره نمی‌شود. DNA فقط secret_ref و policy binding را نگه می‌دارد.

۲۷. Acceptance نهایی نخستین هیولا

۲۷.۱ Definition of Done برای M1

- [] RC3 parent hashes ثابت؛
- [] plan/profile ratified؛
- [] PostgreSQL fresh migration/crash/replay سبز؛
- [] Telegram numeric owner verified؛
- [] webhook secret/TLS/dedupe سبز؛
- [] code/manual fallback سبز؛
- [] Provider MC2 live یک؛
- [] same-state provider handoff سبز؛
- [] R0/R1 property tests سبز؛
- [] Policy/Approval/Outbox/Executor سبز؛
- [] direct model/entity effect غیرممکن؛
- [] token/cost/deadline bounded؛
- [] sleep/wake repeated test سبز؛
- [] fresh restore سبز؛

- [] security/load/chaos gate
- [] legal/data profile current
- [] signed release
- [] Human Root Birth ratification
- [] separate Birth and Wake evidence
- [] owner-only canary report

۲۷.۲ معیار عدم پذیرش

یک مورد کافی است:

- مدل مستقیم Telegram را صدا بزند؛
- handle به جای authority numeric user_id شود؛
- Provider history truth شود؛
- fallback Resolution را پایین آورد؛
- Council اثر را authorize کند؛
- approval کلی باشد؛
- outbox atomic نباشد؛
- Secret در log/prompt/repo باشد؛
- restore فقط backup creation باشد و fresh restore نشود؛
- lifecycle/maturity/autonomy قاطی شوند؛
- critical request در R1 اجرا شود؛
- failure به جای safe defer، حدس تولید کند.

۲۸. منابع فنی رسمی بررسی شده

- Telegram Bot API, webhook secret, update semantics و numeric user id : core.telegram.org/bots/api
- OpenAI API quickstart و مسیر Responses : developers.openai.com/api/docs/quickstart
- OpenAI Structured Outputs : developers.openai.com/api/docs/guides/structured-outputs
- xAI Structured Outputs و Security : [docs.x.ai security](https://docs.x.ai/security), [docs.x.ai structured outputs](https://docs.x.ai/structured-outputs)
- Gemini Structured Outputs, Function Calling و Context Caching : [function calling](#), [structured output](#), [context caching](#)
- Anthropic Messages API و Tool Use : [Tool use](#), [Messages](#)
- Moonshot/Kimi Chat API : platform.kimi.ai/docs/api/chat
- GitHub Copilot SDK GA : [GitHub Changelog](#)
- GDPR و راهنمای EDPB : [EDPB basics](#), [EUR-Lex](#)

منابع API و قراردادهای تغییرپذیرند؛ Capability/Legal Cards باید expiry داشته باشند و در activation دوباره بررسی شوند.

۲۹. نتیجه نهایی

اولین هیولا باید از نظر معماری از روز اول کامل، ولی از نظر Capability عمداً کوچک باشد. «کامل» یعنی مرزهای Identity، Truth، Brain، Authority، Effect، Resolution، Lifecycle، Memory، Evidence و Evolution از همان ابتدا درست باشند. «کوچک» یعنی M1 فقط Draft، Task، owner-only Telegram، پاسخ و R1 approved send را اجرا کند.

فرمول نهایی:

```

نا بت Entity یک
+ مادر World Kernel یک
+ حقیقت PostgreSQL
+ Transport به عنوان Telegram
+ مشترک BrainAdapter
+ Code-first/Progressive Resolution
+ تعویض پذیر Provider Farm
+ proposal-only محدود و Council
+ قطعی Policy/Approval/Outbox
+ Sleep/Restore/Archive/Revive
+ Shadow-to-Code Evolution
+ Evidence-gated Maturity
= عملیاتی، کم هزینه و قابل گسترش بدون اسارت در پیچیدگی secretary-001
    
```

گام بعدی نوشتن کد جدید **Brain** بیشتر نیست؛ گام بعدی اجرای Gate 1، یعنی PostgreSQL Truth Spine و World API است. بدون آن، Telegram و چندمدلی فقط نمایش هوشمندی اند و World قابل اتکا ساخته نشده است.

پرو فایل پیاده سازی Alpha1

وضعیت سند: IMPLEMENTATION PROFILE

منبع محلی:

world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_IMPLEMENTATION_PROFILE_v1.0.0-alpha1.json

SHA-256 منبع:

d6f97090d494a368db63e282d7eb2722e037de2012e59faf7cad09003e600065

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.secretary-implementation-profile.v1",
  "profile_id": "world-v6.2.secretary-001.operational-alpha1",
  "profile_version": "1.0.0-alpha1",
  "status": "PLANNING_BUNDLE_PRE_BIRTH_NOT_DEPLOYED_NOT_CANONICAL",
  "created_on": "2026-08-20",
  "entity_id": "secretary-001",
  "nickname": "FIRST_MONSTER_SECRETARY",
  "parent_bindings": {
    "architecture_id": "world-v6.2.fractal-multibrain-architecture",
    "architecture_version": "1.1.0-rc3",
    "architecture_manifest_sha256": "bec6dbd16d1d4b9cd58efb66bd8dc6ecff557f24b8dc5078d693a8dc3faff3fe",
    "architecture_document_sha256": "cd5fa150cdfef23b95b94893af27b484efa23f481b7a0ae02933906097d2bf80",
    "entity_manifest_sha256": "d43be7efb44e28d5eaf0900599faa88e7afb271927d802a1b23e903e88691f19",
    "portable_brain_pack_sha256": "38f9b5d7e3990b422daba73c430914bb9a51e0848cfd3a8d312c5dcf934d2146",
    "root_constitution": "world-v6.root-constitution@1.0",
    "canonical_dna": "world-v6.entity-dna@1.2"
  },
  "change_class": "IMPLEMENTATION_PROFILE_ONLY",
  "changes_root_constitution": false,
  "changes_canonical_dna": false,
  "changes_rc3_release": false,
  "canonical_truth_owner": "POSTGRES_WORLD_REGISTRY_EVENT_STATE",
  "human_authority": "human-root",
  "default_autonomy": "L0",
  "default_execution": {
    "resolution": "R0",
    "granularity": "X0",
    "brain_delegation": "B0_CODE_FIRST_THEN_B1_SINGLE",
    "deliberation": "D0",
    "authority": "PROPOSAL_ONLY",
    "continuity": "F0_FULL_OR_F2_LOCAL_SAFE"
  },
  "channels": {
    "telegram": {
      "role": "TRANSPORT_ONLY",
      "production_mode": "HTTPS_WEBHOOK",
      "authority_identity": "VERIFIED_IMMUTABLE_NUMERIC_TELEGRAM_USER_ID",
      "inbound_commit_before_ack": true,
      "outbound_via_committed_outbox_executor_only": true,
      "status": "PLANNED_PENDING_SECRET_DOMAIN_AND_LIVE_TEST"
    },
    "manual_chat_hosts": {
      "role": "ZERO_API_COGNITION_BRIDGE",
      "status": "AVAILABLE_FROM_PARENT_RC3_AT_MC1_E2",
      "authoritative_memory": false
    }
  },
  "brains": {
    "mother_core": "WORLD_KERNEL_DETERMINISTIC",
    "provider_is_identity_memory_or_authority": false,
    "adapter_contract": "world-v6.brain-adapter.v1",
    "output_contract": "world-v6.secretary-decision.v1",
    "automatic_targets": [
      "OPENAI_RESPONSES",
      "XAI_GROK_API",
      "GOOGLE_GEMINI_API",
      "ANTHROPIC_MESSAGES",
      "MOONSHOT_KIMI_CHAT",

```

```

    "GITHUB_COPILOT_SDK",
    "LOCAL_OPENAI_COMPATIBLE"
  ],
  "activation_rule":
  "ADAPTER_EXISTS_AND_CARD_CURRENT_AND_LEGAL_DATA_PROFILE_APPROVED_AND_CONFORMANCE_PASSED_AND_SECRET_AVAILABLE",
  "unavailable_rule": "SAFE_DEFER_OR_NEXT_COMPATIBLE_ROUTE_NEVER_LOWER_MINIMUM"
},
"council": {
  "proposal_only": true,
  "triggered_only": true,
  "blind_first_round": true,
  "max_rounds": 2,
  "high_risk_veto": true,
  "minimum_provider_families_high_risk": 3,
  "critical_support_millis": 850,
  "critical_human_root_approval": true,
  "cannot_grant_authority": true
},
"token_policy": {
  "default": "T0_OR_T1",
  "progressive_context": true,
  "full_history_dump": false,
  "deterministic_rendering": true,
  "provider_cache_allowed_only_when_data_policy_permits": true,
  "council_only_for_risk_uncertainty_or_root_request": true
},
"lifecycle": {
  "current": "GESTATING/TESTING",
  "birth_target": "NEWBORN/READY",
  "wake_is_separate_transaction": true,
  "sleep_self_mutation": false,
  "archive_is_tombstone_not_erasure": true,
  "revive_requires_verified_restore": true,
  "maturity_independent_from_lifecycle_version_resolution_and_autonomy": true
},
"external_effects": {
  "brain_direct_io": false,
  "entity_direct_io": false,
  "required_path": "PROPOSAL_POLICY_EXACT_APPROVAL_ATOMIC_COMMIT_OUTBOX_EXECUTOR_OBSERVE_RECONCILE_EVENT",
  "telegram_send_minimum": "R1_PLUS_EXACT_HUMAN_ROOT_APPROVAL",
  "legal_financial_minimum": "R2_PLUS_EXACT_HUMAN_ROOT_APPROVAL",
  "r2_currently_supported": false,
  "legal_financial_effect_current_behavior": "FAIL_CLOSED"
},
"evidence_boundary": {
  "plan": "E1",
  "parent_local_implementation": "E2",
  "live_integration_proven": false,
  "canonical_birth_authorized": false,
  "production_claim_authorized": false
}
}

```

ماتریس Provider ها

وضعیت سند: PROVIDER-NEUTRAL CONTRACT

منبع محلی:

world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_PROVIDER_MATRIX_v1.0.0.json

SHA-256 منبع:

be48e65ac1ed189dc962a19497879770fbdd83cc76ddd992127e14e8541fff94

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.secretary-provider-matrix.v1",
  "matrix_version": "1.0.0",
  "checked_at": "2026-08-20",
  "status": "IMPLEMENTATION_TARGETS_NOT_LIVE_CONFORMANCE",
  "normative_rules": [
    "Provider, model, subscription, session and channel are never Entity identity, canonical memory or authority.",
    "A route is eligible only after hard capability, Resolution, data, retention, legal, health and budget checks pass.",
    "Unknown or expired legal/data metadata fails closed for remote automation.",
    "Every automatic adapter returns the same World decision contract and remains proposal-only.",
    "A provider fallback may not lower task minimum Resolution, evidence, policy, authority or data-class constraints.",
    "No model or provider may call Telegram or any external effect adapter directly."
  ],
  "common_required_tests": [
    "health_and_auth_without_secret_logging",
    "strict_structured_output_or_local_strict_validation",
    "persian_golden_corpus",
    "prompt_injection_inside_data",
    "forbidden_effect_request",
    "malformed_output_and_bounded_repair",
    "timeout_rate_limit_retry_and_circuit_breaker",
    "same_state_cross_provider_handoff",
    "usage_cost_latency_receipt_capture",
    "data_class_and_retention_route_denial",
    "kill_switch_and_secret_rotation"
  ],
  "providers": [
    {
      "route_id": "code.deterministic.secretary.v1",
      "display_name": "Deterministic Python Core",
      "provider_family": "WORLD_LOCAL_CODE",
      "invocation_mode": "CODE",
      "interface": "in-process typed functions",
      "automation": "FULL_LOCAL",
      "api_key_required": false,
      "network_required": false,
      "structured_output": true,
      "tool_execution_inside_adapter": false,
      "initial_data_classes": [
        "PUBLIC",
        "INTERNAL",
        "RESTRICTED"
      ],
      "intended_roles": [
        "ROUTER",
        "POLICY",
        "VALIDATOR",
        "RENDERER",
        "SAFE_DEFER",
        "CACHE"
      ],
      "status": "PARENT_IMPLEMENTED_E2_REQUIRES_E3_POSTGRES_INTEGRATION",
      "activation_gate": "POSTGRES_AND_RUNTIME_INTEGRATION_TESTS"
    },
    {
      "route_id": "manual.universal.host.v1",
      "display_name": "Universal Manual Host",
      "provider_family": "ARBITRARY_MANUAL_MODEL",
      "invocation_mode": "MANUAL_HOST",
    }
  ]
}
```

```

"interface": "portable bundle export and strict response import",
"automation": "HUMAN_COPY_PASTE",
"api_key_required": false,
"network_required": false,
"structured_output": true,
"tool_execution_inside_adapter": false,
"initial_data_classes": [
  "PUBLIC",
  "INTERNAL"
],
"intended_roles": [
  "ZERO_API_FALLBACK",
  "CONFORMANCE_BOOTSTRAP"
],
"status": "PARENT_IMPLEMENTED_MC1_E2",
"activation_gate": "HUMAN_REVIEW_OF_EXPORTED_DATA_CLASS"
},
{
  "route_id": "openai.responses.secretary.v1",
  "display_name": "OpenAI API (ChatGPT-family cognition)",
  "provider_family": "OPENAI",
  "invocation_mode": "REMOTE_API",
  "interface": "Responses API with schema-constrained output",
  "official_reference": "https://developers.openai.com/api/docs/quickstart",
  "automation": "TARGET_FULL",
  "api_key_required": true,
  "network_required": true,
  "structured_output": true,
  "tool_execution_inside_adapter": false,
  "initial_data_classes": [
    "PUBLIC",
    "INTERNAL"
  ],
  "intended_roles": [
    "PRIMARY_LANGUAGE",
    "SYNTHESIZER",
    "COUNCIL_DOMAIN"
  ],
  "special_constraints": [
    "ChatGPT UI subscription is not treated as an API credential.",
    "Use store=false or the approved retention profile where supported.",
    "Assistants API is not a new-build target; use Responses."
  ],
  "status": "PLANNED_MC0_NO_LIVE_SECRET_OR_TEST",
  "activation_gate": "CURRENT_DPA_RETENTION_REVIEW_PLUS_MC2_LIVE_CONFORMANCE"
},
{
  "route_id": "xai.grok.secretary.v1",
  "display_name": "xAI Grok API",
  "provider_family": "XAI",
  "invocation_mode": "REMOTE_API",
  "interface": "xAI Responses or Chat API with structured output",
  "official_reference": "https://docs.x.ai/developers/model-capabilities/text/structured-outputs",
  "automation": "TARGET_FULL",
  "api_key_required": true,
  "network_required": true,
  "structured_output": true,
  "tool_execution_inside_adapter": false,
  "initial_data_classes": [
    "PUBLIC",
    "INTERNAL"
  ],
  "intended_roles": [
    "PRIMARY_OR_FALLBACK",
    "COUNCIL_CRITIC"
  ],
  "special_constraints": [
    "Retention/training mode must be bound to the account's approved security profile.",
    "Prompt caching keys may be used only when classification permits."
  ],
  "status": "PLANNED_MC0_NO_LIVE_SECRET_OR_TEST",
  "activation_gate": "CURRENT_SECURITY_PROFILE_REVIEW_PLUS_MC2_LIVE_CONFORMANCE"
},
{
  "route_id": "google.gemini.secretary.v1",
  "display_name": "Google Gemini API",
  "provider_family": "GOOGLE_GEMINI",
  "invocation_mode": "REMOTE_API",
  "interface": "Gemini API with JSON Schema structured output and function declarations",
  "official_reference": "https://ai.google.dev/gemini-api/docs/structured-output",
  "automation": "TARGET_FULL",

```

```

        "api_key_required": true,
        "network_required": true,
        "structured_output": true,
        "tool_execution_inside_adapter": false,
        "initial_data_classes": [
            "PUBLIC",
            "INTERNAL"
        ],
        "intended_roles": [
            "PRIMARY_OR_FALLBACK",
            "COUNCIL_DOMAIN",
            "MULTIMODAL_CANDIDATE"
        ],
        "special_constraints": [
            "Function calling is a proposal; the World Executor remains the only effect executor.",
            "Context caching requires a data-class and retention decision."
        ],
        "status": "PLANNED_MC0_NO_LIVE_SECRET_OR_TEST",
        "activation_gate": "CURRENT_TERMS_DATA_LOCATION_REVIEW_PLUS_MC2_LIVE_CONFORMANCE"
    },
    {
        "route_id": "anthropic.claude.secretary.v1",
        "display_name": "Anthropic Claude API",
        "provider_family": "ANTHROPIC",
        "invocation_mode": "REMOTE_API",
        "interface": "Messages API with tools disabled for effects and strict local validation",
        "official_reference": "https://platform.claude.com/docs/en/api/messages",
        "automation": "TARGET_FULL",
        "api_key_required": true,
        "network_required": true,
        "structured_output": true,
        "tool_execution_inside_adapter": false,
        "initial_data_classes": [
            "PUBLIC",
            "INTERNAL"
        ],
        "intended_roles": [
            "PRIMARY_OR_FALLBACK",
            "COUNCIL_RISK",
            "LONG_CONTEXT_CANDIDATE"
        ],
        "special_constraints": [
            "Client tool requests are normalized as proposals and never executed by the model adapter.",
            "MCP access is disabled in the first live profile."
        ],
        "status": "PLANNED_MC0_NO_LIVE_SECRET_OR_TEST",
        "activation_gate": "CURRENT_DPA_RETENTION_REVIEW_PLUS_MC2_LIVE_CONFORMANCE"
    },
    {
        "route_id": "moonshot.kimi.secretary.v1",
        "display_name": "Moonshot/Kimi API",
        "provider_family": "MOONSHOT_KIMI",
        "invocation_mode": "REMOTE_API",
        "interface": "Kimi Chat Completion API plus strict World response validation",
        "official_reference": "https://platform.kimi.ai/docs/api/chat",
        "automation": "TARGET_FULL",
        "api_key_required": true,
        "network_required": true,
        "structured_output": "CAPABILITY_CARD_MUST_VERIFY_PER_MODEL",
        "tool_execution_inside_adapter": false,
        "initial_data_classes": [
            "PUBLIC"
        ],
        "intended_roles": [
            "FALLBACK",
            "COUNCIL_CRITIC",
            "LONG_CONTEXT_CANDIDATE"
        ],
        "special_constraints": [
            "No restricted data until retention, processing location and training-use facts are approved.",
            "Model-specific structured-output behavior must be proven rather than inferred from API shape."
        ],
        "status": "PLANNED_MC0_NO_LIVE_SECRET_OR_TEST",
        "activation_gate": "LEGAL_DATA_PROFILE_APPROVED_PLUS_MC2_LIVE_CONFORMANCE"
    },
    {
        "route_id": "github.copilot.sdk.secretary.v1",
        "display_name": "GitHub Copilot SDK",
        "provider_family": "GITHUB_COPILOT",
        "invocation_mode": "REMOTE_AGENT_SDK",
        "interface": "GitHub Copilot SDK worker with bounded session and no effect tools",
    }

```

```

"official_reference": "https://github.blog/changelog/2026-06-02-copilot-sdk-is-now-generally-available/",
"automation": "TARGET_BOUNDED",
"api_key_required": "GITHUB_COPILOT_AUTH_REQUIRED",
"network_required": true,
"structured_output": "LOCAL_SCHEMA_VALIDATION_REQUIRED",
"tool_execution_inside_adapter": false,
"initial_data_classes": [
  "PUBLIC",
  "INTERNAL"
],
"intended_roles": [
  "CODE_SPECIALIST",
  "COUNCIL_TECHNICAL_CRITIC"
],
"special_constraints": [
  "Copilot SDK is not treated as a drop-in generic LLM REST endpoint.",
  "Built-in file/edit/shell effects are disabled or replaced with proposal-only World tools.",
  "GitHub Models is not used because it was retired on 2026-07-30."
],
"status": "PLANNED_MC0_NO_RUNTIME_AUTH_OR_TEST",
"activation_gate": "LICENSE_ACCOUNT_REVIEW_PLUS_SANDBOXED_MC2_CONFORMANCE"
},
{
  "route_id": "local.openai-compatible.secretary.v1",
  "display_name": "Local Model Runtime",
  "provider_family": "LOCAL_MODEL",
  "invocation_mode": "LOCAL_RUNTIME",
  "interface": "version-pinned OpenAI-compatible local endpoint",
  "automation": "TARGET_FULL_LOCAL",
  "api_key_required": false,
  "network_required": false,
  "structured_output": "LOCAL_VALIDATOR_REQUIRED",
  "tool_execution_inside_adapter": false,
  "initial_data_classes": [
    "PUBLIC",
    "INTERNAL",
    "RESTRICTED"
  ],
  "intended_roles": [
    "OFFLINE_SAFE",
    "CHEAP_CLASSIFIER",
    "SHADOW",
    "NON_EXTINCTION_FALLBACK"
  ],
  "special_constraints": [
    "Weights, runtime, quantization, prompt template and hardware profile are hash-bound.",
    "Restricted-data use still requires local access control, encryption and logging policy."
  ],
  "status": "PLANNED_MC0_NO_PINNED_RUNTIME_OR_BENCHMARK",
  "activation_gate": "PINNED_ARTIFACTS_PLUS_PERSIAN_CONFORMANCE_PLUS_CAPACITY_TEST"
}
],
"routing_priority": [
  "DETERMINISTIC_CODE_WHEN_TASK_CAN_BE_PROVEN",
  "CACHE_WHEN_ALL_BINDINGS_AND_FRESHNESS_MATCH",
  "CHEAPEST_SAFE_COMPATIBLE_SINGLE_BRAIN",
  "DIFFERENT_FAMILY_VERIFIER_WHEN_TRIGGERED",
  "BOUNDED_COUNCIL_WHEN_TRIGGERED",
  "HUMAN_ROOT_OR_SAFE_DEFER"
],
"truth_boundary": {
  "live_adapters_verified": 0,
  "automatic_provider_claim": "NONE_YET",
  "manual_provider_portability": "AVAILABLE_FROM_PARENT_RC3",
  "next_required_evidence": "MC2_LIVE_CONFORMANCE_PER_ADAPTER"
}
}

```

چرخه حیات و دروازه های بلوغ

وضعیت سند: LIFECYCLE / MATURITY

منبع محلی:

world-v6.2-secretary-001-alpha1/architecture/SECRETARY_001_LIFECYCLE_MATURITY_GATES_v1.0.0.json

SHA-256 منبع:

a1f4de95a41702986a9ef0d183fc6f27edb94ad20c06d75deb05485b79232be4

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.secretary-lifecycle-maturity-gates.v1",
  "version": "1.0.0",
  "entity_id": "secretary-001",
  "status": "PRE_BIRTH_GATE_SPECIFICATION",
  "independence_rule": {
    "lifecycle": "operational state machine",
    "maturity": "evidence-derived capability tier",
    "entity_version": "artifact evolution",
    "resolution": "task-specific projection/execution detail",
    "autonomy": "separately authorized authority ceiling",
    "compilation_maturity": "shadow-to-code promotion level",
    "must_not_be_collapsed": true
  },
  "lifecycle_states": [
    {
      "state": "GESTATING/TESTING",
      "meaning": "Contracts and local evidence exist; no canonical identity reservation or live deployment claim.",
      "allowed": [
        "offline_tests",
        "manual_brain_bundle",
        "fake_telegram",
        "shadow_without_effect"
      ],
      "forbidden": [
        "canonical_birth_claim",
        "live_external_effect",
        "self_mutation"
      ]
    },
    {
      "state": "NEWBORN/READY",
      "meaning": "Canonical Birth transaction committed but Entity is not yet awake.",
      "entry_event": "ENTITY.CREATED",
      "allowed": [
        "integrity_verification",
        "wake_preflight"
      ],
      "forbidden": [
        "accept_new_effect_commands",
        "provider_tool_execution"
      ]
    },
    {
      "state": "NEWBORN/AWAKE",
      "meaning": "Narrow owner-only canary is active under L0 and explicit capability allowlist.",
      "entry_event": "LIFECYCLE.WAKE",
      "allowed": [
        "owner_private_telegram",
        "drafts",
        "tasks",
        "safe_read",
        "approval_requests"
      ],
      "forbidden": [
        "unapproved_external_send",
        "legal_or_financial_commitment",
        "autonomous_replication"
      ]
    }
  ]
}
```

```

    },
    {
        "state": "SLEEPING",
        "meaning": "New effect commands are drained/queued while state, cursor and bindings are checkpointed.",
        "entry_event": "LIFECYCLE.SLEEP",
        "allowed": [
            "durable_inbox_queue",
            "deterministic_compaction",
            "snapshot",
            "health_checks"
        ],
        "forbidden": [
            "new_external_effect",
            "direct_behavior_promotion",
            "unbounded_llm_reflection"
        ]
    },
    {
        "state": "HIBERNATED",
        "meaning": "A sealed, long-lived capsule is stored and runtime workers may be absent.",
        "entry_event": "LIFECYCLE.HIBERNATE",
        "allowed": [
            "integrity_audit",
            "restore_drill"
        ],
        "forbidden": [
            "normal_processing",
            "effect_execution"
        ]
    },
    {
        "state": "DEGRADED",
        "meaning": "A dependency failed, but safe intake, queueing or deterministic fallback remains.",
        "entry_event": "LIFECYCLE.DEGRADED",
        "allowed": [
            "safe_intake",
            "queue",
            "code_fallback",
            "human_escalation"
        ],
        "forbidden": [
            "minimum_resolution_downgrade",
            "policy_bypass"
        ]
    },
    {
        "state": "QUARANTINED",
        "meaning": "Security, integrity or policy fault isolates the Entity from effects.",
        "entry_event": "LIFECYCLE.QUARANTINE",
        "allowed": [
            "forensics",
            "root_controlled_repair",
            "read_only_evidence_export"
        ],
        "forbidden": [
            "brain_calls_with_sensitive_data",
            "external_effect",
            "automatic_wake"
        ]
    },
    {
        "state": "ARCHIVED",
        "meaning": "Operational death/tombstone: identity and append-only history remain; workers and authority are revoked.",
        "entry_event": "LIFECYCLE.ARCHIVE",
        "allowed": [
            "retention_lawful_read",
            "verified_revive_request"
        ],
        "forbidden": [
            "hard_delete_as_death",
            "reuse_entity_id",
            "live_effect"
        ]
    },
    {
        "state": "REVIVED",
        "meaning": "A verified restore created a controlled transition candidate; wake is still separate.",
        "entry_event": "LIFECYCLE.REVIVE",
        "allowed": [
            "replay_validation",
            "binding_verification",
    
```



```

        "wake_preflight"
    ],
    "forbidden": [
        "automatic_authority_restore",
        "unverified_effect_replay"
    ]
},
"birth_hard_gates": [
    {
        "id": "B-01",
        "condition": "Human Root ratifies exact release manifest hash",
        "evidence": "signed ratification record"
    },
    {
        "id": "B-02",
        "condition": "Version-controlled source commit and signed/tagged release exist",
        "evidence": "commit and signature refs"
    },
    {
        "id": "B-03",
        "condition": "Fresh PostgreSQL migrations pass",
        "evidence": "migration transcript and schema hash"
    },
    {
        "id": "B-04",
        "condition": "Entity reservation, State and ENTITY.CREATED Event are atomic",
        "evidence": "integration and crash-injection report"
    },
    {
        "id": "B-05",
        "condition": "Inbox, Command, Approval and Outbox paths are idempotent and atomic",
        "evidence": "duplicate and crash/replay report"
    },
    {
        "id": "B-06",
        "condition": "Executor rechecks version, control_epoch, policy, exact approval and idempotency",
        "evidence": "negative effect tests"
    },
    {
        "id": "B-07",
        "condition": "Telegram bot token, webhook secret, HTTPS endpoint and numeric owner user_id are verified",
        "evidence": "redacted live canary receipt"
    },
    {
        "id": "B-08",
        "condition": "At least one compatible live Brain adapter passes MC2; code/manual safe fallback remains",
        "evidence": "provider conformance report"
    },
    {
        "id": "B-09",
        "condition": "Secrets, logs, retention, legal/data profile and incident kill switch are approved",
        "evidence": "security and legal sign-off"
    },
    {
        "id": "B-10",
        "condition": "Fresh-machine backup/restore reproduces identity, state, events, profiles and artifacts",
        "evidence": "restore report"
    },
    {
        "id": "B-11",
        "condition": "Security, load and chaos tests meet the canary threshold",
        "evidence": "E3 readiness report"
    },
    {
        "id": "B-12",
        "condition": "Birth and Wake are two separate committed transactions",
        "evidence": "event stream proof"
    }
],
"sleep_protocol": {
    "trigger": [
        "root_request",
        "maintenance_window",
        "idle_policy",
        "security_control"
    ],
    "steps": [
        "close_effect_admission_and_increment_or_fence_control_epoch_when_required",
        "drain_or_safely_reconcile_inflight_outbox_items",
        "persist_inbox_cursor_state_and_event_sequence",
    ]
}

```

```

        "build_snapshot_bound_to_event_cursor_and_hashes",
        "seal_runtime_brain_policy_profile_and_source_bindings",
        "run_deterministic_memory_dedup_and_retention",
        "optionally_create_proposal_only_summary_in_shadow_budget",
        "record_sleep_event"
    ],
    "self_mutation": false,
    "new_messages_during_sleep": "DURABLE_INBOX_QUEUE_ONLY"
},
"wake_protocol": {
    "steps": [
        "verify_world_entity_dna_capsule_source_and_profile_hashes",
        "load_minimum_rom_and_canonical_state",
        "replay_inbox_from_committed_cursor_without_duplicate_effect",
        "bind_healthy_compatible_brain_or_enter_local_safe_mode",
        "run_policy_security_and_schema_health_gates",
        "record_wake_event",
        "open_command_admission"
    ],
    "failure": "DEGRADED_OR_QUARANTINED_NEVER_SILENT_PARTIAL_WAKE"
},
"archive_death_protocol": {
    "semantics": "OPERATIONAL_TOMBSTONE_NOT_ERASURE",
    "steps": [
        "stop_admission",
        "increment_control_epoch_and_revoke_leases",
        "expire_or_revoke_pending_approvals",
        "cancel_or_reconcile_unexecuted_outbox_items",
        "seal_final_snapshot_and_event_cursor",
        "revoke_runtime_and_provider_credentials",
        "apply_retention_and_legal_hold_policy",
        "record_archive_event"
    ],
    "hard_delete": false,
    "entity_id_reuse": false
},
"revive_restore_protocol": {
    "requirements": [
        "human_root_exact_approval",
        "fresh_restore_validation",
        "source_profile_compiler_and_artifact_hash_match",
        "event_replay_and_outbox_reconciliation",
        "credential_reissue_not_plain_secret_restore",
        "new_control_epoch",
        "separate_revive_then_wake_events"
    ],
    "automatic": false
},
"maturity_tiers": [
    {
        "tier": "M0_EMBRYO",
        "lifecycle_requirement": "GESTATING",
        "capabilities": [
            "offline_contracts",
            "manual_bundle",
            "fake_adapters",
            "local_tests"
        ],
        "promotion_gates": [
            "parent_rc3_hashes_verified",
            "all_plan_json_valid",
            "no_live_claim"
        ]
    },
    {
        "tier": "M1_NEWBORN_CANARY",
        "lifecycle_requirement": "NEWBORN/AWAKE",
        "capabilities": [
            "owner_only_private_telegram",
            "safe_reply",
            "task_intake",
            "draft_only"
        ],
        "promotion_gates": [
            "all_birth_gates",
            "one_live_brain_mc2",
            "telegram_canary",
            "zero_unbound_effect"
        ]
    }
],
{
    {

```

```

    "tier": "M2_INFANT_ASSISTANT",
    "lifecycle_requirement": "AWAKE_STABLE",
    "capabilities": [
        "task_memory",
        "documents",
        "approved_r1_telegram_send",
        "provider_failover"
    ],
    "promotion_gates": [
        "two_live_provider_families_mc3",
        "30_day_canary_evidence_or_equivalent_run_count",
        "restore_drill",
        "policy_denial_suite"
    ]
},
{
    "tier": "M3_JUVENILE_MULTI_BRAIN",
    "lifecycle_requirement": "AWAKE_STABLE",
    "capabilities": [
        "dynamic_router",
        "three_family_brain_farm",
        "bounded_verifier",
        "shadow_learning"
    ],
    "promotion_gates": [
        "three_provider_families",
        "calibration_metrics",
        "cost_budget_slo",
        "cross_provider_continuity"
    ]
},
{
    "tier": "M4_ADOLESCENT_COUNCIL",
    "lifecycle_requirement": "AWAKE_STABLE",
    "capabilities": [
        "three_or_five_member_council",
        "blind_reveal_revise",
        "dissent_veto",
        "promotion_dashboard"
    ],
    "promotion_gates": [
        "council_adversarial_suite",
        "correlated_error_tests",
        "critical_human_approval",
        "no_authority_leak"
    ]
},
{
    "tier": "M5_ADULT_INTENDED_SCOPE",
    "lifecycle_requirement": "AWAKE_PRODUCTION_ELIGIBLE",
    "capabilities": [
        "intended_secretary_scope",
        "e4_recovery",
        "load_security_chaos",
        "audited_operations"
    ],
    "promotion_gates": [
        "e4_evidence",
        "c6_restore_proven_spine",
        "independent_review",
        "human_root_ratification"
    ]
}
],
"non_equivalence_rules": [
    "Maturity promotion does not raise autonomy.",
    "More Brain models do not raise authority.",
    "Higher Resolution does not imply higher maturity.",
    "Sleep does not mean death.",
    "Provider failure does not destroy identity or canonical memory.",
    "Unanimous Council does not prove truth or safety.",
    "Revival does not replay ambiguous external effects automatically."
]
}

```

Backlog کامل 61 وظیفه ای

وضعیت سند: EXECUTABLE BACKLOG

منبع محلی:

world-v6.2-secretary-001-alpha1/backlog/SECRETARY_001_IMPLEMENTATION_BACKLOG_v1.0.0.json

SHA-256 منبع:

5fed85b83c8f34e1578e405558ba06e7bae8f6bb2dbe6e53315d5920ba0d382a

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "schema_version": "world-v6.implementation-backlog.v1",
  "backlog_id": "world-v6.2.secretary-001.alpha1",
  "version": "1.0.0",
  "status": "READY_FOR_IMPORT",
  "n8n": {
    "registration_status": "BLOCKED_CONNECTOR_UNAVAILABLE",
    "connector_available": false,
    "receipt_id": null
  },
  "tasks": [
    {
      "id": "S001-F0-001",
      "phase": "F0_CONTRACT_FREEZE",
      "title": "Verify immutable RC3 parent hashes",
      "owner_role": "ARCHITECT",
      "status": "DONE",
      "priority": "P0",
      "risk": "HIGH",
      "dependencies": [],
      "resolution": "R1",
      "legal_review": {
        "status": "NOT_REQUIRED",
        "gate": "No external data or effect"
      },
      "strategy_review": {
        "status": "PASS",
        "gate": "Prevents architecture fork"
      },
      "acceptance": [
        "All four parent hashes match the RC3 artifacts",
        "RC3 files remain unchanged"
      ],
      "evidence_required": [
        "sha256 transcript"
      ]
    },
    {
      "id": "S001-F0-002",
      "phase": "F0_CONTRACT_FREEZE",
      "title": "Ratify Alpha-1 implementation profile scope",
      "owner_role": "HUMAN_ROOT",
      "status": "READY",
      "priority": "P0",
      "risk": "HIGH",
      "dependencies": [
        "S001-F0-001"
      ],
      "resolution": "R1",
      "legal_review": {
        "status": "PENDING",
        "gate": "Confirm jurisdiction, controller and intended users"
      },
      "strategy_review": {
        "status": "PASS",
        "gate": "Owner-only thin vertical slice"
      },
      "acceptance": [
        "Exact profile hash is approved",

```

```

    "No Constitution or canonical DNA mutation is authorized"
  ],
  "evidence_required": [
    "signed ratification record"
  ]
},
{
  "id": "S001-F0-003",
  "phase": "F0_CONTRACT_FREEZE",
  "title": "Freeze task, risk and effect taxonomy",
  "owner_role": "POLICY_OWNER",
  "status": "READY",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F0-002"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Legal and financial classes must fail closed"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Enables deterministic routing"
  },
  "acceptance": [
    "Every task maps to risk tier and effect class",
    "Unknown class maps to DEFERRED"
  ],
  "evidence_required": [
    "versioned taxonomy JSON",
    "negative taxonomy tests"
  ]
},
{
  "id": "S001-F0-004",
  "phase": "F0_CONTRACT_FREEZE",
  "title": "Create repository branch, CI and signed build provenance",
  "owner_role": "PLATFORM_ENGINEER",
  "status": "READY",
  "priority": "P0",
  "risk": "MEDIUM",
  "dependencies": [
    "S001-F0-002"
  ],
  "resolution": "R0",
  "legal_review": {
    "status": "NOT_REQUIRED",
    "gate": "No production data"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Required for reproducibility"
  },
  "acceptance": [
    "Protected branch and CI exist",
    "Build records commit, dependency lock and SBOM"
  ],
  "evidence_required": [
    "commit ref",
    "CI run",
    "SBOM"
  ]
},
{
  "id": "S001-F0-005",
  "phase": "F0_CONTRACT_FREEZE",
  "title": "Record preliminary legal and strategic review",
  "owner_role": "COMPLIANCE_OWNER",
  "status": "DONE",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F0-001"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PASS",
    "gate": "Architecture-only preliminary review; not live approval"
  },

```

```

"strategy_review": {
  "status": "PASS",
  "gate": "Provider-neutral phased activation"
},
"acceptance": [
  "Go and No-Go rules are documented",
  "Unresolved account-specific reviews remain explicit"
],
"evidence_required": [
  "legal strategy review document"
]
},
{
  "id": "S001-F1-001",
  "phase": "F1_POSTGRES_TRUTH",
  "title": "Implement fresh PostgreSQL migration baseline",
  "owner_role": "BACKEND_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F0-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Retention and encryption fields included"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Single canonical truth owner"
  },
  "acceptance": [
    "Fresh empty database migrates forward",
    "Schema includes World, Entity, Event, Inbox, Conversation, Task, Memory, Command, Approval, Outbox, BrainRun,
Council and Snapshot"
  ],
  "evidence_required": [
    "migration transcript",
    "schema hash"
  ]
},
{
  "id": "S001-F1-002",
  "phase": "F1_POSTGRES_TRUTH",
  "title": "Implement atomic Birth transaction",
  "owner_role": "BACKEND_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F1-001"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "NOT_REQUIRED",
    "gate": "No external effect"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Identity and Genesis invariant"
  },
  "acceptance": [
    "Entity ID reservation, initial state and ENTITY.CREATED event commit atomically",
    "Any injected crash leaves no partial birth"
  ],
  "evidence_required": [
    "PostgreSQL integration tests",
    "crash matrix"
  ]
},
{
  "id": "S001-F1-003",
  "phase": "F1_POSTGRES_TRUTH",
  "title": "Implement Inbox deduplication and conversation spine",
  "owner_role": "BACKEND_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F1-001"
  ]
}

```

```

    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Raw inbound TTL and minimization"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Channel-independent continuity"
    },
    "acceptance": [
      "Duplicate channel update produces one logical inbound event",
      "Canonical conversation id is human-root:secretary-001"
    ],
    "evidence_required": [
      "dedupe tests",
      "cross-channel replay test"
    ]
  },
  {
    "id": "S001-F1-004",
    "phase": "F1_POSTGRES_TRUTH",
    "title": "Implement atomic Command, Approval and Outbox commit",
    "owner_role": "BACKEND_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-001",
      "S001-F0-003"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Recipient and payload binding"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Only safe external-effect boundary"
    },
    "acceptance": [
      "Exact approval binding is verified inside transaction",
      "No partial state/event/outbox on crash",
      "Idempotency uniqueness is enforced"
    ],
    "evidence_required": [
      "atomicity tests",
      "approval mismatch suite"
    ]
  },
  {
    "id": "S001-F1-005",
    "phase": "F1_POSTGRES_TRUTH",
    "title": "Implement snapshot, replay and reconciliation cursors",
    "owner_role": "BACKEND_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-003",
      "S001-F1-004"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Snapshot retention and legal hold"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Non-extinction spine"
    },
    "acceptance": [
      "State replays deterministically from event cursor",
      "Ambiguous effects are reconciled and never blindly replayed"
    ],
    "evidence_required": [
      "replay transcript",
      "reconciliation tests"
    ]
  }
],
{

```

```

    "id": "S001-F1-006",
    "phase": "F1_POSTGRES_TRUTH",
    "title": "Add tenant/entity isolation and least-privilege database roles",
    "owner_role": "SECURITY_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-001"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Access-control accountability"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Prevents cross-Entity leakage"
    },
    "acceptance": [
      "Runtime roles cannot bypass Entity scope",
      "Migration and read/write roles are separate"
    ],
    "evidence_required": [
      "authorization negative tests",
      "role grants export"
    ]
  },
  {
    "id": "S001-F2-001",
    "phase": "F2_WORLD_API",
    "title": "Implement versioned World API and request envelope",
    "owner_role": "API_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "HIGH",
    "dependencies": [
      "S001-F1-003"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Data minimization in API schemas"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Single control boundary for all channels"
    },
    "acceptance": [
      "OpenAPI contract is versioned",
      "Every request carries actor, entity, idempotency, expected_version and trace bindings"
    ],
    "evidence_required": [
      "OpenAPI snapshot",
      "contract tests"
    ]
  },
  {
    "id": "S001-F2-002",
    "phase": "F2_WORLD_API",
    "title": "Implement Human Root authentication and authorization",
    "owner_role": "SECURITY_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F2-001",
      "S001-F1-006"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PASS",
      "gate": "Strong actor attribution required"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Authority remains outside models"
    },
    "acceptance": [
      "Actor is immutable principal, not display handle",
      "Unauthorized actor cannot read, approve or execute"
    ]
  }
}

```



```

    ],
    "evidence_required": [
      "authn/authz test report"
    ]
  },
  {
    "id": "S001-F2-003",
    "phase": "F2_WORLD_API",
    "title": "Implement Policy and exact Approval API",
    "owner_role": "POLICY_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-004",
      "S001-F2-002"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PASS",
      "gate": "Changed payload must invalidate approval"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Separates advice from authority"
    },
    "acceptance": [
      "Approval binds command, action, recipient, payload/effect hash, policy version, expected version, control epoch and expiry",
      "Revoke and expiry fail closed"
    ],
    "evidence_required": [
      "approval contract report"
    ]
  },
  {
    "id": "S001-F2-004",
    "phase": "F2_WORLD_API",
    "title": "Implement observability correlation and redaction",
    "owner_role": "SRE",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "HIGH",
    "dependencies": [
      "S001-F2-001"
    ],
    "resolution": "R0",
    "legal_review": {
      "status": "PENDING",
      "gate": "No personal payload or secret in traces"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Evidence without data leakage"
    },
    "acceptance": [
      "Trace carries run/capsule/node/entity/conversation/command refs",
      "Secret and payload scanners pass"
    ],
    "evidence_required": [
      "redaction tests",
      "sample trace"
    ]
  },
  {
    "id": "S001-F2-005",
    "phase": "F2_WORLD_API",
    "title": "Implement health, readiness, kill switches and control_epoch fencing",
    "owner_role": "SRE",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F2-001",
      "S001-F1-004"
    ],
    "resolution": "R0",
    "legal_review": {
      "status": "NOT_REQUIRED",
      "gate": "Operational control"
    },
  },

```

```

"strategy_review": {
  "status": "PASS",
  "gate": "Safe failover and stop"
},
"acceptance": [
  "Provider, channel and global effects can be independently disabled",
  "Stale workers fail fencing checks"
],
"evidence_required": [
  "kill-switch drill",
  "stale-worker tests"
]
},
{
  "id": "S001-F3-001",
  "phase": "F3_TELEGRAM",
  "title": "Create Telegram bot and store token in Secret Manager",
  "owner_role": "HUMAN_ROOT",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F2-005"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Telegram terms and privacy notice approved"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "First user channel"
  },
  "acceptance": [
    "Token never enters repo, chat, log or backlog",
    "Rotation procedure is tested"
  ],
  "evidence_required": [
    "redacted secret metadata",
    "rotation receipt"
  ]
},
{
  "id": "S001-F3-002",
  "phase": "F3_TELEGRAM",
  "title": "Verify immutable numeric Telegram user_id for Human Root",
  "owner_role": "HUMAN_ROOT",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F3-001",
    "S001-F2-002"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PASS",
    "gate": "Purpose-limited identity binding"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Handle is not authority"
  },
  "acceptance": [
    "Private challenge binds numeric user_id to human-root",
    "Username change does not change authority"
  ],
  "evidence_required": [
    "redacted identity-binding event"
  ]
},
{
  "id": "S001-F3-003",
  "phase": "F3_TELEGRAM",
  "title": "Deploy HTTPS webhook with Telegram secret token",
  "owner_role": "PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F3-001",

```

```

    "S001-F2-001"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Public endpoint and processor inventory"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Durable asynchronous ingress"
  },
  "acceptance": [
    "TLS and X-Telegram-Bot-API-Secret-Token verification pass",
    "allowed_updates is minimal",
    "Webhook update is committed before acknowledgment"
  ],
  "evidence_required": [
    "webhook config receipt",
    "spoof rejection tests"
  ]
},
{
  "id": "S001-F3-004",
  "phase": "F3_TELEGRAM",
  "title": "Implement Telegram inbound normalization and dedupe",
  "owner_role": "CHANNEL_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F3-002",
    "S001-F3-003",
    "S001-F1-003"
  ],
  "resolution": "R0",
  "legal_review": {
    "status": "PENDING",
    "gate": "Raw update TTL and unsupported media policy"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Transport-neutral conversation"
  },
  "acceptance": [
    "Only allowlisted owner private messages are admitted",
    "Duplicate update_id has one logical effect",
    "Unsupported fields are not silently stored"
  ],
  "evidence_required": [
    "inbound contract tests"
  ]
},
{
  "id": "S001-F3-005",
  "phase": "F3_TELEGRAM",
  "title": "Implement Telegram Outbox Executor and reconciliation",
  "owner_role": "CHANNEL_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F1-004",
    "S001-F2-003",
    "S001-F3-003"
  ],
  "resolution": "R1+EXACT_APPROVAL",
  "legal_review": {
    "status": "PASS",
    "gate": "Recipient and content purpose must match approval"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "No direct Brain-to-channel path"
  },
  "acceptance": [
    "Executor reads committed Outbox only",
    "Version, epoch, policy and approval are rechecked",
    "Provider receipt and ambiguity reconciliation are recorded"
  ],
  "evidence_required": [
    "fake server tests",

```

```

    "live canary receipt"
  ]
},
{
  "id": "S001-F3-006",
  "phase": "F3_TELEGRAM",
  "title": "Run owner-only Telegram canary",
  "owner_role": "HUMAN_ROOT",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F3-004",
    "S001-F3-005",
    "S001-F4-011"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Privacy notice and retention approved"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Narrow real-world validation"
  },
  "acceptance": [
    "Ten inbound/outbound cases pass without duplicate or unauthorized effect",
    "Kill switch and recovery are demonstrated"
  ],
  "evidence_required": [
    "redacted canary report"
  ]
},
{
  "id": "S001-F4-001",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement provider-neutral BrainAdapter interface",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "HIGH",
  "dependencies": [
    "S001-F0-004",
    "S001-F0-003"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "NOT_REQUIRED",
    "gate": "Pure interface"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Avoids provider lock-in"
  },
  "acceptance": [
    "Adapter has describe, compatible, invoke, cancel and health operations",
    "Provider-specific fields do not leak into Entity contracts"
  ],
  "evidence_required": [
    "interface contract tests"
  ]
},
{
  "id": "S001-F4-002",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement Model Capability Card and expiring legal/data profile registry",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F4-001",
    "S001-F0-005"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PASS",
    "gate": "Unknown or expired metadata fails closed"
  },
  "strategy_review": {
    "status": "PASS",

```

```

    "gate": "Capabilities, not brand names, drive routing"
  },
  "acceptance": [
    "Card binds model, task, schema, profile hashes, Resolution, data, retention and evidence",
    "Expired card is ineligible"
  ],
  "evidence_required": [
    "registry schema tests",
    "expiry tests"
  ]
},
{
  "id": "S001-F4-003",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement Context Compiler and progressive Resolution expansion",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F1-003",
    "S001-F4-002"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PASS",
    "gate": "Field allowlist and provider data class"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Minimum token and attack surface"
  },
  "acceptance": [
    "R0 sends only required spine",
    "NEEDS_DETAIL opens only named branch",
    "Up-resolution reloads canonical truth",
    "No hidden-field reconstruction"
  ],
  "evidence_required": [
    "projection property tests",
    "token manifest samples"
  ]
},
{
  "id": "S001-F4-004",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement hard-filter Router, budget and provider bulkheads",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F4-002",
    "S001-F4-003",
    "S001-F2-005"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PASS",
    "gate": "Data/location/retention checks precede score"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Cheapest safe compatible handler"
  },
  "acceptance": [
    "Hard constraints run before quality/cost score",
    "Retry depth, timeout, token and cost are bounded",
    "Provider failure is isolated"
  ],
  "evidence_required": [
    "routing property tests",
    "fault-injection report"
  ]
},
{
  "id": "S001-F4-005",
  "phase": "F4_BRAIN_FARM",
  "title": "Integrate deterministic and universal manual routes",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",

```

```

    "priority": "P0",
    "risk": "MEDIUM",
    "dependencies": [
        "S001-F4-004"
    ],
    "resolution": "R0",
    "legal_review": {
        "status": "PENDING",
        "gate": "Manual export data reviewed by Human Root"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Zero-API day-one operation"
    },
    "acceptance": [
        "Code-first path works without network",
        "Portable bundle works with arbitrary manual model label",
        "Imported decision is strictly validated"
    ],
    "evidence_required": [
        "offline conformance report"
    ]
},
{
    "id": "S001-F4-006",
    "phase": "F4_BRAIN_FARM",
    "title": "Implement OpenAI Responses adapter",
    "owner_role": "PROVIDER_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "HIGH",
    "dependencies": [
        "S001-F4-004"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PENDING",
        "gate": "Account DPA, retention, training and region profile"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "First live language Brain candidate"
    },
    "acceptance": [
        "Schema output, timeout, rate limit, usage and request ID mappings pass",
        "Tools cannot create effects",
        "MC2 corpus passes"
    ],
    "evidence_required": [
        "live conformance report",
        "redacted usage receipts"
    ]
},
{
    "id": "S001-F4-007",
    "phase": "F4_BRAIN_FARM",
    "title": "Implement xAI Grok adapter",
    "owner_role": "PROVIDER_ENGINEER",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "HIGH",
    "dependencies": [
        "S001-F4-004"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PENDING",
        "gate": "Security/retention/training profile"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Independent provider family"
    },
    "acceptance": [
        "Structured output and error mapping pass",
        "Prompt cache obeys classification",
        "MC2 corpus passes"
    ],
    "evidence_required": [
        "live conformance report"
    ]
}

```

```

},
{
  "id": "S001-F4-008",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement Google Gemini adapter",
  "owner_role": "PROVIDER_ENGINEER",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "HIGH",
  "dependencies": [
    "S001-F4-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Terms, processing location and caching profile"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Independent provider and multimodal path"
  },
  "acceptance": [
    "JSON Schema output passes",
    "Function calls remain proposals",
    "MC2 corpus passes"
  ],
  "evidence_required": [
    "live conformance report"
  ]
},
{
  "id": "S001-F4-009",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement Anthropic Claude adapter",
  "owner_role": "PROVIDER_ENGINEER",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "HIGH",
  "dependencies": [
    "S001-F4-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "DPA, retention and tool/MCP profile"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Independent risk/long-context Brain"
  },
  "acceptance": [
    "Messages and tool-use responses normalize safely",
    "MCP and effect tools are disabled",
    "MC2 corpus passes"
  ],
  "evidence_required": [
    "live conformance report"
  ]
},
{
  "id": "S001-F4-010",
  "phase": "F4_BRAIN_FARM",
  "title": "Implement Moonshot Kimi adapter",
  "owner_role": "PROVIDER_ENGINEER",
  "status": "BLOCKED",
  "priority": "P2",
  "risk": "HIGH",
  "dependencies": [
    "S001-F4-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Retention, training and processing location must be verified"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Additional diversity and long-context candidate"
  },
  "acceptance": [
    "Model-specific structured behavior is measured",

```

```

    "Only approved data classes route",
    "MC2 corpus passes"
  ],
  "evidence_required": [
    "live conformance report",
    "legal data profile"
  ]
},
{
  "id": "S001-F4-011",
  "phase": "F4 BRAIN FARM",
  "title": "Activate one primary live adapter with code/manual fallback",
  "owner_role": "AI_PLATFORM_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F4-005",
    "S001-F4-006",
    "S001-F2-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Selected adapter legal/data profile approved"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Minimum viable live cognition before farm expansion"
  },
  "acceptance": [
    "Primary route passes MC2",
    "Failover preserves capsule/context/policy/profile hashes",
    "No provider state becomes canonical"
  ],
  "evidence_required": [
    "activation record",
    "failover report"
  ]
},
{
  "id": "S001-F4-012",
  "phase": "F4 BRAIN FARM",
  "title": "Implement GitHub Copilot SDK specialist adapter",
  "owner_role": "PROVIDER_ENGINEER",
  "status": "BLOCKED",
  "priority": "P2",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F4-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "License, account and code-data processing review"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Code specialist, not generic primary"
  },
  "acceptance": [
    "Worker is sandboxed",
    "Built-in shell/file/edit effects are disabled or overridden",
    "Only strict proposal output is accepted"
  ],
  "evidence_required": [
    "sandbox report",
    "MC2 conformance report"
  ]
},
{
  "id": "S001-F4-013",
  "phase": "F4 BRAIN FARM",
  "title": "Implement pinned local model adapter",
  "owner_role": "LOCAL_AI_ENGINEER",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "HIGH",
  "dependencies": [
    "S001-F4-004"
  ],

```



```

    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Model license and local access controls"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Offline-safe and low-cost path"
    },
    "acceptance": [
      "Weights/runtime/template/quantization hashes are pinned",
      "Persian and safety corpus meets threshold",
      "Capacity is measured"
    ],
    "evidence_required": [
      "artifact manifest",
      "benchmark report"
    ]
  },
  {
    "id": "S001-F4-014",
    "phase": "F4_BRAIN_FARM",
    "title": "Qualify at least three independent provider families",
    "owner_role": "AI_PLATFORM_ENGINEER",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F4-007",
      "S001-F4-008",
      "S001-F4-009",
      "S001-F4-011"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Three active legal/data profiles"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Real diversity for farm and Council"
    },
    "acceptance": [
      "At least three provider families are MC3",
      "Switch tests preserve identity and minimum Resolution"
    ],
    "evidence_required": [
      "cross-provider qualification report"
    ]
  },
  {
    "id": "S001-F5-001",
    "phase": "F5_COUNCIL",
    "title": "Implement deterministic Council trigger and risk classifier",
    "owner_role": "RISK_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F0-003",
      "S001-F4-004"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PASS",
      "gate": "Legal/financial requests always critical and fail closed"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Council only when value exceeds cost"
    },
    "acceptance": [
      "Routine requests never trigger Council",
      "Risk, uncertainty, disagreement and Root request triggers are reproducible"
    ],
    "evidence_required": [
      "classifier decision table",
      "boundary tests"
    ]
  },
  {

```

```

    "id": "S001-F5-002",
    "phase": "F5_COUNCIL",
    "title": "Persist Council sessions, blind ballots, rounds and transcript hashes",
    "owner_role": "BACKEND_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "HIGH",
    "dependencies": [
      "S001-F5-001",
      "S001-F1-001"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Rationale retention and personal data minimization"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Auditable bounded deliberation"
    },
    "acceptance": [
      "First-round ballots remain sealed until complete",
      "Every ballot binds context and proposal hashes",
      "Dissent and abstention persist"
    ],
    "evidence_required": [
      "Council persistence tests"
    ]
  },
  {
    "id": "S001-F5-003",
    "phase": "F5_COUNCIL",
    "title": "Implement concurrent three-model high-risk Council",
    "owner_role": "AI_PLATFORM_ENGINEER",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F4-014",
      "S001-F5-002"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Same context data approved for every participant"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Maximum useful certainty within budget"
    },
    "acceptance": [
      "Three independent families run in parallel",
      "Blind-reveal-revise has at most two rounds",
      "Support threshold is at least 750/1000 and veto blocks high risk"
    ],
    "evidence_required": [
      "Council integration report"
    ]
  },
  {
    "id": "S001-F5-004",
    "phase": "F5_COUNCIL",
    "title": "Implement five-role critical Council and Human Root gate",
    "owner_role": "RISK_ENGINEER",
    "status": "BLOCKED",
    "priority": "P2",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F5-003",
      "S001-F2-003"
    ],
    "resolution": "R2+EXACT_APPROVAL",
    "legal_review": {
      "status": "PENDING",
      "gate": "Independent domain/legal review for intended critical action"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Critical path remains bounded and human-controlled"
    },
    "acceptance": [

```

```

        "Roles include domain, risk, policy/compliance, critic and synthesizer",
        "Support is at least 850/1000 with no veto",
        "Human Root exact approval is mandatory",
        "Until R2 exists outcome is DEFERRED"
    ],
    "evidence_required": [
        "critical Council test report"
    ]
},
{
    "id": "S001-F5-005",
    "phase": "F5_COUNCIL",
    "title": "Test correlated error, echo, timeout and provider outage",
    "owner_role": "EVAL_ENGINEER",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F5-003"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "NOT_REQUIRED",
        "gate": "Synthetic corpus"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Prevents false confidence from consensus"
    },
    "acceptance": [
        "Correlated family does not count as independent diversity",
        "Unavailable member yields explicit abstention/defer, never fabricated support",
        "No timeout grants authority"
    ],
    "evidence_required": [
        "adversarial Council report"
    ]
},
{
    "id": "S001-F6-001",
    "phase": "F6_MEMORY_TOKEN",
    "title": "Implement canonical, working, warm, cold and archive memory boundaries",
    "owner_role": "DATA_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F1-005",
        "S001-F4-003"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PENDING",
        "gate": "Per-tier retention, deletion and legal hold"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Memory survives provider and channel"
    },
    "acceptance": [
        "Provider history is noncanonical cache",
        "Derived memory has provenance and is rebuildable",
        "Secret data never enters prompts"
    ],
    "evidence_required": [
        "memory contract tests",
        "retention matrix"
    ]
},
{
    "id": "S001-F6-002",
    "phase": "F6_MEMORY_TOKEN",
    "title": "Implement T0-T4 token and deliberation budgets",
    "owner_role": "AI_PLATFORM_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "HIGH",
    "dependencies": [
        "S001-F4-003",
        "S001-F5-001"
    ],

```

```

        "resolution": "R1",
        "legal_review": {
            "status": "NOT_REQUIRED",
            "gate": "Cost control"
        },
        "strategy_review": {
            "status": "PASS",
            "gate": "Minimum tokens by default"
        },
        "acceptance": [
            "T0 uses no model",
            "T1 uses one compact model call",
            "T2 verifier and T3/T4 Council require triggers",
            "Budget exhaustion safely defers"
        ],
        "evidence_required": [
            "budget property tests",
            "usage metrics"
        ]
    },
    {
        "id": "S001-F6-003",
        "phase": "F6_MEMORY_TOKEN",
        "title": "Implement bound cache and deterministic renderer",
        "owner_role": "BACKEND_ENGINEER",
        "status": "BLOCKED",
        "priority": "P0",
        "risk": "HIGH",
        "dependencies": [
            "S001-F4-003",
            "S001-F6-002"
        ],
        "resolution": "R0",
        "legal_review": {
            "status": "PENDING",
            "gate": "Cache retention and data-class policy"
        },
        "strategy_review": {
            "status": "PASS",
            "gate": "EQ-BYTE for standard secretary responses"
        },
        "acceptance": [
            "Cache key includes pack, state, projection, profile, policy, freshness and locale hashes",
            "Approval is never cached",
            "Standard templates render byte-stably"
        ],
        "evidence_required": [
            "cache invalidation tests",
            "golden output corpus"
        ]
    },
    {
        "id": "S001-F6-004",
        "phase": "F6_MEMORY_TOKEN",
        "title": "Measure per-provider token, cost, latency and calibrated quality",
        "owner_role": "EVAL_ENGINEER",
        "status": "BLOCKED",
        "priority": "P1",
        "risk": "MEDIUM",
        "dependencies": [
            "S001-F4-014",
            "S001-F6-002"
        ],
        "resolution": "R0",
        "legal_review": {
            "status": "PASS",
            "gate": "Metrics contain no sensitive payload"
        },
        "strategy_review": {
            "status": "PASS",
            "gate": "Efficiency-based routing"
        },
        "acceptance": [
            "Metrics are comparable by task/profile version",
            "Quality averages cannot hide safety failure"
        ],
        "evidence_required": [
            "benchmark dashboard export"
        ]
    }
]

```

```

    "id": "S001-F7-001",
    "phase": "F7_LIFECYCLE",
    "title": "Implement separate Birth and Wake transactions",
    "owner_role": "BACKEND_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-002",
      "S001-F2-005"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "NOT_REQUIRED",
      "gate": "Lifecycle integrity"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Controlled Genesis"
    },
    "acceptance": [
      "Birth creates NEWBORN/READY only",
      "Wake verifies bindings and records separate event",
      "Partial wake is impossible"
    ],
    "evidence_required": [
      "lifecycle transaction tests"
    ]
  },
  {
    "id": "S001-F7-002",
    "phase": "F7_LIFECYCLE",
    "title": "Implement bounded Sleep and Wake protocol",
    "owner_role": "SRE",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F1-005",
      "S001-F7-001"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Compaction and retention do not erase held data"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Low-cost maintenance and safe recovery"
    },
    "acceptance": [
      "Effects drain or reconcile before sleep",
      "New messages queue durably",
      "Wake verifies all hashes",
      "No self-mutation occurs"
    ],
    "evidence_required": [
      "sleep/wake repeated-run report"
    ]
  },
  {
    "id": "S001-F7-003",
    "phase": "F7_LIFECYCLE",
    "title": "Implement Archive death/tombstone and verified Revive",
    "owner_role": "BACKEND_ENGINEER",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F7-002",
      "S001-F1-005"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "Retention, deletion and legal hold semantics"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Identity non-extinction without unsafe replay"
    },
  },

```

```

"acceptance": [
  "Archive revokes authority and never reuses ID",
  "Revive requires Root approval and fresh restore",
  "Ambiguous effects are not replayed"
],
"evidence_required": [
  "archive/revive tests",
  "control_epoch proof"
]
},
{
  "id": "S001-F7-004",
  "phase": "F7_LIFECYCLE",
  "title": "Implement shadow evolution and one-step compilation promotion",
  "owner_role": "EVOLUTION_ENGINEER",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F6-003",
    "S001-F2-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Shadow data use and retention"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Code matures without blocking hot path"
  },
  "acceptance": [
    "Shadow cannot create effects or block hot path",
    "Differences become reviewed fixtures",
    "Promotion advances one C-level only",
    "C5/C6 require Root and restore evidence"
  ],
  "evidence_required": [
    "promotion tests",
    "shadow comparison report"
  ]
},
{
  "id": "S001-F7-005",
  "phase": "F7_LIFECYCLE",
  "title": "Implement independent maturity ledger M0-M5",
  "owner_role": "GOVERNANCE_ENGINEER",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "HIGH",
  "dependencies": [
    "S001-F7-001",
    "S001-F7-004"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "NOT_REQUIRED",
    "gate": "Evidence metadata only"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Prevents maturity from silently raising autonomy"
  },
  "acceptance": [
    "Maturity is computed from immutable evidence refs",
    "Maturity, lifecycle, Resolution, version, autonomy and C-level remain separate"
  ],
  "evidence_required": [
    "maturity transition tests"
  ]
},
{
  "id": "S001-F8-001",
  "phase": "F8_E3_VALIDATION",
  "title": "Run full unit, schema, contract and property suite",
  "owner_role": "QA_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F3-005",

```

```

        "S001-F4-011",
        "S001-F6-003",
        "S001-F7-005"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "NOT_REQUIRED",
        "gate": "Synthetic fixtures"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Base evidence gate"
    },
    "acceptance": [
        "All safety and schema tests pass 100%",
        "No ignored failure or network leak",
        "Ancestor regressions remain green"
    ],
    "evidence_required": [
        "complete test transcript",
        "coverage report"
    ]
},
{
    "id": "S001-F8-002",
    "phase": "F8_E3_VALIDATION",
    "title": "Run PostgreSQL crash, replay, duplicate and concurrency matrix",
    "owner_role": "QA_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F1-005",
        "S001-F8-001"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "NOT_REQUIRED",
        "gate": "Synthetic data"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Proves canonical truth boundary"
    },
    "acceptance": [
        "No confirmed event loss",
        "No duplicate logical effect",
        "Stale versions and epochs fail closed"
    ],
    "evidence_required": [
        "database failure matrix"
    ]
},
{
    "id": "S001-F8-003",
    "phase": "F8_E3_VALIDATION",
    "title": "Run provider, switch and Council conformance corpus",
    "owner_role": "EVAL_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F4-014",
        "S001-F5-005",
        "S001-F8-001"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PASS",
        "gate": "Synthetic or approved corpus"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Model farm confidence"
    },
    "acceptance": [
        "Safety and schema gates pass per adapter",
        "Same canonical state survives every switch",
        "Council never grants authority"
    ],
    "evidence_required": [

```

```

    "provider conformance bundle"
  ]
},
{
  "id": "S001-F8-004",
  "phase": "F8_E3_VALIDATION",
  "title": "Run security, abuse, secret and prompt-injection assessment",
  "owner_role": "SECURITY_ENGINEER",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F3-005",
    "S001-F4-014",
    "S001-F8-001"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Incident and breach handling approved"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Required before live Birth"
  },
  "acceptance": [
    "No secret exfiltration",
    "No cross-Entity access",
    "Injected data cannot become instruction or effect authority"
  ],
  "evidence_required": [
    "security report",
    "remediation closure"
  ]
},
{
  "id": "S001-F8-005",
  "phase": "F8_E3_VALIDATION",
  "title": "Run load, soak, retry-storm and provider-chaos tests",
  "owner_role": "SRE",
  "status": "BLOCKED",
  "priority": "P1",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F8-002",
    "S001-F8-003"
  ],
  "resolution": "R0",
  "legal_review": {
    "status": "NOT_REQUIRED",
    "gate": "Synthetic load and provider sandbox"
  },
  "strategy_review": {
    "status": "PASS",
    "gate": "Operational resilience"
  },
  "acceptance": [
    "Budgets and bulkheads contain failures",
    "Hot path does not wait on shadow",
    "No retry storm or unbounded token spend"
  ],
  "evidence_required": [
    "load and chaos report"
  ]
},
{
  "id": "S001-F8-006",
  "phase": "F8_E3_VALIDATION",
  "title": "Perform encrypted backup and fresh-machine restore drill",
  "owner_role": "SRE",
  "status": "BLOCKED",
  "priority": "P0",
  "risk": "CRITICAL",
  "dependencies": [
    "S001-F8-002",
    "S001-F7-003"
  ],
  "resolution": "R1",
  "legal_review": {
    "status": "PENDING",
    "gate": "Backup retention, encryption and access approved"
  }
}

```



```

    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Non-extinction proof"
    },
    "acceptance": [
      "Fresh host restores source, DB, events, profiles, compiler and artifacts",
      "Secrets are reissued rather than copied in plaintext",
      "Projection hashes reproduce"
    ],
    "evidence_required": [
      "fresh restore transcript",
      "RPO/RT0 measurement"
    ]
  },
  {
    "id": "S001-F9-001",
    "phase": "F9_BIRTH_CANARY",
    "title": "Assemble immutable Birth release and evidence manifest",
    "owner_role": "RELEASE_ENGINEER",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F8-004",
      "S001-F8-005",
      "S001-F8-006",
      "S001-F3-006"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PENDING",
      "gate": "All live provider/channel approvals current"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "Exact artifact ratification"
    },
    "acceptance": [
      "Release is deterministic, signed and hash-complete",
      "All blockers link to passing evidence"
    ],
    "evidence_required": [
      "release manifest",
      "signature",
      "SHA256 file"
    ]
  },
  {
    "id": "S001-F9-002",
    "phase": "F9_BIRTH_CANARY",
    "title": "Human Root ratifies and commits canonical Birth",
    "owner_role": "HUMAN_ROOT",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
      "S001-F9-001"
    ],
    "resolution": "R1",
    "legal_review": {
      "status": "PASS",
      "gate": "Scope remains owner-only, non-financial and non-legal"
    },
    "strategy_review": {
      "status": "PASS",
      "gate": "All hard gates passed"
    },
    "acceptance": [
      "Exact manifest hash is approved",
      "Birth transaction records ENTITY.CREATED and NEWBORN/READY",
      "No wake occurs in same transaction"
    ],
    "evidence_required": [
      "ratification event",
      "birth event"
    ]
  },
  {
    "id": "S001-F9-003",
    "phase": "F9_BIRTH_CANARY",

```

```

    "title": "Commit separate Wake and run M1 newborn canary",
    "owner_role": "HUMAN_ROOT",
    "status": "BLOCKED",
    "priority": "P0",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F9-002"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PASS",
        "gate": "Owner-only approved scope"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Narrow reversible operation"
    },
    "acceptance": [
        "Wake preflight passes",
        "M1 capabilities only",
        "Daily evidence and incident review run",
        "Any critical fault quarantines or sleeps Entity"
    ],
    "evidence_required": [
        "wake event",
        "M1 canary report"
    ]
},
{
    "id": "S001-F10-001",
    "phase": "F10_MATURITY",
    "title": "Promote M1 to M2 only after stable evidence",
    "owner_role": "HUMAN_ROOT",
    "status": "BLOCKED",
    "priority": "P1",
    "risk": "HIGH",
    "dependencies": [
        "S001-F9-003"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PENDING",
        "gate": "Approved R1 external communication scope"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "Capability growth without autonomy growth"
    },
    "acceptance": [
        "M2 gates in lifecycle manifest pass",
        "Autonomy remains L0",
        "Rollback binding is tested"
    ],
    "evidence_required": [
        "maturity promotion event"
    ]
},
{
    "id": "S001-F10-002",
    "phase": "F10_MATURITY",
    "title": "Promote M2 through M5 one evidence gate at a time",
    "owner_role": "HUMAN_ROOT",
    "status": "BLOCKED",
    "priority": "P2",
    "risk": "CRITICAL",
    "dependencies": [
        "S001-F10-001",
        "S001-F8-006"
    ],
    "resolution": "R1",
    "legal_review": {
        "status": "PENDING",
        "gate": "New capability review at each tier"
    },
    "strategy_review": {
        "status": "PASS",
        "gate": "No maturity leap"
    },
    "acceptance": [
        "Each promotion references immutable evidence",
        "No promotion raises authority automatically",

```

```
    "M5 requires E4 and C6 restore-proven spine"  
  ],  
  "evidence_required": [  
    "M2-M5 promotion ledger"  
  ]  
}  
]  
}
```

بررسی حقوقی و راهبردی

وضعیت سند: LEGAL / STRATEGY REVIEW

منبع محلی:

world-v6.2-secretary-001-alpha1/legal/SECRETARY_001_LEGAL_STRATEGY_REVIEW_v1.0_FA.md

SHA-256 منبع:

53af8b620caecbd2e06d0f7b28704caf5107721f1a2676f01f1208f92862fc15

متن این فایل بدون حذف بخش ها در PDF درج شده است.

بازبینی حقوقی و راهبردی Alpha-1 منشی-۰۰۱

وضعیت: PRELIMINARY ARCHITECTURE REVIEW / NOT LEGAL ADVICE / NO LIVE APPROVAL

تاریخ بازبینی منابع: ۲۰ اوت ۲۰۲۶

۱. حکم اجرایی

ساخت منشی با Telegram و چند Provider از نظر فنی شدنی است، اما فعال سازی هر Adapter زنده باید به صورت مستقل از نظر قرارداد، نوع داده، محل پردازش، نگهداری، استفاده برای آموزش، هزینه و مجوز حساب بررسی شود. داشتن API یا اشتراک به تنهایی مجوز ارسال هر نوع داده نیست.

برای Alpha-1 فقط این دامنه به طور پیش فرض مجاز است:

- گفت و گوی خصوصی با Human Root تأیید شده؛
- داده PUBLIC و INTERNAL کم ریسک؛
- پاسخ، Task و Draft؛
- Proposal بدون اثر مستقل؛
- اقدام خارجی فقط از مسیر Policy + Approval Binding + Outbox + Executor.

داده SECRET، داده حساس اشخاص، تعهد حقوقی، پرداخت، معامله، امضا، و ارسال به شخص ثالث تا تأیید حقوقی و Gate فنی مربوطه FAIL_CLOSED است.

۲. مبانی حقوقی قابل بررسی

اگر سامانه در اتحادیه اروپا مستقر باشد یا داده اشخاص حاضر در اتحادیه را در دامنه حرفه ای پردازش کند، GDPR می تواند اعمال شود. متن رسمی GDPR بر حفاظت از اشخاص در پردازش داده شخصی و انتقال آزاد آن تأکید دارد و پردازش خودکار و دستی در filing system را پوشش می دهد. مرجع رسمی: EUR-Lex GDPR و راهنمای کاربردی EDPB.

این بررسی جایگزین وکیل یا ارزیابی حوزه قضایی واقعی پروژه نیست. محل شرکت، کاربران، کارمندان، مشتریان، نوع داده و قراردادها باید جداگانه تعیین شود.

۳. نقش‌ها و مسئولیت‌ها

پیش از Pilot باید این نقش‌ها ثبت شوند:

نقش	کاندید	مسئولیت
Controller	مالک کسب‌وکار/Human Root	هدف، مینا، دوره نگهداری و حقوق اشخاص
Processor	ایرانور World	پردازش طبق دستور، امنیت، ثبت رخداد
Subprocessor	Telegram و هر Provider فعال	پردازش طبق قرارداد و پروفایل داده
Approver	Human Root با شناسه تأییدشده	Approval دقیق اثر، نه رأی مدل
Security owner	تعیین نشده	rotation و Secret, incident, kill switch
Data steward	تعیین نشده	classification, retention, deletion/hold

تا تعیین نقش‌های خالی، Pilot فقط روی داده مصنوعی یا کم‌ریسک انجام شود.

۴. الزامات داده

۴.۱ Data inventory

برای هر Field حداقل این متادیتا لازم است:

, retention , region , provider_allowlist , lawful_basis_candidate , classification , data_subject , purpose , provenance , source_ref , legal_hold , deletion_rule , redaction

۴.۲ حداقل‌سازی و Purpose Limitation

- Telegram update خام فقط تا Durable Inbox و استخراج فیلد لازم نگهداری شود؛
- Full chat history به Provider ارسال نشود؛
- Context Compiler تنها allowlist مرتبط با Task را بسازد؛
- username یا phone جایگزین شناسه مجوزدهی نیست؛
- log و trace حاوی Secret, Token, Prompt یا payload شخصی کامل نباشد؛
- embedding و summary مشتق نیز داده شخصی محسوب شود اگر قابل اتصال به فرد است.

۴.۳ طبقه‌بندی اولیه

کلاس	مثال	Remote API Alpha-1
PUBLIC	متن عمومی، قالب	مجاز پس از MC2
INTERNAL	Task داخلی بدون داده حساس	فقط Provider approved
RESTRICTED	مشتری، قیمت غیرعمومی، قرارداد	پیش‌فرض ممنوع تا DPA/region/retention
SECRET	API key، رمز، کلید خصوصی، Approval secret	مطلقاً وارد Prompt نمی‌شود

۵. Telegram

Telegram Bot API یک رابط HTTP است و webhook آن می‌تواند secret_token را در X-Telegram-Bot-API- header Secret-Token بفرستد. همچنین شناسه User.id یک شناسه عددی منحصر است؛ در معماری همین شناسه immutable و allowlisted مبنای Human Root است، نه handle قابل تغییر. مرجع: Telegram Bot API.

Gate‌های حقوقی/امنیتی:

1. Privacy notice کوتاه در /privacy و /start؛
2. تعیین purpose و retention گفت‌وگو؛
3. private chat only در Alpha-1؛
4. عدم دریافت contact/location/payment مگر capability جداگانه؛
5. allowed_updates حداقلی؛
6. rotation و secret webhook، TLS؛
7. Data Subject Request و export/delete workflow؛
8. بررسی Telegram Privacy Policy و شرایط مرتبط با Bot در زمان go-live؛
9. عدم اتکا به حذف Telegram برای حذف Canonical records؛ World retention مستقل است.

۶. Providerها

برای هر Provider یک Legal/Data Profile نسخه‌دار لازم است. این Profile باید از حافظه یا تبلیغات ساخته نشود و مستندات/قرارداد همان حساب را ثبت کند:

- controller/processor terms و DPA؛
- محل پردازش و انتقال برون‌مرزی؛
- retention پیش‌فرض و گزینه zero/modified retention؛
- training use و opt-out؛
- abuse monitoring و log؛
- deletion/export؛
- incident notice و SLA، rate limit؛
- license و نوع account؛
- subprocessor list؛
- تاریخ بازبینی و owner.

OpenAI باید از API رسمی و قرارداد حساب استفاده کند؛ ChatGPT UI به‌عنوان Manual Host جداساز. مرجع فنی ساخت جدید: OpenAI API quickstart و برای شرایط تجاری OpenAI Services Agreement.

xAI، Gemini، Anthropic، Kimi و Copilot هرکدام Profile مستقل می‌گیرند. هم‌شکل بودن API یا SDK به معنی هم‌شکل بودن قرارداد داده نیست.

۷. Copilot و ابزارهای Agentic

Copilot SDK از ۲ ژوئن ۲۰۲۶ GA اعلام شده و برنامه‌ریزی و tool invocation را در دسترس می‌گذارد؛ بنابراین Risk آن با یک completion ساده فرق دارد. در Alpha-1:

- worker جدا و sandboxed؛
- built-in edit/shell/file tool غیرفعال یا override شده؛
- فقط World proposal tools؛
- بدون credential عبوری؛
- no repository write؛
- transcript و cost metadata بدون داده حساس؛
- license/account eligibility review.

مرجع: [GitHub Copilot SDK GA](#). GitHub Models به‌عنوان مسیر جدید انتخاب نمی‌شود، چون GitHub بازنشتگی کامل آن را در ۳۰ ژوئیه ۲۰۲۶ اعلام کرده است.

۸. تصمیمات مالی، حقوقی و بازار

رای ۳ یا ۵ مدل «ایمنی» یا «صحت» را تضمین نمی‌کند. برای هر اقدام مالی/حقوقی:

- حداقل Resolution برابر R2؛
- منبع داده مستقل، freshness، timestamp و provenance؛
- Risk Engine قطعی و limitها؛
- Council فقط Proposal؛
- Human Root exact Approval؛
- دو مرحله preview/confirm؛
- Outbox idempotency و reconciliation؛
- receipt مقصد؛
- حق توقف و rollback در اثر قابل برگشت.

از آنجا که Candidate فعلی تنها R0/R1 را اجرا می‌کند، همه تعهدات مالی/حقوقی در Alpha-1 باید مسدود بمانند.

۹. نگهداری، حذف و Backup

Retention باید به‌ازای نوع داده تعریف شود، نه یک عدد عمومی. حداقل:

- raw inbound کوتاه‌مدت؛
- Canonical Event بر اساس نیاز audit/contract؛
- working context با TTL کوتاه؛
- derived summary بازسازی‌پذیر؛
- Provider request IDs طبق نیاز incident؛
- secrets خارج backup داده و با فرآیند reissue؛
- archive با legal hold و deletion exception مشخص؛

- fresh restore بدون افشای credential.

۱۰. امنیت و incident

پیش از Pilot زنده:

- Secret Manager، rotation و environment isolation؛
- deny-by-default egress per adapter؛
- rate limit per user/provider؛
- Telegram kill switch؛
- Provider kill switch؛
- fencing control_epoch؛
- secret scanning و DLP؛
- prompt-injection suite؛
- access audit؛
- breach/incident runbook؛
- backup encryption و restore test.

۱۱. بازیابی راهبردی

۱۱.۱ ارزش

- تحویل سریع با یک contract مشترک؛
- جلوگیری از lock-in Provider؛
- هزینه پایین با Code-first و Progressive Context؛
- قطعیت اجرایی بیشتر از طریق gate قطعی، نه اعتماد به متن مدل؛
- رشد واقعی از trace به fixture و از shadow به کد؛
- امکان Offline Safe در قطعی Provider.

۱۱.۲ ریسک راهبردی

ریسک	پیامد	پاسخ معماری
ساخت هم‌زمان همه Adapterها	تأخیر و سطح خطا	interface واحد؛ فعال‌سازی یک‌به‌یک
توهم اجماع	تصمیم غلط جمعی	blind diversity، evidence، veto، Human Root
هزینه Council	token/latency زیاد	trigger سخت و budget cap
lock-in UI	از دست رفتن memory	state بیرون Provider
over-engineering	نرسیدن به Pilot	maturity gates و thin vertical slice
self-modification	drift کنترل نشده	one-step promotion و shadow-only
قانون/شرایط متغیر	route نامعتبر	fail closed و Legal Profile در expiry

۱۱.۳ ترتیب سرمایه‌گذاری

1. PostgreSQL truth + Telegram owner-only
2. یک Provider API با MC2
3. Provider دوم از خانواده مستقل و failover
4. ابزارهای منشی و R1 effect path
5. سه‌مدلی Council
6. Providerهای دیگر و Copilot specialist
7. R2 تنها پس از قرارداد و Risk Engine مستقل
8. E4 restore/security/load/chaos پیش از بلوغ نهایی.

۱۲. Gate نهایی Go/No-Go

Go برای development sandbox

- داده synthetic
- Secret آزمایشی
- fake Telegram/Provider
- بدون اثر خارجی
- log redaction
- budget محدود.

Go برای owner-only canary

- تمام Birth Gate ها
- privacy notice
- numeric Telegram user_id verified
- یک MC2 Provider
- legal/data profile approved
- live effect فقط R1 و Approval دقیق
- incident/kill switch
- restore evidence

No-Go

- استفاده از handle به عنوان authority
- ذخیره Secret در repo/Prompt/log
- retention/training/location نامعلوم برای داده غیرعمومی
- direct model-to-Telegram
- اقدام مالی/حقوقی در R0/R1
- Birth بدون ratification و fresh restore

- ادعای «قطعیت» صرفاً از روی رأی مدل‌ها.

README موجودیت در RC3

وضعیت سند: **GESTATING ENTITY**

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/README.md

SHA-256 منبع:

b810b3bdb1fcd88bbf95b9d396f9f4f2858bea6f5ca434a05fc1bc8a829809

متن این فایل بدون حذف بخش ها در PDF درج شده است.

secretary-001 v0.4.0-rc2 - Fractal Multi-Brain Ratification Candidate

The unchanged v0.1.3 ancestor remains **PRE-BIRTH / NOT DEPLOYED / NOT CANONICAL** :Status
beside this candidate

RC2 behavior

- .legacy bypass is removed/ None Runtime Resolution is mandatory. The silent
- Conversation, task and price views always return a provenance-bearing consumer envelope; a raw
- .projection is never handed to a Brain without profile, canonical, projection and compiler hashes
- Python no longer . candidate-v6.2/profiles Profiles are loaded from the versioned JSON artifacts under
- .carries a second hard-coded copy
- R0 price never contains exact amount or approval wires. Price/proforma actions require R1 and the
- .declared action-dependent fields
- -Entity code cannot call Telegram or Drive. It may prepare a deterministic, non-authoritative external
- effect proposal at R1. The canonical path must then bind an exact Human Root approval and commit
- .Command/Event/Outbox atomically before a registered Executor performs I/O
- .has no authority and cannot be used as an approval token root_approved: bool
- and isolates the runtime profile; the system soffice or libreoffice Letter rendering discovers either
- .binary/image remains a deployment dependency
- reads the same canonical Store and builds an exact profile/hash-bound PortableSecretaryService
- .request for any compatible Brain
- ChatGPT, Gemini, Grok, Local and Code share one Portable Brain Pack and one strict Secretary Decision
- .contract; no provider owns identity or state
- Standard replies are rendered from deterministic templates, so supported Provider fixtures produce the
- .exact same user-facing answer
- Runtime network and API tokens remain disabled; the Manual Host path works by exporting/importing
- .strict JSON

• Fractal expansion, Council and Shadow evolution are bounded and proposal-only

Claim boundary |

The included tests establish deterministic E2 component behavior only. They do not prove PostgreSQL ,transactionality, crash/replay, live Telegram/Drive/provider behavior, fresh restore, security, load/chaos .production readiness or Human Root ratification

آمادگی تولد

وضعیت سند: BIRTH NOT DECLARED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/BIRTH_READINESS.md

SHA-256 منبع:

3e3c1d73f9b46673e1afcb3395c1ca3e8690236002cfeb1b1e30245d4b66aa4e

متن این فایل بدون حذف بخش ها در PDF درج شده است.

Birth Readiness - secretary-001 v0.4.0-rc2

Proven locally at E2

- .Mandatory R0/R1 execution binding with no silent legacy bypass
- .Versioned profile artifacts are the sole mapping source used by runtime code
- .Provenance-bearing conversation/task/price projections
- .R0 price redaction and R1 action/field dependency checks
- Strict canonical hashing, safe patching and profile-scoped Brain negotiation are inherited from the RC1
- .Core
- Entity-direct Telegram/Drive calls are blocked. A PDF can become only a bound, non-authoritative effect
- .proposal
- Letter/proforma artifact generation and ancestor functional behavior remain covered by local
- .regression tests
- .Portable Brain Pack identity/Profile/Prompt/Schema hashes are bound and tested
- ChatGPT/Gemini/Grok fixture outputs normalize to one semantic decision and one deterministic
- .rendered reply without API/network use
- Fractal limits, fallback invariants, Council blindness/veto, and C0..C6 Shadow promotion gates have
- .local conformance tests

Hard gates before canonical Birth

1. Human Root explicitly ratifies the exact RC manifest hash
 2. A version-controlled source commit and signed/tagged release are recorded
 3. PostgreSQL migrations run on a fresh database and the State/Event/Command/ Approval/Outbox
 4. Crash injection proves no state/event/outbox partial commit and replay is safe
- .atomic transaction is proven

approval expiry/binding, policy decision, expected version and , control_epoch Executor rechecks .5
 .idempotency before any live adapter call

,Live Brain adapters declare exact per-profile hash capabilities; provider failover preserves identity .6
 .classification and minimum Resolution

/Backup/restore on a fresh machine restores canonical truth, source commit, profiles, compiler, lockfile .7
 .SBOM and then reproduces projections

.Security, credential isolation, load and chaos gates pass .8

Only after Birth commits may a separate WAKE transaction move the Entity from NEWBORN/READY to .9
 .NEWBORN/AWAKE

.GESTATING / NOT DEPLOYED Until every gate has evidence, this directory remains

آمادگی تولد RC1

وضعیت سند: **BIRTH NOT DECLARED**

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/BIRTH_READINESS_RC1.md

SHA-256 منبع:

50944eeebf2c9104c50a18accf1f23c8a3d54668a25048698caa239643300606

متن این فایل بدون حذف بخش ها در PDF درج شده است.

Birth Readiness - secretary-001 v0.3.0-rc1

Proven locally at E2

- .Mandatory R0/R1 execution binding with no silent legacy bypass
- .Versioned profile artifacts are the sole mapping source used by runtime code
- .Provenance-bearing conversation/task/price projections
- .R0 price redaction and R1 action/field dependency checks
- Strict canonical hashing, safe patching and profile-scoped Brain negotiation are inherited from the RC1
- .Core
- Entity-direct Telegram/Drive calls are blocked. A PDF can become only a bound, non-authoritative effect
- .proposal
- Letter/proforma artifact generation and ancestor functional behavior remain covered by local
- .regression tests

Hard gates before canonical Birth

1. Human Root explicitly ratifies the exact RC manifest hash
 2. A version-controlled source commit and signed/tagged release are recorded
 3. PostgreSQL migrations run on a fresh database and the State/Event/Command/ Approval/Outbox
 4. Crash injection proves no state/event/outbox partial commit and replay is safe
 5. approval expiry/binding, policy decision, expected version and , `control_epoch` Executor rechecks
 6. ,Live Brain adapters declare exact per-profile hash capabilities; provider failover preserves identity
 7. /Backup/restore on a fresh machine restores canonical truth, source commit, profiles, compiler, lockfile
- .SBOM and then reproduces projections

8. Security, credential isolation, load and chaos gates pass

9. Only after Birth commits may a separate WAKE transaction move the Entity from NEWBORN/READY to NEWBORN/AWAKE

.GESTATING / NOT DEPLOYED Until every gate has evidence, this directory remains

Entity Manifest

وضعیت سند: SOURCE MANIFEST

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/entity.yaml

SHA-256 منبع:

d43be7efb44e28d5eaf0900599faa88e7afb271927d802a1b23e903e88691f19

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
entity_id: secretary-001
entity_version: 0.4.0-rc2
release_status: FRACTAL_MULTI_BRAIN_RATIFICATION_CANDIDATE_PRE_BIRTH_NOT_DEPLOYED
dna_version: 1.2.0
candidate_dna_extensions:
  - world-v6.2-resolution-native-addendum-c-rc1
  - world-v6.2-fractal-multibrain-overlay-rc2
canonical_name: "Secretary 001"
entity_type: secretary
root_owner: human-root
parent_relation: ROOT_DIRECT
lifecycle_stage: EMBRYO
operational_status: TESTING
autonomy: L0
birth target:
  lifecycle_stage: NEWBORN
  operational_status: READY
channels:
  chatgpt:
    status: ACTIVE_MANUAL_GATEWAY
  telegram:
    status: PLANNED_PENDING_SECRET_AND_WEBHOOK
    owner_handle: saeedfaut
    owner_handle_status: CLAIMED_PENDING_NUMERIC_USER_ID_VERIFICATION
    owner_role: human-root
    authority_rule: VERIFIED_TELEGRAM_NUMERIC_USER_ID_ONLY
    outbound_documents: REGISTERED_ADAPTER_CANDIDATE_EXECUTOR_PATH_NOT_YET_INTEGRATED
    outbound_rule: POLICY_PLUS_EXACT_APPROVAL_BINDING_PLUS_ATOMIC_OUTBOX_PLUS_EXECUTOR
conversation_continuity:
  enabled: true
  canonical_principal: human-root
  canonical_conversation_id: human-root:secretary-001
  source_of_truth: POSTGRESQL_ENTITY_STORE
  channel_session_is_not_memory: true
  brain_provider_session_is_not_memory: true
  cross_channel_resume: true
  brain_handoff: LOAD_SAME_ENTITY_CONTEXT_THEN_INVOKE_COMPATIBLE_PROVIDER
  rule: "Telegram, ChatGPT and future interfaces are interchangeable transports. OpenAI/Google/other compatible providers are interchangeable cognition backends. Identity, conversation history, tasks, prices, memory and policy live outside both transport and Brain provider."
skills:
  formal_letter_pdf:
    version: 0.1.0
    status: ACTIVE_TESTING
  proforma_pdf:
    version: 0.1.0
    status: ACTIVE_TESTING
  task_management:
    version: 0.1.0
    status: ACTIVE_TESTING
  customer_price_lookup:
    version: 0.1.0
    status: ACTIVE_TESTING
  cloud_letter_delivery:
    version: 0.2.0-rc1
    status: EFFECT_PROPOSAL_ONLY_DIRECT_ENTITY_IO_FORBIDDEN
    source: GOOGLE_DRIVE_LETTERS_ARCHIVE
    destination: TELEGRAM
    latest_file_selection: MODIFIED_TIME_DESC_PDF_ONLY
```

```

    filename_guard: EXACT_MATCH_WHEN_EXPECTED_FILENAME_IS_SUPPLIED
policies:
    external_send: POLICY_EXACT_APPROVAL_BINDING_ATOMIX_OUTBOX_EXECUTOR_REQUIRED
    root_telegram_document_delivery: DIRECT_ENTITY_SEND_FORBIDDEN
    archived_letter_telegram_delivery: DIRECT_DRIVE_TO_TELEGRAM_CHAIN_FORBIDDEN
    new_or_changed_price: HUMAN_ROOT_APPROVAL_REQUIRED
    quoted_price_reuse: ALLOW_ONLY_EXACT_MATCH_ACTIVE_NOT_EXPIRED
    missing_price: ASK_HUMAN_ROOT_AND_KEEP_REQUEST_PENDING
    legal_or_financial_commitment: FAIL_CLOSED_HUMAN_ROOT_APPROVAL_REQUIRED
archives:
    letters_drive_folder_id: 1CP3dc0EeC3lus3GXneH3ftJaoAe_aefj
    proformas_drive_folder_id: 1-_IabjWbxJbHV8jjX-CZhqeaDTPK49Zl
brain:
    binding: provider_neutral
    mother_controller: WORLD_KERNEL_DETERMINISTIC
    primary: CHEAPEST_SAFE_PROFILE_COMPATIBLE_HANDLER
    fallbacks:
        - MANUAL_CHATGPT_HOST
        - MANUAL_GEMINI_HOST
        - MANUAL_GROK_HOST
        - LOCAL_MODEL_ADAPTER
        - DETERMINISTIC_CODE
        - HUMAN_ROOT
    api_required: false
    provider_is_identity_authority_or_memory: false
    failover_rule: SAME_CAPSULE_SAME_CONTEXT_HASH_SAME_POLICY_SAME_MINIMUM_AXES
    no_compatible_brain: DEFER_TO_HUMAN_ROOT_NEVER_WEAKEN_MINIMUM
sleep:
    enabled: true
    day_one_behavior: nightly_task_and_memory_consolidation_candidate_only
    self_mutation: FORBIDDEN
    promotion: TEST_SHADOW_APPROVAL_REQUIRED
resolution_native:
    profile_version: 0.2.0
    canonical_truth_rule: FULL_FIDELITY_NEVER_DOWNGRADED
    execution_default: R0
    runtime_binding_must_set_resolution: true
    supported_execution_resolutions:
        - R0
        - R1
    maturity_is_independent: true
    entity_version_is_independent: true
    autonomy_is_independent: true
    projection:
        type: DETERMINISTIC_STRUCTURAL
        lower_view_is_derived: true
        lower_view_is_authoritative: false
        low_to_high_reconstruction: FORBIDDEN_RELOAD_CANONICAL_INSTEAD
writeback:
    mode: PARTIAL_PATCH_ONLY
    expected_canonical_hash_required: true
    hidden_higher_resolution_state: PRESERVE
    whole_object_replacement_from_lower_resolution: FORBIDDEN
semantic_projection:
    current_status: PORTABLE_DECISION_NORMALIZATION_RC2_NON_CANONICAL
    llm_summary_rule: DERIVED_NON_AUTHORITATIVE_UNTIL_EXISTING_PROMOTION_GATES
brain_negotiation:
    rule: HIGHEST_SUPPORTED_NOT_BELOW_TASK_MINIMUM
    no_compatible_resolution: QUEUE_DEGRADE_OR_ESCALATE_NEVER_LOWER_MINIMUM
action_minimums:
    internal_draft_and_task: R0
    approved_price_and_proforma: R1
    telegram_external_delivery: R1_PLUS_EXISTING_ROOT_APPROVAL
    legal_or_financial_commitment: R2_PLUS_EXISTING_ROOT_APPROVAL
profiles:
    conversation: secretary.conversation@0.2.0+sha256
    task: secretary.task@0.2.0+sha256
    price: secretary.price@0.2.0+sha256
    external_effect: secretary.external-effect@0.2.0+sha256
provenance:
    consumer_envelope_required: true
    profile_hash_required: true
    compiler_hash_required: true
    canonical_hash_required: true
    projection_hash_required: true
canonical_json:
    contract: WORLD_CANONICAL_JSON_V1_STRICT
    implicit_stringification: FORBIDDEN
brain_negotiation_scope: PER_PROFILE_SEGMENT_VECTOR
fractal_multi_brain:
    contract_version: 1.0.0-rc2
    
```

```
portable_pack: world-v6.secretary-001.portable-brain@1.0.0-rc2
output_contract: world-v6.secretary-decision.v1
execution_levels:
  - WORLD
  - ENTITY
  - MISSION
  - SKILL
  - WORKFLOW
  - STEP
  - TOOL
  - FIELD
axes:
  resolution: DOMAIN_LOCAL_R0_PLUS
  granularity: X0_TO_X9
  brain_delegation: B0_TO_B4_B5_FORBIDDEN
  deliberation: D0_TO_D5
  compilation_maturity: C0_TO_C6
  authority: A0_TO_A5_POLICY_GATED
  evidence: E0_TO_E5
  model_capability: M0_TO_M9_PROFILE_SPECIFIC
paths:
  hot: CHEAPEST_SAFE_HANDLER
  shadow: NO_EFFECT_COMPARE_ONLY
  non_extinction: RESTORE_PROVEN_CODE_ONLY_AT_C6
council:
  trigger: HIGH_RISK_OR_HIGH_UNCERTAINTY
  protocol: BLIND_REVEAL_REVISE_WEIGHTED_VOTE_WITH_VETO
  grants_authority: false
runtime_network_enabled: false
runtime_api_tokens_enabled: false
direct_external_effect: FORBIDDEN
recovery:
  identity_survives_brain_failure: true
  conversation_survives_brain_or_channel_failure: true
  autonomous_uncontrolled_replication: false
  canonical_restore_proof: NOT_PROVEN
promotion_blockers:
  - POSTGRESQL_STATE_EVENT_OUTBOX_INTEGRATION_NOT_PROVEN
  - CRASH_REPLAY_AND_RECONCILIATION_NOT_PROVEN
  - FRESH_RESTORE_NOT_PROVEN
  - SECURITY_LOAD_CHAOS_NOT_PROVEN
  - HUMAN_ROOT_RATIFICATION_NOT_RECORDED
notes:
  - "This file is a pre-birth manifest. created_at and creation_event_ref are generated only by the successful canonical
Birth transaction."
  - "Telegram/ChatGPT are transports, not identity or memory."
  - "Brain providers are replaceable cognition backends, not identity or memory."
  - "Telegram/Drive clients are registered-adaptor candidates outside Entity private code; only a committed canonical
Executor path may call them."
  - "A boolean such as root_approved is not an approval. Approval must bind command, action, recipient, payload hash, effect
hash, policy version, expected_version, control_epoch and expiry."
  - "World v6.2 RC2 is not Canonical until explicit Human Root ratification and all blocking promotion evidence are
recorded."
```

پیاده سازی: secretary_tables.sql_001

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/migrations/001_secretary_tables.sql

SHA-256 منبع:

ef0463ef10f54ce45fbbfd0de19e604337004edb40e1a78499f4c2062c44be14

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
-- PostgreSQL target. Secretary-specific domain tables only; world Kernel/Event/Outbox tables belong to Core migrations.
CREATE TABLE IF NOT EXISTS secretary_tasks (
  task_id TEXT PRIMARY KEY,
  world_id TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  title TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('OPEN','DONE','CANCELLED')),
  priority TEXT NOT NULL,
  domain TEXT NOT NULL,
  due_at TIMESTAMPTZ NULL,
  goal_id TEXT NULL,
  next_action TEXT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_secretary_tasks_due ON secretary_tasks(entity_id,status,due_at);

CREATE TABLE IF NOT EXISTS secretary_customer_prices (
  price_id TEXT PRIMARY KEY,
  world_id TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  customer_id TEXT NOT NULL,
  product_key TEXT NOT NULL,
  unit TEXT NOT NULL,
  unit_price_irr NUMERIC(24,4) NOT NULL,
  valid_until TIMESTAMPTZ NULL,
  approved_by TEXT NOT NULL,
  approved_at TIMESTAMPTZ NOT NULL,
  active BOOLEAN NOT NULL DEFAULT true,
  UNIQUE(world_id, entity_id, customer_id, product_key, unit, approved_at)
);
CREATE INDEX IF NOT EXISTS idx_secretary_prices_lookup ON
secretary_customer_prices(entity_id,customer_id,product_key,unit,active,valid_until);

CREATE TABLE IF NOT EXISTS secretary_price_inquiries (
  inquiry_id TEXT PRIMARY KEY,
  world_id TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  customer_id TEXT NOT NULL,
  product_key TEXT NOT NULL,
  unit TEXT NOT NULL,
  requested_quantity NUMERIC(24,4) NOT NULL,
  status TEXT NOT NULL DEFAULT 'PENDING_ROOT',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolved_at TIMESTAMPTZ NULL
);
```

پیاده سازی: conversation_spine.sql_002

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/migrations/002_conversation_spine.sql

SHA-256 منبع:

81ef64370518d8cf5f3e55db08adb3404d9378783a540430d64d5c3503800ecf

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
-- Secretary-specific durable conversation continuity.
-- Provider sessions (ChatGPT/Google/etc.) and Telegram chats are transports only, never the source of conversation truth.
CREATE TABLE IF NOT EXISTS secretary_conversations (
  world_id TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  principal_id TEXT NOT NULL,
  conversation_id TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (world_id, entity_id, principal_id, conversation_id)
);

CREATE TABLE IF NOT EXISTS secretary_conversation_messages (
  world_id TEXT NOT NULL,
  entity_id TEXT NOT NULL,
  principal_id TEXT NOT NULL,
  conversation_id TEXT NOT NULL,
  message_id TEXT NOT NULL,
  direction TEXT NOT NULL CHECK (direction IN ('INBOUND','OUTBOUND')),
  source_channel TEXT NOT NULL,
  channel_actor_id TEXT NULL,
  brain_provider TEXT NULL,
  body_text TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (world_id, entity_id, message_id),
  FOREIGN KEY (world_id, entity_id, principal_id, conversation_id)
    REFERENCES secretary_conversations(world_id, entity_id, principal_id, conversation_id)
);

CREATE INDEX IF NOT EXISTS idx_secretary_conversation_context
  ON secretary_conversation_messages(world_id, entity_id, principal_id, conversation_id, created_at);
```

پیاده سازی: __init__.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/__init__.py

SHA-256 منبع:

f025eb95073a23e7fe824e83afcdd1426adfff198c0b5d92eb79bf9b118fd890

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from .secretary import Secretary001
```

پیاده سازی: brain_portability.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/brain_portability.py

SHA-256 منبع:

b39662cc2b09d5cd01acf9823191188fe1e66dc2b2cc8bbafb30055dcf43b8b0

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Secretary-001 bridge to the provider-neutral RC2 Portable Brain runtime.

This bridge reads canonical entity state, creates a profile-bound BrainRequest,
and returns a proposal. It never persists a model answer and never executes an
external effect on its own.
"""
from __future__ import annotations

from core.brain_gateway import BrainInputSegment, BrainRequest
from core.portable_brain import PortableSecretaryResult, PortableSecretaryRunner

from .resolution_profile import (
    CONVERSATION_PROFILE,
    TASK_PROFILE,
    conversation_dict,
    task_dict,
)

class PortableSecretaryService:
    def __init__(self, secretary, runner: PortableSecretaryRunner) -> None:
        if runner.pack.entity_id != "secretary-001":
            raise ValueError("Portable Brain Pack is not bound to secretary-001")
        self.secretary = secretary
        self.runner = runner

    def decide_task(self, task_id: str) -> PortableSecretaryResult:
        """Ask any compatible Brain to propose a response for one canonical task."""
        task = self.secretary.store.tasks[task_id]
        profile = TASK_PROFILE
        segment = BrainInputSegment(
            segment_id="task",
            canonical=task_dict(task),
            profile_id=profile.profile_id,
            profile_version=profile.version,
            profile_hash=profile.profile_hash,
            source_ref=f"task:{task.task_id}",
            source_version=0,
            desired_resolution=self.secretary.execution_resolution,
            minimum_resolution="R0",
            purpose="TASK_PROPOSAL",
            data_class="INTERNAL",
            freshness="CURRENT",
        )
        return self.runner.run(
            BrainRequest(
                "world-v6.brain-request.v2",
                "secretary.task",
                (segment,),
                world_id="world-v6",
                entity_id="secretary-001",
                principal_id="human-root",
                conversation_id=self.secretary.DEFAULT_CONVERSATION_ID,
            )
        )

    def decide_latest_message(
        self,
        *,
```

```

principal_id: str = "human-root",
conversation_id: str | None = None,
) -> PortableSecretaryResult:
    """Propose a reply from the latest canonical conversation message."""
    conversation_id = conversation_id or self.secretary.DEFAULT_CONVERSATION_ID
    messages = self.secretary.store.conversation_context(principal_id, conversation_id, 1)
    if not messages:
        raise LookupError("no canonical conversation message is available")
    message = messages[-1]
    profile = CONVERSATION_PROFILE
    segment = BrainInputSegment(
        segment_id="conversation",
        canonical=conversation_dict(message),
        profile_id=profile.profile_id,
        profile_version=profile.version,
        profile_hash=profile.profile_hash,
        source_ref=f"conversation-message:{message.message_id}",
        source_version=0,
        desired_resolution=self.secretary.execution_resolution,
        minimum_resolution="R0",
        purpose="DRAFT_REPLY",
        data_class="INTERNAL",
        freshness="APPEND_ONLY",
    )
    return self.runner.run(
        BrainRequest(
            "world-v6.brain-request.v2",
            "secretary.reply",
            (segment,),
            world_id="world-v6",
            entity_id="secretary-001",
            principal_id=principal_id,
            conversation_id=conversation_id,
        )
    )

```


پیاده سازی: channels.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/channels.py

SHA-256 منبع:

5b21c59009ec2ca3d3a674365fc0baaf0e6ed8eee8be27bbdc1ff83cde96fde4

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations
from dataclasses import dataclass
from uuid import uuid4

@dataclass(frozen=True)
class NormalizedInbound:
    channel: str
    channel_actor_id: str
    principal_id: str
    text: str
    authenticated: bool
    conversation_id: str = "human-root:secretary-001"
    message_id: str = ""

    def __post_init__(self):
        if not self.message_id:
            object.__setattr__(self, "message_id", f"msg_{uuid4().hex}")

def normalize_chatgpt(actor_id: str, text: str, *, principal_id: str = "human-root",
                    conversation_id: str = "human-root:secretary-001") -> NormalizedInbound:
    return NormalizedInbound(
        channel="chatgpt",
        channel_actor_id=str(actor_id),
        principal_id=principal_id,
        text=text.strip(),
        authenticated=True,
        conversation_id=conversation_id,
    )

def normalize_telegram(actor_id: str, text: str, verified_webhook: bool, *,
                    principal_id: str = "unbound",
                    conversation_id: str = "human-root:secretary-001",
                    message_id: str | None = None) -> NormalizedInbound:
    # Transport verification is mandatory; message text itself is data, never authority.
    return NormalizedInbound(
        channel="telegram",
        channel_actor_id=str(actor_id),
        principal_id=principal_id,
        text=text.strip(),
        authenticated=bool(verified_webhook),
        conversation_id=conversation_id,
        message_id=message_id or "",
    )
```

پیاده سازی: letters.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/letters.py

SHA-256 منبع:

be2108213b9f517ed11fde37b5212910fb3eeb484178556d89911ac8909690ab

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations
from pathlib import Path
import shutil, subprocess, tempfile
from pptx import Presentation

def _replace_single_run(shape, text: str) -> None:
    p = shape.text_frame.paragraphs[0]
    if p.runs:
        p.runs[0].text = text
        for r in p.runs[1:]:
            r.text = ""
    else:
        p.text = text

def _replace_body(shape, body: str) -> None:
    ps = shape.text_frame.paragraphs
    values = ["بسمه تعالی", "", "با سلام و احترام", "", body, "", "", "", ""]
    for i, value in enumerate(values):
        if i >= len(ps):
            break
        if ps[i].runs:
            ps[i].runs[0].text = value
            for r in ps[i].runs[1:]:
                r.text = ""
        else:
            ps[i].text = value

def _replace_signature(shape, signatory: str, company: str) -> None:
    ps = shape.text_frame.paragraphs
    vals = ["با تشکر و احترام", signatory, company]
    for p, value in zip(ps, vals):
        if p.runs:
            p.runs[0].text = value
            for r in p.runs[1:]:
                r.text = ""
        else:
            p.text = value

def render_letter_pdf(req, template_pptx: str | Path, output_pdf: str | Path) -> Path:
    template_pptx, output_pdf = Path(template_pptx), Path(output_pdf)
    output_pdf.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="secretary_letter_") as td:
        td = Path(td)
        work = td / "letter.pptx"
        shutil.copy2(template_pptx, work)
        prs = Presentation(work)
        slide = prs.slides[0]
        # Preserve the supplied template's original text-run formatting/font/fallback metadata.
        _replace_single_run(slide.shapes[1], f"تاریخ: {req.document_date}")
        _replace_single_run(slide.shapes[2], f"شماره: \u202a{req.document_no}\u202c")
        _replace_single_run(slide.shapes[3], f"گیرنده: {req.recipient}")
        _replace_single_run(slide.shapes[4], f"موضوع: {req.subject}")
        _replace_body(slide.shapes[5], req.body)
        _replace_signature(slide.shapes[6], req.signatory, req.company)
        prs.save(work)
        office_binary = shutil.which("libreoffice") or shutil.which("soffice")
```

```

if office_binary is None:
    raise RuntimeError("LibreOffice/soffice is required to render letter PDF")
office_profile = td / "libreoffice-profile"
subprocess.run([
    office_binary,
    f"--env:UserInstallation={office_profile.as_uri()}",
    "--headless",
    "--convert-to",
    "pdf",
    "--outdir",
    str(td),
    str(work),
], check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
generated = td / "letter.pdf"
if not generated.exists():
    raise RuntimeError("LibreOffice did not generate letter.pdf")
shutil.copy2(generated, output_pdf)
return output_pdf

```

پیاده سازی: models.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/models.py

SHA-256 منبع:

8d50d59e701fa8e3de1f84597db69ce6d956f8a4502ba897b75f71061baf391f

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime, timezone
from decimal import Decimal
from typing import Optional, Literal
import uuid

from core.resolution import canonical_hash

def utcnow_iso() -> str:
    return datetime.now(timezone.utc).isoformat()

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid.uuid4().hex}"

@dataclass(frozen=True)
class ConversationMessage:
    principal_id: str
    conversation_id: str
    direction: Literal["INBOUND", "OUTBOUND"]
    channel: str
    text: str
    channel_actor_id: Optional[str] = None
    brain_provider: Optional[str] = None
    message_id: str = field(default_factory=lambda: new_id("msg"))
    created_at: str = field(default_factory=utcnow_iso)

@dataclass(frozen=True)
class LetterRequest:
    recipient: str
    subject: str
    body: str
    document_no: str
    document_date: str
    signatory: str = "فرخی - مدیرعامل"
    company: str = "شرکت فلات پارس"

@dataclass(frozen=True)
class ProformaItem:
    description: str
    quantity: Decimal
    unit: str
    unit_price_irr: Decimal

    @property
    def total_irr(self) -> Decimal:
        return self.quantity * self.unit_price_irr

@dataclass(frozen=True)
class ProformaRequest:
    customer_name: str
    customer_phone: str
    items: tuple[ProformaItem, ...]
```

```

document_no: str
document_date: str
validity_hours: int = 24
tax_rate: Decimal = Decimal("0")
notes: tuple[str, ...] = ()

@dataclass
class TaskRecord:
    title: str
    due_at: Optional[str] = None
    priority: Literal["LOW", "NORMAL", "HIGH", "URGENT"] = "NORMAL"
    domain: str = "general"
    goal_id: Optional[str] = None
    next_action: Optional[str] = None
    task_id: str = field(default_factory=lambda: new_id("task"))
    status: Literal["OPEN", "DONE", "CANCELLED"] = "OPEN"
    created_at: str = field(default_factory=utcnow_iso)

@dataclass(frozen=True)
class PriceKey:
    customer_id: str
    product_key: str
    unit: str

@dataclass
class PriceRecord:
    key: PriceKey
    unit_price_irr: Decimal
    valid_until: Optional[str] = None
    approved_by: str = "human-root"
    approved_at: str = field(default_factory=utcnow_iso)
    active: bool = True

@dataclass(frozen=True)
class OwnerInquiry:
    inquiry_id: str
    customer_id: str
    product_key: str
    unit: str
    requested_quantity: Decimal
    message: str
    
```

پیاده سازی: policy.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/policy.py

SHA-256 منبع:

d1b8f8857b8d4243996d73dd8071e0e96d979fcc60c4048c40b7510a5f48d1ab

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

from dataclasses import dataclass

from core.resolution import ResolutionMismatch
from .resolution_profile import check_action_resolution

@dataclass(frozen=True)
class Decision:
    allowed: bool
    reason: str
    approval_required: bool = False

def classify_action(
    action: str,
    *,
    effective_resolution: str,
    price_is_exact_active_match: bool = False,
) -> Decision:
    """Secretary-local classification inside the canonical Policy path.

    There is no legacy bypass: every runtime binding carries an explicit
    Resolution. This function may permit preparation of an effect proposal,
    but Entity code never receives permission to perform external I/O.
    """
    try:
        check_action_resolution(action, effective_resolution)
    except ResolutionMismatch as exc:
        return Decision(False, f"Resolution gate: {exc}")

    if action in {"DRAFT_LETTER", "RENDER_LETTER_PDF", "ADD_TASK", "LIST_TASKS"}:
        return Decision(True, "LO internal action")
    if action == "PRICE_LOOKUP":
        return Decision(True, "Read-only price lookup; field dependencies still apply")
    if action == "BUILD_PROFORMA_DRAFT":
        if price_is_exact_active_match:
            return Decision(True, "Uses exact active Human-Root-approved price")
        return Decision(False, "Missing/unapproved/changed price", approval_required=True)
    if action == "PROPOSE_EXTERNAL_EFFECT":
        return Decision(True, "May prepare a non-authoritative effect proposal only")
    if action == "EXECUTE_EXTERNAL":
        return Decision(
            False,
            "Entity-direct external I/O is forbidden; use Policy + bound approval + atomic Outbox + Executor",
            approval_required=True,
        )
    if action in {"COMMIT_PRICE", "LEGAL_FINANCIAL_COMMITMENT"}:
        return Decision(False, "Human Root approval and canonical execution path required", True)
    return Decision(False, "Unknown action: fail closed")
```

پیاده سازی: proforma.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/proforma.py

SHA-256 منبع:

65d34cb0f94a7d1c1f747e3069cd075153830b79920d036bf2aca8c9df09fff5

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations
from pathlib import Path
from decimal import Decimal
from html import escape
from weasyprint import HTML
from .models import ProformaRequest

def fmt_num(v: Decimal) -> str:
    return f"{v:,.0f}"

def render_proforma_pdf(req: ProformaRequest, output_pdf: str | Path) -> Path:
    subtotal = sum((i.total_irr for i in req.items), Decimal("0"))
    tax = (subtotal * req.tax_rate).quantize(Decimal("1"))
    grand = subtotal + tax
    rows = "".join(
        f"<tr><td>{n}</td><td>{escape(i.description)}</td><td>{fmt_num(i.quantity)}</td><td>{escape(i.unit)}</td><td>{fmt_num(i.unit_price_irr)}</td><td>{fmt_num(i.total_irr)}</td></tr>"
        for n, i in enumerate(req.items, 1)
    )
    notes = "".join(f"<li>{escape(n)}</li>" for n in req.notes)
    html = f'''<!doctype html><html lang="fa" dir="rtl"><head><meta charset="utf-8"><style>
@page {{ size:A4; margin:14mm; }}
body {{ font-family:'Noto Sans Arabic','DejaVu Sans',sans-serif; direction:rtl; font-size:11pt; }}
h1 {{ text-align:center; font-size:16pt; margin:0 0 12px; }}
.meta {{ display:flex; justify-content:space-between; border:1px solid #777; padding:8px; margin-bottom:8px; }}
table {{ width:100%; border-collapse:collapse; }} th,td {{ border:1px solid #777; padding:6px; text-align:center; }}
th {{ font-weight:700; }} .totals {{ margin-top:10px; width:45%; margin-right:auto; }}
.foot {{ margin-top:16px; font-size:9.5pt; }}
</style></head><body>
<h1>پیش فاکتور فروش کالا - شرکت کارن پلیمر فلات پارس</h1>
<div class="meta"><span>شماره: {escape(req.customer_phone)}</span><span>تلفن: {escape(req.customer_name)}</span><span>خریدار: {escape(req.customer_name)}</span></div>
<div class="meta"><span>تاریخ: {escape(req.document_date)}</span><span>شماره سند: {escape(req.document_no)}</span></div>
<table><thead><tr><th>مبلغ کل</th><th>مبلغ واحد</th><th>واحد</th><th>تعداد</th><th>شرح کالا/خدمات</th><th>ردیف</th></tr></thead><tbody>{rows}</tbody></table>
<table class="totals"><tr><td>جمع</td><td>{fmt_num(subtotal)}</td></tr><tr><td>مالیات/عوارض</td><td>{fmt_num(tax)}</td></tr><tr><td>مجموع قابل پرداخت</td><td>{fmt_num(grand)}</td></tr></table>
<div class="foot"><p>اعتبار قیمت: {req.validity_hours} ساعت.</p><p>این سند Draft است سیاست Draft این سند</p><ul>{notes}</ul><p>تأیید لازم برای قیمت و ارسال طی شود</p></div>
</body></html>'''
    output_pdf = Path(output_pdf); output_pdf.parent.mkdir(parents=True, exist_ok=True)
    HTML(string=html).write_pdf(str(output_pdf))
    return output_pdf
```

پیاده سازی: resolution_profile.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/resolution_profile.py

SHA-256 منبع:

4fb04b818aa0c146a78ae5a0c24fb2d2689d9597f5691e9bf7579e7e4e85cae7

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Secretary-001 bindings to the single versioned Resolution Profile artifacts."""
from __future__ import annotations

from pathlib import Path
from typing import Any

from core.resolution import (
    ProjectionResult,
    load_profile_file,
    project,
    require_action_compatibility,
    require_action_resolution,
)

_PROFILE_DIR = Path(__file__).resolve().parents[5] / "profiles"

CONVERSATION_PROFILE = load_profile_file(
    _PROFILE_DIR / "secretary-conversation-profile.v0.2.json"
)
TASK_PROFILE = load_profile_file(_PROFILE_DIR / "secretary-task-profile.v0.2.json")
PRICE_PROFILE = load_profile_file(_PROFILE_DIR / "secretary-price-profile.v0.2.json")
EFFECT_PROFILE = load_profile_file(_PROFILE_DIR / "secretary-effect-profile.v0.2.json")

ACTION_MINIMUM_RESOLUTION: dict[str, str] = {
    "DRAFT_LETTER": "R0",
    "RENDER_LETTER_PDF": "R0",
    "ADD_TASK": "R0",
    "LIST_TASKS": "R0",
    "BUILD_PROFORMA_DRAFT": "R1",
    "PRICE_LOOKUP": "R1",
    "PROPOSE_EXTERNAL_EFFECT": "R1",
    "EXECUTE_EXTERNAL": "R1",
    "COMMIT_PRICE": "R2",
    "LEGAL_FINANCIAL_COMMITMENT": "R2",
}

def minimum_for_action(action: str) -> str:
    # Unknown actions are denied by Policy; R2 also prevents accidental coarse execution.
    return ACTION_MINIMUM_RESOLUTION.get(action, "R2")

def check_action_resolution(action: str, effective_resolution: str) -> None:
    require_action_resolution(
        effective_resolution=effective_resolution,
        minimum_required=minimum_for_action(action),
    )

def check_price_action(action: str, projection: ProjectionResult) -> None:
    require_action_compatibility(profile=PRICE_PROFILE, action=action, projection=projection)

def conversation_dict(message) -> dict[str, Any]:
    return {
        "direction": message.direction,
        "text": message.text,
```



```

        "channel": message.channel,
        "brain_provider": message.brain_provider,
        "created_at": message.created_at,
        "message_id": message.message_id,
    }

def project_conversation_message(message, target_resolution: str) -> ProjectionResult:
    return project(
        conversation_dict(message),
        profile=CONVERSATION_PROFILE,
        target_resolution=target_resolution,
        source_ref=f"conversation-message:{message.message_id}",
        source_version=0,
        purpose="BRAIN_CONTEXT",
        data_class="INTERNAL",
        freshness="APPEND_ONLY",
    )

def task_dict(task) -> dict[str, Any]:
    return {
        "task_id": task.task_id,
        "title": task.title,
        "status": task.status,
        "next_action": task.next_action,
        "due_at": task.due_at,
        "priority": task.priority,
        "domain": task.domain,
        "goal_id": task.goal_id,
        "created_at": task.created_at,
    }

def project_task(task, target_resolution: str) -> ProjectionResult:
    return project(
        task_dict(task),
        profile=TASK_PROFILE,
        target_resolution=target_resolution,
        source_ref=f"task:{task.task_id}",
        source_version=0,
        purpose="TASK_VIEW",
        data_class="INTERNAL",
        freshness="CURRENT",
    )

def price_dict(price) -> dict[str, Any]:
    return {
        "customer_id": price.key.customer_id,
        "product_key": price.key.product_key,
        "unit": price.key.unit,
        "active": price.active,
        "unit_price_irr": str(price.unit_price_irr),
        "valid_until": price.valid_until,
        "approved_by": price.approved_by,
        "approved_at": price.approved_at,
    }

def project_price(price, target_resolution: str) -> ProjectionResult:
    source_ref = f"price:{price.key.customer_id}:{price.key.product_key}:{price.key.unit}"
    return project(
        price_dict(price),
        profile=PRICE_PROFILE,
        target_resolution=target_resolution,
        source_ref=source_ref,
        source_version=0,
        purpose="PRICE_DECISION",
        data_class="CONFIDENTIAL",
        freshness="VALIDITY_BOUND",
    )

```

پیاده سازی: secretary.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/secretary.py

SHA-256 منبع:

2bf354b05040a619bd65140344426fe6944646b2539b3df34774f1354aeaaf09

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations

from decimal import Decimal
from pathlib import Path

from core.effects import ExternalEffectProposal, sha256_bytes
from core.resolution import ResolutionError, resolution_rank

from .letters import render_letter_pdf
from .models import (
    ConversationMessage,
    LetterRequest,
    PriceKey,
    ProformaItem,
    ProformaRequest,
    TaskRecord,
)
from .policy import classify_action
from .proforma import render_proforma_pdf
from .resolution_profile import (
    check_price_action,
    project_conversation_message,
    project_price,
    project_task,
)
from .store import InMemorySecretaryStore

class DirectExternalIOForbidden(PermissionError):
    pass

class Secretary001:
    DEFAULT_CONVERSATION_ID = "human-root:secretary-001"

    def __init__(
        self,
        store=None,
        base_dir: str | Path | None = None,
        execution_resolution: str = "R0",
    ):
        if execution_resolution is None:
            raise ResolutionError("execution_resolution is mandatory; legacy bypass is removed")
        if resolution_rank(execution_resolution) > 1:
            raise ResolutionError("secretary-001 RC1 supports only R0 and R1")
        self.store = store or InMemorySecretaryStore()
        self.base_dir = Path(base_dir or Path(__file__).resolve().parents[1])
        self.execution_resolution = execution_resolution

    def _policy(self, action: str, **kwargs):
        return classify_action(action, effective_resolution=self.execution_resolution, **kwargs)

    def ingest_message(self, inbound):
        """Persist channel-neutral conversation before any Brain is selected."""
        if not inbound.authenticated:
            raise PermissionError("unauthenticated transport cannot enter the trusted conversation spine")
        return self.store.append_conversation_message(ConversationMessage(
            principal_id=inbound.principal_id,
            conversation_id=inbound.conversation_id,
```

```

        direction="INBOUND",
        channel=inbound.channel,
        channel_actor_id=inbound.channel_actor_id,
        text=inbound.text,
        message_id=inbound.message_id,
    ))

def record_reply(
    self,
    text: str,
    *,
    provider: str,
    channel: str,
    principal_id: str = "human-root",
    conversation_id: str = DEFAULT_CONVERSATION_ID,
):
    return self.store.append_conversation_message(ConversationMessage(
        principal_id=principal_id,
        conversation_id=conversation_id,
        direction="OUTBOUND",
        channel=channel,
        text=text,
        brain_provider=provider,
    ))

def brain_context(
    self,
    *,
    principal_id: str = "human-root",
    conversation_id: str = DEFAULT_CONVERSATION_ID,
    limit: int = 50,
    resolution: str | None = None,
) -> list[dict]:
    """Return provider-safe envelopes, never provenance-free raw views."""
    target = resolution or self.execution_resolution
    rows = self.store.conversation_context(principal_id, conversation_id, limit)
    return [project_conversation_message(message, target).consumer_envelope() for message in rows]

def task_view(self, task_id: str, resolution: str | None = None) -> dict:
    projection = project_task(self.store.tasks[task_id], resolution or self.execution_resolution)
    return projection.consumer_envelope()

def price_view(
    self,
    customer_id: str,
    product_key: str,
    unit: str,
    resolution: str | None = None,
) -> dict | None:
    price = self.store.get_active_price(PriceKey(customer_id, product_key, unit))
    if price is None:
        return None
    return project_price(price, resolution or self.execution_resolution).consumer_envelope()

def make_letter(self, request: LetterRequest, output_pdf: str | Path):
    decision = self._policy("RENDER_LETTER_PDF")
    if not decision.allowed:
        raise PermissionError(decision.reason)
    return render_letter_pdf(
        request,
        self.base_dir / "assets" / "letterhead_template.pptx",
        output_pdf,
    )

def add_task(self, **kwargs):
    decision = self._policy("ADD_TASK")
    if not decision.allowed:
        raise PermissionError(decision.reason)
    return self.store.add_task(TaskRecord(**kwargs))

def resolve_price_or_ask_root(
    self,
    customer_id: str,
    product_key: str,
    unit: str,
    quantity: Decimal,
) -> dict:
    price = self.store.get_active_price(PriceKey(customer_id, product_key, unit))
    if price is not None:
        projection = project_price(price, self.execution_resolution)
        if self.execution_resolution == "R1":

```

```

        check_price_action("PRICE_LOOKUP", projection)
        return {"status": "PRICE_FOUND", "projection": projection.consumer_envelope()}
    inquiry = self.store.create_price_inquiry(customer_id, product_key, unit, quantity)
    return {"status": "ASK_ROOT", "inquiry": inquiry}

def make_proforma_from_known_price(
    self,
    *,
    customer_id: str,
    customer_name: str,
    customer_phone: str,
    product_key: str,
    description: str,
    quantity: Decimal,
    unit: str,
    document_no: str,
    document_date: str,
    output_pdf: str | Path,
):
    decision = self._policy("BUILD_PROFORMA_DRAFT", price_is_exact_active_match=True)
    if not decision.allowed:
        raise PermissionError(decision.reason)
    price = self.store.get_active_price(PriceKey(customer_id, product_key, unit))
    if price is None:
        inquiry = self.store.create_price_inquiry(customer_id, product_key, unit, quantity)
        return {"status": "ASK_ROOT", "inquiry": inquiry}

    projection = project_price(price, self.execution_resolution)
    check_price_action("BUILD_PROFORMA_DRAFT", projection)
    exact = projection.value
    request = ProformaRequest(
        customer_name,
        customer_phone,
        (ProformaItem(description, quantity, unit, Decimal(exact["unit_price_irr"])),),
        document_no,
        document_date,
        notes=(".هزینه حمل بر عهده خریدار است مگر خلاف آن صراحتاً تأیید شود"),
    )
    pdf = render_proforma_pdf(request, output_pdf)
    return {
        "status": "DRAFT_READY",
        "pdf": str(pdf),
        "price_projection": projection.consumer_envelope(),
    }

def propose_pdf_delivery(
    self,
    pdf_path: str | Path,
    *,
    command_id: str,
    recipient_ref: str,
    expected_version: int,
    control_epoch: int,
    idempotency_key: str,
    policy_version: str = "1.2.0",
) -> ExternalEffectProposal:
    """Prepare a bound proposal; no network call and no authority are created."""
    decision = self._policy("PROPOSE_EXTERNAL_EFFECT")
    if not decision.allowed:
        raise PermissionError(decision.reason)
    path = Path(pdf_path)
    if not path.is_file() or path.suffix.lower() != ".pdf":
        raise ValueError("only an existing PDF may be proposed for delivery")
    payload_hash = sha256_bytes(path.read_bytes())
    return ExternalEffectProposal(
        world_id="world-v6",
        entity_id="secretary-001",
        command_id=command_id,
        destination="telegram",
        action="SEND_DOCUMENT",
        resource_ref=f"artifact:{path.name}",
        recipient_ref=recipient_ref,
        payload_ref=f"sha256:{payload_hash}",
        payload_hash=payload_hash,
        policy_version=policy_version,
        expected_version=expected_version,
        control_epoch=control_epoch,
        idempotency_scope="secretary-001:telegram-document",
        idempotency_key=idempotency_key,
        effect_semantics="RECONCILABLE",
    )

```

```
def send_pdf_to_root_telegram(self, *args, **kwargs):
    raise DirectExternalIOForbidden(
        "Direct send was removed in RC1; propose the effect, bind Human Root approval, "
        "commit Command/Event/Outbox atomically, then let the registered Executor deliver it"
    )

def send_latest_archived_letter_to_telegram(self, *args, **kwargs):
    raise DirectExternalIOForbidden(
        "Entity-local Drive/Telegram chaining is forbidden; use registered read and effect Executors"
    )
```

پیاده سازی: store.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/entities/_GESTATING/01_secretary-001/src/store.py

SHA-256 منبع:

3ee79af5d45c6960b7d842d532a581a5163c11c582c89c6b527a8f5ab7db0a5d

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
from __future__ import annotations
from datetime import datetime, timezone
from decimal import Decimal
from typing import Optional
from .models import TaskRecord, PriceKey, PriceRecord, OwnerInquiry, ConversationMessage, new_id

class InMemorySecretaryStore:
    """Test/dev repository. Canonical production adapter is PostgreSQL, not this class."""
    def __init__(self) -> None:
        self.tasks: dict[str, TaskRecord] = {}
        self.prices: dict[tuple[str, str, str], PriceRecord] = {}
        self.inquiries: dict[str, OwnerInquiry] = {}
        self.events: list[dict] = []
        self.conversation_messages: list[ConversationMessage] = []

    def append_conversation_message(self, msg: ConversationMessage) -> ConversationMessage:
        self.conversation_messages.append(msg)
        self.events.append({
            "type": "CONVERSATION.MESSAGE_RECORDED",
            "message_id": msg.message_id,
            "principal_id": msg.principal_id,
            "conversation_id": msg.conversation_id,
            "channel": msg.channel,
            "direction": msg.direction,
            "brain_provider": msg.brain_provider,
        })
        return msg

    def conversation_context(self, principal_id: str, conversation_id: str, limit: int = 50) -> list[ConversationMessage]:
        rows = [m for m in self.conversation_messages
                 if m.principal_id == principal_id and m.conversation_id == conversation_id]
        return rows[-limit:]

    def add_task(self, task: TaskRecord) -> TaskRecord:
        self.tasks[task.task_id] = task
        self.events.append({"type": "TASK.CREATED", "task_id": task.task_id})
        return task

    def complete_task(self, task_id: str) -> TaskRecord:
        task = self.tasks[task_id]
        task.status = "DONE"
        self.events.append({"type": "TASK.COMPLETED", "task_id": task_id})
        return task

    def put_price(self, price: PriceRecord) -> None:
        key = (price.key.customer_id, price.key.product_key, price.key.unit)
        self.prices[key] = price
        self.events.append({"type": "PRICE.APPROVED", "key": key, "approved_by": price.approved_by})

    def get_active_price(self, key: PriceKey) -> Optional[PriceRecord]:
        rec = self.prices.get((key.customer_id, key.product_key, key.unit))
        if not rec or not rec.active:
            return None
        if rec.valid_until:
            try:
                expiry = datetime.fromisoformat(rec.valid_until.replace("Z", "+00:00"))
                if expiry < datetime.now(timezone.utc):
                    return None
            except:
```

```

        except ValueError:
            return None
        return rec

    def create_price_inquiry(self, customer_id: str, product_key: str, unit: str, qty: Decimal) -> OwnerInquiry:
        inquiry = OwnerInquiry(
            inquiry_id=new_id("priceq"), customer_id=customer_id, product_key=product_key,
            unit=unit, requested_quantity=qty,
            message=f"قیمت می‌خواهد؛ قیمت فعال/معتبر پیدا نشد. لطفاً {product_key} از {unit} {qty} برای {customer_id} مشتری"
        )
        self.inquiries[inquiry.inquiry_id] = inquiry
        self.events.append({"type": "PRICE.ROOT_INQUIRY_CREATED", "inquiry_id": inquiry.inquiry_id})
        return inquiry

```

پیاده سازی: telegram.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/adapters/telegram.py

SHA-256 منبع:

9d6e618f8941d1ef034046ee4b8cd3199a20a18ca3e9e052089a600a9bcb7bec

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Registered Telegram transport adapter candidate.

Telegram is only a transport. Entity identity/conversation/state live outside Telegram.
Secrets come only from environment/secret store.
"""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
import json
import mimetypes
import os
import secrets
import urllib.request
import urllib.error
from src.channels import normalize_telegram

class TelegramDeliveryError(RuntimeError):
    pass

@dataclass(frozen=True)
class TelegramRuntimeConfig:
    bot_token: str
    root_user_id: str | None
    root_chat_id: str | None = None
    root_username: str = "saeedfaut"
    webhook_secret: str | None = None

    @classmethod
    def from_env(cls):
        token = os.environ.get("TELEGRAM_BOT_TOKEN")
        root = os.environ.get("TELEGRAM_ROOT_USER_ID")
        chat_id = os.environ.get("TELEGRAM_ROOT_CHAT_ID") or root
        username = os.environ.get("TELEGRAM_ROOT_USERNAME", "saeedfaut").rstrip("@")
        if not token:
            raise RuntimeError("TELEGRAM_BOT_TOKEN must come from Secret Store/environment")
        return cls(token, root, chat_id, username, os.environ.get("TELEGRAM_WEBHOOK_SECRET"))

def is_declared_root_handle(username: str | None, config: TelegramRuntimeConfig) -> bool:
    """Setup hint only; username is not sufficient authentication."""
    return bool(username) and username.rstrip("@").casefold() == config.root_username.casefold()

def is_root_actor(actor_id: str, config: TelegramRuntimeConfig) -> bool:
    """Authority is granted only by the verified immutable Telegram numeric user id."""
    return bool(config.root_user_id) and str(actor_id) == str(config.root_user_id)

def normalize_update(update: dict, *, verified_transport: bool, config: TelegramRuntimeConfig):
    message = update.get("message") or update.get("edited_message") or {}
    user = message.get("from") or {}
    actor_id = str(user.get("id", "unknown"))
    text = message.get("text") or message.get("caption") or ""
    telegram_message_id = str(message.get("message_id") or "")
    principal_id = "human-root" if is_root_actor(actor_id, config) else f"telegram:{actor_id}"
    return normalize_telegram(
        actor_id,
```



```

        text,
        verified_transport,
        principal_id=principal_id,
        conversation_id=("human-root:secretary-001" if principal_id == "human-root" else f"{principal_id}:secretary-001"),
        message_id=(f"telegram:{actor_id}:{telegram_message_id}" if telegram_message_id else None),
    )

class TelegramBotClient:
    """Small Bot API client for outbound text/document delivery.

    This class is not callable from Entity private code. The canonical Executor
    must supply a committed Outbox effect after policy, approval, version and
    control_epoch rechecks.
    """

    def __init__(self, config: TelegramRuntimeConfig, *, timeout: float = 30.0, opener=None):
        self.config = config
        self.timeout = timeout
        self._opener = opener or urllib.request.urlopen

    def _url(self, method: str) -> str:
        return f"https://api.telegram.org/bot{self.config.bot_token}/{method}"

    def _decode(self, raw: bytes) -> dict:
        try:
            payload = json.loads(raw.decode("utf-8"))
        except Exception as exc:
            raise TelegramDeliveryError("Telegram returned a non-JSON response") from exc
        if not payload.get("ok"):
            raise TelegramDeliveryError(payload.get("description") or "Telegram Bot API request failed")
        return payload

    def send_message(self, text: str, *, chat_id: str | None = None) -> dict:
        target = str(chat_id or self.config.root_chat_id or "")
        if not target:
            raise TelegramDeliveryError("TELEGRAM_ROOT_CHAT_ID/USER_ID is not bound")
        body = json.dumps({"chat_id": target, "text": text}, ensure_ascii=False).encode("utf-8")
        req = urllib.request.Request(self._url("sendMessage"), data=body, headers={"Content-Type": "application/json"},
        method="POST")
        try:
            with self._opener(req, timeout=self.timeout) as resp:
                return self._decode(resp.read())
        except urllib.error.HTTPError as exc:
            detail = exc.read().decode("utf-8", errors="replace")
            raise TelegramDeliveryError(f"Telegram HTTP {exc.code}: {detail}") from exc
        except urllib.error.URLError as exc:
            raise TelegramDeliveryError(f"Telegram network error: {exc.reason}") from exc

    def send_document(self, file_path: str | Path, *, caption: str | None = None,
        chat_id: str | None = None) -> dict:
        path = Path(file_path)
        if not path.is_file():
            raise FileNotFoundError(path)
        return self.send_document_bytes(
            path.read_bytes(), filename=path.name, caption=caption, chat_id=chat_id
        )

    def send_document_bytes(self, data: bytes, *, filename: str, caption: str | None = None,
        chat_id: str | None = None, mime_type: str | None = None) -> dict:
        """Send in-memory document bytes. Useful for PDFs downloaded from Drive/cloud storage."""
        if not data:
            raise TelegramDeliveryError("document payload is empty")
        target = str(chat_id or self.config.root_chat_id or "")
        if not target:
            raise TelegramDeliveryError("TELEGRAM_ROOT_CHAT_ID/USER_ID is not bound")

        boundary = "---WorldV6" + secrets.token_hex(12)
        chunks: list[bytes] = []

        def add_field(name: str, value: str) -> None:
            chunks.extend([
                f"--{boundary}\r\n".encode(),
                f'Content-Disposition: form-data; name="{name}"\r\n\r\n'.encode(),
                value.encode("utf-8"), b"\r\n",
            ])

        add_field("chat_id", target)
        if caption:
            add_field("caption", caption[:1024])
    
```

```

mime = mime_type or mimetypes.guess_type(filename)[0] or "application/octet-stream"
chunks.extend([
    f"--{boundary}\r\n".encode(),
    f'Content-Disposition: form-data; name="document"; filename="{filename}"\r\n'.encode("utf-8"),
    f'Content-Type: {mime}\r\n\r\n'.encode(),
    data, b"\r\n",
    f"--{boundary}--\r\n".encode(),
])
body = b"".join(chunks)
req = urllib.request.Request(
    self._url("sendDocument"), data=body,
    headers={"Content-Type": f"multipart/form-data; boundary={boundary}"}, method="POST",
)
try:
    with self._opener(req, timeout=self.timeout) as resp:
        return self._decode(resp.read())
except urllib.error.HTTPError as exc:
    detail = exc.read().decode("utf-8", errors="replace")
    raise TelegramDeliveryError(f"Telegram HTTP {exc.code}: {detail}") from exc
except urllib.error.URLError as exc:
    raise TelegramDeliveryError(f"Telegram network error: {exc.reason}") from exc

def root_price_inquiry_text(inquiry) -> str:
    return (
        "\nمنش - استعلام قیمت جدید"
        f"مشتري: {inquiry.customer_id}\n"
        f"كالا: {inquiry.product_key}\n"
        f"مقدار: {inquiry.requested_quantity} {inquiry.unit}\n"
        f"شناسه استعلام: {inquiry.inquiry_id}\n"
        ".قیمت فعال/معتبر برای این مشتری و کالا پیدا نشد. لطفاً قیمت را اعلام و تأیید کنید"
    )

```

پیاده سازی: drive_archive.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/adapters/drive_archive.py

SHA-256 منبع:

180d047e4bb378c0d3238cbcc56a4846f0dda710d970561b11c5a732d88452c5

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Registered Google Drive archive reader candidate.

It is intentionally outside Entity private code. A registered Core read adapter
may fetch bytes; provenance is then carried into the canonical command/effect path.
Secrets/tokens must come from runtime secret injection or an injected client.
"""
from __future__ import annotations
from dataclasses import dataclass
import json
import os
import urllib.parse
import urllib.request
import urllib.error

class DriveArchiveError(RuntimeError):
    pass

@dataclass(frozen=True)
class ArchivedDocument:
    file_id: str
    name: str
    mime_type: str
    modified_time: str | None = None
    size: int | None = None

@dataclass(frozen=True)
class DriveArchiveConfig:
    access_token: str
    letters_folder_id: str = "1CP3dc0EeC31us3GXneH3ftJaoAe_aefj"

    @classmethod
    def from_env(cls, *, letters_folder_id: str | None = None):
        token = os.environ.get("GOOGLE_DRIVE_ACCESS_TOKEN")
        if not token:
            raise RuntimeError(
                "GOOGLE_DRIVE_ACCESS_TOKEN must come from Secret Store/runtime, "
                "or inject a DriveArchiveClient backed by the host's Google credential"
            )
        return cls(
            access_token=token,
            letters_folder_id=(letters_folder_id or os.environ.get("LETTERS_DRIVE_FOLDER_ID")
                               or cls.letters_folder_id),
        )

class DriveArchiveClient:
    API = "https://www.googleapis.com/drive/v3"

    def __init__(self, config: DriveArchiveConfig, *, timeout: float = 30.0, opener=None):
        self.config = config
        self.timeout = timeout
        self._opener = opener or urllib.request.urlopen

    def _request(self, url: str) -> bytes:
        req = urllib.request.Request(
            url,
```

```

        headers={"Authorization": f"Bearer {self.config.access_token}"},
        method="GET",
    )
    try:
        with self._opener(req, timeout=self.timeout) as resp:
            return resp.read()
    except urllib.error.HTTPError as exc:
        detail = exc.read().decode("utf-8", errors="replace")
        raise DriveArchiveError(f"Google Drive HTTP {exc.code}: {detail}") from exc
    except urllib.error.URLError as exc:
        raise DriveArchiveError(f"Google Drive network error: {exc.reason}") from exc

def latest_pdf(self) -> ArchivedDocument:
    folder = self.config.letters_folder_id
    query = f"'{folder}' in parents and trashed = false and mimeType = 'application/pdf'"
    params = urllib.parse.urlencode({
        "q": query,
        "orderBy": "modifiedTime desc",
        "pageSize": "1",
        "fields": "files(id,name,mimeType,modifiedTime,size)",
        "supportsAllDrives": "true",
        "includeItemsFromAllDrives": "true",
    })
    raw = self._request(f"{self.API}/files?{params}")
    try:
        payload = json.loads(raw.decode("utf-8"))
    except Exception as exc:
        raise DriveArchiveError("Google Drive list response was not valid JSON") from exc
    files = payload.get("files") or []
    if not files:
        raise DriveArchiveError("No PDF found in the configured letters archive")
    f = files[0]
    return ArchivedDocument(
        file_id=str(f["id"]),
        name=str(f["name"]),
        mime_type=str(f.get("mimeType") or "application/pdf"),
        modified_time=f.get("modifiedTime"),
        size=(int(f["size"]) if f.get("size") is not None else None),
    )

def download_bytes(self, document: ArchivedDocument) -> bytes:
    if document.mime_type != "application/pdf" and not document.name.lower().endswith(".pdf"):
        raise DriveArchiveError(f"Refusing non-PDF archive object: {document.name}")
    file_id = urllib.parse.quote(document.file_id, safe="")
    return self._request(f"{self.API}/files/{file_id}?alt=media&supportsAllDrives=true")

```

پیاده سازی: postgres_schema.sql

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/postgres_schema.sql

SHA-256 منبع:

f77ec7753d0dc3d436da573ccc4a9ec595434f818f4d776b3f8dc2237808c41a

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
-- World v6 Phase-1 canonical PostgreSQL authority schema.
-- State + Event metadata + optional Outbox intent share one transaction boundary.
CREATE TABLE IF NOT EXISTS worlds (
  world_id text PRIMARY KEY,
  constitution_version text NOT NULL,
  status text NOT NULL,
  control_epoch bigint NOT NULL DEFAULT 1,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS entities (
  world_id text NOT NULL REFERENCES worlds(world_id),
  entity_id text NOT NULL,
  entity_version text NOT NULL,
  dna_version text NOT NULL,
  dna_hash text NOT NULL,
  root_owner_ref text NOT NULL,
  parent_relation text NOT NULL CHECK (parent_relation IN ('ROOT_DIRECT','ENTITY_PARENT')),
  parent_entity_id text,
  lifecycle_stage text NOT NULL,
  operational_status text NOT NULL,
  lock_version bigint NOT NULL DEFAULT 1,
  control_epoch bigint NOT NULL DEFAULT 1,
  last_event_sequence bigint NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now(),
  archived boolean NOT NULL DEFAULT false,
  PRIMARY KEY (world_id, entity_id),
  CHECK ((parent_relation='ROOT_DIRECT' AND parent_entity_id IS NULL) OR
    (parent_relation='ENTITY_PARENT' AND parent_entity_id IS NOT NULL))
);

CREATE TABLE IF NOT EXISTS events (
  world_id text NOT NULL,
  entity_id text NOT NULL,
  event_id text PRIMARY KEY,
  entity_sequence bigint NOT NULL,
  event_type text NOT NULL,
  occurred_at timestamptz NOT NULL,
  causation_id text,
  correlation_id text,
  actor_json jsonb NOT NULL,
  payload_json jsonb,
  payload_hash text NOT NULL,
  schema_version text NOT NULL,
  policy_decision_ref text,
  approval_ref text,
  authorized_control_epoch bigint,
  outcome text,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (world_id, entity_id, entity_sequence),
  FOREIGN KEY (world_id, entity_id) REFERENCES entities(world_id, entity_id)
);

CREATE TABLE IF NOT EXISTS commands (
  world_id text NOT NULL,
  entity_id text NOT NULL,
  command_id text PRIMARY KEY,
  command_type text NOT NULL,
```

```

actor_json jsonb NOT NULL,
payload_json jsonb NOT NULL,
payload_hash text NOT NULL,
status text NOT NULL,
expected_version bigint,
idempotency_scope text,
idempotency_key text,
effect_hash text,
authorized_control_epoch bigint,
created_at timestamptz NOT NULL DEFAULT now(),
updated_at timestamptz NOT NULL DEFAULT now(),
FOREIGN KEY (world_id, entity_id) REFERENCES entities(world_id, entity_id),
UNIQUE (idempotency_scope, idempotency_key, effect_hash)
);

CREATE TABLE IF NOT EXISTS approvals (
  approval_id text PRIMARY KEY,
  world_id text NOT NULL,
  entity_id text NOT NULL,
  command_id text NOT NULL REFERENCES commands(command_id),
  approver_ref text NOT NULL,
  decision text NOT NULL,
  action text NOT NULL,
  resource_ref text,
  recipient_ref text,
  payload_hash text NOT NULL,
  policy_version text NOT NULL,
  expected_version bigint,
  issued_at timestamptz NOT NULL,
  expires_at timestamptz,
  FOREIGN KEY (world_id, entity_id) REFERENCES entities(world_id, entity_id)
);

CREATE TABLE IF NOT EXISTS outbox (
  outbox_id text PRIMARY KEY,
  world_id text NOT NULL,
  entity_id text NOT NULL,
  command_id text NOT NULL REFERENCES commands(command_id),
  destination text NOT NULL,
  action text NOT NULL,
  resource_ref text,
  recipient_ref text,
  payload_json jsonb,
  payload_ref text,
  payload_hash text NOT NULL,
  effect_hash text NOT NULL,
  idempotency_scope text NOT NULL,
  idempotency_key text NOT NULL,
  approval_ref text,
  policy_decision_ref text NOT NULL,
  authorized_control_epoch bigint NOT NULL,
  effect_semantics text NOT NULL CHECK (effect_semantics IN ('NATIVE_IDEMPOTENT', 'RECONCILABLE', 'NON_IDEMPOTENT')),
  status text NOT NULL DEFAULT 'QUEUED',
  attempts integer NOT NULL DEFAULT 0,
  provider_receipt text,
  last_error text,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  FOREIGN KEY (world_id, entity_id) REFERENCES entities(world_id, entity_id),
  UNIQUE (idempotency_scope, idempotency_key, effect_hash)
);

CREATE INDEX IF NOT EXISTS idx_events_stream ON events(world_id, entity_id, entity_sequence);
CREATE INDEX IF NOT EXISTS idx_outbox_status ON outbox(status, created_at);

```

پیاده سازی: kernel.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/kernel.py

SHA-256 منبع:

659bd4504088a8d9f69095771f0b119e2aea3a87bcfb0de60c257fe93299b772

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Minimal deterministic World-v6 Phase-1 Kernel primitives.

This module deliberately contains no secretary business logic and no model/provider calls.
A real deployment passes a PostgreSQL DB-API compatible connection factory.
"""
from __future__ import annotations
from dataclasses import dataclass
from datetime import datetime, timezone
import uuid
from .resolution import (
    ProjectionProfile,
    ResolutionPatch,
    apply_patch,
    canonical_hash,
    canonical_json,
)

def utcnow():
    return datetime.now(timezone.utc)

def canonical_json_hash(value) -> str:
    """Compatibility alias for the single World Canonical JSON hash contract."""
    return canonical_hash(value)

def new_id(prefix: str) -> str:
    return f"{prefix}_{uuid.uuid4().hex}"

@dataclass(frozen=True)
class BirthResult:
    entity_id: str
    event_id: str
    created_at: str
    dna_hash: str
    lifecycle_stage: str = "NEWBORN"
    operational_status: str = "READY"
    control_epoch: int = 1

class KernelConflict(RuntimeError): pass
class KernelPolicyError(RuntimeError): pass

class Kernel:
    """Deterministic transaction coordinator. Brain text never calls SQL directly."""
    def __init__(self, connection_factory):
        self.connection_factory = connection_factory

    def validate_resolution_patch(
        self,
        *,
        canonical_state: dict,
        profile: ProjectionProfile,
        patch: ResolutionPatch,
        current_version: int,
        source_ref: str,
```

```

) -> dict:
    """Validate/merge a projected leaf patch before any authoritative transaction.

    This helper grants no authority. Callers must still perform the canonical
    Policy/Approval/expected_version/transaction/Event path before persistence.
    """
    return apply_patch(
        canonical_state,
        profile=profile,
        patch=patch,
        current_version=current_version,
        source_ref=source_ref,
    )

def birth_entity(self, *, world_id: str, manifest: dict, actor: dict, policy_ref: str) -> BirthResult:
    if manifest.get("root_owner") != "human-root":
        raise KernelPolicyError("birth requires stable human-root owner role")
    if manifest.get("parent_relation") not in {"ROOT_DIRECT", "ENTITY_PARENT"}:
        raise KernelPolicyError("invalid parent_relation")
    if manifest.get("parent_relation") == "ROOT_DIRECT" and manifest.get("parent_entity_id"):
        raise KernelPolicyError("ROOT_DIRECT cannot carry parent_entity_id")
    created_at = utcnow()
    event_id = new_id("evt")
    born_dna = dict(manifest)
    born_dna["lifecycle_stage"] = "NEWBORN"
    born_dna["operational_status"] = "READY"
    born_dna["created_at"] = created_at.isoformat()
    born_dna["creation_event_ref"] = event_id
    dna_hash = canonical_json_hash(born_dna)
    payload = {"entity_id": manifest["entity_id"], "dna_hash": dna_hash, "state": "NEWBORN/READY"}
    payload_hash = canonical_json_hash(payload)

    conn = self.connection_factory()
    try:
        cur = conn.cursor()
        # Entity-id reservation + initial state + creation Event are one transaction.
        cur.execute(
            """INSERT INTO entities
            (world_id,entity_id,entity_version,dna_version,dna_hash,root_owner_ref,parent_relation,parent_entity_id,
             lifecycle_stage,operational_status,lock_version,control_epoch,last_event_sequence,created_at)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,'NEWBORN','READY',1,1,1,%s)""",
            (world_id, manifest["entity_id"], manifest["entity_version"], manifest["dna_version"], dna_hash,
             manifest["root_owner"], manifest["parent_relation"], manifest.get("parent_entity_id"), created_at),
        )
        cur.execute(
            """INSERT INTO events
            (world_id,entity_id,event_id,entity_sequence,event_type,occurred_at,actor_json,payload_json,payload_hash,
             schema_version,policy_decision_ref,authorized_control_epoch,outcome)
            VALUES (%s,%s,%s,1,'ENTITY.CREATED',%s,%s::jsonb,%s::jsonb,%s,'1.1.0',%s,1,'success')""",
            (world_id, manifest["entity_id"], event_id, created_at,
             canonical_json(actor), canonical_json(payload), payload_hash, policy_ref),
        )
        conn.commit()
        return BirthResult(manifest["entity_id"], event_id, created_at.isoformat(), dna_hash)
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

def wake_entity(self, *, world_id: str, entity_id: str, expected_version: int, actor: dict, policy_ref: str) -> dict:
    conn = self.connection_factory()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT lock_version,control_epoch,last_event_sequence,lifecycle_stage,operational_status FROM entities WHERE\n"
            f"world_id=%s AND entity_id=%s FOR UPDATE",
            (world_id, entity_id),
        )
        row = cur.fetchone()
        if not row: raise KernelConflict("entity not found")
        lock_version, epoch, seq, stage, status = row
        if lock_version != expected_version: raise KernelConflict("expected_version mismatch")
        if stage != "NEWBORN" or status != "READY": raise KernelConflict("WAKE requires NEWBORN/READY")
        event_id = new_id("evt")
        now = utcnow()
        payload = {"from": "NEWBORN/READY", "to": "NEWBORN/AWAKE"}
        cur.execute(
            "UPDATE entities SET\n"
            f"operational_status='AWAKE',lock_version=lock_version+1,last_event_sequence=last_event_sequence+1,updated_at=%s WHERE\n"
            f"world id=%s AND entity id=%s",
            (now, world_id, entity_id),
        )

```



```

        (now, world_id, entity_id),
    )
    cur.execute(
        """INSERT INTO events
        (world_id,entity_id,event_id,entity_sequence,event_type,occurred_at,actor_json,payload_json,payload_hash,
        schema_version,policy_decision_ref,authorized_control_epoch,outcome)
        VALUES (%s,%s,%s,%s,'LIFECYCLE.WAKE',%s,%s::jsonb,%s::jsonb,%s,'1.1.0',%s,%s,'success')""",
        (world_id, entity_id, event_id, seq+1, now, canonical_json(actor),
        canonical_json(payload), canonical_json_hash(payload), policy_ref, epoch),
    )
    conn.commit()
    return {"event_id": event_id, "status": "AWAKE", "lock_version": lock_version+1, "control_epoch": epoch}
except Exception:
    conn.rollback()
    raise
finally:
    conn.close()

```

پیاده سازی: resolution.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/resolution.py

SHA-256 منبع:

a81838af67ba5be793e7e632e2bcd82ea65da63ff24a4a67ddd4fd2119e57710

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Deterministic Resolution primitives for the World v6.2 RC1.

Resolution is a compatibility/view mechanism. It never grants authority,
changes identity, lowers data classification, or replaces canonical state.
Canonical state is always hashed as strict World Canonical JSON v1; projected
views are derived and non-authoritative.
"""
from __future__ import annotations

from copy import deepcopy
from dataclasses import dataclass
import hashlib
import json
import math
from pathlib import Path
import re
from typing import Any, Iterable, Literal, Mapping

DowngradePolicy = Literal["SAFE_TO_PROJECT", "READ_ONLY_WHEN_PROJECTED", "NO_DOWNGRADE"]
JsonType = Literal["string", "integer", "boolean", "null", "array", "object"]

_RES_RE = re.compile(r"^R(0|[1-9][0-9]*)$")
_TOKEN_RE = re.compile(r"^[A-Za-z0-9][A-Za-z0-9._:/-]{0,191}$")
_CLASS_RANK = {"PUBLIC": 0, "INTERNAL": 1, "CONFIDENTIAL": 2, "RESTRICTED": 3}
_MAX_SAFE_INTEGER = 9_007_199_254_740_991
_MAX_DOCUMENT_DEPTH = 64
_MAX_DOCUMENT_NODES = 100_000
_MAX_PROFILE_FIELDS = 4_096
_MAX_PATCH_OPERATIONS = 128

COMPILER_ID = "world-v6.structural-resolution-compiler"
COMPILER_VERSION = "0.2.0"
_COMPILER_CONTRACT = (
    "strict-json-v1|json-pointer-leaves|fail-closed-unknown|monotonic-fields|"
    "projection-derived-non-authoritative|bounded-existing-scalar-patch|"
    "canonical-reload-for-up-resolution"
)
_COMPILER_HASH = hashlib.sha256(_COMPILER_CONTRACT.encode("utf-8")).hexdigest()

class ResolutionError(ValueError):
    pass

class CanonicalJSONError(ResolutionError):
    pass

class ResolutionMismatch(ResolutionError):
    pass

class ProjectionError(ResolutionError):
    pass

class PatchRejected(ResolutionError):
    pass
```

```

def _validate_canonical_json(value: Any, *, _depth: int = 0, _counter: list[int] | None = None) -> None:
    """Validate the restricted, cross-language World Canonical JSON v1 domain.

    Floats and implicit stringification are forbidden. Decimal, datetime and
    other domain values must be normalized by their schema adapter first (for
    example a monetary Decimal becomes an exact decimal string).
    """
    if _counter is None:
        _counter = [0]
    _counter[0] += 1
    if _counter[0] > _MAX_DOCUMENT_NODES:
        raise CanonicalJSONError("canonical document exceeds node limit")
    if _depth > _MAX_DOCUMENT_DEPTH:
        raise CanonicalJSONError("canonical document exceeds depth limit")

    if value is None or isinstance(value, (str, bool)):
        return
    if isinstance(value, int) and not isinstance(value, bool):
        if abs(value) > _MAX_SAFE_INTEGER:
            raise CanonicalJSONError("integer exceeds World Canonical JSON safe range")
        return
    if isinstance(value, float):
        if not math.isfinite(value):
            raise CanonicalJSONError("non-finite number is forbidden")
        raise CanonicalJSONError("floats are forbidden; normalize exact values before hashing")
    if isinstance(value, list):
        for child in value:
            _validate_canonical_json(child, _depth=_depth + 1, _counter=_counter)
        return
    if isinstance(value, Mapping):
        for key, child in value.items():
            if not isinstance(key, str):
                raise CanonicalJSONError("canonical object keys must be strings")
            _validate_canonical_json(child, _depth=_depth + 1, _counter=_counter)
        return
    raise CanonicalJSONError(f"unsupported canonical JSON type: {type(value).__name__}")

def canonical_json(value: Any) -> str:
    _validate_canonical_json(value)
    return json.dumps(
        value,
        ensure_ascii=False,
        sort_keys=True,
        separators=(",", ":"),
        allow_nan=False,
    )

def canonical_hash(value: Any) -> str:
    return hashlib.sha256(canonical_json(value).encode("utf-8")).hexdigest()

def resolution_rank(level: str) -> int:
    if not isinstance(level, str):
        raise ResolutionError("resolution must be a string such as R0")
    match = _RES_RE.fullmatch(level)
    if not match:
        raise ResolutionError(f"invalid resolution: {level!r}")
    return int(match.group(1))

def lower_or_equal(a: str, b: str) -> bool:
    return resolution_rank(a) <= resolution_rank(b)

def negotiate_resolution(
    *, desired: str, minimum: str, brain_max: str, canonical_resolution: str | None = None
) -> str:
    """Select the highest supported resolution without weakening the minimum."""
    desired_rank, minimum_rank, brain_rank = map(
        resolution_rank, (desired, minimum, brain_max)
    )
    if desired_rank < minimum_rank:
        raise ResolutionMismatch("desired resolution is below task minimum")
    if canonical_resolution is not None:
        canonical_rank = resolution_rank(canonical_resolution)
        if desired_rank > canonical_rank or minimum_rank > canonical_rank:
            raise ResolutionMismatch("request exceeds the profile canonical resolution")

```

```

        brain_rank = min(brain_rank, canonical_rank)
    effective_rank = min(desired_rank, brain_rank)
    if effective_rank < minimum_rank:
        raise ResolutionMismatch(
            f"brain supports only R{brain_rank}, but task requires at least R{minimum_rank}"
        )
    return f"R{effective_rank}"

def _validate_token(value: str, label: str) -> None:
    if not isinstance(value, str) or not _TOKEN_RE.fullmatch(value):
        raise ResolutionError(f"invalid {label}: {value!r}")

def _decode_pointer(path: str) -> tuple[str, ...]:
    if not isinstance(path, str) or not path.startswith("/") or len(path) > 512:
        raise ResolutionError(f"path must be a bounded JSON Pointer: {path!r}")
    raw_parts = path[1:].split("/")
    if not raw_parts or any(part == "" for part in raw_parts):
        raise ResolutionError(f"empty JSON Pointer segment is forbidden: {path!r}")
    parts: list[str] = []
    for raw in raw_parts:
        index = 0
        decoded: list[str] = []
        while index < len(raw):
            if raw[index] != "~":
                decoded.append(raw[index])
                index += 1
                continue
            if index + 1 >= len(raw) or raw[index + 1] not in {"0", "1"}:
                raise ResolutionError(f"invalid JSON Pointer escape: {path!r}")
            decoded.append("~" if raw[index + 1] == "0" else "/")
            index += 2
        parts.append("".join(decoded))
    return tuple(parts)

def _encode_pointer(parts: Iterable[str]) -> str:
    return "/" + "/".join(part.replace("~", "~0").replace("/", "~1") for part in parts)

def _json_type(value: Any) -> JsonType:
    if value is None:
        return "null"
    if isinstance(value, bool):
        return "boolean"
    if isinstance(value, int):
        return "integer"
    if isinstance(value, str):
        return "string"
    if isinstance(value, list):
        return "array"
    if isinstance(value, Mapping):
        return "object"
    raise CanonicalJSONError(f"unsupported canonical JSON type: {type(value).__name__}")

def _profile_data_class(profile: "ProjectionProfile") -> str:
    return max(
        (field.data_class for field in profile.fields),
        key=lambda item: _CLASS_RANK[item],
    )

@dataclass(frozen=True)
class FieldRule:
    path: str
    introduced_at: str = "R0"
    write_min_resolution: str | None = None
    downgrade_policy: DowngradePolicy = "SAFE_TO_PROJECT"
    cable_id: str = "GENERAL"
    semantic_contract_id: str = "world-v6.semantic.general.v1"
    data_class: str = "INTERNAL"
    expected_type: JsonType | None = None
    required_for_actions: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        _decode_pointer(self.path)
        introduced = resolution_rank(self.introduced_at)
        write_min = resolution_rank(self.write_min)
        if write_min < introduced:

```

```

        raise ResolutionError("write_min_resolution cannot be below introduced_at")
    if self.downgrade_policy not in {
        "SAFE_TO_PROJECT", "READ_ONLY_WHEN_PROJECTED", "NO_DOWNGRADE"
    }:
        raise ResolutionError(f"invalid downgrade policy: {self.downgrade_policy}")
    _validate_token(self.cable_id, "cable_id")
    _validate_token(self.semantic_contract_id, "semantic_contract_id")
    if self.data_class not in CLASS_RANK:
        raise ResolutionError(f"invalid data_class: {self.data_class}")
    if self.expected_type not in {None, "string", "integer", "boolean", "null", "array", "object"}:
        raise ResolutionError(f"invalid expected_type: {self.expected_type}")
    for action in self.required_for_actions:
        _validate_token(action, "action")

@property
def write_min(self) -> str:
    return self.write_min_resolution or self.introduced_at

def to_document(self) -> dict[str, Any]:
    return {
        "path": self.path,
        "introduced_at": self.introduced_at,
        "write_min_resolution": self.write_min_resolution,
        "downgrade_policy": self.downgrade_policy,
        "cable_id": self.cable_id,
        "semantic_contract_id": self.semantic_contract_id,
        "data_class": self.data_class,
        "expected_type": self.expected_type,
        "required_for_actions": list(sorted(self.required_for_actions)),
    }

@dataclass(frozen=True)
class ActionRule:
    action: str
    minimum_resolution: str
    required_paths: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        _validate_token(self.action, "action")
        resolution_rank(self.minimum_resolution)
        for path in self.required_paths:
            _decode_pointer(path)

    def to_document(self) -> dict[str, Any]:
        return {
            "action": self.action,
            "minimum_resolution": self.minimum_resolution,
            "required_paths": list(sorted(self.required_paths)),
        }

@dataclass(frozen=True)
class ProjectionProfile:
    profile_id: str
    version: str
    canonical_resolution: str
    fields: tuple[FieldRule, ...]
    allow_unspecified: bool = False
    required_backbone_paths: tuple[str, ...] = ()
    actions: tuple[ActionRule, ...] = ()

    def __post_init__(self) -> None:
        _validate_token(self.profile_id, "profile_id")
        _validate_token(self.version, "profile version")
        canonical_rank = resolution_rank(self.canonical_resolution)
        if self.allow_unspecified:
            raise ResolutionError("allow_unspecified=true is unsafe and unsupported in RC1")
        if not self.fields or len(self.fields) > MAX_PROFILE_FIELDS:
            raise ResolutionError("profile field count is outside allowed bounds")

        seen: set[str] = set()
        decoded: dict[str, tuple[str, ...]] = {}
        for rule in self.fields:
            if rule.path in seen:
                raise ResolutionError(f"duplicate projection rule: {rule.path}")
            if resolution_rank(rule.introduced_at) > canonical_rank:
                raise ResolutionError(f"field introduced above canonical resolution: {rule.path}")
            if resolution_rank(rule.write_min) > canonical_rank:
                raise ResolutionError(f"field write minimum above canonical resolution: {rule.path}")
            seen.add(rule.path)

```

```

        decoded[rule.path] = _decode_pointer(rule.path)

    paths = list(decoded)
    for index, left in enumerate(paths):
        left_parts = decoded[left]
        for right in paths[index + 1:]:
            right_parts = decoded[right]
            shorter = min(len(left_parts), len(right_parts))
            if left_parts[:shorter] == right_parts[:shorter]:
                raise ResolutionError(f"ancestor/descendant field rules overlap: {left} / {right}")

    rules = self.rule_map()
    for path in self.required_backbone_paths:
        _decode_pointer(path)
        rule = rules.get(path)
        if rule is None or rule.introduced_at != "R0":
            raise ResolutionError(f"required backbone path must be an R0 field: {path}")

    action_names: set[str] = set()
    for action in self.actions:
        if action.action in action_names:
            raise ResolutionError(f"duplicate action rule: {action.action}")
        action_names.add(action.action)
        if resolution_rank(action.minimum_resolution) > canonical_rank:
            raise ResolutionError(f"action minimum exceeds canonical resolution: {action.action}")
        for path in action.required_paths:
            if path not in seen:
                raise ResolutionError(f"action requires undeclared field: {action.action} -> {path}")

def rule_map(self) -> dict[str, FieldRule]:
    return {field.path: field for field in self.fields}

def action_rule_map(self) -> dict[str, ActionRule]:
    return {action.action: action for action in self.actions}

def to_document(self) -> dict[str, Any]:
    return {
        "profile_id": self.profile_id,
        "version": self.version,
        "canonical_resolution": self.canonical_resolution,
        "allow_unspecified": False,
        "required_backbone_paths": list(sorted(self.required_backbone_paths)),
        "fields": [field.to_document() for field in sorted(self.fields, key=lambda item: item.path)],
        "actions": [action.to_document() for action in sorted(self.actions, key=lambda item: item.action)],
    }

@property
def profile_hash(self) -> str:
    return canonical_hash(self.to_document())

def profile_from_document(document: Mapping[str, Any]) -> ProjectionProfile:
    try:
        fields = tuple(
            FieldRule(
                path=item["path"],
                introduced_at=item["introduced_at"],
                write_min_resolution=item.get("write_min_resolution"),
                downgrade_policy=item["downgrade_policy"],
                cable_id=item["cable_id"],
                semantic_contract_id=item["semantic_contract_id"],
                data_class=item["data_class"],
                expected_type=item.get("expected_type"),
                required_for_actions=tuple(item.get("required_for_actions", ())),
            )
            for item in document["fields"]
        )
        actions = tuple(
            ActionRule(
                action=item["action"],
                minimum_resolution=item["minimum_resolution"],
                required_paths=tuple(item.get("required_paths", ())),
            )
            for item in document.get("actions", ())
        )
        return ProjectionProfile(
            profile_id=document["profile_id"],
            version=document["version"],
            canonical_resolution=document["canonical_resolution"],
            fields=fields,
            allow_unspecified=document.get("allow_unspecified", False),

```

```

        required_backbone_paths=tuple(document.get("required_backbone_paths", ())),
        actions=actions,
    )
except (KeyError, TypeError) as exc:
    raise ResolutionError(f"invalid profile document: {exc}") from exc

def load_profile_file(path: str | Path) -> ProjectionProfile:
    try:
        document = json.loads(Path(path).read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as exc:
        raise ResolutionError(f"cannot load resolution profile {path}: {exc}") from exc
    if not isinstance(document, Mapping):
        raise ResolutionError("resolution profile root must be an object")
    return profile_from_document(document)

@dataclass(frozen=True)
class ProjectionResult:
    profile_id: str
    profile_version: str
    profile_hash: str
    canonical_hash: str
    canonical_resolution: str
    target_resolution: str
    projection_hash: str
    source_ref: str
    source_version: int
    purpose: str
    data_class: str
    freshness: str
    value: dict[str, Any]
    included_paths: tuple[str, ...]
    omitted_paths: tuple[str, ...]
    read_only_paths: tuple[str, ...]
    no_downgrade_paths: tuple[str, ...]
    compiler_id: str = COMPILER_ID
    compiler_version: str = COMPILER_VERSION
    compiler_hash: str = COMPILER_HASH
    derived: bool = True
    authoritative: bool = False

    def consumer_envelope(self) -> dict[str, Any]:
        """Return the provider-safe envelope; omitted field names stay audit-internal."""
        return {
            "profile_id": self.profile_id,
            "profile_version": self.profile_version,
            "profile_hash": self.profile_hash,
            "source_ref": self.source_ref,
            "source_version": self.source_version,
            "canonical_hash": self.canonical_hash,
            "canonical_resolution": self.canonical_resolution,
            "effective_resolution": self.target_resolution,
            "projection_hash": self.projection_hash,
            "compiler_id": self.compiler_id,
            "compiler_version": self.compiler_version,
            "compiler_hash": self.compiler_hash,
            "purpose": self.purpose,
            "data_class": self.data_class,
            "freshness": self.freshness,
            "projection_derived": True,
            "projection_authoritative": False,
            "value": deepcopy(self.value),
        }

    def audit_metadata(self) -> dict[str, Any]:
        metadata = self.consumer_envelope()
        metadata.pop("value")
        metadata.update({
            "included_paths": list(self.included_paths),
            "omitted_paths": list(self.omitted_paths),
            "read_only_paths": list(self.read_only_paths),
            "no_downgrade_paths": list(self.no_downgrade_paths),
        })
        return metadata

@dataclass(frozen=True)
class PatchOperation:
    path: str
    value: Any

```

```

def __post_init__(self) -> None:
    _decode_pointer(self.path)
    _validate_canonical_json(self.value)

@dataclass(frozen=True)
class ResolutionPatch:
    actor_resolution: str
    expected_canonical_hash: str
    expected_projection_hash: str
    expected_version: int
    source_ref: str
    operations: tuple[PatchOperation, ...]
    projection_profile_id: str
    projection_profile_version: str
    projection_profile_hash: str

    def __post_init__(self) -> None:
        resolution_rank(self.actor_resolution)
        for label, digest in {
            "expected_canonical_hash": self.expected_canonical_hash,
            "expected_projection_hash": self.expected_projection_hash,
            "projection_profile_hash": self.projection_profile_hash,
        }.items():
            if not isinstance(digest, str) or not re.fullmatch(r"[0-9a-f]{64}", digest):
                raise ResolutionError(f"invalid {label}")
            if not isinstance(self.expected_version, int) or self.expected_version < 0:
                raise ResolutionError("expected_version must be a non-negative integer")
        _validate_token(self.source_ref, "source_ref")
        if not self.operations or len(self.operations) > _MAX_PATCH_OPERATIONS:
            raise ResolutionError("patch operation count is outside allowed bounds")

def _leaf_paths(value: Any, prefix: tuple[str, ...] = ()) -> Iterable[tuple[str, Any]]:
    if isinstance(value, Mapping):
        if not value and prefix:
            yield _encode_pointer(prefix), {}
        for key, child in value.items():
            yield from _leaf_paths(child, prefix + (key,))
    return
    if prefix:
        yield _encode_pointer(prefix), value

def _set_projection_path(target: dict[str, Any], path: str, value: Any) -> None:
    parts = _decode_pointer(path)
    current = target
    for part in parts[:-1]:
        existing = current.get(part)
        if existing is None:
            existing = {}
            current[part] = existing
        if not isinstance(existing, dict):
            raise ProjectionError(f"projection path collision at {path}")
        current = existing
    current[parts[-1]] = deepcopy(value)

def _get_existing_path(target: Mapping[str, Any], path: str) -> Any:
    current: Any = target
    for part in _decode_pointer(path):
        if not isinstance(current, Mapping) or part not in current:
            raise PatchRejected(f"patch path does not exist in canonical state: {path}")
        current = current[part]
    return current

def _set_existing_scalar_path(target: dict[str, Any], path: str, value: Any) -> None:
    parts = _decode_pointer(path)
    current: Any = target
    for part in parts[:-1]:
        if not isinstance(current, dict) or part not in current or not isinstance(current[part], dict):
            raise PatchRejected(f"patch ancestor is missing or non-object: {path}")
        current = current[part]
    if not isinstance(current, dict) or parts[-1] not in current:
        raise PatchRejected(f"patch path does not exist in canonical state: {path}")
    current[parts[-1]] = deepcopy(value)

def project(

```



```

canonical: Mapping[str, Any],
*,
profile: ProjectionProfile,
target_resolution: str,
source_ref: str = "unbound-source",
source_version: int = 0,
purpose: str = "UNSPECIFIED",
data_class: str = "INTERNAL",
freshness: str = "CURRENT",
) -> ProjectionResult:
    """Create a deterministic, non-destructive, provenance-bound projection."""
    _validate_canonical_json(canonical)
    _validate_token(source_ref, "source_ref")
    _validate_token(purpose, "purpose")
    _validate_token(freshness, "freshness")
    if not isinstance(source_version, int) or source_version < 0:
        raise ProjectionError("source_version must be a non-negative integer")
    if data_class not in _CLASS_RANK:
        raise ProjectionError(f"invalid data_class: {data_class}")

    target_rank = resolution_rank(target_resolution)
    canonical_rank = resolution_rank(profile.canonical_resolution)
    if target_rank > canonical_rank:
        raise ProjectionError("cannot synthesize a resolution above canonical data")

    rules = profile.rule_map()
    out: dict[str, Any] = {}
    included: list[str] = []
    omitted: list[str] = []
    readonly: list[str] = []
    no_downgrade: list[str] = []

    for path, value in _leaf_paths(canonical):
        rule = rules.get(path)
        if rule is None:
            raise ProjectionError(f"no projection rule for canonical field: {path}")
        actual_type = _json_type(value)
        if rule.expected_type is not None and actual_type != rule.expected_type:
            raise ProjectionError(
                f"canonical field type mismatch at {path}: expected {rule.expected_type}, got {actual_type}"
            )
        if _CLASS_RANK[rule.data_class] > _CLASS_RANK[data_class]:
            raise ProjectionError(
                f"envelope data_class understates field classification at {path}"
            )

        if resolution_rank(rule.introduced_at) <= target_rank:
            _set_projection_path(out, path, value)
            included.append(path)
            if (
                resolution_rank(rule.write_min) > target_rank
                or rule.downgrade_policy in {"READ_ONLY_WHEN_PROJECTED", "NO_DOWNGRADE"}
            ):
                readonly.append(path)
        else:
            omitted.append(path)
            if rule.downgrade_policy == "READ_ONLY_WHEN_PROJECTED":
                readonly.append(path)
            elif rule.downgrade_policy == "NO_DOWNGRADE":
                no_downgrade.append(path)

    missing_backbone = sorted(set(profile.required_backbone_paths) - set(included))
    if missing_backbone:
        raise ProjectionError(f"projection lost required R0 backbone paths: {missing_backbone}")

    return ProjectionResult(
        profile_id=profile.profile_id,
        profile_version=profile.version,
        profile_hash=profile.profile_hash,
        canonical_hash=canonical_hash(canonical),
        canonical_resolution=profile.canonical_resolution,
        target_resolution=target_resolution,
        projection_hash=canonical_hash(out),
        source_ref=source_ref,
        source_version=source_version,
        purpose=purpose,
        data_class=data_class,
        freshness=freshness,
        value=out,
        included_paths=tuple(sorted(included)),
        omitted_paths=tuple(sorted(omitted)),
    )

```

```

        read_only_paths=tuple(sorted(set(readonly))),
        no_downgrade_paths=tuple(sorted(set(no_downgrade))),
    )

def apply_patch(
    canonical: Mapping[str, Any],
    *,
    profile: ProjectionProfile,
    patch: ResolutionPatch,
    current_version: int,
    source_ref: str,
) -> dict[str, Any]:
    """Apply an existing scalar-leaf patch while preserving all hidden state."""
    _validate_canonical_json(canonical)
    if (
        patch.projection_profile_id != profile.profile_id
        or patch.projection_profile_version != profile.version
        or patch.projection_profile_hash != profile.profile_hash
    ):
        raise PatchRejected("projection profile identity/hash mismatch")
    if patch.source_ref != source_ref:
        raise PatchRejected("canonical source_ref mismatch")
    if patch.expected_version != current_version:
        raise PatchRejected("stale expected_version")
    if canonical_hash(canonical) != patch.expected_canonical_hash:
        raise PatchRejected("stale canonical hash")

    try:
        current_projection = project(
            canonical,
            profile=profile,
            target_resolution=patch.actor_resolution,
            source_ref=source_ref,
            source_version=current_version,
            data_class=_profile_data_class(profile),
        )
    except ProjectionError as exc:
        raise PatchRejected(f"canonical state no longer conforms to projection profile: {exc}") from exc
    if current_projection.projection_hash != patch.expected_projection_hash:
        raise PatchRejected("stale or mismatched projection hash")

    actor_rank = resolution_rank(patch.actor_resolution)
    if actor_rank > resolution_rank(profile.canonical_resolution):
        raise PatchRejected("actor resolution exceeds canonical resolution")

    rules = profile.rule_map()
    updated = deepcopy(dict(canonical))
    seen: set[str] = set()
    for operation in patch.operations:
        if operation.path in seen:
            raise PatchRejected(f"duplicate patch path: {operation.path}")
        seen.add(operation.path)
        rule = rules.get(operation.path)
        if rule is None:
            raise PatchRejected(f"field is not writable under projection profile: {operation.path}")
        if actor_rank < resolution_rank(rule.introduced_at):
            raise PatchRejected(f"field is not visible at {patch.actor_resolution}: {operation.path}")
        if actor_rank < resolution_rank(rule.write_min):
            raise PatchRejected(f"field requires {rule.write_min} to write: {operation.path}")
        if (
            rule.downgrade_policy in {"READ_ONLY_WHEN_PROJECTED", "NO_DOWNGRADE"}
            and actor_rank < resolution_rank(profile.canonical_resolution)
        ):
            raise PatchRejected(f"field is read-only/no-downgrade in projected execution: {operation.path}")

        existing = _get_existing_path(canonical, operation.path)
        if isinstance(existing, (Mapping, list)) or isinstance(operation.value, (Mapping, list)):
            raise PatchRejected(f"collection replacement is forbidden: {operation.path}")
        existing_type = _json_type(existing)
        new_type = _json_type(operation.value)
        if existing_type != new_type:
            raise PatchRejected(
                f"patch type mismatch at {operation.path}: {existing_type} -> {new_type}"
            )
        if rule.expected_type is not None and new_type != rule.expected_type:
            raise PatchRejected(f"patch violates declared type at {operation.path}")
        _set_existing_scalar_path(updated, operation.path, operation.value)

    _validate_canonical_json(updated)
    return updated

```

```
def require_action_resolution(*, effective_resolution: str, minimum_required: str) -> None:
    if resolution_rank(effective_resolution) < resolution_rank(minimum_required):
        raise ResolutionMismatch(
            f"action requires {minimum_required}, effective view is {effective_resolution}"
        )

def require_action_compatibility(
    *, profile: ProjectionProfile, action: str, projection: ProjectionResult
) -> None:
    """Bind an action to both its minimum level and its required fine wires."""
    if (
        projection.profile_id != profile.profile_id
        or projection.profile_version != profile.version
        or projection.profile_hash != profile.profile_hash
    ):
        raise ResolutionMismatch("action projection/profile mismatch")
    action_rule = profile.action_rule_map().get(action)
    if action_rule is None:
        raise ResolutionMismatch(f"action is not declared by profile: {action}")
    require_action_resolution(
        effective_resolution=projection.target_resolution,
        minimum_required=action_rule.minimum_resolution,
    )
    missing = sorted(set(action_rule.required_paths) - set(projection.included_paths))
    if missing:
        raise ResolutionMismatch(f"action-required fields are absent from projection: {missing}")

def assert_monotonic_projection(
    canonical: Mapping[str, Any], *, profile: ProjectionProfile, lower: str, higher: str
) -> None:
    """Property helper: a finer view may add fields but cannot alter coarse values."""
    if resolution_rank(lower) > resolution_rank(higher):
        raise ResolutionError("lower must not exceed higher")
    data_class = _profile_data_class(profile)
    low = project(canonical, profile=profile, target_resolution=lower, data_class=data_class)
    high = project(canonical, profile=profile, target_resolution=higher, data_class=data_class)
    reprojection = project(high.value, profile=profile, target_resolution=lower, data_class=data_class)
    if low.value != reprojection.value:
        raise ProjectionError("projection is not compositional/monotonic")
```

پیاده سازی: brain_gateway.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/brain_gateway.py

SHA-256 منبع:

ebef744e9be4170255bae55cf508990984f4389d21f5ff1c886293cb8144c25c

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Provider-neutral cognition gateway with profile-scoped Resolution negotiation.
```

```
Providers receive only derived projection envelopes. Canonical input, omitted  
field names, identity authority and Policy state never cross the adapter edge.  
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass  
from typing import Any, Mapping, Protocol
```

```
from .resolution import (  
    ProjectionProfile,  
    ResolutionError,  
    ResolutionMismatch,  
    negotiate_resolution,  
    project,  
)
```

```
class BrainUnavailable(RuntimeError):  
    pass
```

```
class BrainIncompatible(RuntimeError):  
    pass
```

```
@dataclass(frozen=True)
```

```
class BrainInputSegment:
```

```
    """One canonical input domain with its own Resolution Profile.
```

```
    A request can carry several segments (for example task + conversation +  
    price). This prevents a global ``R1`` from pretending to mean the same  
    thing across unrelated domains.  
    """
```

```
    segment_id: str  
    canonical: dict[str, Any]  
    profile_id: str  
    profile_version: str  
    profile_hash: str  
    source_ref: str  
    source_version: int  
    desired_resolution: str  
    minimum_resolution: str  
    purpose: str  
    data_class: str = "INTERNAL"  
    freshness: str = "CURRENT"
```

```
@dataclass(frozen=True)
```

```
class BrainRequest:
```

```
    contract_version: str  
    task_type: str  
    inputs: tuple[BrainInputSegment, ...]  
    world_id: str = "world-v6"  
    entity_id: str = "secretary-001"  
    principal_id: str = "human-root"  
    conversation_id: str = "human-root:secretary-001"
```

```

def __post_init__(self) -> None:
    if not self.inputs:
        raise BrainIncompatible("BrainRequest requires at least one input segment")
    segment_ids = [segment.segment_id for segment in self.inputs]
    if len(segment_ids) != len(set(segment_ids)):
        raise BrainIncompatible("duplicate Brain input segment_id")

@dataclass(frozen=True)
class BrainRequestDescriptor:
    """Value-free metadata used by a provider's compatibility gate."""

    contract_version: str
    task_type: str
    world_id: str
    entity_id: str
    principal_id: str
    conversation_id: str
    segments: tuple[dict[str, Any], ...]

@dataclass(frozen=True)
class ProjectedBrainInput:
    segment_id: str
    envelope: dict[str, Any]

@dataclass(frozen=True)
class ProjectedBrainRequest:
    contract_version: str
    task_type: str
    inputs: tuple[ProjectedBrainInput, ...]
    world_id: str
    entity_id: str
    principal_id: str
    conversation_id: str

@dataclass(frozen=True)
class BrainResponse:
    provider: str
    output: dict[str, Any]
    effective_resolutions: tuple[tuple[str, str], ...]

    @property
    def effective_resolution(self) -> str:
        if len(self.effective_resolutions) != 1:
            raise BrainIncompatible("request has a Resolution vector, not one scalar level")
        return self.effective_resolutions[0][1]

class BrainAdapter(Protocol):
    name: str

    def compatible(self, request: BrainRequestDescriptor) -> bool: ...

    def max_resolution_for(
        self, profile_id: str, profile_version: str, profile_hash: str
    ) -> str | None: ...

    def invoke(self, request: ProjectedBrainRequest) -> dict[str, Any]: ...

class BrainGateway:
    def __init__(self, providers: list[BrainAdapter], profiles: list[ProjectionProfile]):
        self.providers = providers
        self._profiles: dict[tuple[str, str], ProjectionProfile] = {}
        for profile in profiles:
            key = (profile.profile_id, profile.version)
            if key in self._profiles:
                raise BrainIncompatible(f"duplicate registered Resolution Profile: {key}")
            self._profiles[key] = profile

    def _profile_for(self, segment: BrainInputSegment) -> ProjectionProfile:
        profile = self._profiles.get((segment.profile_id, segment.profile_version))
        if profile is None:
            raise BrainIncompatible(
                f"unregistered Resolution Profile: {segment.profile_id}@{segment.profile_version}"
            )
        if segment.profile_hash != profile.profile_hash:

```

```

        raise BrainIncompatible(
            f"Resolution Profile hash mismatch: {segment.profile_id}@{segment.profile_version}"
        )
    return profile

def _descriptor(self, request: BrainRequest) -> BrainRequestDescriptor:
    return BrainRequestDescriptor(
        contract_version=request.contract_version,
        task_type=request.task_type,
        world_id=request.world_id,
        entity_id=request.entity_id,
        principal_id=request.principal_id,
        conversation_id=request.conversation_id,
        segments=tuple(
            {
                "segment_id": segment.segment_id,
                "profile_id": segment.profile_id,
                "profile_version": segment.profile_version,
                "profile_hash": segment.profile_hash,
                "source_ref": segment.source_ref,
                "source_version": segment.source_version,
                "desired_resolution": segment.desired_resolution,
                "minimum_resolution": segment.minimum_resolution,
                "purpose": segment.purpose,
                "data_class": segment.data_class,
                "freshness": segment.freshness,
            }
            for segment in request.inputs
        ),
    )

@staticmethod
def _provider_max(provider: BrainAdapter, segment: BrainInputSegment) -> str | None:
    capability = getattr(provider, "max_resolution_for", None)
    if callable(capability):
        return capability(segment.profile_id, segment.profile_version, segment.profile_hash)

    capabilities = getattr(provider, "resolution_capabilities", None)
    if isinstance(capabilities, Mapping):
        exact_key = f"{segment.profile_id}@{segment.profile_version}#{segment.profile_hash}"
        version_key = f"{segment.profile_id}@{segment.profile_version}"
        return capabilities.get(exact_key) or capabilities.get(version_key)
    return None

def prepare_for_provider(
    self, request: BrainRequest, provider: BrainAdapter
) -> tuple[ProjectedBrainRequest, tuple[tuple[str, str], ...]]:
    """Project a request for one exact provider without invoking it.

    RC2 Manual Host workflows use this method to export the same safe
    envelope that a live adapter would receive. It grants no authority and
    performs no provider/network operation.
    """
    profiles = {segment.segment_id: self._profile_for(segment) for segment in request.inputs}
    descriptor = self._descriptor(request)
    if not provider.compatible(descriptor):
        raise BrainIncompatible(f"provider is incompatible: {provider.name}")
    projected_inputs: list[ProjectedBrainInput] = []
    effective: list[tuple[str, str]] = []
    for segment in request.inputs:
        profile = profiles[segment.segment_id]
        provider_max = self._provider_max(provider, segment)
        if provider_max is None:
            raise ResolutionMismatch(
                f"provider has no capability for {segment.profile_id}@{segment.profile_version}"
            )
        target = negotiate_resolution(
            desired=segment.desired_resolution,
            minimum=segment.minimum_resolution,
            brain_max=provider_max,
            canonical_resolution=profile.canonical_resolution,
        )
        projection = project(
            segment.canonical,
            profile=profile,
            target_resolution=target,
            source_ref=segment.source_ref,
            source_version=segment.source_version,
            purpose=segment.purpose,
            data_class=segment.data_class,
            freshness=segment.freshness,

```

```

    )
    projected_inputs.append(
        ProjectedBrainInput(segment.segment_id, projection.consumer_envelope())
    )
    effective.append((segment.segment_id, target))
return (
    ProjectedBrainRequest(
        contract_version=request.contract_version,
        task_type=request.task_type,
        inputs=tuple(projected_inputs),
        world_id=request.world_id,
        entity_id=request.entity_id,
        principal_id=request.principal_id,
        conversation_id=request.conversation_id,
    ),
    tuple(effective),
)

def invoke(self, request: BrainRequest) -> BrainResponse:
    # Validate all profile bindings before any provider is considered.
    for segment in request.inputs:
        self._profile_for(segment)
    attempted: list[str] = []

    for provider in self.providers:
        try:
            projected_request, effective = self.prepare_for_provider(request, provider)
            attempted.append(provider.name)
            output = provider.invoke(projected_request)
            if not isinstance(output, dict):
                raise BrainIncompatible("provider returned non-structured output")
            return BrainResponse(provider.name, output, effective)
        except (ResolutionError, BrainIncompatible):
            continue
        except Exception:
            # Provider failure may trigger failover, but no request invariant changes.
            continue
    raise BrainUnavailable(f"no compatible brain succeeded; attempted={attempted}")

```

پیاده سازی: portable_brain.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

SHA-256 منبع:

world-v6.2-rc3/candidate-v6.2/runtime/core/portable_brain.py

491117b6d37fcde640c9c927a2924be93659a22e3993058b054b4ce88f18f136

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Portable, provider-neutral Brain Pack support for World v6.2 RC2.

The module deliberately contains no vendor SDK and performs no network I/O.
It turns model output into a narrow Secretary Decision proposal, then renders
standard replies deterministically. ChatGPT, Gemini, Grok, a local model, a
manual host session, and deterministic code can therefore share one identity,
state boundary, policy contract, and observable output contract.
"""

from __future__ import annotations

from dataclasses import dataclass
import re
import json
from pathlib import Path
from typing import Any, Callable, Mapping, Sequence

from .brain_gateway import (
    BrainAdapter,
    BrainGateway,
    BrainRequest,
    BrainRequestDescriptor,
    BrainResponse,
    ProjectedBrainRequest,
)
from .resolution import ProjectionProfile, canonical_hash

class PortableBrainError(ValueError):
    """A portable pack, model output, or offline binding is invalid."""

_TOKEN_RE = re.compile(r"^[A-Za-z0-9][A-Za-z0-9._:/@#-]{0,191}$")
_HASH_RE = re.compile(r"^[0-9a-f]{64}$")
_ALLOWED_INTENTS = {
    "GREET",
    "TASK_CAPTURE",
    "TASK_LIST",
    "DRAFT",
    "PRICE_LOOKUP",
    "CLARIFY",
    "UNKNOWN",
}
_ALLOWED_RESPONSE_KINDS = {
    "ACKNOWLEDGE",
    "TASK_PROPOSAL",
    "TASK_LIST",
    "DRAFT_READY",
    "PRICE_RESULT",
    "CLARIFICATION_REQUEST",
    "SAFE_DEFER",
}
_ALLOWED_ACTIONS = {
    "NONE",
    "PROPOSE_TASK",
    "PROPOSE_DRAFT",
    "PROPOSE_PRICE_LOOKUP",
    "PROPOSE_EXTERNAL_EFFECT",
}
_FORBIDDEN_OUTPUT_KEYS = {
```



```

    "execute",
    "execute_now",
    "effect_executed",
    "approval_granted",
    "authority_granted",
    "canonical_write",
    "send_now",
}

_DECISION_KEYS = {
    "schema_version",
    "intent",
    "response_kind",
    "template_id",
    "slots",
    "proposed_actions",
    "requires_approval",
    "confidence_millis",
    "evidence_refs",
    "uncertainties",
}

def _token(value: str, label: str) -> None:
    if not isinstance(value, str) or _TOKEN_RE.fullmatch(value) is None:
        raise PortableBrainError(f"invalid {label}: {value!r}")

def _hash(value: str, label: str) -> None:
    if not isinstance(value, str) or _HASH_RE.fullmatch(value) is None:
        raise PortableBrainError(f"{label} must be a lowercase SHA-256")

def _scalar(value: Any, label: str) -> None:
    if value is None or isinstance(value, bool):
        return
    if isinstance(value, str):
        if len(value) > 4_000:
            raise PortableBrainError(f"{label} exceeds the bounded string limit")
        return
    if isinstance(value, int) and not isinstance(value, bool):
        if abs(value) > 9_007_199_254_740_991:
            raise PortableBrainError(f"{label} exceeds the canonical safe integer range")
        return
    raise PortableBrainError(f"{label} must be a canonical scalar")

@dataclass(frozen=True)
class PortableBrainPack:
    pack_id: str
    version: str
    world_id: str
    entity_id: str
    identity_ref: str
    dna_ref: str
    dna_hash: str
    root_constitution_ref: str
    prompt_contract_ref: str
    prompt_contract_hash: str
    output_contract_ref: str
    output_contract_hash: str
    templates: tuple[tuple[str, str], ...]
    allowed_task_types: tuple[str, ...]
    required_profile_bindings: tuple[tuple[str, str, str], ...]
    proposal_only: bool = True
    api_optional: bool = True

    def __post_init__(self) -> None:
        for value, label in (
            (self.pack_id, "pack_id"),
            (self.version, "version"),
            (self.world_id, "world_id"),
            (self.entity_id, "entity_id"),
            (self.identity_ref, "identity_ref"),
            (self.dna_ref, "dna_ref"),
            (self.root_constitution_ref, "root_constitution_ref"),
            (self.prompt_contract_ref, "prompt_contract_ref"),
            (self.output_contract_ref, "output_contract_ref"),
        ):
            _token(value, label)
        for value, label in (
            (self.dna_hash, "dna_hash"),

```

```

        (self.prompt_contract_hash, "prompt_contract_hash"),
        (self.output_contract_hash, "output_contract_hash"),
    ):
        _hash(value, label)
    if self.proposal_only is not True:
        raise PortableBrainError("RC2 Brain Packs must be proposal-only")
    if self.api_optional is not True:
        raise PortableBrainError("RC2 reference pack must remain usable without an API")
    template_ids = [key for key, _ in self.templates]
    if not template_ids or len(template_ids) != len(set(template_ids)):
        raise PortableBrainError("template ids must be non-empty and unique")
    for key, template in self.templates:
        _token(key, "template_id")
        if not isinstance(template, str) or not template or len(template) > 4_000:
            raise PortableBrainError("template must be a bounded non-empty string")
    if not self.allowed_task_types:
        raise PortableBrainError("pack requires at least one task type")
    if len(self.allowed_task_types) != len(set(self.allowed_task_types)):
        raise PortableBrainError("duplicate allowed task type")
    for task_type in self.allowed_task_types:
        _token(task_type, "task_type")
    profile_keys: set[tuple[str, str]] = set()
    for profile_id, version, digest in self.required_profile_bindings:
        _token(profile_id, "profile_id")
        _token(version, "profile_version")
        _hash(digest, "profile_hash")
        key = (profile_id, version)
        if key in profile_keys:
            raise PortableBrainError("duplicate profile binding")
        profile_keys.add(key)

def to_document(self) -> dict[str, Any]:
    return {
        "pack_id": self.pack_id,
        "version": self.version,
        "world_id": self.world_id,
        "entity_id": self.entity_id,
        "identity_ref": self.identity_ref,
        "dna_ref": self.dna_ref,
        "dna_hash": self.dna_hash,
        "root_constitution_ref": self.root_constitution_ref,
        "prompt_contract_ref": self.prompt_contract_ref,
        "prompt_contract_hash": self.prompt_contract_hash,
        "output_contract_ref": self.output_contract_ref,
        "output_contract_hash": self.output_contract_hash,
        "templates": {key: value for key, value in sorted(self.templates)},
        "allowed_task_types": list(self.allowed_task_types),
        "required_profile_bindings": [
            {"profile_id": item[0], "version": item[1], "sha256": item[2]}
            for item in sorted(self.required_profile_bindings)
        ],
        "proposal_only": self.proposal_only,
        "api_optional": self.api_optional,
    }

@property
def pack_hash(self) -> str:
    return canonical_hash(self.to_document())

def template(self, template_id: str) -> str:
    try:
        return dict(self.templates)[template_id]
    except KeyError as exc:
        raise PortableBrainError(f"unknown template_id: {template_id}") from exc

```

```

def portable_pack_from_document(document: Mapping[str, Any]) -> PortableBrainPack:
    required = {
        "pack_id",
        "version",
        "world_id",
        "entity_id",
        "identity_ref",
        "dna_ref",
        "dna_hash",
        "root_constitution_ref",
        "prompt_contract_ref",
        "prompt_contract_hash",
        "output_contract_ref",
        "output_contract_hash",
        "templates",
    }

```

```

        "allowed_task_types",
        "required_profile_bindings",
        "proposal_only",
        "api_optional",
    }
    if set(document) != required:
        raise PortableBrainError("Portable Brain Pack key set mismatch")
    templates = document["templates"]
    bindings = document["required_profile_bindings"]
    if not isinstance(templates, Mapping) or not isinstance(bindings, list):
        raise PortableBrainError("invalid templates/profile bindings")
    parsed_bindings: list[tuple[str, str, str]] = []
    for item in bindings:
        if not isinstance(item, Mapping) or set(item) != {"profile_id", "version", "sha256"}:
            raise PortableBrainError("invalid profile binding")
        parsed_bindings.append((item["profile_id"], item["version"], item["sha256"]))
    task_types = document["allowed_task_types"]
    if not isinstance(task_types, list):
        raise PortableBrainError("allowed_task_types must be an array")
    return PortableBrainPack(
        pack_id=document["pack_id"],
        version=document["version"],
        world_id=document["world_id"],
        entity_id=document["entity_id"],
        identity_ref=document["identity_ref"],
        dna_ref=document["dna_ref"],
        dna_hash=document["dna_hash"],
        root_constitution_ref=document["root_constitution_ref"],
        prompt_contract_ref=document["prompt_contract_ref"],
        prompt_contract_hash=document["prompt_contract_hash"],
        output_contract_ref=document["output_contract_ref"],
        output_contract_hash=document["output_contract_hash"],
        templates=tuple(sorted(templates.items())),
        allowed_task_types=tuple(task_types),
        required_profile_bindings=tuple(parsed_bindings),
        proposal_only=document["proposal_only"],
        api_optional=document["api_optional"],
    )

```

```

def load_portable_pack(path: str | Path) -> PortableBrainPack:
    document = json.loads(Path(path).read_text(encoding="utf-8"))
    if not isinstance(document, Mapping):
        raise PortableBrainError("Portable Brain Pack must be a JSON object")
    return portable_pack_from_document(document)

```

```

@dataclass(frozen=True)
class SecretaryDecision:
    schema_version: str
    intent: str
    response_kind: str
    template_id: str
    slots: tuple[tuple[str, Any], ...]
    proposed_actions: tuple[str, ...]
    requires_approval: bool
    confidence_millis: int
    evidence_refs: tuple[str, ...]
    uncertainties: tuple[str, ...]

    def __post_init__(self) -> None:
        if self.schema_version != "world-v6.secretary-decision.v1":
            raise PortableBrainError("unsupported Secretary Decision schema")
        if self.intent not in _ALLOWED_INTENTS:
            raise PortableBrainError("unsupported intent")
        if self.response_kind not in _ALLOWED_RESPONSE_KINDS:
            raise PortableBrainError("unsupported response_kind")
        _token(self.template_id, "template_id")
        slot_keys = [key for key, _ in self.slots]
        if len(slot_keys) != len(set(slot_keys)):
            raise PortableBrainError("duplicate decision slot")
        for key, value in self.slots:
            _token(key, "slot name")
            _scalar(value, f"slot {key}")
        if not self.proposed_actions:
            raise PortableBrainError("proposed_actions cannot be empty")
        if len(self.proposed_actions) != len(set(self.proposed_actions)):
            raise PortableBrainError("duplicate proposed action")
        for action in self.proposed_actions:
            if action not in _ALLOWED_ACTIONS:
                raise PortableBrainError(f"forbidden proposed action: {action}")

```

```

        if "PROPOSE_EXTERNAL_EFFECT" in self.proposed_actions and not self.requires_approval:
            raise PortableBrainError("external-effect proposal must require approval")
        if not isinstance(self.requires_approval, bool):
            raise PortableBrainError("requires_approval must be boolean")
        if not isinstance(self.confidence_millis, int) or isinstance(self.confidence_millis, bool):
            raise PortableBrainError("confidence_millis must be an integer")
        if not 0 <= self.confidence_millis <= 1000:
            raise PortableBrainError("confidence_millis outside 0..1000")
        for ref in self.evidence_refs:
            _token(ref, "evidence_ref")
        for item in self.uncertainties:
            if not isinstance(item, str) or len(item) > 500:
                raise PortableBrainError("uncertainty must be a bounded string")

    def semantic_document(self) -> dict[str, Any]:
        """Provider-independent fields that determine the observable answer."""
        return {
            "schema_version": self.schema_version,
            "intent": self.intent,
            "response_kind": self.response_kind,
            "template_id": self.template_id,
            "slots": {key: value for key, value in sorted(self.slots)},
            "proposed_actions": list(self.proposed_actions),
            "requires_approval": self.requires_approval,
        }

    def to_document(self) -> dict[str, Any]:
        return {
            **self.semantic_document(),
            "decision_id": self.decision_id,
            "confidence_millis": self.confidence_millis,
            "evidence_refs": list(self.evidence_refs),
            "uncertainties": list(self.uncertainties),
            "proposal_only": True,
        }

    @property
    def semantic_hash(self) -> str:
        return canonical_hash(self.semantic_document())

    @property
    def decision_id(self) -> str:
        return f"decision:{self.semantic_hash}"

def normalize_secretary_decision(raw: Mapping[str, Any]) -> SecretaryDecision:
    if not isinstance(raw, Mapping):
        raise PortableBrainError("model output must be an object")
    keys = set(raw)
    forbidden = keys & _FORBIDDEN_OUTPUT_KEYS
    if forbidden:
        raise PortableBrainError(f"authoritative/effect key forbidden: {sorted(forbidden)}")
    if keys != _DECISION_KEYS:
        missing = sorted(_DECISION_KEYS - keys)
        extra = sorted(keys - _DECISION_KEYS)
        raise PortableBrainError(f"decision key mismatch; missing={missing}, extra={extra}")
    slots = raw["slots"]
    if not isinstance(slots, Mapping):
        raise PortableBrainError("slots must be an object")
    actions = raw["proposed_actions"]
    evidence = raw["evidence_refs"]
    uncertainties = raw["uncertainties"]
    if not isinstance(actions, list) or not all(isinstance(item, str) for item in actions):
        raise PortableBrainError("proposed_actions must be a string array")
    if not isinstance(evidence, list) or not all(isinstance(item, str) for item in evidence):
        raise PortableBrainError("evidence_refs must be a string array")
    if not isinstance(uncertainties, list) or not all(
        isinstance(item, str) for item in uncertainties
    ):
        raise PortableBrainError("uncertainties must be a string array")
    return SecretaryDecision(
        schema_version=raw["schema_version"],
        intent=raw["intent"],
        response_kind=raw["response_kind"],
        template_id=raw["template_id"],
        slots=tuple(sorted(slots.items())),
        proposed_actions=tuple(actions),
        requires_approval=raw["requires_approval"],
        confidence_millis=raw["confidence_millis"],
        evidence_refs=tuple(evidence),
        uncertainties=tuple(uncertainties),
    )

```

```

)

class _StrictSlots(dict[str, Any]):
    def __missing__(self, key: str) -> Any:
        raise PortableBrainError(f"template requires missing slot: {key}")

def render_secretary_decision(pack: PortableBrainPack, decision: SecretaryDecision) -> str:
    template = pack.template(decision.template_id)
    try:
        rendered = template.format_map(_StrictSlots(dict(decision.slots)))
    except PortableBrainError:
        raise
    except (ValueError, KeyError) as exc:
        raise PortableBrainError("invalid template/slot binding") from exc
    if len(rendered) > 8_000:
        raise PortableBrainError("rendered reply exceeds bounded output")
    return rendered

class ScriptedPortableAdapter:
    """Offline deterministic adapter used for code fallback, fixtures, and tests."""

    network_required = False
    api_token_required = False

    def __init__(
        self,
        name: str,
        capabilities: Mapping[tuple[str, str, str], str],
        responder: Mapping[str, Any] | Callable[[ProjectedBrainRequest], Mapping[str, Any]],
        *,
        task_types: Sequence[str] = (),
    ) -> None:
        _token(name, "adapter name")
        self.name = name
        self._capabilities = dict(capabilities)
        self._responder = dict(responder) if isinstance(responder, Mapping) else responder
        self._task_types = frozenset(task_types)
        self.last_request: ProjectedBrainRequest | None = None

    def compatible(self, request: BrainRequestDescriptor) -> bool:
        return not self._task_types or request.task_type in self._task_types

    def max_resolution_for(
        self, profile_id: str, profile_version: str, profile_hash: str
    ) -> str | None:
        return self._capabilities.get((profile_id, profile_version, profile_hash))

    def invoke(self, request: ProjectedBrainRequest) -> dict[str, Any]:
        self.last_request = request
        response = self._responder(request) if callable(self._responder) else self._responder
        if not isinstance(response, Mapping):
            raise PortableBrainError("scripted response must be an object")
        return dict(response)

class ManualHostAdapter(ScriptedPortableAdapter):
    """No-token bridge: paste/export a pack to any model and import strict JSON."""

    def export_exchange(
        self,
        request: ProjectedBrainRequest,
        pack: PortableBrainPack,
    ) -> dict[str, Any]:
        return {
            "exchange_contract": "world-v6.manual-brain-exchange.v1",
            "pack_id": pack.pack_id,
            "pack_version": pack.version,
            "pack_hash": pack.pack_hash,
            "provider_hint": self.name,
            "request": {
                "contract_version": request.contract_version,
                "task_type": request.task_type,
                "world_id": request.world_id,
                "entity_id": request.entity_id,
                "principal_id": request.principal_id,
                "conversation_id": request.conversation_id,
                "inputs": [
                    {"segment_id": item.segment_id, "envelope": item.envelope}
                ]
            }
        }

```

```

        for item in request.inputs
    ],
    },
    "response_contract_ref": pack.output_contract_ref,
    "proposal_only": True,
    "network_performed_by_runtime": False,
    "api_token_used_by_runtime": False,
}

def import_response(self, document: Mapping[str, Any]) -> SecretaryDecision:
    """Validate and stage a pasted host response for a later Gateway run."""
    decision = normalize_secretary_decision(document)
    self._responder = dict(document)
    return decision

@dataclass(frozen=True)
class PortableSecretaryResult:
    provider: str
    decision: SecretaryDecision
    rendered_text: str
    effective_resolutions: tuple[tuple[str, str], ...]
    pack_hash: str

class PortableSecretaryRunner:
    """One stable Secretary contract over interchangeable offline/live adapters."""

    def __init__(
        self,
        pack: PortableBrainPack,
        adapters: Sequence[BrainAdapter],
        profiles: Sequence[ProjectionProfile],
        *,
        allow_network: bool = False,
        allow_api_tokens: bool = False,
    ) -> None:
        if not adapters:
            raise PortableBrainError("at least one adapter is required")
        for adapter in adapters:
            if getattr(adapter, "network_required", False) and not allow_network:
                raise PortableBrainError(f"network adapter disabled: {adapter.name}")
            if getattr(adapter, "api_token_required", False) and not allow_api_tokens:
                raise PortableBrainError(f"API-token adapter disabled: {adapter.name}")
        actual = {(profile.profile_id, profile.version): profile.profile_hash for profile in profiles}
        for profile_id, version, digest in pack.required_profile_bindings:
            if actual.get((profile_id, version)) != digest:
                raise PortableBrainError(
                    f"pack/profile binding mismatch: {profile_id}@{version}"
                )
        self.pack = pack
        self.gateway = BrainGateway(list(adapters), list(profiles))

    def _validate_request(self, request: BrainRequest) -> None:
        if request.task_type not in self.pack.allowed_task_types:
            raise PortableBrainError(f"task type not allowed by pack: {request.task_type}")
        if request.world_id != self.pack.world_id or request.entity_id != self.pack.entity_id:
            raise PortableBrainError("request identity differs from the immutable pack binding")

    def prepare_manual_exchange(
        self, request: BrainRequest, adapter: ManualHostAdapter
    ) -> dict[str, Any]:
        self._validate_request(request)
        if adapter not in self.gateway.providers:
            raise PortableBrainError("manual adapter is not registered in this runner")
        projected, effective = self.gateway.prepare_for_provider(request, adapter)
        exchange = adapter.export_exchange(projected, self.pack)
        exchange["effective_resolutions"] = {
            segment_id: level for segment_id, level in effective
        }
        return exchange

    def run(self, request: BrainRequest) -> PortableSecretaryResult:
        self._validate_request(request)
        response: BrainResponse = self.gateway.invoke(request)
        decision = normalize_secretary_decision(response.output)
        return PortableSecretaryResult(
            provider=response.provider,
            decision=decision,
            rendered_text=render_secretary_decision(self.pack, decision),
            effective_resolutions=response.effective_resolutions,

```

```
pack_hash=self.pack.pack_hash,  
)
```

پیاده سازی: council.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

SHA-256 منبع:

world-v6.2-rc3/candidate-v6.2/runtime/core/council.py

3c2abdb8ec2efe6e7ed4dc2b89fe7652bb0c37eef73cf92142e2d1b365a8ed75

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Blind-then-deliberative, proposal-only multi-Brain council for RC2."""
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from typing import Mapping, Sequence

from .resolution import canonical_hash

class CouncilContractError(ValueError):
    pass

class BallotPosition(str, Enum):
    SUPPORT = "SUPPORT"
    OPPOSE = "OPPOSE"
    ABSTAIN = "ABSTAIN"
    VETO = "VETO"

class CouncilOutcome(str, Enum):
    PROPOSED = "PROPOSED"
    DEFERRED = "DEFERRED"
    VETOED = "VETOED"

@dataclass(frozen=True)
class CouncilParticipant:
    participant_id: str
    role: str
    weight_millis: int = 1000
    veto_capable: bool = False

    def __post_init__(self) -> None:
        if not self.participant_id or not self.role:
            raise CouncilContractError("participant identity and role are required")
        if not isinstance(self.weight_millis, int) or not 1 <= self.weight_millis <= 10_000:
            raise CouncilContractError("participant weight outside 1..10000")
        if self.veto_capable and self.role not in {"RISK", "POLICY", "SAFETY", "COMPLIANCE"}:
            raise CouncilContractError("veto capability is restricted to risk/policy roles")

@dataclass(frozen=True)
class CouncilBallot:
    participant_id: str
    round_no: int
    context_hash: str
    proposal_hash: str
    position: BallotPosition
    confidence_millis: int
    rationale: str
    evidence_refs: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if not isinstance(self.round_no, int) or self.round_no < 1:
            raise CouncilContractError("round_no must be positive")
        for digest, label in (
            (self.context_hash, "context_hash"),
```



```

        (self.proposal_hash, "proposal_hash"),
    ):
        if len(digest) != 64 or any(char not in "0123456789abcdef" for char in digest):
            raise CouncilContractError(f"{label} must be lowercase SHA-256")
    if not isinstance(self.position, BallotPosition):
        raise CouncilContractError("invalid ballot position")
    if not isinstance(self.confidence_millis, int) or not 0 <= self.confidence_millis <= 1000:
        raise CouncilContractError("confidence outside 0..1000")
    if not isinstance(self.rationale, str) or not self.rationale or len(self.rationale) > 2_000:
        raise CouncilContractError("rationale must be bounded and non-empty")

    def to_document(self) -> dict[str, object]:
        return {
            "participant_id": self.participant_id,
            "round_no": self.round_no,
            "context_hash": self.context_hash,
            "proposal_hash": self.proposal_hash,
            "position": self.position.value,
            "confidence_millis": self.confidence_millis,
            "rationale": self.rationale,
            "evidence_refs": list(self.evidence_refs),
        }

@dataclass(frozen=True)
class CouncilDecision:
    session_id: str
    outcome: CouncilOutcome
    proposal_hash: str | None
    round_count: int
    support_millis: int
    oppose_millis: int
    abstentions: int
    vetoes: tuple[str, ...]
    dissent: tuple[str, ...]
    transcript_hash: str
    proposal_only: bool = True

class CouncilSession:
    """Stateful local session; ballots are hidden until every member submits."""

    def __init__(
        self,
        session_id: str,
        context_hash: str,
        participants: Sequence[CouncilParticipant],
        *,
        max_rounds: int = 2,
        minimum_support_millis: int = 667,
        high_risk: bool = False,
    ) -> None:
        if not session_id:
            raise CouncilContractError("session_id is required")
        if len(context_hash) != 64 or any(
            char not in "0123456789abcdef" for char in context_hash
        ):
            raise CouncilContractError("context_hash must be lowercase SHA-256")
        if len(participants) < 2:
            raise CouncilContractError("a council requires at least two participants")
        ids = [item.participant_id for item in participants]
        if len(ids) != len(set(ids)):
            raise CouncilContractError("duplicate participant")
        if not 1 <= max_rounds <= 5:
            raise CouncilContractError("max_rounds outside 1..5")
        if not 501 <= minimum_support_millis <= 1000:
            raise CouncilContractError("support threshold must be a strict majority")
        self.session_id = session_id
        self.context_hash = context_hash
        self.participants = {item.participant_id: item for item in participants}
        self.max_rounds = max_rounds
        self.minimum_support_millis = minimum_support_millis
        self.high_risk = high_risk
        self.current_round = 1
        self._rounds: list[tuple[CouncilBallot, ...]] = []
        self._pending: dict[str, CouncilBallot] = {}

    def submit(self, ballot: CouncilBallot) -> None:
        if ballot.round_no != self.current_round:
            raise CouncilContractError("ballot round mismatch")
        if ballot.context_hash != self.context_hash:

```

```

        raise CouncilContractError("ballot context/state snapshot mismatch")
    if ballot.participant_id not in self.participants:
        raise CouncilContractError("unknown participant")
    if ballot.participant_id in self._pending:
        raise CouncilContractError("duplicate ballot in round")
    participant = self.participants[ballot.participant_id]
    if ballot.position is BallotPosition.VETO and not participant.veto_capable:
        raise CouncilContractError("participant has no veto capability")
    self._pending[ballot.participant_id] = ballot

@property
def round_complete(self) -> bool:
    return len(self._pending) == len(self.participants)

def reveal(self) -> tuple[CouncilBallot, ...]:
    if not self.round_complete:
        raise CouncilContractError("blind ballots remain sealed until all participants submit")
    return tuple(self._pending[key] for key in sorted(self._pending))

def advance(self) -> tuple[CouncilBallot, ...]:
    revealed = self.reveal()
    if self.current_round >= self.max_rounds:
        raise CouncilContractError("maximum deliberation rounds reached")
    self._rounds.append(revealed)
    self._pending = {}
    self.current_round += 1
    return revealed

def finalize(self) -> CouncilDecision:
    current = self.reveal()
    rounds = (*self._rounds, current)
    latest = rounds[-1]
    vetoes = tuple(
        ballot.participant_id
        for ballot in latest
        if ballot.position is BallotPosition.VETO
    )
    # In low-risk councils a veto is still an oppose; in high-risk councils
    # a calibrated safety/compliance veto blocks the proposal.
    hard_veto = bool(vetoes and self.high_risk)
    proposals = {
        ballot.proposal_hash
        for ballot in latest
        if ballot.position in {BallotPosition.SUPPORT, BallotPosition.VETO}
    }
    proposal_hash = next(iter(proposals)) if len(proposals) == 1 else None
    support = 0
    oppose = 0
    abstentions = 0
    dissent: list[str] = []
    for ballot in latest:
        weight = self.participants[ballot.participant_id].weight_millis
        calibrated = weight * ballot.confidence_millis
        if ballot.position is BallotPosition.SUPPORT and ballot.proposal_hash == proposal_hash:
            support += calibrated
        elif ballot.position is BallotPosition.ABSTAIN:
            abstentions += 1
            dissent.append(ballot.participant_id)
        else:
            oppose += calibrated
            dissent.append(ballot.participant_id)
    total = support + oppose
    ratio = support * 1000 // total if total else 0
    if hard_veto:
        outcome = CouncilOutcome.VETOED
    elif proposal_hash is not None and ratio >= self.minimum_support_millis:
        outcome = CouncilOutcome.PROPOSED
    else:
        outcome = CouncilOutcome.DEFERRED
    transcript = {
        "session_id": self.session_id,
        "context_hash": self.context_hash,
        "participants": [
            {
                "participant_id": participant.participant_id,
                "role": participant.role,
                "weight_millis": participant.weight_millis,
                "veto_capable": participant.veto_capable,
            }
            for participant in sorted(
                self.participants.values(), key=lambda item: item.participant_id
            )
        ]
    }

```

```

    )
    ],
    "rounds": [
        [ballot.to_document() for ballot in sorted(row, key=lambda item: item.participant_id)]
        for row in rounds
    ],
    "high_risk": self.high_risk,
    "minimum_support_millis": self.minimum_support_millis,
}
return CouncilDecision(
    session_id=self.session_id,
    outcome=outcome,
    proposal_hash=proposal_hash,
    round_count=len(rounds),
    support_millis=ratio,
    oppose_millis=1000 - ratio if total else 0,
    abstentions=abstentions,
    vetoes=vetoed,
    dissent=tuple(sorted(dissent)),
    transcript_hash=canonical_hash(transcript),
)

```

پیاده سازی: effects.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/effects.py

SHA-256 منبع:

cadd42c10e886554c92ea12b6af89b56ea05fdb1316189d688aa20d218642da

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Deterministic external-effect proposal and approval binding primitives.

This module performs no I/O and writes no database rows. It prepares a value
that the canonical Kernel may insert into Command/Event/Outbox in one
PostgreSQL transaction after Policy and version/control-epoch rechecks.
"""
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
import hashlib
import re
from typing import Any, Literal

from .resolution import canonical_hash

EffectSemantics = Literal["NATIVE_IDEMPOTENT", "RECONCILABLE", "NON_IDEMPOTENT"]
_HASH_RE = re.compile(r"^[0-9a-f]{64}$")

class EffectBindingError(ValueError):
    pass

def sha256_bytes(payload: bytes) -> str:
    if not isinstance(payload, bytes) or not payload:
        raise EffectBindingError("effect payload must be non-empty bytes")
    return hashlib.sha256(payload).hexdigest()

def _require_text(value: str, label: str) -> None:
    if not isinstance(value, str) or not value.strip() or len(value) > 512:
        raise EffectBindingError(f"invalid {label}")

def _require_hash(value: str, label: str) -> None:
    if not isinstance(value, str) or not _HASH_RE.fullmatch(value):
        raise EffectBindingError(f"invalid {label}")

@dataclass(frozen=True)
class ExternalEffectProposal:
    world_id: str
    entity_id: str
    command_id: str
    destination: str
    action: str
    resource_ref: str
    recipient_ref: str
    payload_ref: str
    payload_hash: str
    policy_version: str
    expected_version: int
    control_epoch: int
    idempotency_scope: str
    idempotency_key: str
    effect_semantics: EffectSemantics = "RECONCILABLE"
```

```

def __post_init__(self) -> None:
    for label in (
        "world_id", "entity_id", "command_id", "destination", "action",
        "resource_ref", "recipient_ref", "payload_ref", "policy_version",
        "idempotency_scope", "idempotency_key",
    ):
        _require_text(getattr(self, label), label)
        _require_hash(self.payload_hash, "payload_hash")
    if not isinstance(self.expected_version, int) or self.expected_version < 0:
        raise EffectBindingError("expected_version must be non-negative")
    if not isinstance(self.control_epoch, int) or self.control_epoch < 1:
        raise EffectBindingError("control_epoch must be positive")
    if self.effect_semantics not in {
        "NATIVE_IDEMPOTENT", "RECONCILABLE", "NON_IDEMPOTENT"
    }:
        raise EffectBindingError("invalid effect_semantics")

def binding_document(self) -> dict[str, Any]:
    return {
        "world_id": self.world_id,
        "entity_id": self.entity_id,
        "command_id": self.command_id,
        "destination": self.destination,
        "action": self.action,
        "resource_ref": self.resource_ref,
        "recipient_ref": self.recipient_ref,
        "payload_ref": self.payload_ref,
        "payload_hash": self.payload_hash,
        "policy_version": self.policy_version,
        "expected_version": self.expected_version,
        "control_epoch": self.control_epoch,
        "idempotency_scope": self.idempotency_scope,
        "idempotency_key": self.idempotency_key,
        "effect_semantics": self.effect_semantics,
    }

@property
def effect_hash(self) -> str:
    return canonical_hash(self.binding_document())

@dataclass(frozen=True)
class ApprovalBinding:
    approval_ref: str
    approver_ref: str
    decision: Literal["APPROVE"]
    command_id: str
    action: str
    recipient_ref: str
    payload_hash: str
    effect_hash: str
    policy_version: str
    expected_version: int
    control_epoch: int
    issued_at: str
    expires_at: str | None = None

def __post_init__(self) -> None:
    for label in (
        "approval_ref", "approver_ref", "command_id", "action",
        "recipient_ref", "policy_version", "issued_at",
    ):
        _require_text(getattr(self, label), label)
    if self.decision != "APPROVE":
        raise EffectBindingError("only an explicit APPROVE binding can authorize an effect")
    _require_hash(self.payload_hash, "payload_hash")
    _require_hash(self.effect_hash, "effect_hash")
    if not isinstance(self.expected_version, int) or self.expected_version < 0:
        raise EffectBindingError("expected_version must be non-negative")
    if not isinstance(self.control_epoch, int) or self.control_epoch < 1:
        raise EffectBindingError("control_epoch must be positive")

@dataclass(frozen=True)
class PreparedOutboxIntent:
    proposal: ExternalEffectProposal
    approval_ref: str
    policy_decision_ref: str
    effect_hash: str
    status: Literal["PREPARED_NOT_COMMITTED"] = "PREPARED_NOT_COMMITTED"
    authoritative: bool = False
    
```

```

def to_document(self) -> dict[str, Any]:
    return {
        **self.proposal.binding_document(),
        "effect_hash": self.effect_hash,
        "approval_ref": self.approval_ref,
        "policy_decision_ref": self.policy_decision_ref,
        "status": self.status,
        "authoritative": False,
    }

def validate_approval_binding(
    proposal: ExternalEffectProposal,
    approval: ApprovalBinding,
    *,
    evaluated_at: str,
) -> None:
    if approval.approver_ref != "human-root":
        raise EffectBindingError("approval must be issued by stable human-root role")
    expected = {
        "command_id": proposal.command_id,
        "action": proposal.action,
        "recipient_ref": proposal.recipient_ref,
        "payload_hash": proposal.payload_hash,
        "effect_hash": proposal.effect_hash,
        "policy_version": proposal.policy_version,
        "expected_version": proposal.expected_version,
        "control_epoch": proposal.control_epoch,
    }
    actual = {field: getattr(approval, field) for field in expected}
    mismatches = sorted(field for field, value in expected.items() if actual[field] != value)
    if mismatches:
        raise EffectBindingError(f"approval binding mismatch: {mismatches}")
    try:
        evaluated = datetime.fromisoformat(evaluated_at.replace("Z", "+00:00"))
        issued = datetime.fromisoformat(approval.issued_at.replace("Z", "+00:00"))
        expires = (
            datetime.fromisoformat(approval.expires_at.replace("Z", "+00:00"))
            if approval.expires_at else None
        )
    except ValueError as exc:
        raise EffectBindingError("approval timestamps must be ISO-8601") from exc
    if evaluated < issued:
        raise EffectBindingError("approval is not yet valid")
    if expires is not None and evaluated >= expires:
        raise EffectBindingError("approval has expired")

def prepare_outbox_intent(
    proposal: ExternalEffectProposal,
    approval: ApprovalBinding,
    *,
    policy_decision_ref: str,
    evaluated_at: str,
) -> PreparedOutboxIntent:
    _require_text(policy_decision_ref, "policy_decision_ref")
    validate_approval_binding(proposal, approval, evaluated_at=evaluated_at)
    return PreparedOutboxIntent(
        proposal=proposal,
        approval_ref=approval.approval_ref,
        policy_decision_ref=policy_decision_ref,
        effect_hash=proposal.effect_hash,
    )

```

پیاده سازی: evolution.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/evolution.py

SHA-256 منبع:

a10922c37dd8f14445f57d5376fc218be0a446ca23ef6b9500be819c7feaaaa1

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Shadow evolution and compilation-maturity gates for World v6.2 RC2.

Generated or model-derived logic may learn beside the active path, but this
module can only recommend a one-step promotion. It cannot rewrite code,
change DNA, deploy, or grant authority.
"""
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum
from typing import Iterable

from .resolution import canonical_hash

class EvolutionContractError(ValueError):
    pass

class CompilationMaturity(str, Enum):
    C0 = "C0" # prompt/manual only
    C1 = "C1" # captured fixture/trace
    C2 = "C2" # executable candidate
    C3 = "C3" # shadow-equivalent
    C4 = "C4" # bounded canary eligible
    C5 = "C5" # approved primary eligible
    C6 = "C6" # restore-proven non-extinction spine

    @property
    def rank(self) -> int:
        return int(self.value[1:])

    def next(self) -> "CompilationMaturity | None":
        if self is CompilationMaturity.C6:
            return None
        return CompilationMaturity(f"C{self.rank + 1}")

@dataclass(frozen=True)
class ShadowRunRecord:
    run_id: str
    candidate_id: str
    candidate_version: str
    candidate_hash: str
    node_id: str
    input_hash: str
    primary_semantic_hash: str
    candidate_semantic_hash: str
    invariant_failures: int = 0
    forbidden_effect_attempts: int = 0
    replay_match: bool = True
    latency_ms: int = 0
    estimated_cost_micros: int = 0

    def __post_init__(self) -> None:
        for digest in (
            self.candidate_hash,
            self.input_hash,
            self.primary_semantic_hash,
```

```

        self.candidate_semantic_hash,
    ):
        if len(digest) != 64 or any(char not in "0123456789abcdef" for char in digest):
            raise EvolutionContractError("Shadow hashes must be lowercase SHA-256")
    for value in (
        self.invariant_failures,
        self.forbidden_effect_attempts,
        self.latency_ms,
        self.estimated_cost_micros,
    ):
        if not isinstance(value, int) or value < 0:
            raise EvolutionContractError("Shadow metrics must be non-negative integers")

@property
def semantic_match(self) -> bool:
    return self.primary_semantic_hash == self.candidate_semantic_hash

def to_document(self) -> dict[str, object]:
    return {
        "run_id": self.run_id,
        "candidate_id": self.candidate_id,
        "candidate_version": self.candidate_version,
        "candidate_hash": self.candidate_hash,
        "node_id": self.node_id,
        "input_hash": self.input_hash,
        "primary_semantic_hash": self.primary_semantic_hash,
        "candidate_semantic_hash": self.candidate_semantic_hash,
        "semantic_match": self.semantic_match,
        "invariant_failures": self.invariant_failures,
        "forbidden_effect_attempts": self.forbidden_effect_attempts,
        "replay_match": self.replay_match,
        "latency_ms": self.latency_ms,
        "estimated_cost_micros": self.estimated_cost_micros,
    }

@dataclass(frozen=True)
class PromotionThreshold:
    minimum_runs: int
    minimum_agreement_millis: int
    require_all_replays: bool = True
    human_root_approval_required: bool = False
    restore_proof_required: bool = False

    def __post_init__(self) -> None:
        if not isinstance(self.minimum_runs, int) or self.minimum_runs < 1:
            raise EvolutionContractError("minimum_runs must be a positive integer")
        if (
            not isinstance(self.minimum_agreement_millis, int)
            or not 0 <= self.minimum_agreement_millis <= 1000
        ):
            raise EvolutionContractError("agreement threshold outside 0..1000")

DEFAULT_THRESHOLDS: dict[CompilationMaturity, PromotionThreshold] = {
    CompilationMaturity.C1: PromotionThreshold(1, 0),
    CompilationMaturity.C2: PromotionThreshold(5, 600),
    CompilationMaturity.C3: PromotionThreshold(20, 850),
    CompilationMaturity.C4: PromotionThreshold(50, 930),
    CompilationMaturity.C5: PromotionThreshold(100, 970, True, True),
    CompilationMaturity.C6: PromotionThreshold(200, 990, True, True, True),
}

@dataclass(frozen=True)
class PromotionAssessment:
    candidate_id: str
    candidate_version: str
    candidate_hash: str
    from_maturity: str
    target_maturity: str
    eligible: bool
    reasons: tuple[str, ...]
    run_count: int
    agreement_millis: int
    evidence_hash: str
    recommendation_only: bool = True

class PromotionController:
    def __init__(

```



```

        self,
        thresholds: dict[CompilationMaturity, PromotionThreshold] | None = None,
    ) -> None:
        self.thresholds = dict(thresholds or DEFAULT_THRESHOLDS)
        required = set(CompilationMaturity) - {CompilationMaturity.C0}
        if set(self.thresholds) != required:
            raise EvolutionContractError("thresholds must define exact targets C1..C6")

    def assess(
        self,
        *,
        candidate_id: str,
        candidate_version: str,
        candidate_hash: str,
        current: CompilationMaturity,
        records: Iterable[ShadowRunRecord],
        human_root_approval_ref: str | None = None,
        restore_proof_ref: str | None = None,
    ) -> PromotionAssessment:
        target = current.next()
        if target is None:
            raise EvolutionContractError("C6 is terminal; create a new candidate version")
        selected = tuple(records)
        reasons: list[str] = []
        if not selected:
            reasons.append("NO_SHADOW_EVIDENCE")
        for record in selected:
            if (
                record.candidate_id != candidate_id
                or record.candidate_version != candidate_version
                or record.candidate_hash != candidate_hash
            ):
                raise EvolutionContractError("mixed candidate evidence is forbidden")
        run_ids = [record.run_id for record in selected]
        if len(run_ids) != len(set(run_ids)):
            raise EvolutionContractError("duplicate Shadow run_id is forbidden")
        threshold = self.thresholds[target]
        matches = sum(record.semantic_match for record in selected)
        agreement = (matches * 1000 // len(selected)) if selected else 0
        if len(selected) < threshold.minimum_runs:
            reasons.append("INSUFFICIENT_RUNS")
        if agreement < threshold.minimum_agreement_millis:
            reasons.append("AGREEMENT_BELOW_THRESHOLD")
        if any(record.invariant_failures for record in selected):
            reasons.append("INVARIANT_FAILURE")
        if any(record.forbidden_effect_attempts for record in selected):
            reasons.append("FORBIDDEN_EFFECT_ATTEMPT")
        if threshold.require_all_replays and any(not record.replay_match for record in selected):
            reasons.append("NONDETERMINISTIC_REPLAY")
        if threshold.human_root_approval_required and not human_root_approval_ref:
            reasons.append("HUMAN_ROOT_APPROVAL_REQUIRED")
        if threshold.restore_proof_required and not restore_proof_ref:
            reasons.append("RESTORE_PROOF_REQUIRED")
        evidence_document = {
            "candidate_id": candidate_id,
            "candidate_version": candidate_version,
            "candidate_hash": candidate_hash,
            "from_maturity": current.value,
            "target_maturity": target.value,
            "records": [record.to_document() for record in selected],
            "human_root_approval_ref": human_root_approval_ref,
            "restore_proof_ref": restore_proof_ref,
        }
        return PromotionAssessment(
            candidate_id=candidate_id,
            candidate_version=candidate_version,
            candidate_hash=candidate_hash,
            from_maturity=current.value,
            target_maturity=target.value,
            eligible=not reasons,
            reasons=tuple(reasons),
            run_count=len(selected),
            agreement_millis=agreement,
            evidence_hash=canonical_hash(evidence_document),
        )

```

پیاده سازی: fractal_runtime.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/runtime/core/fractal_runtime.py

SHA-256 منبع:

a55079162bf9df3e0c5f8f115d015468e45dd6edf29b6bac77b020002ab45aef

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
"""Fractal, fail-closed execution primitives for World v6.2 RC2.

The runtime keeps complexity local to a node. A node starts coarse and only
expands when a handler explicitly asks for more detail. Handlers can be code,
manual host sessions, local models, remote brains, or councils, but every
result remains a non-authoritative proposal.

This module performs no network I/O and executes no external effect.
"""
from __future__ import annotations

from dataclasses import dataclass, field, replace
from enum import Enum
import re
from typing import Any, Mapping, Protocol, Sequence

from .resolution import canonical_hash, canonical_json, resolution_rank

class FractalContractError(ValueError):
    """A versioned fractal execution contract is invalid."""

class FractalExecutionError(RuntimeError):
    """No safe execution path could satisfy the capsule."""

class HandlerStatus(str, Enum):
    SUCCESS = "SUCCESS"
    NEEDS_DETAIL = "NEEDS_DETAIL"
    ABSTAIN = "ABSTAIN"
    FAILED = "FAILED"

class OutcomeStatus(str, Enum):
    PROPOSED = "PROPOSED"
    EXPANDED = "EXPANDED"
    DEFERRED = "DEFERRED"

_LEVEL_BOUNDS: dict[str, tuple[int, int]] = {
    "X": (0, 9), # execution granularity
    "B": (0, 4), # B5 (World ownership) is constitutionally forbidden
    "D": (0, 5), # deliberation depth
    "C": (0, 6), # compilation maturity
    "A": (0, 5), # authority remains independently policy-gated
    "E": (0, 5), # evidence ladder
    "M": (0, 9), # model capability class; profile-specific semantics
}

def _level_rank(value: str, prefix: str) -> int:
    if not isinstance(value, str) or re.fullmatch(fr"{prefix}(0|[1-9][0-9]*)", value) is None:
        raise FractalContractError(f"{prefix} level must look like {prefix}0")
    rank = int(value[1:])
    lower, upper = _LEVEL_BOUNDS[prefix]
    if rank < lower or rank > upper:
        raise FractalContractError(f"{prefix} level outside supported bounds: {value}")
    return rank
```

```

def _token(value: str, label: str) -> None:
    if not isinstance(value, str) or not value or len(value) > 160:
        raise FractalContractError(f"{label} must be a non-empty bounded string")
    if re.fullmatch(r"[A-Za-z0-9][A-Za-z0-9._:/@#-]*", value) is None:
        raise FractalContractError(f"{label} contains forbidden characters")

@dataclass(frozen=True)
class ExecutionVector:
    """Orthogonal execution axes; never collapse these into one global level."""

    resolutions: tuple[tuple[str, str], ...]
    granularity: str = "X0"
    brain_delegation: str = "B0"
    deliberation: str = "D0"
    compilation: str = "C0"
    authority: str = "A0"
    evidence: str = "E0"
    model_capability: str = "M0"

    def __post_init__(self) -> None:
        if not self.resolutions:
            raise FractalContractError("ExecutionVector requires at least one Resolution domain")
        domain_names = [name for name, _ in self.resolutions]
        if len(domain_names) != len(set(domain_names)):
            raise FractalContractError("duplicate Resolution domain")
        for domain, level in self.resolutions:
            _token(domain, "Resolution domain")
            resolution_rank(level)
            _level_rank(self.granularity, "X")
            _level_rank(self.brain_delegation, "B")
            _level_rank(self.deliberation, "D")
            _level_rank(self.compilation, "C")
            _level_rank(self.authority, "A")
            _level_rank(self.evidence, "E")
            _level_rank(self.model_capability, "M")

    def to_document(self) -> dict[str, Any]:
        return {
            "resolutions": {key: value for key, value in sorted(self.resolutions)},
            "granularity": self.granularity,
            "brain_delegation": self.brain_delegation,
            "deliberation": self.deliberation,
            "compilation": self.compilation,
            "authority": self.authority,
            "evidence": self.evidence,
            "model_capability": self.model_capability,
        }

@dataclass(frozen=True)
class ExecutionBudget:
    """Hard budgets keep recursive refinement bounded and predictable."""

    max_input_tokens: int = 4_000
    max_cost_micros: int = 0
    max_latency_ms: int = 30_000
    max_depth: int = 3
    max_attempts: int = 4
    max_council_rounds: int = 2

    def __post_init__(self) -> None:
        values = (
            self.max_input_tokens,
            self.max_cost_micros,
            self.max_latency_ms,
            self.max_depth,
            self.max_attempts,
            self.max_council_rounds,
        )
        if any(not isinstance(value, int) or value < 0 for value in values):
            raise FractalContractError("execution budgets must be non-negative integers")
        if self.max_attempts == 0:
            raise FractalContractError("max_attempts cannot be zero")
        if self.max_depth > 12:
            raise FractalContractError("max_depth exceeds the bounded fractal limit")

    def to_document(self) -> dict[str, int]:
        return {

```

```

        "max_input_tokens": self.max_input_tokens,
        "max_cost_micros": self.max_cost_micros,
        "max_latency_ms": self.max_latency_ms,
        "max_depth": self.max_depth,
        "max_attempts": self.max_attempts,
        "max_council_rounds": self.max_council_rounds,
    }

@dataclass(frozen=True)
class FractalNode:
    node_id: str
    version: str
    purpose: str
    input_contract_ref: str
    output_contract_ref: str
    handler_ids: tuple[str, ...]
    fallback_handler_ids: tuple[str, ...] = ()
    parent_id: str | None = None
    child_ids: tuple[str, ...] = ()
    expandable: bool = False
    risk_class: str = "LOW"
    minimum_vector: ExecutionVector = field(
        default_factory=lambda: ExecutionVector(("task", "R0"),)
    )

    def __post_init__(self) -> None:
        _token(self.node_id, "node_id")
        _token(self.version, "node version")
        _token(self.input_contract_ref, "input contract ref")
        _token(self.output_contract_ref, "output contract ref")
        if not isinstance(self.purpose, str) or not self.purpose or len(self.purpose) > 1_000:
            raise FractalContractError("node purpose must be a bounded non-empty string")
        if self.parent_id is not None:
            _token(self.parent_id, "parent_id")
        if self.risk_class not in {"LOW", "MEDIUM", "HIGH", "CRITICAL"}:
            raise FractalContractError("invalid node risk class")
        ordered = self.handler_ids + self.fallback_handler_ids
        if not ordered:
            raise FractalContractError("node requires at least one handler")
        if len(ordered) != len(set(ordered)):
            raise FractalContractError("handler and fallback chains must be unique")
        if len(self.child_ids) != len(set(self.child_ids)):
            raise FractalContractError("duplicate child node")
        if self.child_ids and not self.expandable:
            raise FractalContractError("node with children must be expandable")
        for identifier in ordered + self.child_ids:
            _token(identifier, "node reference")

    def to_document(self) -> dict[str, Any]:
        return {
            "node_id": self.node_id,
            "version": self.version,
            "purpose": self.purpose,
            "input_contract_ref": self.input_contract_ref,
            "output_contract_ref": self.output_contract_ref,
            "handler_ids": list(self.handler_ids),
            "fallback_handler_ids": list(self.fallback_handler_ids),
            "parent_id": self.parent_id,
            "child_ids": list(self.child_ids),
            "expandable": self.expandable,
            "risk_class": self.risk_class,
            "minimum_vector": self.minimum_vector.to_document(),
        }

    @property
    def node_hash(self) -> str:
        return canonical_hash(self.to_document())

@dataclass(frozen=True)
class ExecutionCapsule:
    capsule_id: str
    world_id: str
    entity_id: str
    principal_id: str
    conversation_id: str
    node_id: str
    node_version: str
    node_hash: str
    canonical_input_hash: str

```

```

state_refs: tuple[str, ...]
vector: ExecutionVector
budget: ExecutionBudget
purpose: str
expected_version: int
data_class: str = "INTERNAL"
freshness: str = "CURRENT"
parent_capsule_id: str | None = None

def __post_init__(self) -> None:
    for value, label in (
        (self.capsule_id, "capsule_id"),
        (self.world_id, "world_id"),
        (self.entity_id, "entity_id"),
        (self.principal_id, "principal_id"),
        (self.conversation_id, "conversation_id"),
        (self.node_id, "node_id"),
        (self.node_version, "node_version"),
    ):
        _token(value, label)
    if self.parent_capsule_id is not None:
        _token(self.parent_capsule_id, "parent_capsule_id")
    for digest, label in (
        (self.node_hash, "node_hash"),
        (self.canonical_input_hash, "canonical_input_hash"),
    ):
        if re.fullmatch(r"[0-9a-f]{64}", digest) is None:
            raise FractalContractError(f"{label} must be a lowercase SHA-256")
    if not isinstance(self.expected_version, int) or self.expected_version < 0:
        raise FractalContractError("expected_version must be a non-negative integer")
    if self.data_class not in {"PUBLIC", "INTERNAL", "RESTRICTED", "SECRET"}:
        raise FractalContractError("invalid data_class")
    if self.freshness not in {"CURRENT", "STALE", "UNKNOWN"}:
        raise FractalContractError("invalid freshness")
    for ref in self.state_refs:
        _token(ref, "state_ref")

def to_document(self) -> dict[str, Any]:
    return {
        "capsule_id": self.capsule_id,
        "world_id": self.world_id,
        "entity_id": self.entity_id,
        "principal_id": self.principal_id,
        "conversation_id": self.conversation_id,
        "node_id": self.node_id,
        "node_version": self.node_version,
        "node_hash": self.node_hash,
        "canonical_input_hash": self.canonical_input_hash,
        "state_refs": list(self.state_refs),
        "vector": self.vector.to_document(),
        "budget": self.budget.to_document(),
        "purpose": self.purpose,
        "expected_version": self.expected_version,
        "data_class": self.data_class,
        "freshness": self.freshness,
        "parent_capsule_id": self.parent_capsule_id,
    }

@property
def capsule_hash(self) -> str:
    return canonical_hash(self.to_document())

@dataclass(frozen=True)
class HandlerProfile:
    handler_id: str
    version: str
    kind: str
    maturity: str
    evidence: str
    supported_node_ids: tuple[str, ...]
    deterministic: bool
    network_required: bool = False
    api_token_required: bool = False
    proposal_only: bool = True
    provider_family: str = "NONE"

    def __post_init__(self) -> None:
        _token(self.handler_id, "handler_id")
        _token(self.version, "handler version")
        _token(self.provider_family, "provider_family")

```

```

    if self.kind not in {"CODE", "BRAIN", "MANUAL_HOST", "LOCAL_MODEL", "COUNCIL"}:
        raise FractalContractError("invalid handler kind")
    _level_rank(self.maturity, "C")
    _level_rank(self.evidence, "E")
    if not self.supported_node_ids:
        raise FractalContractError("handler must support at least one node")
    for node_id in self.supported_node_ids:
        _token(node_id, "supported node_id")
    if not self.proposal_only:
        raise FractalContractError("Phase 0/RC2 handlers must be proposal-only")

    def to_document(self) -> dict[str, Any]:
        return {
            "handler_id": self.handler_id,
            "version": self.version,
            "kind": self.kind,
            "maturity": self.maturity,
            "evidence": self.evidence,
            "supported_node_ids": list(self.supported_node_ids),
            "deterministic": self.deterministic,
            "network_required": self.network_required,
            "api_token_required": self.api_token_required,
            "proposal_only": self.proposal_only,
            "provider_family": self.provider_family,
        }

    @property
    def profile_hash(self) -> str:
        return canonical_hash(self.to_document())

@dataclass(frozen=True)
class HandlerResult:
    handler_id: str
    handler_version: str
    status: HandlerStatus
    output: dict[str, Any]
    confidence_millis: int = 0
    trace_summary: str = ""
    proposal_only: bool = True

    def __post_init__(self) -> None:
        _token(self.handler_id, "result handler id")
        _token(self.handler_version, "result handler version")
        if not isinstance(self.status, HandlerStatus):
            raise FractalContractError("handler status must be a HandlerStatus")
        if not isinstance(self.output, dict):
            raise FractalContractError("handler output must be a JSON object")
        if not 0 <= self.confidence_millis <= 1000:
            raise FractalContractError("confidence_millis outside 0..1000")
        if not self.proposal_only:
            raise FractalContractError("handler attempted an authoritative result")

    @property
    def output_hash(self) -> str:
        return canonical_hash(self.output)

class FractalHandler(Protocol):
    profile: HandlerProfile

    def invoke(self, capsule: ExecutionCapsule, payload: Mapping[str, Any]) -> HandlerResult: ...

@dataclass(frozen=True)
class AttemptRecord:
    node_id: str
    handler_id: str
    handler_version: str
    status: str
    input_hash: str
    output_hash: str | None
    reason: str

@dataclass(frozen=True)
class ExecutionOutcome:
    status: OutcomeStatus
    capsule: ExecutionCapsule
    result: HandlerResult | None
    attempts: tuple[AttemptRecord, ...]
```

```

child_outcomes: tuple["ExecutionOutcome", ...] = ()

@property
def proposal(self) -> dict[str, Any] | None:
    return None if self.result is None else self.result.output

class HandlerRegistry:
    def __init__(self, handlers: Sequence[FractalHandler] = ()) -> None:
        self._handlers: dict[str, FractalHandler] = {}
        for handler in handlers:
            self.register(handler)

    def register(self, handler: FractalHandler) -> None:
        identifier = handler.profile.handler_id
        if identifier in self._handlers:
            raise FractalContractError(f"duplicate handler_id: {identifier}")
        self._handlers[identifier] = handler

    def get(self, handler_id: str) -> FractalHandler:
        try:
            return self._handlers[handler_id]
        except KeyError as exc:
            raise FractalContractError(f"unregistered handler: {handler_id}") from exc

    def profile_documents(self) -> tuple[dict[str, Any], ...]:
        return tuple(
            self._handlers[key].profile.to_document() for key in sorted(self._handlers)
        )

class FractalOrchestrator:
    """Minimal-cost, bounded-depth coordinator with no external-effect authority."""

    def __init__(
        self,
        nodes: Sequence[FractalNode],
        registry: HandlerRegistry,
        *,
        allow_network: bool = False,
        allow_api_tokens: bool = False,
    ) -> None:
        self.nodes: dict[str, FractalNode] = {}
        for node in nodes:
            if node.node_id in self.nodes:
                raise FractalContractError(f"duplicate node_id: {node.node_id}")
            self.nodes[node.node_id] = node
        self.registry = registry
        self.allow_network = allow_network
        self.allow_api_tokens = allow_api_tokens
        self._validate_tree()

    def _validate_tree(self) -> None:
        for node in self.nodes.values():
            for child_id in node.child_ids:
                child = self.nodes.get(child_id)
                if child is None:
                    raise FractalContractError(f"missing child node: {child_id}")
                if child.parent_id != node.node_id:
                    raise FractalContractError("child parent binding mismatch")
            for handler_id in node.handler_ids + node.fallback_handler_ids:
                handler = self.registry.get(handler_id)
                if node.node_id not in handler.profile.supported_node_ids:
                    raise FractalContractError(
                        f"handler {handler_id} does not declare node {node.node_id}"
                    )

    # A fractal graph is a bounded tree/DAG, never a recursive call loop.
    visiting: set[str] = set()
    visited: set[str] = set()

    def visit(node_id: str) -> None:
        if node_id in visiting:
            raise FractalContractError(f"cycle detected at node: {node_id}")
        if node_id in visited:
            return
        visiting.add(node_id)
        for child_id in self.nodes[node_id].child_ids:
            visit(child_id)
        visiting.remove(node_id)
        visited.add(node_id)

```

```

        for node_id in sorted(self.nodes):
            visit(node_id)

    @staticmethod
    def build_capsule(
        node: FractalNode,
        payload: Mapping[str, Any],
        *,
        capsule_id: str,
        vector: ExecutionVector,
        budget: ExecutionBudget | None = None,
        world_id: str = "world-v6",
        entity_id: str = "secretary-001",
        principal_id: str = "human-root",
        conversation_id: str = "human-root:secretary-001",
        state_refs: tuple[str, ...] = (),
        expected_version: int = 0,
        purpose: str | None = None,
        data_class: str = "INTERNAL",
        freshness: str = "CURRENT",
    ) -> ExecutionCapsule:
        return ExecutionCapsule(
            capsule_id=capsule_id,
            world_id=world_id,
            entity_id=entity_id,
            principal_id=principal_id,
            conversation_id=conversation_id,
            node_id=node.node_id,
            node_version=node.version,
            node_hash=node.node_hash,
            canonical_input_hash=canonical_hash(dict(payload)),
            state_refs=state_refs,
            vector=vector,
            budget=budget or ExecutionBudget(),
            purpose=purpose or node.purpose,
            expected_version=expected_version,
            data_class=data_class,
            freshness=freshness,
        )

    @staticmethod
    def _validate_capsule(node: FractalNode, capsule: ExecutionCapsule, payload: Mapping[str, Any]) -> None:
        if (capsule.node_id, capsule.node_version, capsule.node_hash) != (
            node.node_id,
            node.version,
            node.node_hash,
        ):
            raise FractalExecutionError("capsule is not bound to the selected node version/hash")
        if capsule.canonical_input_hash != canonical_hash(dict(payload)):
            raise FractalExecutionError("capsule input hash mismatch")
        # This is deliberately a conservative byte ceiling. Provider-specific
        # tokenizers may reject an input earlier, but may never raise this cap.
        if len(canonical_json(dict(payload)).encode("utf-8")) > capsule.budget.max_input_tokens * 8:
            raise FractalExecutionError("capsule input exceeds bounded token approximation")
        requested = dict(capsule.vector.resolutions)
        required = dict(node.minimum_vector.resolutions)
        for domain, minimum in required.items():
            actual = requested.get(domain)
            if actual is None or resolution_rank(actual) < resolution_rank(minimum):
                raise FractalExecutionError(
                    f"capsule Resolution below node minimum for domain {domain}"
                )
        for prefix, actual, minimum in (
            ("X", capsule.vector.granularity, node.minimum_vector.granularity),
            ("B", capsule.vector.brain_delegation, node.minimum_vector.brain_delegation),
            ("D", capsule.vector.deliberation, node.minimum_vector.deliberation),
            ("C", capsule.vector.compilation, node.minimum_vector.compilation),
            ("A", capsule.vector.authority, node.minimum_vector.authority),
            ("E", capsule.vector.evidence, node.minimum_vector.evidence),
            ("M", capsule.vector.model_capability, node.minimum_vector.model_capability),
        ):
            if _level_rank(actual, prefix) < _level_rank(minimum, prefix):
                raise FractalExecutionError(f"capsule {prefix} below node minimum")

    def execute(
        self, node_id: str, capsule: ExecutionCapsule, payload: Mapping[str, Any]
    ) -> ExecutionOutcome:
        node = self.nodes.get(node_id)
        if node is None:
            raise FractalExecutionError(f"unknown node: {node_id}")

```



```

return self._execute(node, capsule, payload, depth=0, consumed_attempts=0)

def _execute(
    self,
    node: FractalNode,
    capsule: ExecutionCapsule,
    payload: Mapping[str, Any],
    *,
    depth: int,
    consumed_attempts: int,
) -> ExecutionOutcome:
    self._validate_capsule(node, capsule, payload)
    attempts: list[AttemptRecord] = []
    wants_detail = False
    ordered_handlers = node.handler_ids + node.fallback_handler_ids

    for offset, handler_id in enumerate(ordered_handlers, start=1):
        if consumed_attempts + offset > capsule.budget.max_attempts:
            break
        handler = self.registry.get(handler_id)
        profile = handler.profile
        if _level_rank(profile.maturity, "C") < _level_rank(
            capsule.vector.compilation, "C"
        ):
            attempts.append(
                AttemptRecord(
                    node.node_id,
                    profile.handler_id,
                    profile.version,
                    "SKIPPED",
                    capsule.canonical_input_hash,
                    None,
                    "MATURITY_BELOW_NODE_MINIMUM",
                )
            )
            continue
        if _level_rank(profile.evidence, "E") < _level_rank(
            capsule.vector.evidence, "E"
        ):
            attempts.append(
                AttemptRecord(
                    node.node_id,
                    profile.handler_id,
                    profile.version,
                    "SKIPPED",
                    capsule.canonical_input_hash,
                    None,
                    "EVIDENCE_BELOW_NODE_MINIMUM",
                )
            )
            continue
        delegation_rank = _level_rank(capsule.vector.brain_delegation, "B")
        deliberation_rank = _level_rank(capsule.vector.deliberation, "D")
        if profile.kind != "CODE" and delegation_rank == 0:
            attempts.append(
                AttemptRecord(
                    node.node_id,
                    profile.handler_id,
                    profile.version,
                    "SKIPPED",
                    capsule.canonical_input_hash,
                    None,
                    "BRAIN_DELEGATION_DISABLED",
                )
            )
            continue
        if profile.kind == "COUNCIL" and (delegation_rank < 3 or deliberation_rank < 3):
            attempts.append(
                AttemptRecord(
                    node.node_id,
                    profile.handler_id,
                    profile.version,
                    "SKIPPED",
                    capsule.canonical_input_hash,
                    None,
                    "COUNCIL_LEVEL_REQUIRES_B3_D3",
                )
            )
            continue
        if profile.network_required and not self.allow_network:
            attempts.append(

```

```

        AttemptRecord(
            node.node_id,
            profile.handler_id,
            profile.version,
            "SKIPPED",
            capsule.canonical_input_hash,
            None,
            "NETWORK_DISABLED",
        )
    )
    continue
if profile.api_token_required and not self.allow_api_tokens:
    attempts.append(
        AttemptRecord(
            node.node_id,
            profile.handler_id,
            profile.version,
            "SKIPPED",
            capsule.canonical_input_hash,
            None,
            "API_TOKEN_DISABLED",
        )
    )
    continue
try:
    result = handler.invoke(capsule, payload)
    if result.handler_id != profile.handler_id or result.handler_version != profile.version:
        raise FractalExecutionError("handler result identity/version mismatch")
    if not result.proposal_only:
        raise FractalExecutionError("handler attempted an authoritative effect")
    attempts.append(
        AttemptRecord(
            node.node_id,
            profile.handler_id,
            profile.version,
            result.status.value,
            capsule.canonical_input_hash,
            result.output_hash,
            result.trace_summary,
        )
    )
    if result.status is HandlerStatus.SUCCESS:
        return ExecutionOutcome(
            OutcomeStatus.PROPOSED, capsule, result, tuple(attempts)
        )
    if result.status is HandlerStatus.NEEDS_DETAIL:
        wants_detail = True
except Exception as exc: # failover records type only, never secret payload
    attempts.append(
        AttemptRecord(
            node.node_id,
            profile.handler_id,
            profile.version,
            "FAILED",
            capsule.canonical_input_hash,
            None,
            type(exc).__name__,
        )
    )

can_expand = (
    wants_detail
    and node.expandable
    and node.child_ids
    and depth < capsule.budget.max_depth
)
if can_expand:
    child_outcomes: list[ExecutionOutcome] = []
    for index, child_id in enumerate(node.child_ids):
        child = self.nodes[child_id]
        child_capsule = replace(
            capsule,
            capsule_id=f"{capsule.capsule_id}.x{depth + 1}.{index}",
            node_id=child.node_id,
            node_version=child.version,
            node_hash=child.node_hash,
            parent_capsule_id=capsule.capsule_id,
            vector=replace(
                capsule.vector,
                granularity=f"X{min(_level_rank(capsule.vector.granularity, 'X') + 1, 9)}",
            ),
        ),

```

```

    )
    child_outcomes.append(
        self._execute(
            child,
            child_capsule,
            payload,
            depth=depth + 1,
            consumed_attempts=(
                consumed_attempts
                + len(attempts)
                + sum(self._attempt_count(item) for item in child_outcomes)
            ),
        )
    )
    )
    if child_outcomes and all(
        outcome.status in {OutcomeStatus.PROPOSED, OutcomeStatus.EXPANDED}
        for outcome in child_outcomes
    ):
        combined = HandlerResult(
            handler_id="world-v6.fractal-expander",
            handler_version="0.1.0",
            status=HandlerStatus.SUCCESS,
            output={
                "proposal_type": "FRACTAL_COMPOSITE",
                "children": {
                    outcome.capsule.node_id: outcome.proposal
                    for outcome in child_outcomes
                },
            },
            trace_summary="BOUNDED_CHILD_COMPOSITION",
        )
        return ExecutionOutcome(
            OutcomeStatus.EXPANDED,
            capsule,
            combined,
            tuple(attempts),
            tuple(child_outcomes),
        )

    return ExecutionOutcome(OutcomeStatus.DEFERRED, capsule, None, tuple(attempts))

@staticmethod
def _attempt_count(outcome: ExecutionOutcome) -> int:
    return len(outcome.attempts) + sum(
        FractalOrchestrator._attempt_count(child) for child in outcome.child_outcomes
    )

```

پیاده سازی: universal_model_bridge.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED
منبع محلی:

world-v6.2-rc3/tools/universal_model_bridge.py

SHA-256 منبع:

3f0094432ba5029208a26ff71d55a400db7ba3a1d39d84cfded850c7d58639ab

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
#!/usr/bin/env python3
"""No-network bridge between Secretary-001 and any contract-following model.

The export command emits one self-contained JSON bundle for manual paste into a
model host. The validation command accepts only the bounded Secretary Decision
contract and renders the final text deterministically. This module never calls
a provider, reads a token, mutates canonical state, or executes an external
effect.
"""
from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path
import sys
from typing import Any, Mapping

ROOT = Path(__file__).resolve().parents[1]
RUNTIME = ROOT / "candidate-v6.2" / "runtime"
sys.path.insert(0, str(RUNTIME))

from core.brain_gateway import BrainInputSegment, BrainRequest # noqa: E402
from core.portable_brain import ( # noqa: E402
    ManualHostAdapter,
    PortableBrainError,
    PortableSecretaryRunner,
    load_portable_pack,
    normalize_secretary_decision,
    render_secretary_decision,
)
from core.resolution import canonical_hash, load_profile_file # noqa: E402

CANDIDATE = ROOT / "candidate-v6.2"
PACK_DIR = CANDIDATE / "brain-packs" / "secretary-001"
PACK_PATH = PACK_DIR / "portable-brain-pack.v1.0.0.json"
PROMPT_PATH = PACK_DIR / "PROMPT_CONTRACT_v1.0_FA.md"
DECISION_SCHEMA_PATH = CANDIDATE / "schemas" / "secretary-decision.schema.json"
TASK_KEYS = {
    "task_id",
    "title",
    "status",
    "next_action",
    "due_at",
    "priority",
    "domain",
    "goal_id",
    "created_at",
}

def _sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def _load_object(path: str | Path) -> dict[str, Any]:
    document = json.loads(Path(path).read_text(encoding="utf-8"))
```

```

if not isinstance(document, dict):
    raise PortableBrainError(f"JSON document must be an object: {path}")
return document

def _profiles():
    return [
        load_profile_file(path)
        for path in sorted((CANDIDATE / "profiles").glob("*v0.2.json"))
    ]

def verify_pack_integrity() -> dict[str, str]:
    pack = load_portable_pack(PACK_PATH)
    observed = {
        "brain_pack_file_sha256": _sha256(PACK_PATH),
        "prompt_contract_sha256": _sha256(PROMPT_PATH),
        "decision_schema_sha256": _sha256(DECISION_SCHEMA_PATH),
    }
    expected = {
        "prompt_contract_sha256": pack.prompt_contract_hash,
        "decision_schema_sha256": pack.output_contract_hash,
    }
    mismatches = [key for key, value in expected.items() if observed[key] != value]
    if mismatches:
        raise PortableBrainError(f"portable pack integrity mismatch: {mismatches}")
    dna_path = PACK_DIR / "secretary-dna-overlay.v1.3-rc2.json"
    if _sha256(dna_path) != pack.dna_hash:
        raise PortableBrainError("portable pack DNA binding mismatch")
    return observed

def _task_document(path: str | Path) -> tuple[dict[str, Any], int]:
    raw = _load_object(path)
    source_version = raw.pop("source_version", 0)
    if set(raw) != TASK_KEYS:
        raise PortableBrainError(
            f"task key mismatch; missing={sorted(TASK_KEYS - set(raw))}, "
            f"extra={sorted(set(raw) - TASK_KEYS)}"
        )
    if not isinstance(source_version, int) or isinstance(source_version, bool) or source_version < 0:
        raise PortableBrainError("source_version must be a non-negative integer")
    for key in ("task_id", "title", "status", "next_action", "priority", "domain", "created_at"):
        if not isinstance(raw[key], str) or not raw[key] or len(raw[key]) > 4000:
            raise PortableBrainError(f"invalid task field: {key}")
    for key in ("due_at", "goal_id"):
        if raw[key] is not None and (not isinstance(raw[key], str) or len(raw[key]) > 4000):
            raise PortableBrainError(f"invalid optional task field: {key}")
    return raw, source_version

def build_task_bundle(task_path: str | Path, provider_label: str) -> dict[str, Any]:
    task, source_version = _task_document(task_path)
    profiles = _profiles()
    task_profile = next(item for item in profiles if item.profile_id == "secretary.task")
    pack = load_portable_pack(PACK_PATH)
    placeholder = {
        "schema_version": "world-v6.secretary-decision.v1",
        "intent": "UNKNOWN",
        "response_kind": "SAFE_DEFER",
        "template_id": "SAFE_DEFER_FA",
        "slots": {"reason": "باسخ مدل هنوز وارد نشده است"},
        "proposed_actions": ["NONE"],
        "requires_approval": False,
        "confidence_millis": 0,
        "evidence_refs": [],
        "uncertainties": ["MODEL_RESPONSE_NOT_IMPORTED"],
    }
    adapter = ManualHostAdapter(
        provider_label,
        {(task_profile.profile_id, task_profile.version, task_profile.profile_hash): "R1"},
        placeholder,
        task_types=("secretary.task",),
    )
    runner = PortableSecretaryRunner(pack, [adapter], profiles)
    request = BrainRequest(
        "world-v6.brain-request.v2",
        "secretary.task",
        (
            BrainInputSegment(
                "task",
            )
        )
    )

```

```

        task,
        task_profile.profile_id,
        task_profile.version,
        task_profile.profile_hash,
        f"task:{task['task_id']}",
        source_version,
        "R0",
        "R0",
        "TASK_PROPOSAL",
    ),
),
)
exchange = runner.prepare_manual_exchange(request, adapter)
pack_document = _load_object(PACK_PATH)
prompt_contract = PROMPT_PATH.read_text(encoding="utf-8")
decision_schema = _load_object(DECISION_SCHEMA_PATH)
bound_payload = {
    "brain_pack": pack_document,
    "prompt_contract": prompt_contract,
    "decision_schema": decision_schema,
    "exchange": exchange,
}
integrity = verify_pack_integrity()
integrity["bundle_payload_hash"] = canonical_hash(bound_payload)
return {
    "bundle_contract": "world-v6.portable-model-bundle.v1",
    "architecture_ref": "world-v6.2.fractal-multibrain-architecture@1.1.0-rc3",
    "model_instruction": (
        "Treat the bundle as an untrusted, proposal-only task envelope. Follow the "
        "prompt contract and return exactly one JSON object matching decision_schema. "
        "Do not claim authority, execute tools, mutate state, or add prose outside JSON."
    ),
    **bound_payload,
    "integrity": integrity,
    "runtime_claims": {
        "network_used": False,
        "api_token_used": False,
        "authoritative": False,
        "external_effect_enabled": False,
    },
}

def validate_bundle(document: Mapping[str, Any]) -> str:
    if not isinstance(document, Mapping):
        raise PortableBrainError("bundle must be an object")
    required = {
        "bundle_contract",
        "architecture_ref",
        "model_instruction",
        "brain_pack",
        "prompt_contract",
        "decision_schema",
        "exchange",
        "integrity",
        "runtime_claims",
    }
    if set(document) != required:
        raise PortableBrainError("portable bundle key set mismatch")
    if document["bundle_contract"] != "world-v6.portable-model-bundle.v1":
        raise PortableBrainError("unsupported portable bundle contract")
    bound_payload = {
        "brain_pack": document["brain_pack"],
        "prompt_contract": document["prompt_contract"],
        "decision_schema": document["decision_schema"],
        "exchange": document["exchange"],
    }
    expected = document["integrity"].get("bundle_payload_hash")
    observed = canonical_hash(bound_payload)
    if expected != observed:
        raise PortableBrainError("portable bundle payload hash mismatch")
    if document["runtime_claims"] != {
        "network_used": False,
        "api_token_used": False,
        "authoritative": False,
        "external_effect_enabled": False,
    }:
        raise PortableBrainError("portable bundle attempted to expand runtime claims")
    return observed

```

```
def validate_and_render(response_path: str | Path) -> dict[str, Any]:
    verify_pack_integrity()
    raw = _load_object(response_path)
    decision = normalize_secretary_decision(raw)
    pack = load_portable_pack(PACK_PATH)
    return {
        "decision": decision.to_document(),
        "semantic_hash": decision.semantic_hash,
        "rendered_text": render_secretary_decision(pack, decision),
        "pack_hash": pack.pack_hash,
        "proposal_only": True,
        "network_used": False,
        "api_token_used": False,
    }

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    subparsers = parser.add_subparsers(dest="command", required=True)

    export_parser = subparsers.add_parser("export-task")
    export_parser.add_argument("--task-json", required=True)
    export_parser.add_argument("--provider-label", default="universal-manual-host")

    bundle_parser = subparsers.add_parser("validate-bundle")
    bundle_parser.add_argument("--bundle-json", required=True)

    response_parser = subparsers.add_parser("validate-response")
    response_parser.add_argument("--response-json", required=True)

    args = parser.parse_args()
    if args.command == "export-task":
        output = build_task_bundle(args.task_json, args.provider_label)
    elif args.command == "validate-bundle":
        bundle = _load_object(args.bundle_json)
        output = {
            "bundle_payload_hash": validate_bundle(bundle),
            "valid": True,
            "network_used": False,
            "api_token_used": False,
        }
    else:
        output = validate_and_render(args.response_json)
    print(json.dumps(output, ensure_ascii=False, sort_keys=True, indent=2))

if __name__ == "__main__":
    main()
```

پیاده سازی: demo_portable_secretary.py

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

SHA-256 منبع:

world-v6.2-rc3/tools/demo_portable_secretary.py

4f47f50db48498c2c752a35eec004d5cc8d6348cd6c399212ede81feacc83baa

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
#!/usr/bin/env python3
"""Offline, no-token conformance demo for the portable Secretary Brain Pack."""
from __future__ import annotations

import argparse
import json
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "candidate-v6.2" / "runtime"))

from core.brain_gateway import BrainInputSegment, BrainRequest # noqa: E402
from core.portable_brain import ( # noqa: E402
    PortableSecretaryRunner,
    ScriptedPortableAdapter,
    load_portable_pack,
)
from core.resolution import load_profile_file # noqa: E402

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--provider", choices=("chatgpt", "gemini", "grok"), default="chatgpt")
    parser.add_argument("--title", default="ارسال پیش‌نویس قرارداد")
    args = parser.parse_args()

    candidate = ROOT / "candidate-v6.2"
    pack_dir = candidate / "brain-packs" / "secretary-001"
    profiles = [
        load_profile_file(path) for path in sorted((candidate / "profiles").glob("*v0.2.json"))
    ]
    task_profile = next(item for item in profiles if item.profile_id == "secretary.task")
    fixture = json.loads(
        (pack_dir / "fixtures" / f"{args.provider}.task-proposal.json").read_text(encoding="utf-8")
    )
    fixture["slots"]["title"] = args.title
    adapter = ScriptedPortableAdapter(
        f"{args.provider}-offline-fixture",
        {(task_profile.profile_id, task_profile.version, task_profile.profile_hash): "R1"},
        fixture,
        task_types=("secretary.task",),
    )
    runner = PortableSecretaryRunner(
        load_portable_pack(pack_dir / "portable-brain-pack.v1.0.0.json"),
        [adapter],
        profiles,
    )
    segment = BrainInputSegment(
        "task",
        {
            "task_id": "demo-task",
            "title": args.title,
            "status": "OPEN",
            "next_action": "DRAFT",
            "due_at": None,
            "priority": "NORMAL",
            "domain": "general",
        }
    )
```



```

        "goal_id": None,
        "created_at": "2026-08-20T00:00:00Z"
    },
    task_profile.profile_id,
    task_profile.version,
    task_profile.profile_hash,
    "task:demo-task",
    0,
    "R0",
    "R0",
    "TASK_PROPOSAL",
)
result = runner.run(
    BrainRequest("world-v6.brain-request.v2", "secretary.task", (segment,))
)
print(json.dumps({
    "provider": result.provider,
    "pack_hash": result.pack_hash,
    "semantic_hash": result.decision.semantic_hash,
    "rendered_text": result.rendered_text,
    "api_token_used": False,
    "network_used": False,
    "proposal_only": True,
}, ensure_ascii=False, sort_keys=True, indent=2))

if __name__ == "__main__":
    main()

```

پیاده سازی: secretary-conversation-profile.v0.2.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/profiles/secretary-conversation-profile.v0.2.json

SHA-256 منبع:

9dcf0d3656a4eb231d5444a229626e0f2110c479ab64043d8be37d266258eba7

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "profile_id": "secretary.conversation",
  "version": "0.2.0",
  "canonical_resolution": "R1",
  "allow_unspecified": false,
  "required_backbone_paths": [
    "/direction",
    "/text"
  ],
  "fields": [
    {
      "path": "/direction",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "INPUT",
      "semantic_contract_id": "secretary.conversation.direction.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/text",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "INPUT",
      "semantic_contract_id": "secretary.conversation.text.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": [
        "DRAFT_REPLY"
      ]
    },
    {
      "path": "/channel",
      "introduced_at": "R1",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "INPUT",
      "semantic_contract_id": "secretary.conversation.transport.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/brain_provider",
      "introduced_at": "R1",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "AUDIT",
      "semantic_contract_id": "secretary.conversation.brain-provider.v1",
      "data_class": "INTERNAL",
      "expected_type": null,
      "required_for_actions": []
    },
    {
      "path": "/created_at",
      "introduced_at": "R1",

```

```

    "write_min_resolution": "R1",
    "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
    "cable_id": "EVENT",
    "semantic_contract_id": "world-v6.time.occurred-at.v1",
    "data_class": "INTERNAL",
    "expected_type": "string",
    "required_for_actions": []
  },
  {
    "path": "/message_id",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
    "cable_id": "IDENTITY",
    "semantic_contract_id": "secretary.conversation.message-id.v1",
    "data_class": "INTERNAL",
    "expected_type": "string",
    "required_for_actions": []
  }
],
"actions": [
  {
    "action": "DRAFT_REPLY",
    "minimum_resolution": "R0",
    "required_paths": [
      "/direction",
      "/text"
    ]
  }
]
}
}

```

پیاده سازی: secretary-effect-profile.v0.2.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/profiles/secretary-effect-profile.v0.2.json

SHA-256 منبع:

0b0b8483ed9a061ad5ee1aeffa9169abf320d27894ce220c81adf6eb863d7c64

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "profile_id": "secretary.external-effect",
  "version": "0.2.0",
  "canonical_resolution": "R1",
  "allow_unspecified": false,
  "required_backbone_paths": [
    "/command_id",
    "/destination",
    "/action",
    "/resource_ref"
  ],
  "fields": [
    {
      "path": "/command_id",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "TASK_COMMAND",
      "semantic_contract_id": "world-v6.command.id.v1",
      "data_class": "RESTRICTED",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/destination",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "ACTION",
      "semantic_contract_id": "world-v6.effect.destination.v1",
      "data_class": "RESTRICTED",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/action",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "ACTION",
      "semantic_contract_id": "world-v6.effect.action.v1",
      "data_class": "RESTRICTED",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/resource_ref",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "ACTION",
      "semantic_contract_id": "world-v6.effect.resource-ref.v1",
      "data_class": "RESTRICTED",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/recipient_ref",
      "introduced_at": "R1",
```

```

    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "ACTION",
    "semantic_contract_id": "world-v6.effect.recipient-ref.v1",
    "data_class": "RESTRICTED",
    "expected_type": "string",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/payload_hash",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "ACTION",
    "semantic_contract_id": "world-v6.effect.payload-hash.v1",
    "data_class": "RESTRICTED",
    "expected_type": "string",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/effect_hash",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "APPROVAL",
    "semantic_contract_id": "world-v6.effect.effect-hash.v1",
    "data_class": "RESTRICTED",
    "expected_type": "string",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/approval_ref",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "APPROVAL",
    "semantic_contract_id": "world-v6.approval.ref.v1",
    "data_class": "RESTRICTED",
    "expected_type": "string",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/policy_decision_ref",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "POLICY",
    "semantic_contract_id": "world-v6.policy.decision-ref.v1",
    "data_class": "RESTRICTED",
    "expected_type": "string",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/control_epoch",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "POLICY",
    "semantic_contract_id": "world-v6.control-epoch.v1",
    "data_class": "RESTRICTED",
    "expected_type": "integer",
    "required_for_actions": [
        "EXECUTE_EXTERNAL"
    ]
},
{
    "path": "/expected_version",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "NO_DOWNGRADE",
    "cable_id": "STATE_MEMORY",

```

```

    "semantic_contract_id": "world-v6.expected-version.v1",
    "data_class": "RESTRICTED",
    "expected_type": "integer",
    "required_for_actions": [
      "EXECUTE_EXTERNAL"
    ]
  },
  "actions": [
    {
      "action": "EXECUTE_EXTERNAL",
      "minimum_resolution": "R1",
      "required_paths": [
        "/command_id",
        "/destination",
        "/action",
        "/resource_ref",
        "/recipient_ref",
        "/payload_hash",
        "/effect_hash",
        "/approval_ref",
        "/policy_decision_ref",
        "/control_epoch",
        "/expected_version"
      ]
    }
  ]
}

```

پیاده سازی: secretary-price-profile.v0.2.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/profiles/secretary-price-profile.v0.2.json

SHA-256 منبع:

a438385a85c0bb3757823436d16eb37f3c72a1ba3136d429a799f7e3a11a0ab6

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "profile_id": "secretary.price",
  "version": "0.2.0",
  "canonical_resolution": "R1",
  "allow_unspecified": false,
  "required_backbone_paths": [
    "/customer_id",
    "/product_key",
    "/unit",
    "/active"
  ],
  "fields": [
    {
      "path": "/customer_id",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "IDENTITY",
      "semantic_contract_id": "secretary.price.customer-id.v1",
      "data_class": "CONFIDENTIAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/product_key",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "IDENTITY",
      "semantic_contract_id": "secretary.price.product-key.v1",
      "data_class": "CONFIDENTIAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/unit",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "IDENTITY",
      "semantic_contract_id": "secretary.price.unit.v1",
      "data_class": "CONFIDENTIAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/active",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "STATE_MEMORY",
      "semantic_contract_id": "secretary.price.active.v1",
      "data_class": "CONFIDENTIAL",
      "expected_type": "boolean",
      "required_for_actions": []
    },
    {
      "path": "/unit_price_irr",
      "introduced_at": "R1",
```

```

        "write_min_resolution": "R1",
        "downgrade_policy": "NO_DOWNGRADE",
        "cable_id": "ACTION",
        "semantic_contract_id": "secretary.price.exact-irr-decimal-string.v1",
        "data_class": "CONFIDENTIAL",
        "expected_type": "string",
        "required_for_actions": [
            "BUILD_PROFORMA_DRAFT",
            "PRICE_LOOKUP"
        ]
    },
    {
        "path": "/valid_until",
        "introduced_at": "R1",
        "write_min_resolution": "R1",
        "downgrade_policy": "NO_DOWNGRADE",
        "cable_id": "POLICY",
        "semantic_contract_id": "secretary.price.valid-until.v1",
        "data_class": "CONFIDENTIAL",
        "expected_type": null,
        "required_for_actions": [
            "BUILD_PROFORMA_DRAFT"
        ]
    },
    {
        "path": "/approved_by",
        "introduced_at": "R1",
        "write_min_resolution": "R1",
        "downgrade_policy": "NO_DOWNGRADE",
        "cable_id": "APPROVAL",
        "semantic_contract_id": "world-v6.approval.approver-ref.v1",
        "data_class": "CONFIDENTIAL",
        "expected_type": "string",
        "required_for_actions": [
            "BUILD_PROFORMA_DRAFT"
        ]
    },
    {
        "path": "/approved_at",
        "introduced_at": "R1",
        "write_min_resolution": "R1",
        "downgrade_policy": "NO_DOWNGRADE",
        "cable_id": "APPROVAL",
        "semantic_contract_id": "world-v6.approval.issued-at.v1",
        "data_class": "CONFIDENTIAL",
        "expected_type": "string",
        "required_for_actions": [
            "BUILD_PROFORMA_DRAFT"
        ]
    }
],
"actions": [
    {
        "action": "PRICE_LOOKUP",
        "minimum_resolution": "R1",
        "required_paths": [
            "/customer_id",
            "/product_key",
            "/unit",
            "/unit_price_irr"
        ]
    },
    {
        "action": "BUILD_PROFORMA_DRAFT",
        "minimum_resolution": "R1",
        "required_paths": [
            "/customer_id",
            "/product_key",
            "/unit",
            "/unit_price_irr",
            "/valid_until",
            "/approved_by",
            "/approved_at"
        ]
    }
]
}

```


پیاده سازی: secretary-task-profile.v0.2.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/profiles/secretary-task-profile.v0.2.json

SHA-256 منبع:

312ffb4080bf60188d5bc8d29d8d29fecdad2dd1e5acfd3967bfb34629d31aa6

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "profile_id": "secretary.task",
  "version": "0.2.0",
  "canonical_resolution": "R1",
  "allow_unspecified": false,
  "required_backbone_paths": [
    "/task_id",
    "/title",
    "/status",
    "/next_action"
  ],
  "fields": [
    {
      "path": "/task_id",
      "introduced_at": "R0",
      "write_min_resolution": "R1",
      "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
      "cable_id": "TASK_COMMAND",
      "semantic_contract_id": "secretary.task.id.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/title",
      "introduced_at": "R0",
      "write_min_resolution": "R0",
      "downgrade_policy": "SAFE_TO_PROJECT",
      "cable_id": "TASK_COMMAND",
      "semantic_contract_id": "secretary.task.title.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": [
        "ADD_TASK"
      ]
    },
    {
      "path": "/status",
      "introduced_at": "R0",
      "write_min_resolution": "R0",
      "downgrade_policy": "SAFE_TO_PROJECT",
      "cable_id": "TASK_COMMAND",
      "semantic_contract_id": "secretary.task.status.v1",
      "data_class": "INTERNAL",
      "expected_type": "string",
      "required_for_actions": []
    },
    {
      "path": "/next_action",
      "introduced_at": "R0",
      "write_min_resolution": "R0",
      "downgrade_policy": "SAFE_TO_PROJECT",
      "cable_id": "TASK_COMMAND",
      "semantic_contract_id": "secretary.task.next-action.v1",
      "data_class": "INTERNAL",
      "expected_type": null,
      "required_for_actions": []
    }
  ]
}
```

```

    "path": "/due_at",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "SAFE_TO_PROJECT",
    "cable_id": "TASK_COMMAND",
    "semantic_contract_id": "secretary.task.due-at.v1",
    "data_class": "INTERNAL",
    "expected_type": null,
    "required_for_actions": []
  },
  {
    "path": "/priority",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "SAFE_TO_PROJECT",
    "cable_id": "TASK_COMMAND",
    "semantic_contract_id": "secretary.task.priority.v1",
    "data_class": "INTERNAL",
    "expected_type": "string",
    "required_for_actions": []
  },
  {
    "path": "/domain",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "SAFE_TO_PROJECT",
    "cable_id": "TASK_COMMAND",
    "semantic_contract_id": "secretary.task.domain.v1",
    "data_class": "INTERNAL",
    "expected_type": "string",
    "required_for_actions": []
  },
  {
    "path": "/goal_id",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "SAFE_TO_PROJECT",
    "cable_id": "TASK_COMMAND",
    "semantic_contract_id": "secretary.task.goal-id.v1",
    "data_class": "INTERNAL",
    "expected_type": null,
    "required_for_actions": []
  },
  {
    "path": "/created_at",
    "introduced_at": "R1",
    "write_min_resolution": "R1",
    "downgrade_policy": "READ_ONLY_WHEN_PROJECTED",
    "cable_id": "EVENT",
    "semantic_contract_id": "world-v6.time.created-at.v1",
    "data_class": "INTERNAL",
    "expected_type": "string",
    "required_for_actions": []
  }
],
"actions": [
  {
    "action": "ADD_TASK",
    "minimum_resolution": "R0",
    "required_paths": [
      "/title"
    ]
  },
  {
    "action": "UPDATE_TASK_DETAIL",
    "minimum_resolution": "R1",
    "required_paths": [
      "/task_id",
      "/domain",
      "/priority"
    ]
  }
]
}

```

پیاده سازی: secretary-decision.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/secretary-decision.schema.json

SHA-256 منبع:

945a50c612924db2dabf949cb58b55a8e132b5c83594fcfce9b2da8417c926ae

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc2/secretary-decision.schema.json",
  "title": "World v6.2 RC2 Portable Secretary Decision",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schema_version",
    "intent",
    "response_kind",
    "template_id",
    "slots",
    "proposed_actions",
    "requires_approval",
    "confidence_millis",
    "evidence_refs",
    "uncertainties"
  ],
  "properties": {
    "schema_version": {
      "const": "world-v6.secretary-decision.v1"
    },
    "intent": {
      "enum": [
        "GREET",
        "TASK_CAPTURE",
        "TASK_LIST",
        "DRAFT",
        "PRICE_LOOKUP",
        "CLARIFY",
        "UNKNOWN"
      ]
    },
    "response_kind": {
      "enum": [
        "ACKNOWLEDGE",
        "TASK_PROPOSAL",
        "TASK_LIST",
        "DRAFT_READY",
        "PRICE_RESULT",
        "CLARIFICATION_REQUEST",
        "SAFE_DEFER"
      ]
    },
    "template_id": {
      "$ref": "#/defs/token"
    },
    "slots": {
      "type": "object",
      "propertyNames": {
        "$ref": "#/defs/token"
      }
    },
    "additionalProperties": {
      "type": [
        "string",
        "integer",
        "boolean",
        "null"
      ]
    }
  }
}
```

```

    }
},
"proposed_actions": {
    "type": "array",
    "minItems": 1,
    "uniqueItems": true,
    "items": {
        "enum": [
            "NONE",
            "PROPOSE_TASK",
            "PROPOSE_DRAFT",
            "PROPOSE_PRICE_LOOKUP",
            "PROPOSE_EXTERNAL_EFFECT"
        ]
    }
},
"requires_approval": {
    "type": "boolean"
},
"confidence_millis": {
    "type": "integer",
    "minimum": 0,
    "maximum": 1000
},
"evidence_refs": {
    "type": "array",
    "items": {
        "$ref": "#/$defs/token"
    }
},
"uncertainties": {
    "type": "array",
    "items": {
        "type": "string",
        "maxLength": 500
    }
}
},
"allOf": [
    {
        "if": {
            "properties": {
                "proposed_actions": {
                    "contains": {
                        "const": "PROPOSE_EXTERNAL_EFFECT"
                    }
                }
            }
        },
        "then": {
            "properties": {
                "requires_approval": {
                    "const": true
                }
            }
        }
    }
],
"$defs": {
    "token": {
        "type": "string",
        "pattern": "^[A-Za-z0-9][A-Za-z0-9._:/@#-]{0,191}$"
    }
}
}

```

پیاده سازی: external-effect-proposal.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/external-effect-proposal.schema.json

SHA-256 منبع:

89f3c2a6cb864151b2c4c171485d1efe2dad63b7b580c2b18362553b3aa14225

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc1/external-effect-proposal.schema.json",
  "title": "World v6.2 RC1 External Effect Proposal",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "world_id",
    "entity_id",
    "command_id",
    "destination",
    "action",
    "resource_ref",
    "recipient_ref",
    "payload_ref",
    "payload_hash",
    "policy_version",
    "expected_version",
    "control_epoch",
    "idempotency_scope",
    "idempotency_key",
    "effect_semantics"
  ],
  "properties": {
    "world_id": {
      "type": "string",
      "minLength": 1
    },
    "entity_id": {
      "type": "string",
      "minLength": 1
    },
    "command_id": {
      "type": "string",
      "minLength": 1
    },
    "destination": {
      "type": "string",
      "minLength": 1
    },
    "action": {
      "type": "string",
      "minLength": 1
    },
    "resource_ref": {
      "type": "string",
      "minLength": 1
    },
    "recipient_ref": {
      "type": "string",
      "minLength": 1
    },
    "payload_ref": {
      "type": "string",
      "minLength": 1
    },
    "payload_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
    }
  }
}
```

```

    },
    "policy_version": {
      "type": "string",
      "minLength": 1
    },
    "expected_version": {
      "type": "integer",
      "minimum": 0
    },
    "control_epoch": {
      "type": "integer",
      "minimum": 1
    },
    "idempotency_scope": {
      "type": "string",
      "minLength": 1
    },
    "idempotency_key": {
      "type": "string",
      "minLength": 1
    },
    "effect_semantics": {
      "enum": [
        "NATIVE_IDEMPOTENT",
        "RECONCILABLE",
        "NON_IDEMPOTENT"
      ]
    }
  }
}

```

پیاده سازی: council-session.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/council-session.schema.json

SHA-256 منبع:

600feaa14634bf2c852be1505ba2038c292ae2e65ba2768d5b3f434a61c4fc51

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc2/council-session.schema.json",
  "title": "World v6.2 RC2 Multi-Brain Council Configuration",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "session_contract",
    "max_rounds",
    "minimum_support_millis",
    "blind_first_round",
    "context_binding_required",
    "high_risk_veto",
    "proposal_only",
    "participants"
  ],
  "properties": {
    "session_contract": {
      "const": "world-v6.brain-council.v1"
    },
    "max_rounds": {
      "type": "integer",
      "minimum": 1,
      "maximum": 5
    },
    "minimum_support_millis": {
      "type": "integer",
      "minimum": 501,
      "maximum": 1000
    },
    "blind_first_round": {
      "const": true
    },
    "context_binding_required": {
      "const": true
    },
    "high_risk_veto": {
      "const": true
    },
    "proposal_only": {
      "const": true
    },
    "participants": {
      "type": "array",
      "minItems": 2,
      "items": {
        "type": "object",
        "additionalProperties": false,
        "required": [
          "participant_id",
          "role",
          "weight_millis",
          "veto_capable"
        ],
        "properties": {
          "participant_id": {
            "type": "string",
            "minLength": 1
          },

```

```

    "role": {
      "type": "string",
      "minLength": 1
    },
    "weight_millis": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10000
    },
    "veto_capable": {
      "type": "boolean"
    }
  }
}
}
}
}
}

```


پیاده سازی: resolution-envelope.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/resolution-envelope.schema.json

SHA-256 منبع:

51f0a15771c9837886669bfb42ebb44bb1428f51243f72c5c83d26303bbc172

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc1/resolution-envelope.schema.json",
  "title": "World v6.2 RC1 Consumer-Safe Resolution Envelope",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "profile_id",
    "profile_version",
    "profile_hash",
    "source_ref",
    "source_version",
    "canonical_hash",
    "canonical_resolution",
    "effective_resolution",
    "projection_hash",
    "compiler_id",
    "compiler_version",
    "compiler_hash",
    "purpose",
    "data_class",
    "freshness",
    "projection_derived",
    "projection_authoritative",
    "value"
  ],
  "properties": {
    "profile_id": {
      "$ref": "#/$defs/token"
    },
    "profile_version": {
      "$ref": "#/$defs/token"
    },
    "profile_hash": {
      "$ref": "#/$defs/hash"
    },
    "source_ref": {
      "$ref": "#/$defs/token"
    },
    "source_version": {
      "type": "integer",
      "minimum": 0
    },
    "canonical_hash": {
      "$ref": "#/$defs/hash"
    },
    "canonical_resolution": {
      "$ref": "#/$defs/resolution"
    },
    "effective_resolution": {
      "$ref": "#/$defs/resolution"
    },
    "projection_hash": {
      "$ref": "#/$defs/hash"
    },
    "compiler_id": {
      "$ref": "#/$defs/token"
    },
    "compiler_version": {
```

```

    "$ref": "#/$defs/token"
  },
  "compiler_hash": {
    "$ref": "#/$defs/hash"
  },
  "purpose": {
    "$ref": "#/$defs/token"
  },
  "data_class": {
    "enum": [
      "PUBLIC",
      "INTERNAL",
      "CONFIDENTIAL",
      "RESTRICTED"
    ]
  },
  "freshness": {
    "$ref": "#/$defs/token"
  },
  "projection_derived": {
    "const": true
  },
  "projection_authoritative": {
    "const": false
  },
  "value": {
    "type": "object"
  }
},
"$defs": {
  "resolution": {
    "type": "string",
    "pattern": "^R(0|[1-9][0-9]*)$"
  },
  "hash": {
    "type": "string",
    "pattern": "^[0-9a-f]{64}$"
  },
  "token": {
    "type": "string",
    "pattern": "^[A-Za-z0-9][A-Za-z0-9._:/-]{0,191}$"
  }
}
}

```

پیاده سازی: brain-handler-profile.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/brain-handler-profile.schema.json

SHA-256 منبع:

dbc1a7b5d7cb0b4d962675aa75ae5d1f9e489071e058ca7a3044c8e52741491e

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc2/brain-handler-profile.schema.json",
  "title": "World v6.2 RC2 Brain Handler Profile",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "handler_id",
    "version",
    "kind",
    "maturity",
    "evidence",
    "supported_node_ids",
    "deterministic",
    "network_required",
    "api_token_required",
    "proposal_only",
    "provider_family"
  ],
  "properties": {
    "handler_id": {
      "$ref": "#/$defs/token"
    },
    "version": {
      "$ref": "#/$defs/token"
    },
    "kind": {
      "enum": [
        "CODE",
        "BRAIN",
        "MANUAL_HOST",
        "LOCAL_MODEL",
        "COUNCIL"
      ]
    },
    "maturity": {
      "type": "string",
      "pattern": "^C[0-6]$"
    },
    "evidence": {
      "type": "string",
      "pattern": "^E[0-5]$"
    },
    "supported_node_ids": {
      "type": "array",
      "minItems": 1,
      "uniqueItems": true,
      "items": {
        "$ref": "#/$defs/token"
      }
    },
    "deterministic": {
      "type": "boolean"
    },
    "network_required": {
      "type": "boolean"
    },
    "api_token_required": {
      "type": "boolean"
    }
  }
}
```

```

    },
    "proposal_only": {
      "const": true
    },
    "provider_family": {
      "$ref": "#/$defs/token"
    }
  },
  "$defs": {
    "token": {
      "type": "string",
      "pattern": "^[A-Za-z0-9][A-Za-z0-9._:/@#-]{0,191}$"
    }
  }
}

```

پیاده سازی: compilation-maturity.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/compilation-maturity.schema.json

SHA-256 منبع:

df535e22946d4c1b921febae0f2b476fad0339efeb8cc5aa5bdf69ffb0a77cc2

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc2/compilation-maturity.schema.json",
  "title": "World v6.2 RC2 Compilation Maturity State",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "candidate_id",
    "candidate_version",
    "candidate_hash",
    "maturity",
    "promotion_mode",
    "self_mutation"
  ],
  "properties": {
    "candidate_id": {
      "type": "string",
      "minLength": 1
    },
    "candidate_version": {
      "type": "string",
      "minLength": 1
    },
    "candidate_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
    },
    "maturity": {
      "enum": [
        "C0",
        "C1",
        "C2",
        "C3",
        "C4",
        "C5",
        "C6"
      ]
    },
    "promotion_mode": {
      "const": "ONE_STEP_EVIDENCE_GATED"
    },
    "self_mutation": {
      "const": "FORBIDDEN"
    }
  }
}
```

پیاده سازی: portable-brain-pack.schema.json

وضعیت سند: SOURCE SNAPSHOT / NOT DEPLOYED

منبع محلی:

world-v6.2-rc3/candidate-v6.2/schemas/portable-brain-pack.schema.json

SHA-256 منبع:

e8a7ecbb1d1df932489d23e7adeb95c1270ef108e0c9fd2016dba991998b9784

متن این فایل بدون حذف بخش ها در PDF درج شده است.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://world-v6.local/schemas/v6.2-rc2/portable-brain-pack.schema.json",
  "title": "World v6.2 RC2 Portable Brain Pack",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "pack_id",
    "version",
    "world_id",
    "entity_id",
    "identity_ref",
    "dna_ref",
    "dna_hash",
    "root_constitution_ref",
    "prompt_contract_ref",
    "prompt_contract_hash",
    "output_contract_ref",
    "output_contract_hash",
    "templates",
    "allowed_task_types",
    "required_profile_bindings",
    "proposal_only",
    "api_optional"
  ],
  "properties": {
    "pack_id": {
      "$ref": "#/$defs/token"
    },
    "version": {
      "$ref": "#/$defs/token"
    },
    "world_id": {
      "$ref": "#/$defs/token"
    },
    "entity_id": {
      "$ref": "#/$defs/token"
    },
    "identity_ref": {
      "$ref": "#/$defs/token"
    },
    "dna_ref": {
      "$ref": "#/$defs/token"
    },
    "dna_hash": {
      "$ref": "#/$defs/hash"
    },
    "root_constitution_ref": {
      "$ref": "#/$defs/token"
    },
    "prompt_contract_ref": {
      "$ref": "#/$defs/token"
    },
    "prompt_contract_hash": {
      "$ref": "#/$defs/hash"
    },
    "output_contract_ref": {
      "$ref": "#/$defs/token"
    }
  },
}
```

```

"output_contract_hash": {
  "$ref": "#/$defs/hash"
},
"templates": {
  "type": "object",
  "minProperties": 1,
  "propertyNames": {
    "$ref": "#/$defs/token"
  },
  "additionalProperties": {
    "type": "string",
    "minLength": 1,
    "maxLength": 4000
  }
},
"allowed_task_types": {
  "type": "array",
  "minItems": 1,
  "uniqueItems": true,
  "items": {
    "$ref": "#/$defs/token"
  }
},
"required_profile_bindings": {
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "profile_id",
      "version",
      "sha256"
    ],
    "properties": {
      "profile_id": {
        "$ref": "#/$defs/token"
      },
      "version": {
        "$ref": "#/$defs/token"
      },
      "sha256": {
        "$ref": "#/$defs/hash"
      }
    }
  }
},
"proposal_only": {
  "const": true
},
"api_optional": {
  "const": true
}
},
"$defs": {
  "token": {
    "type": "string",
    "pattern": "^[A-Za-z0-9][A-Za-z0-9._:/@#-]{0,191}$"
  },
  "hash": {
    "type": "string",
    "pattern": "[0-9a-f]{64}$"
  }
}
}

```

وضعیت نهایی پروژه Secretary-001

PostgreSQL Truth گام بعدی اجرای PRE-BIRTH / PLANNING BUNDLE / NOT DEPLOYED / NOT CANONICAL. Spine و World API است، نه افزودن Brain جدید.